Stefan Björnander

C++17 By Example

Practical projects to get you up and running with C++17



Packt>

C++17 By Example

版权所有 © 2018 Packt Publishing

保留所有权利。未经出版商事先书面许可，不得以任何形式或任何方式复制、存储于检索系统或以任何方式传播本书的任何部分，但在评论性文章或评论中引用少量片段的情形除外。

本书在编写过程中已尽一切努力确保所提供信息的准确性。但是，本书所含信息在销售时不附带任何明示或默示的担保。对于直接或间接因本书引起的或声称因本书引起的任何损失，作者、Packt Publishing 及其经销商和分销商均不承担责任。

Packt Publishing 已尽力通过恰当使用首字母大写等方式提供本书中提及的所有公司和产品的商标信息。但 Packt Publishing 无法保证这些信息的准确性。

组稿编辑：Merint Mathew
策划编辑：Chaitanya Nair
内容开发编辑：Lawrence Veigas
技术编辑：Adhithya Haridas
文字编辑：Safis Editing
项目协调：Prajakta Naik
审读：Safis Editing
索引编制：Aishwarya Gangawane
图形：Jisha Chirayil
出版协调：Deepika Naik

首次出版：2018 年 2 月

出版参考号：1220218

出版方：Packt Publishing Ltd.
Livery Place
35 Livery Street
Birmingham
B3 2PB, UK.

ISBN 978-1-78839-181-8

www.packtpub.com

我谨将本书献给我的父母拉尔夫(Ralf)与古妮拉(Gunilla)、我的妹妹卡塔琳娜(Catharina)、她的丈夫马格努斯（Magnus），以及他们的儿子埃米尔（Emil）和拉穆斯（Rasmus)。



mapt.io

Mapt 是一家在线数字图书馆，让你可以完整访问 5,000 多本图书和视频，以及业内领先的工具，帮助你规划个人发展、推进职业生涯。欲了解更多信息，请访问我们的网站。

为什么要订阅?

- 借助来自 4,000 多位业内专业人士的实用电子书和视频，少花时间学习，多花时间编码
- 配合为你量身打造的学习计划（Skill Plans），提升学习效果
- 每月获赠一本免费电子书或视频
- Mapt 的内容可全文检索
- 可以复制粘贴、打印并收藏内容

PacktPub.com

你知不知道 Packt 为每一本出版的图书都提供电子书版本，有 PDF 和 EPUB 文件可供选择? 你可以在 www.PacktPub.com 升级到电子书版本；作为纸质书读者，你还可以享受电子书副本的折扣。欲了解详情，请致电 service@packtpub.com 与我们联系。

在 www.PacktPub.com 上，你还可以阅读一系列免费的技术文章、订阅多种免费新闻通讯，并获得 Packt 图书和电子书的专属折扣与优惠。

贡献者

关于作者

斯蒂芬 · 比约南德（Stefan Björnander）著有《Microsoft Windows C++》与《C++ Windows Programming》两本书。他拥有工程硕士学位与计算机科学副博士学位，多年来一直从事软件开发工作，也担任计算机科学与数学教师。

关于审校

马克 · 埃尔顿（Mark Elston）是一家自动化测试设备公司的软件架构师，主要工作于 IC 与移动设备测试领域。他有 30 年的从业经验，包括为美国空军和海军开发飞机与导弹仿真、为 NASA 开发硬件控制系统，以及为商业产品开发测试器操作系统。出于兴趣，他还开发过若干 Android 应用。他最近的热情所在是探索函数式编程与设计的世界。

我要感谢我的妻子在我需要完成一章审校时给予的理解。我也要感谢 Packt 团队给我机会参与这个项目，这既开阔眼界，又充满乐趣。最后，我要感谢作者，他连我最细小的意见都认真考虑。能参与一个重视你投入的项目，实在是一件乐事。

Packt 正在寻找像你一样的作者

如果你有兴趣成为 Packt 的作者，请访问 authors.packtpub.com 立即报名。我们已经与成千上万像你一样的开发者和技术专业人员合作，帮助他们把自己的见解分享给全球技术社区。你可以提交一般申请，也可以申请我们正在招募作者的特定热门主题，或者提交你自己的想法。

目录

关于作者 **5**
关于审校 **5**
Packt 正在寻找像你一样的作者 **5**
前言 **1**
联系我们 **4**
留下一条书评 4
Chapter 1: 初识 C++ **5**
掷骰子 **5**
理解类——Car 类 **12**
扩展 Car 类 17
类层次结构——Person、Student 与 Employee 类 **20**
简单数据类型——栈 **24**
更高级的数据类型——队列 **27**
小结 **30**
Chapter 2: 数据结构与算法 **31**
列表类 **31**
Cell 类 32
Iterator 类 33
LinkedList 类 34
向已有列表加入列表 38
从列表中删除值 41
集合类 **44**
并集、交集与差集运算 49
基础搜索与排序 **51**
选择排序算法 52
插入排序算法 53
冒泡排序算法 54
扩展的列表类 **56**
ReverseIterator 类 59
扩展的集合类 **72**
并集、交集与差集 74

高级搜索与排序 **79**
归并排序算法 80
快速排序算法 81
小结 **82**
Chapter 3: 构建图书馆管理系统 **84**
Book 类 **84**
写入图书 87
读取图书 88
图书的借出与预约 88
显示图书 89
Customer 类 **90**
从文件读取读者 91
将读者写入文件 92
读者借阅与预约图书 93
显示读者 93
Library 类 **94**
查找图书与读者 98
添加图书 99
删除图书 99
列出图书 100
添加读者 101
删除读者 101
列出读者 102
借出图书 102
预约图书 104
归还图书 105
将图书馆信息保存到文件 107
从文件加载图书馆信息 107
main 函数 **108**
小结 **109**
Chapter 4: 基于指针的图书馆管理系统 **110**
Book 类 **110**
读取与写入图书 112
借阅与预约图书 113
显示图书 113
Customer 类 **114**

读取与写入顾客 116
借阅与预约图书 116
显示顾客 116
Library 类 117
查找图书与顾客 120
新增图书 121
删除图书 121
列出图书 123
新增顾客 123
删除顾客 124
列出顾客 125
借阅图书 125
预约图书 126
归还图书 128
查找图书与顾客（索引与指针） 129
Marshmallowing（棉花糖化） 130
将图书馆信息保存到文件 131
从文件加载图书馆信息 134
释放内存 137
main 函数 137
小结 138
Chapter 5: Qt 图形应用 139
创建时钟应用程序 139
环境搭建 139
Clock 类 140
main 函数 144
为窗口和部件搭建可复用的类 146
添加监听器 147
基窗口类 147
基部件类 154
构建画图程序 164
Figure 基类 164
Line 子类 167
Rectangle 子类 172
Ellipse 子类 175
绘制窗口 176

绘制部件 179
main 函数 187
构建编辑器 188
Caret 类 188
绘制编辑器窗口 190
绘制编辑器部件 192
main 函数 201
小结 202
Chapter 6: 增强 Qt 图形应用 203
改进时钟 203
Clock 类 203
main 函数 206
改进画图程序 207
Figure 类 207
Line 类 209
Rectangle 类 215
Ellipse 类 221
DrawingWindow 类 225
DrawingWidget 类 227
main 函数 242
改进编辑器 242
EditorWindow 类 243
EditorWidget 类 245
main 函数 264
小结 265
Chapter 7: 游戏 266
奥赛罗棋 266
游戏部件 267
圈叉棋 282
NaCWindow 类 282
小结 289
Chapter 8: 计算机对弈 291
奥赛罗棋 291
OthelloWindow 类 291
圈叉棋 303

NaCWindow 类 303
小结 314
Chapter 9: 领域特定语言 315
引入源语言——一个简单示例 315
源语言的文法 317
目标语言 319
颜色 320
错误处理 321
值 323
扫描器 325
构建解析器 333
解析语言中的指令 335
解析语言中的表达式 337
表达式的类型检查 341
求值表达式的值 342
查看器 343
main 函数 348
小结 349
Chapter 10: 高级领域特定语言 350
通过示例改进源语言 351
改进文法 353
Token 与扫描器 353
解析器 354
求值器 363
main 函数 370
小结 371
其他你可能会喜欢的书籍 372
索引 375

前言

C++ 是一种通用编程语言，最初的设计偏向嵌入式编程与系统编程。多年来，C++ 不断演进，如今被用于开发众多不同领域的软件。凭借其多面性与健壮性，C++ 是开启编程之旅的绝佳语言。本书涵盖用 C++ 构建的一系列精彩项目，展示如何在不同场景中运用这门语言。在开发这些项目的过程中，你不仅会学到语言的各个构造，还会学到如何用 C++ 满足自己的软件需求。

本书研究一组用 C++ 编写的应用程序，涵盖抽象数据类型、图书馆管理系统、图形应用、游戏以及领域特定语言（DSL）。

本书面向的读者

本书面向想用 C++ 开发软件的开发者。具备基本的编程经验会是额外的优势。

本书涵盖的内容

第 1 章「初识 C++」，介绍 C++ 的面向对象编程。我们先研究一个掷骰子的简单程序：编写代码，然后编译、链接并执行。接着构造一个简单的面向对象层次结构，涉及指针与动态绑定。最后创建两个简单的抽象数据类型：栈与队列。栈是自底向上排列的一组值，只有栈顶的值可访问；队列则是传统意义上的队列，在前端读取值、在后端加入值。

第 2 章「数据结构与算法」，在上章所学的基础上展开，特别是列表与集合抽象数据类型。我们还引入模板与运算符重载，并研究线性搜索与二分搜索算法，以及插入排序、选择排序、冒泡排序、归并排序与快速排序算法。

第 3 章「构建图书馆管理系统」，帮助你开发一个真实系统：一个由图书和读者组成的图书馆管理系统。图书记录哪些读者借阅和预约了它，读者则记录自己借阅和预约了哪些图书。

第 4 章「基于指针的图书馆管理系统」，进一步发展图书馆管理系统。上一章中每本图书和每个读者都用整数编号标识；而在本章中，我们改用指针。每本图书持有指向借阅或预约它的读者的指针，每个读者则持有指向自己借阅或预约的图书的指针。

第 5 章「Qt 图形应用」，深入三个用 Qt 图形库开发的图形应用：一个带时针、分针和秒针的模拟时钟，一个用不同颜色绘制直线、矩形和椭圆的画图程序，以及一个让用户输入并编辑文本的编辑器。我们将学习在 Qt 库中处理窗口与部件、菜单与工具栏，还会学习如何绘制图形、书写文本，以及如何捕获鼠标和键盘输入。

第 6 章「增强 Qt 图形应用」，进一步发展三个图形应用：模拟时钟、画图程序和编辑器。我们

为钟面添加数字，为画图程序添加移动、修改与剪切粘贴图形的功能，并为编辑器添加更改字体与文本对齐方式的功能。

第 7 章「游戏」，介绍基础的游戏开发。我们在本章用 Qt 库开发奥赛罗棋和圈叉棋两种游戏。在奥赛罗棋中，双方轮流在棋盘上放置黑色和白色的棋子，以包围对手的棋子；在圈叉棋中，双方轮流放置圆圈和叉号，以将五个棋子连成一行。

第 8 章「计算机对弈」，让计算机与人类玩家对弈。在奥赛罗棋中，计算机试图放置棋子以包围尽可能多的对手棋子；在圈叉棋中，计算机试图放置棋子使自己的五个棋子连成一行，同时阻止对手连成一行。

第 9 章「领域特定语言」，教你开发领域特定语言（DSL），即面向特定领域的语言。更具体地说，我们开发一种在 Qt 部件上书写图形对象的语言。我们先用文法形式化地定义这门语言，然后编写识别有意义字符序列的扫描器、检查源代码是否符合文法的解析器，以及显示图形对象的查看器。

第 10 章「高级领域特定语言」，从多个方面改进我们的领域特定语言：添加改变程序流程的选择与迭代，添加可在程序执行期间赋值的变量，并添加带参数与返回值的函数。

充分利用本书

本书面向每一位读者，从初学者到较为熟练的 C++ 程序员皆可阅读。不过，具备一些 C++ 经验会更有帮助。

本书的示例在 Visual Studio 与 Qt Creator 中开发。

下载示例代码文件

你可以从 www.packtpub.com 的个人账户下载本书的示例代码文件。如果你是在别处购买的本书，可以访问 www.packtpub.com/support 并注册，以便将文件直接发送到你的邮箱。

你可以按以下步骤下载代码文件：

1. 登录或注册 www.packtpub.com。
2. 选择 SUPPORT 标签页。
3. 点击 Code Downloads & Errata。
4. 在搜索框输入书名，并按照屏幕上的指示操作。

文件下载完成后，请确保用以下软件的最新版本解压文件夹：

- Windows：WinRAR/7-Zip
- Mac：Zipeg/iZip/UnRarX
- Linux：7-Zip/PeaZip

本书的代码包也托管在 GitHub：https://github.com/PacktPublishing/CPP17-By-Example。我们丰富的图书和视频目录中的其他代码包也发布在 https://github.com/PacktPublishing/。欢迎查看！

下载彩色图片

我们还提供一个 PDF 文件，收录本书中使用的截图/示意图的彩色版本，可在此下载：https://www.packtpub.com/sites/default/files/downloads/CPP17ByExample_ColorImages.pdf。

本书中使用的约定

本书使用了若干文本约定。

`CodeInText`：表示正文中的代码字词。例如：「我们继续看一个类层次结构，其中 `Person` 是基类，`Student` 与 `Employee` 是它的派生类：」

代码块按如下方式排版：

```
class Person {
   public:
     Person(string name);
     virtual void print();
   private:
     string m_name;
};
```

当我们希望你注意代码块中的特定部分时，相关的行或条目会以加粗显示（原书中 `print` 一行为加粗等字体；本译稿统一以正体呈现）：

```
class Person {
   public:
     Person(string name);
     virtual void print();
   private:
     string m_name;
};
```

Bold：表示新术语、重要词语，或者你在屏幕上看到的字词。例如，菜单或对话框中的词语在正文中显示如下：「在第一个对话框中，我们只需按下 Next 按钮。」



重要提示或注意事项以此样式出现。



提示与技巧以此样式出现。

联系我们

我们始终欢迎读者的反馈。

一般反馈：请发邮件至 feedback@packtpub.com，并在邮件主题中注明书名。如果你对本书的任何方面有疑问，请发邮件至 questions@packtpub.com。

勘误：尽管我们在内容的准确性上已经尽最大努力，错误仍在所难免。如果你发现书中有错误，恳请告知。请访问 www.packtpub.com/submit-errata，选择你的书籍，点击勘误提交表单链接，并输入详情。

盗版：如果你在互联网上遇到任何形式的非法复制本，恳请告知我们地址或网站名称，并附上相关链接，联系 copyright@packtpub.com。

成为作者：如果你精通某个主题并有兴趣写书或参与写作，请访问 authors.packtpub.com。

留下一条书评

请留下一条书评。读完并使用本书之后，不妨在你购买本书的站点上留下一条书评。这样，潜在读者可以看到并借鉴你的客观意见，做出购买决定；我们 Packt 也能了解你对产品的看法，我们的作者也能看到你对其著作的反馈。谢谢！

要了解更多关于 Packt 的信息，请访问 packtpub.com。

1
初识 C++

本章介绍 C++ 的面向对象编程（Object-Oriented Programming，OOP）。我们先研究一个掷骰子的简单程序：编写代码，然后编译、链接并执行程序。

接着构造一个简单的面向对象层次结构，涉及基类 `Person` 及其两个派生类 `Student` 和 `Employee`。我们还会研究指针与动态绑定。

最后，我们创建两个简单的数据类型——栈和队列。栈由自底向上排列的一组值构成，我们只对栈顶的值感兴趣。队列则是传统意义上的值队列：在后端加入值，在前端读取值。

本章将涵盖以下主题：

- 我们先实现一个简单的游戏：掷骰子。它的主要目的是让你熟悉开发环境，教会你如何建立工程，以及如何编译、链接并执行程序。
- 然后开始接触面向对象编程：编写一个类层次结构，以 `Person` 为基类，`Student` 和 `Employee` 为派生类。这会介绍继承、封装与动态绑定。
- 最后，我们为抽象数据类型栈和队列编写类。栈是一种既在顶端加入值、又从顶端移除值的结构；而队列更接近传统意义上的队列：在后端加入值，从前端移除值。

掷骰子

作为入门，我们先编写一个掷骰子的程序。我们使用内置的随机数生成器产生 1 到 6 之间（含两端）的整数值：

`Main.cpp`

```
#include <CStdLib>
#include <CTime>
#include <IOStream>
```

```
using namespace std;

void main() {
  srand((int) time(nullptr));
  int dice = (rand() % 6 ) + 1;
  cout << "Dice: " << dice << endl;
}
```

在前面的程序中，开头的 include 指令用来包含头文件，头文件里大多存放着标准库的声明。我们需要 CStdLib 头文件来使用随机数生成器，需要 CTime 头文件用当前时间初始化随机数生成器，还需要 IOStream 头文件来输出结果。

标准库存放在名为 std 的命名空间中。命名空间可以看作一个存放代码的容器。我们通过 using namespace 指令获得对标准库的访问。

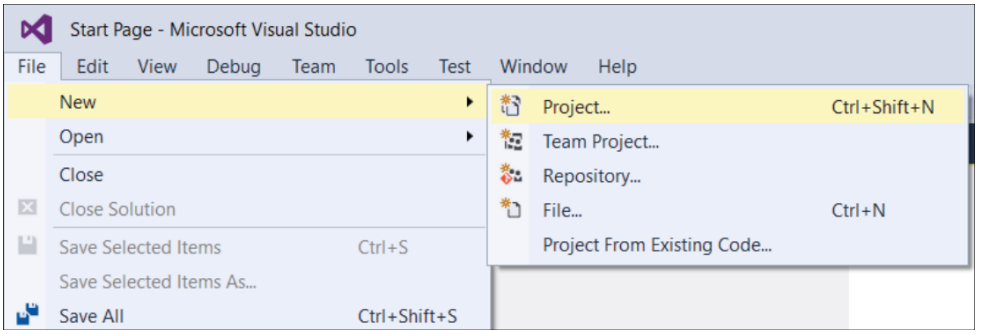
每个 C++ 程序都有且只有一个 main 函数，程序的执行总是从 main 函数开始。我们用标准函数 srand 和 time 初始化随机数生成器，用 rand 产生实际的随机值。百分号（%）是取模运算符，它把两个整数相除并给出余数。这样，整型变量 dice 的值就总在 1 到 6 之间。

最后，我们用 cout 输出变量 dice 的值。cout 是标准库用来输出文本和值的对象。

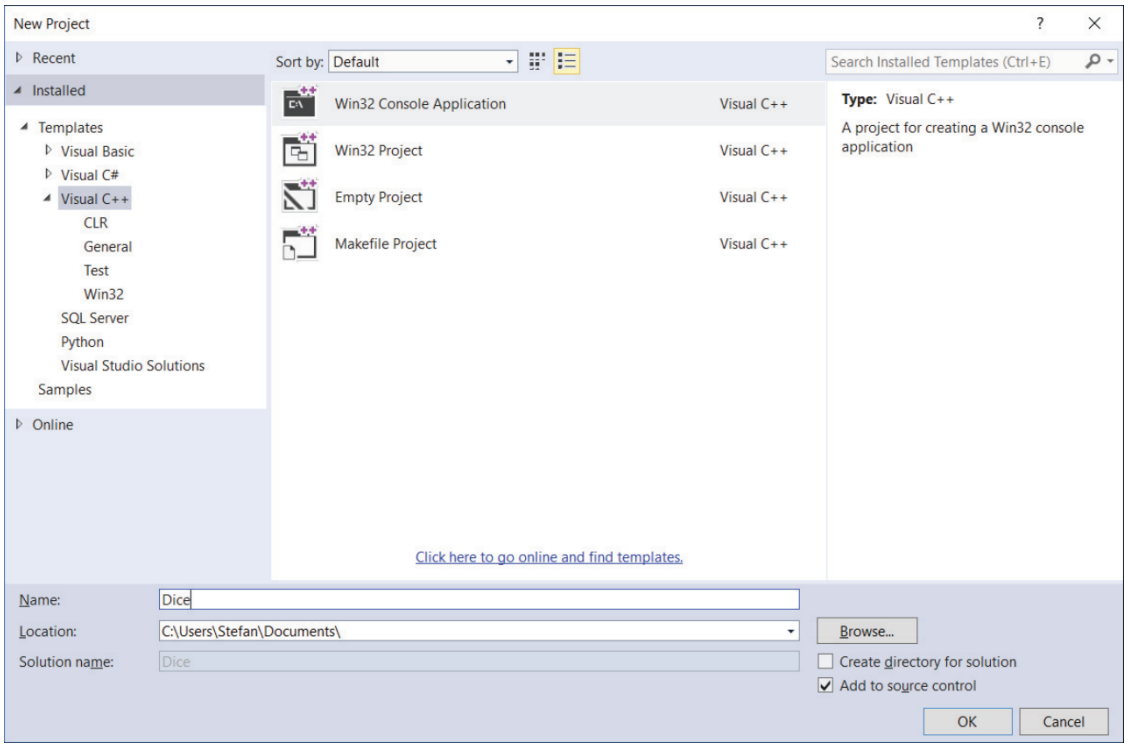
前四章的程序使用 Visual Studio 编写，其余各章的程序则使用 Qt Creator 编写。

下面说明如何创建工程、编写代码并运行应用程序。启动 Visual Studio 后，我们按以下步骤创建工程：

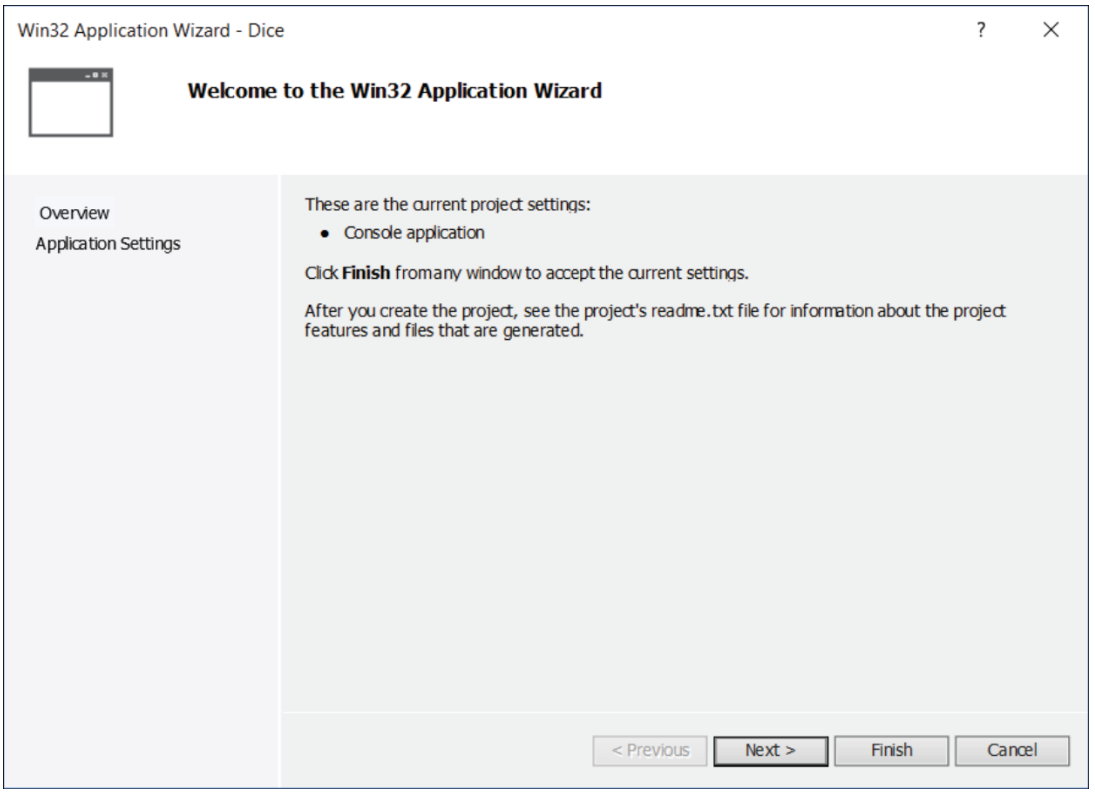
1. 首先，在 File 菜单中选择 New 和 Project 两个菜单项，如下面的截图所示：



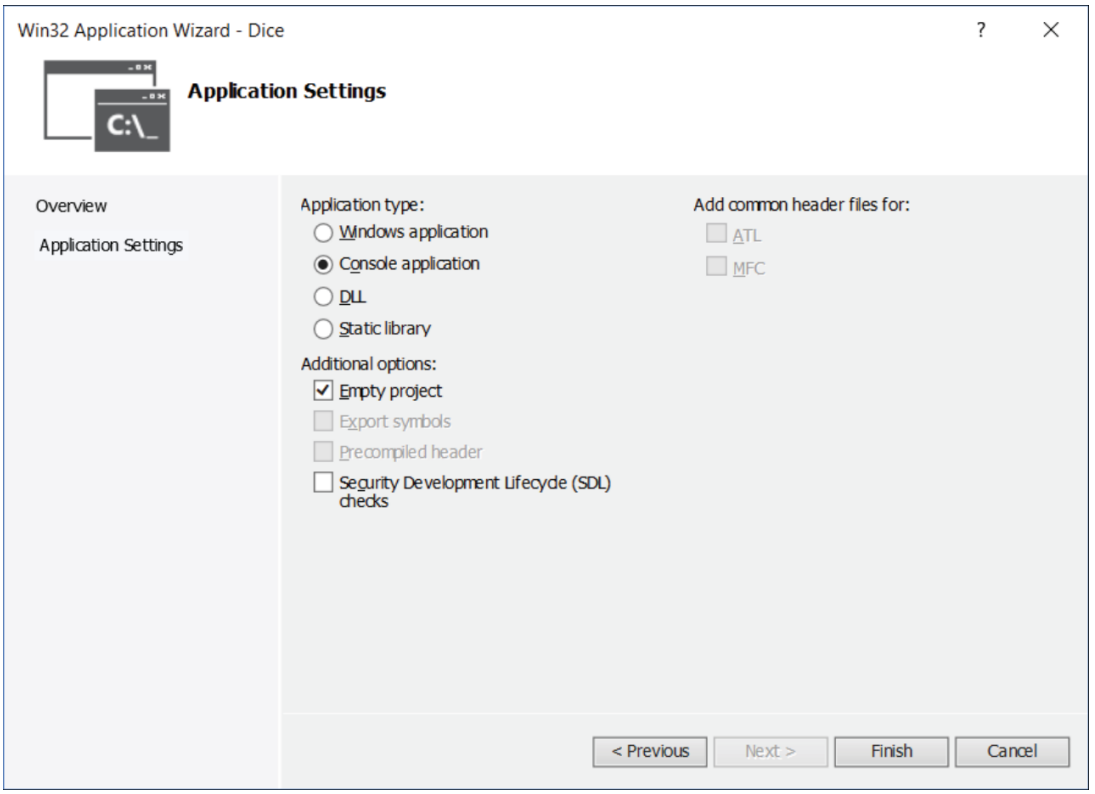
2. 我们选择 Win32 Console Application 类型，并把工程命名为 Dice：



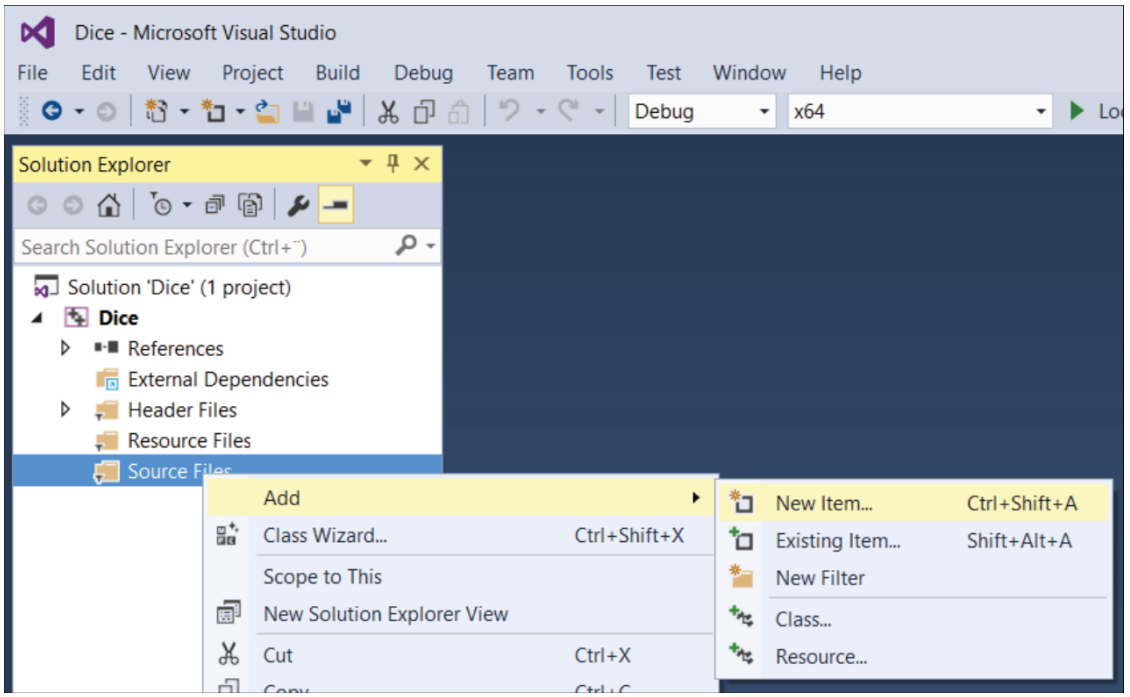
3. 在第一个对话框中，我们只需按下 Next 按钮：



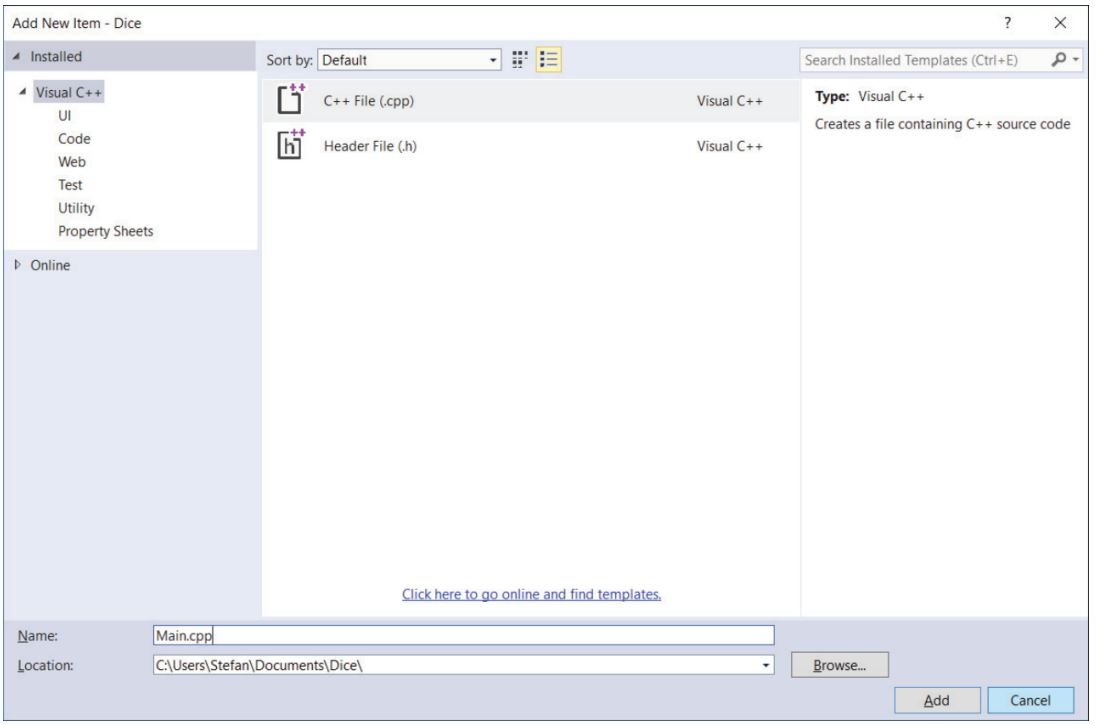
4. 在第二个对话框中，我们勾选 Empty project 复选框并点击 Finish 按钮。这样将创建一个不含文件的工程：

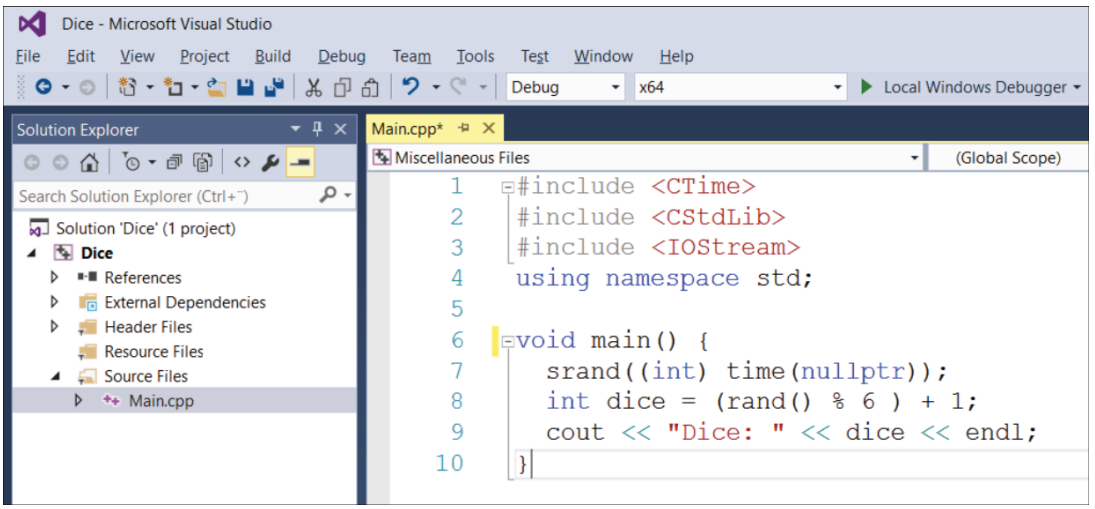


5. 创建好工程后，我们需要添加一个文件：



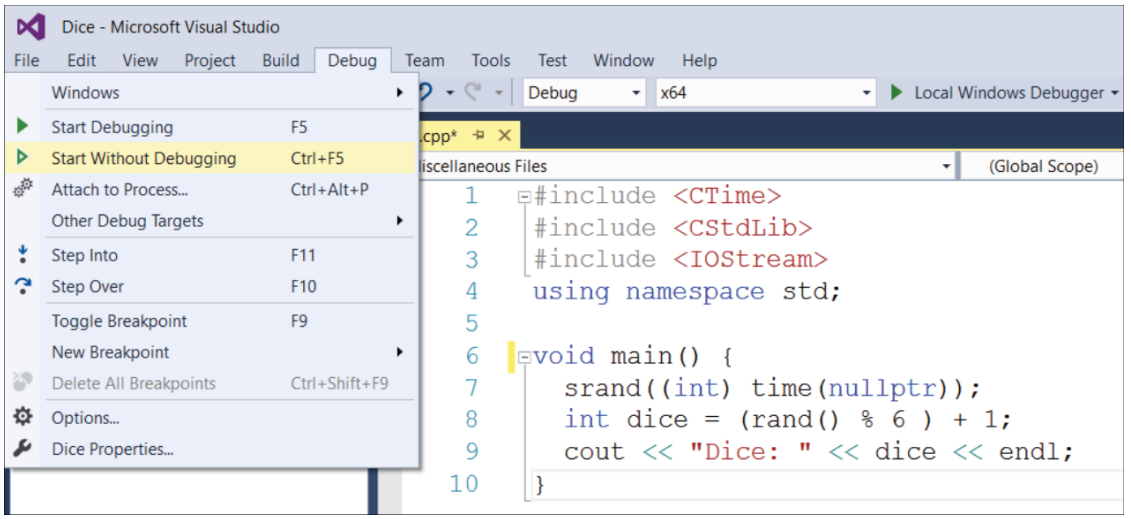
6. 我们选择 C++ File(.cpp) 类型，并把它命名为 Main.cpp：



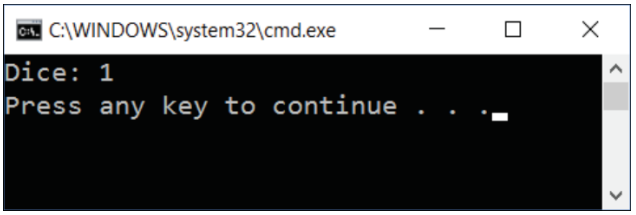


7. 然后，我们在 Main.cpp 文件中输入代码：

8. 最后，我们运行程序。最简单的方法是选择 Start Debugging 或 Start Without Debugging 菜单选项。这样程序就会被编译、链接并执行：



9. 程序的输出显示在一个命令窗口中：



理解类——Car 类

我们继续来看一个处理汽车的简单类，包括汽车的速度和方向。类是面向对象语言中一个非常核心的特性。在 C++ 中，类的规格说明由两部分组成——定义与实现。定义部分通常放在头文件（后缀为 .h）中，而实现部分放在后缀为 .cpp 的文件中，如 Car.h 和 Car.cpp 文件。不过，模板类（将在第 3 章「构建图书馆管理系统」中介绍）只存放在一个文件里。

类由其成员组成，成员可以是字段或方法。字段存放某个特定类型的值。方法是一种数学抽象，可以接受输入值并返回一个值。方法的输入值称为参数。不过在 C++ 中，也可以定义没有参数、没有返回类型的函数。

对象是类的实例；同一个类可以创建许多对象。方法可以分为以下几类：

- **构造函数**：在对象被创建时调用
- **读取器**：读取器读取类的字段
- **修改器**：修改器修改字段的值

- **析构函数**：在对象被销毁时调用

理想情况下，类的方法不应提供对字段的直接访问，否则一旦字段发生变化，方法的名称/类型也得跟着变。方法应当访问的是类的属性。属性是类的概念性要素，未必对应单个字段。类的每个成员都是 public、protected 或 private 的：

- **public 成员**：程序的所有其他部分都可以访问。
- **protected 成员**：只有它自己的成员或其派生类（下一节介绍）的成员才能访问。
- **private 成员**：只有它自己的成员才能访问。但这并不完全成立——类可以邀请其他类成为它的友元（friend），此时后者就能访问它的 private 和 protected 成员。我们将在下一章讨论友元。

下面的 `Car` 类定义有两个构造函数和一个析构函数。在本例中，它们总是与 `Car` 类同名。析构函数前面带有波浪号（~）。没有参数的构造函数称为默认构造函数。



多个方法可以同名，只要它们的参数列表不同即可，这称为重载。更准确地说，这称为上下文无关重载。还有上下文相关重载：两个方法同名且参数列表相同，但返回类型不同。不过，C++ 不支持上下文相关重载。

因此，一个类可以拥有多个构造函数，只要它们的参数列表不同。但析构函数不允许带参数，所以一个类只能有一个析构函数：

`Car.h`

```
class Car {
  public:
    Car();
    Car(int speed, int direction);
    ~Car();
```

`getSpeed` 和 `getDirection` 方法是读取器，返回汽车当前的速度和方向。返回值的类型是 `int`，即整数（integer）的缩写。它们用 `const` 关键字标记为常量方法，因为它们不修改类的字段。不过，构造函数和析构函数不能是常量的：

```
int getSpeed() const;
int getDirection() const;
```

`accelerate`、`decelerate`、`turnLeft` 和 `turnRight` 方法是修改器，设置汽车当前的速度和方向。它们会修改类的字段，因此不能标记为常量：

```
void accelerate(int speed);
void decelerate(int speed);
void turnLeft(int degrees);
void turnRight(int degrees);
```

m_speed 和 m_direction 字段存放汽车当前的速度和方向。前缀 m_ 表示它们是类的成员，而不是方法内的局部变量：

```
  private:
    int m_speed, m_direction;
};
```

在实现文件中，我们必须包含 Car.h 头文件。#include 指令属于预处理器的组成部分，它只是把 Car.h 文件的内容包含进来。上一节中我们用尖括号（< 和 >）包含系统文件，而这里我们用引号（"）包含本地文件。系统包含文件（尖括号）包含的是语言本身自带的系统代码，而本地包含文件（引号）包含的是我们自己在工程中编写的代码。从技术上讲，系统包含文件通常从文件系统的某个特殊目录中包含进来，而本地包含文件则通常在文件系统的本地位置包含：

Car.cpp

```
#include "Car.h"
```

默认构造函数把速度和方向都初始化为 0。冒号（:）记法用来初始化字段。两条斜线（//）与行尾之间的文本称为行注释，会被忽略：

```
Car::Car()
  :m_speed(0),
   m_direction(0) {
  // Empty.
}
```

第二个构造函数把速度和方向都初始化为给定的参数值：

```
Car::Car(int speed, int direction)
  :m_speed(speed),
   m_direction(direction) {
  // Empty.
}
```

在前面的构造函数中，其实也可以用赋值运算符（=）代替类初始化记法，如下所示。不过这种写法被认为效率不高，因为代码可以用前面的初始化记法进行优化。注意，赋值时我们用一个等号（=）；比较两个值时，我们用两个等号（==），后者将在第 2 章「数据结构与算法」中介绍：

```
Car::Car() {
  m_speed = 0;
  m_direction = 0;
}
```

这个类的析构函数什么也不做，包含它只是为了完整起见：

```
Car::~Car() {
  // Empty.
```

```
}
```

getSpeed 和 getDirection 方法只是返回汽车当前的速度和方向：

```
int Car::getSpeed() const {
  return m_speed;
}

int Car::getDirection() const {
  return m_direction;
}
```

紧跟着等号的加号称为复合赋值运算，它把右边的值加到左边的值上。同样，紧跟着等号的减号把右边的值从左边的值中减去。

斜线（/）后紧跟星号（*）、星号后紧跟斜线之间的文本称为块注释，会被忽略：

```
void Car::accelerate(int speed) {
  m_speed += speed; /* Same effect as: m_speed = m_speed + speed;
*/
}

void Car::decelerate(int speed) {
  m_speed -= speed;
}

void Car::turnLeft(int degrees) {
  m_direction -= degrees;
}

void Car::turnRight(int degrees) {
  m_direction += degrees;
}
```

现在可以测试我们的类了。为此，我们像在 Car.cpp 文件中那样包含 Car.h 文件，同时还要包含系统头文件 IOStream。和上一节一样，系统头文件用尖括号（< 和 >）括起来。我们还需要使用命名空间 std 来访问其功能：

Main.cpp

```
#include <IOStream>
using namespace std;
#include "Car.h"
```

在 C++ 中，函数可以属于某个类，也可以不带类而独立存在。类的函数通常称为方法。函数是一种数学抽象，它有输入值（称为参数）并返回一个值。不过在 C++ 中，函数允许没有参数，也可以返回特殊类型 void，表示不返回任何值。

如上一节所述，程序的执行总是从名为 main 的函数开始，而且每个程序必须恰好有一个名为 main 的函数。与其他一些语言不同，文件名不必叫 Main。

不过为方便起见，本书中每个含有 main 函数的文件都命名为 Main.cpp。void 关键字表示 main 不返回值。注意，构造函数和析构函数从不返回值，也不标记 void；而不返回值的其他方法和函数则必须标记 void：

```
void main() {
```

我们创建一个 Car 类的对象，命名为 redVolvo。对象是类的实例；redVolvo 是众多汽车中的一辆：

```
Car redVolvo;
```

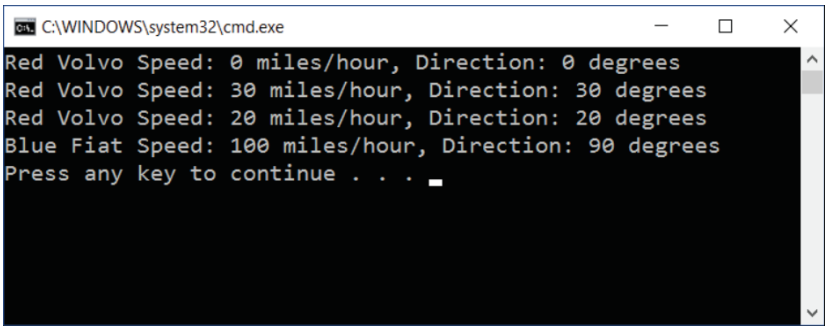
输出信息时，我们使用 cout 对象（console output 的缩写），它通常把内容写到文本窗口。由两个左尖括号（<<）组成的运算符称为输出流运算符。endl 指令让下一次输出从下一行的开头开始：

```
cout << "Red Volvo Speed: " << redVolvo.getSpeed()
     << " miles/hour" << ", Direction: "
     << redVolvo.getDirection() << " degrees" << endl;
redVolvo.accelerate(30);
redVolvo.turnRight(30);
cout << "Red Volvo Speed: " << redVolvo.getSpeed()
     << " miles/hour" << ", Direction: "
     << redVolvo.getDirection() << " degrees" << endl;
redVolvo.decelerate(10);
redVolvo.turnLeft(10);
cout << "Red Volvo Speed: " << redVolvo.getSpeed()
     << " miles/hour" << ", Direction: "
     << redVolvo.getDirection() << " degrees" << endl;
```

blueFiat 对象是 Car 类的常量对象。这意味着它只能由某个构造函数初始化，之后只能读取而不能修改。更准确地说，只有常量方法才能在常量对象上调用，而只有不修改对象字段的方法才能是常量的：

```
  const Car blueFiat(100, 90);
  cout << "Blue Fiat Speed: " << blueFiat.getSpeed()
       << " miles/hour" << ", Direction: "
       << blueFiat.getDirection() << " degrees" << endl;
}
```

运行代码后，输出显示在一个命令窗口中：



扩展 Car 类

本节我们修改 Car 类。在早先的版本中，我们在构造函数里初始化字段。另一种初始化字段的方式是直接在类定义中初始化。不过这一特性应谨慎使用，因为它可能导致不必要的初始化。如果调用的是 Car 类的第二个构造函数，字段就会被初始化两次，效率不高：

Car.h

```
class Car {
  public:
    // ...
  private:
    int m_speed = 0, m_direction = 0;
};
```

Car 类定义在 Car.h 文件中，而其方法定义在 Car.cpp 文件中。注意我们首先要包含 Car.h 文件，以便方法的定义与 Car.h 中的声明一致：

Car.cpp

```
#include "Car.h"

Car::Car() {
  // Empty.
}

Car::Car(int speed, int direction)
  :m_speed(speed),
   m_direction(direction) {
  // Empty.
}
```

此外，上一节的 Car 类有一些局限：

- 可以无限制地加速汽车，而且可以把汽车减速到负速度。
- 可以把汽车转到方向为负或超过 360 度。

我们先把汽车的最大速度设为 200 英里/小时。如果速度超过 200 英里/小时，就把它设为 200 英里/小时。我们使用 `if` 语句：它接受一个条件，并在条件为真时执行后面的语句。这里的语句（`m_speed = 200;`）用花括号括起来。这并非必需，因为只有一条语句；但如果有多条语句就有必要了。为清晰起见，本书中无论语句多少，我们总是使用花括号：

```
void Car::accelerate(int speed) {
  m_speed += speed;
  if (m_speed > 200) {
    m_speed = 200;
  }
}
```

如果速度变成负值，我们就改变速度的符号使其变为正值。注意，我们不能写 `m_speed -= m_speed`，那会把速度减成零，因为它会从自身中减去自身。

由于值是负的，改变符号后它就变成正值。我们还要把汽车转 180 度来改变方向。注意，这种情况下也必须检查汽车是否超过速度上限。

另外注意，我们还必须检查方向是否小于 180 度。如果是，就加上 180 度；否则减去 180 度，以把方向保持在 0 到 360 度之间。我们用 `if...else` 语句来做这件事。如果 `if` 语句的条件不为真，就执行 `else` 关键字之后的语句：

```
void Car::decelerate(int speed) {
  m_speed -= speed;
  if (m_speed < 0) {
    m_speed = -m_speed;
    if (m_speed > 200) {
      m_speed = 200;
    }
    if (m_direction < 180) {
      m_direction += 180;
    }
    else {
      m_direction -= 180;
    }
  }
}
```

转动汽车时，我们使用取模（`%`）运算符。除以 360 时，取模运算符给出除法的余数。例如，370 除以 360 的余数是 10：

```
void Car::turnLeft(int degrees) {
  m_direction -= degrees;
  m_direction %= 360;
```

```
  if (m_direction < 0) {
    m_direction += 360;
  }
}

void Car::turnRight(int degrees) {
  m_direction += degrees;
  m_direction %= 360;
}
```

main 函数创建一个 Car 类的对象——redVolvo。我们先输出它的速度和方向，然后加速并向左转，再输出速度和方向。最后，减速并向右转，最后一次输出速度和方向：

Main.cpp

```
#include <IOStream>
using namespace std;
#include "Car.h"

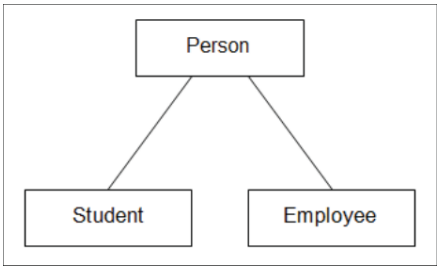
void main() {
  Car redVolvo(20, 30);
  cout << "Red Volvo Speed: " << redVolvo.getSpeed()
       << " miles/hour" << ", Direction: "
       << redVolvo.getDirection() << " degrees" << endl;
  redVolvo.accelerate(30);
  redVolvo.turnLeft(60);
  cout << "Red Volvo Speed: " << redVolvo.getSpeed()
       << " miles/hour" << ", Direction: "
       << redVolvo.getDirection() << " degrees" << endl;
  redVolvo.decelerate(60);
  redVolvo.turnRight(50);
  cout << "Red Volvo Speed: " << redVolvo.getSpeed()
       << " miles/hour" << ", Direction: "
       << redVolvo.getDirection() << " degrees" << endl;
}
```

运行代码后，输出显示在一个命令窗口中，如下所示：

```
C:\WINDOWS\system32\cmd.exe
Red Volvo Speed: 20 miles/hour, Direction: 30 degrees
Red Volvo Speed: 50 miles/hour, Direction: 330 degrees
Red Volvo Speed: 10 miles/hour, Direction: 200 degrees
Press any key to continue . . .
```

类层次结构——Person、Student 与 Employee 类

我们继续看一个类层次结构，其中 Person 是基类，Student 和 Employee 是它的派生类：



由于人有名字，我们使用 C++ 标准类 string 来存放名字。virtual 关键字标记 print 方法将进行动态绑定，本节稍后会讨论：

Person.h

```
class Person {
  public:
    Person(string name);
    virtual void print();
  private:
    string m_name;
};
```

我们包含 String 头文件，它让我们可以使用 string 类：

Person.cpp

```
#include <String>
#include <IOStream>
using namespace std;
#include "Person.h"

Person::Person(string name)
  :m_name(name) {
  // Empty.
}

void Person::print() {
  cout << "Person " << m_name << endl;
}
```

Student 和 Employee 类是 Person 的派生类，它们以 public 方式继承 Person。有时人们用「扩展」一词代替「继承」。继承方式可以是 public、protected 或 private 的：

- public 继承：基类的所有成员对派生类保持相同的可访问性。
- protected 继承：基类的所有 public 成员在派生类中变为 protected。
- private 继承：基类的所有 public 和 protected 成员在派生类中变为 private。

Student 和 Employee 类有文本字段 m_university 和 m_company：

Student.h

```
class Student : public Person {
  public:
    Student(string name, string university);
    void print();
  private:
    string m_university;
};
```

文件 Student.cpp 定义 Student 类的方法：

Student.cpp

```
#include <String>
#include <IOStream>
using namespace std;
#include "Person.h"
#include "Student.h"
```

派生类可以通过冒号（:）记法写出基类构造函数的名字来调用基类的构造函数。Student 的构造函数把名字作为参数传给 Person 的构造函数：

```
Student::Student(string name, string university)
  :Person(name),
   m_university(university) {
  // Empty.
}
```

我们必须用双冒号（::）记法表明调用的是 Person 中的 print，而不是 Student 的：

```
void Student::print() {
  Person::print();
  cout << "University " << m_university << endl;
}
```

Employee 类与 Student 类似，不过它持有的字段是 m_company 而不是 m_university。

Employee.h

```
class Employee : public Person {
  public:
    Employee(string name, string company);
    void print();
  private:
    string m_company;
};
```

文件 Employee.cpp 定义 Employee 类的方法：

Employee.cpp

```
#include <String>
#include <IOStream>
using namespace std;
#include "Person.h"
#include "Employee.h"
```

构造函数初始化人名与其供职的公司：

```
Employee::Employee(string name, string company)
  :Person(name),
   m_company(company) {
  // Empty.
}

void Employee::print() {
  Person::print();
  cout << "Company " << m_company << endl;
}
```

最后，main 函数先包含系统头文件 String 和 IOStream，它们存放着字符串处理以及输入输出流的声明。由于所有标准头文件都在标准命名空间中，我们用 using namespace 指令获得对系统声明的访问：

Main.cpp

```
#include <String>
#include <IOStream>
using namespace std;
#include "Person.h"
#include "Student.h"
#include "Employee.h"
```

我们定义三个对象 Monica、Demi 和 Charles，并对每个对象调用 print。在这三种情况下，分别调用 Person、Student 和 Employee 类的 print 方法：

```
void main() {
  Person monica("Monica");
  person.print();
  Student demi("Demi", "MIT");
  student.print();
  Employee charles("Charles", "Microsoft");
  employee.print();
```

星号（*）表示 personPtr 是指向 Person 对象的指针，而不是 Person 对象。指向对象的指针存放的是对象的内存地址，而不是对象本身。不过此刻它根本没有存放任何地址。我们很快会把它赋值为某个对象的地址：

```
Person *personPtr;
```

取地址符（&）是一个运算符，它给出对象的地址，该地址被赋给指针 personPtr。我们依次把 Person、Student 和 Employee 对象的地址赋给 personPtr，并在每种情况下调用 print。由于 print 在 Person 中被标记为 virtual，调用的是指针当前所指向对象所属类中的 print。既然 print 在基类 Person 中被标记为 virtual，就不必在派生类 Student 和 Employee 中再把 print 标记为 virtual。访问指针所指对象的成员时，我们使用箭头（->）运算符，而不是句点运算符。

当 personPtr 指向一个 Person 对象时，调用的是 Person 中的 print：

```
personPtr = &person;
personPtr->print();
```

当 personPtr 指向一个 Student 对象时，调用的是 Student 中的 print：

```
personPtr = &student;
personPtr->print();
```

当 personPtr 指向一个 Employee 对象时，调用的是 Employee 中的 print：

```
  personPtr = &employee;
  personPtr->print();
}
```

这个过程称为动态绑定。如果我们在 Person 中省略 virtual 标记，就会发生静态绑定，所有情况下调用的都是 Person 中的 print。

面向对象编程的概念建立在三大基石之上：封装、继承和动态绑定。不支持这三个特性中任何一个的语言，都称不上是面向对象的语言。

简单数据类型——栈

栈是一种简单的数据类型：我们把值加到顶端，移除顶端的值，且只能读取栈顶的值。本节我们实现一个整数栈。下一章我们会研究能存放任意类型值的模板类。我们使用链表——一种让一个指针指向链表的第一个单元格，而每个单元格又持有指向链表中下一个单元格的指针的结构。自然，链表最终必须结束。我们用 nullptr 标记链表的末尾，它是 C++ 中指向特殊空地址的标准指针。

首先，我们需要一个类来存放链表的每个单元格。单元格存放一个整数值和一个指向链表中下一个单元格的指针；如果它是链表的最后一个单元格，则该指针为 nullptr。下面的章节中，我们会研究同时持有前一个和下一个单元格指针的单元格类。

Cell.h

```
class Cell {
  public:
    Cell(int value, Cell *next);
```

可以直接在类定义中实现方法，这样的方法称为内联方法。不过通常只对短方法这样做。经验法则是：内联方法不应超过一行：

```
int value() const { return m_value; }
Cell *next() const { return m_next; }
```

每个单元格存放一个值和链表中下一个单元格的地址：

```
  private:
    int m_value;
    Cell *m_next;
};
```

Cell.cpp

```
#include "Cell.h"
```

单元格用一个值和一个指向链表中下一个单元格的指针来初始化。注意，如果该单元格是链表中最后一个单元格，则 m_next 的值为 nullptr：

```
Cell::Cell(int value, Cell *next)
  :m_value(value),
   m_next(next) {
  // Empty.
}
```

对栈来说，我们只对栈顶的值感兴趣。默认构造函数把栈初始化为空。push 在栈顶加入一个值，top 返回栈顶的值，pop 移除栈顶的值，size 返回栈中值的个数，empty 在栈为空时返回 true。bool 类型是一种逻辑类型，可以存放 true 或 false 值。

Stack.h

```
class Stack {
  public:
    Stack();
    void push(int value);
    int top();
    void pop();
    int size() const;
    bool empty() const;
```

m_firstCellPtr 字段是指向链表第一个单元格的指针，链表中存放着栈的值。栈为空时，m_firstCellPtr 将持有 nullptr 值。m_size 字段存放栈的当前大小：

```
  private:
    Cell *m_firstCellPtr;
    int m_size;
};
```

包含 CAssert 头文件是为了使用 assert 宏，它用于测试某些条件是否为真。宏是预处理器的一部分，执行特定的文本替换。

Stack.cpp

```
#include <CAssert>
using namespace std;
#include "Cell.h"
#include "Stack.h"
```

默认构造函数把指向第一个单元格的指针初始化为 nullptr、把大小初始化为零，从而把栈设为空：

```
Stack::Stack()
  :m_firstCellPtr(nullptr),
   m_size(0) {
  // Empty.
}
```

向栈顶压入新值时，我们用 new 运算符动态分配单元格所需的内存。如果内存耗尽，就返回 nullptr，assert 宏会对它进行测试。如果 m_firstCellPtr 等于 nullptr，执行将带着错误信息中止。感叹号（!）后跟等号（=）构成不等运算符。两个加号（++）构成自增运算符，表示把值加一。

自增运算符实际上有两个版本——前缀（++m_size）和后缀（m_size++）。前缀形式先把值加一再返回新值；后缀形式同样把值加一，但返回的是原值。不过在这里，用哪个版本都没有关系，因为我们只关心结果——m_size 的值加了一：

```
void Stack::push(int value) {
  m_firstCellPtr = new Cell(value, m_firstCellPtr);
  ++m_size;
}
```

返回栈顶值之前，必须先检查栈非空，因为返回空栈的栈顶值是不合逻辑的。如果栈为空，执行将带着错误信息中止。单个感叹号（!）是逻辑非运算符。我们返回栈顶的值，它存放在链表的第一个单元格中：

```
int Stack::top() {
  assert(!empty());
  return m_firstCellPtr->getValue();
}
```

弹出栈顶值时，同样必须检查栈非空。我们把指向链表第一个单元格的指针改为指向下一个单元格。但在此之前，必须先保存第一个指针 deleteCellPtr，以便释放它所指向的单元格的内存。

我们用 delete 运算符释放内存：

```
void Stack::pop() {
  assert(!empty());
  Cell *deleteCellPtr = m_firstCellPtr;
  m_firstCellPtr = m_firstCellPtr->getNext();
  delete deleteCellPtr;
```

与上面的自增运算符一样，两个减号（--）构成自减运算符，表示把值减一：

```
  --m_size;
}
```

size 方法只是返回 m_size 字段的值：

```
int Stack::size() const {
  return m_size;
}
```

当指向第一个单元格的指针等于 nullptr 时，栈为空。通俗地说，如果指针等于 nullptr，我们就说这个指针是空的：

```
bool Stack::empty() const {
  return (m_firstCellPtr == nullptr);
}
```

我们通过压入、读取和弹出一些值来测试栈：

Main.cpp

```
#include <String>
#include <IOStream>
using namespace std;
#include "Cell.h"
#include "Stack.h"

void main() {
  Stack s;
  s.push(1);
  s.push(2);
  s.push(3);
```

打印布尔值时，流运算符输出的不是 true 或 false，而是 true 打印 1、false 打印 0。为了真正打印 true 或 false，我们使用条件运算符。它接受三个值，用问号（?）和冒号（:）分隔。如果第一个值为 true，返回第二个值；如果第一个值为 false，返回第三个值：

```
  cout << "top " << s.top() << ", size " << s.size()
       << ", empty " << (s.empty() ? "true" : "false") << endl;
  s.pop();
  s.pop();
  s.push(4);
  cout << "top " << s.top() << ", size " << s.size()
       << ", empty " << (s.empty() ? "true" : "false") << endl;
}
```

更高级的数据类型——队列

队列是传统队列的模型：我们在后端进入值，在前端读取并移除值。还可以知道它容纳的值的个数，以及它是否为空。

与上一节的栈类似，我们用链表实现队列。我们复用 Cell 类；不过对队列来说，我们需要设

置单元格的 next 链接。因此，我们把 next 改名为 `getNext`，并添加新的 `setNext` 方法：

Cell.h

```
class Cell {
  public:
    Cell(int value, Cell *next);
    int value() const {return m_value;}
    Cell *getNext() const { return m_next; }
    void setNext(Cell* next) { m_next = next; }
  private:
    int m_value;
    Cell *m_next;
};
```

我们用链表、以与栈类似的方式实现队列。构造函数初始化一个空队列，enter 在队列后端进入一个值，remove 移除队列前端的值，size 返回队列的当前大小，empty 在队列为空时返回 true：

Queue.h

```
class Queue {
  public:
    Queue();
    void enter(int value);
    int first();
    void remove();
    int size() const;
    bool empty() const;
```

栈的场合我们只对栈顶感兴趣，它存放在链表的开头。队列的场合我们既关心前端也关心后端，这意味着需要访问链表的第一个和最后一个单元格。因此我们有两个指针 `m_firstCellPtr` 和 `m_lastCellPtr`，分别指向链表中的第一个和最后一个单元格：

```
  private:
    Cell *m_firstCellPtr, *m_lastCellPtr;
    int m_size;
};
```

Queue.cpp

```
#include <CAssert>
using namespace std;
#include "Cell.h"
#include "Queue.h"
```

队列创建时是空的：指针为空，大小为零。由于链表中还没有任何单元格，两个单元格指针都

指向 nullptr：

```
Queue::Queue()
  :m_firstCellPtr(nullptr),
   m_lastCellPtr(nullptr),
   m_size(0) {
  // Empty.
}
```

在队列后端进入新值时，我们检查队列是否为空。如果为空，两个指针都指向新单元格。如果不为空，先把最后一个单元格的 next 指针指向新单元格，然后把最后一个单元格指针设为新单元格：

```
void Queue::enter(int value) {
  Cell *newCellPtr = new Cell(value, nullptr);
  if (empty()) {
    m_firstCellPtr = m_lastCellPtr = newCellPtr;
  }
  else {
    m_lastCellPtr->setNext(newCellPtr);
    m_lastCellPtr = newCellPtr;
  }
  ++m_size;
}
```

first 方法只是返回链表中第一个单元格的值：

```
int Queue::first() {
  assert(!empty());
  return m_firstCellPtr->value();
}
```

remove 方法让第一个单元格指向第二个单元格。不过首先必须保存它的地址，以便用 C++ 标准的 delete 运算符释放其内存：

```
void Queue::remove() {
  assert(!empty());
  Cell *deleteCellPtr = m_firstCellPtr;
  m_firstCellPtr = m_firstCellPtr->getNext();
  delete deleteCellPtr;
  --m_size;
}

int Queue::size() const {
  return m_size;
}
```

```
bool Queue::empty() const {
  return (m_firstCellPtr == nullptr);
}
```

我们通过进入并移除一些值来测试队列。我们进入值 1、2、3，它们按该顺序排入队列。然后移除前两个值，再进入值 4。此时队列中保存着值 3 和 4：

Main.cpp

```
#include <CMath>
#include <String>
#include <IOStream>
using namespace std;
#include "Cell.h"
#include "Queue.h"

void main() {
  Queue q;
  q.enter(1);
  q.enter(2);
  q.enter(3);
  cout << "first " << q.first() << ", size " << q.size()
       << ", empty " << (q.empty() ? "true" : "false") << endl;
  q.remove();
  q.remove();
  q.enter(4);
  cout << "first " << q.first() << ", size " << q.size()
       << ", empty " << (q.empty() ? "true" : "false") << endl;
}
```

小结

本章我们研究了面向对象编程的基础。我们从创建工程、执行掷骰子程序开始。我们还创建了类层次结构，包括基类 Person 及其两个派生类 Student 和 Employee。通过定义指向对象的指针，我们实现了动态绑定。

最后，我们创建了两个数据类型——栈和队列。栈是这样一种结构：我们只关心顶端的值，可以在顶端加入值、读取栈顶值并移除栈顶值。队列则是传统意义上的队列：在后端进入值，在前端读取并移除值。

下一章我们将继续创建数据类型以及更高级的数据类型，如列表和集合。我们还会研究 C++ 的更多高级特性。

2
数据结构与算法

上一章中，我们为栈和队列这两个抽象数据类型创建了类。本章我们将继续处理列表和集合这两个抽象数据类型。

与上一章的栈和队列类似，列表（list）是一种有头有尾的有序结构。不过，列表允许在任意位置加入和移除值，还可以遍历列表。

而集合（set）则是一种由值组成的无序结构。对于集合，我们唯一能说的是某个值是否存在；我们无法说出某个值相对于其他任何值的位置关系。

本章将涵盖以下主题：

- 我们先实现相当简单但低效的列表类和集合类，并研究搜索与排序的基础算法。
- 然后创建更高级版本的列表类和集合类，研究更高级的搜索与排序算法。我们还会引入一些新概念，如模板、运算符重载、异常和引用重载。
- 我们还将研究线性搜索（linear search）和二分搜索（binary search）这两种搜索算法：前者可用于任何序列（无论有序还是无序），但效率较低；后者效率更高，但只能用于有序序列。
- 最后，我们将研究插入排序（insert sort）、选择排序（select sort）和冒泡排序（bubble sort）这些相当简单的排序算法，以及更高级、更高效的归并排序（merge sort）和快速排序（quick sort）算法。

列表类

链表类比栈和队列更复杂，是一种更复杂的抽象数据类型。我们可以在列表的任意位置加入和移除值，也可以遍历列表。

Cell 类

本节的单元格是栈和队列两节中单元格的扩展。与它们类似，单元格存放一个值和一个指向下一个单元格的指针；不过，这个版本还持有一个指向前一个单元格的指针，这使得本节的链表成为双向链表（double-linked list）。

注意，这里的构造函数是 private 的，这意味着单元格对象只能由它自己的成员函数创建。不过，有一种办法可以绕过这一限制：我们可以把某个类或函数定义为 LinkedList 的友元（friend）。这里，我们把 LinkedList 定义为 Cell 的友元，也就是说，LinkedList 可以访问 Cell 的所有 private 和 protected 成员——包括构造函数——从而可以创建 Cell 对象。

Cell.h

```
class Cell {
  private:
    Cell(double value, Cell *previous, Cell *next);
    friend class LinkedList;

  public:
    double getValue() const { return m_value; }
    void setValue(double value) { m_value = value; }

    Cell *getPrevious() const { return m_previous; }
    void setPrevious(Cell *previous) { m_previous = previous; }

    Cell *getNext() const { return m_next; }
    void setNext(Cell *getNext) { m_next = getNext; }

  private:
    double m_value;
    Cell *m_previous, *m_next;
};
```

Cell.cpp

```
#include "Cell.h"

Cell::Cell(double value, Cell *previous, Cell *next)
 :m_value(value),
  m_previous(previous),
  m_next(next) {
  // Empty.
}
```

Iterator 类

遍历列表时，我们需要一个迭代器（iterator），它被初始化为列表的开头，然后一步一步地移向列表末尾。与前面的单元格类似，Iterator 的构造函数也是 private 的，不过我们同样把 LinkedList 定义为 Iterator 的友元。

Iterator.h

```
class Iterator {
  private:
    Iterator(Cell *cellPtr);
    friend class LinkedList;

  public:
    Iterator();
```

第三个构造函数是复制构造函数（copy constructor）。它接受另一个迭代器并复制它。我们不能直接接受迭代器对象作为参数，而是要定义一个引用参数。& 符号表明该参数是对迭代器对象的引用，而不是迭代器对象本身。这样，传递给参数的是迭代器的内存地址，而不是对象本身。我们还声明被引用的对象为常量，这样它就不会被构造函数修改。

在本例中必须使用引用参数。如果参数只是一个简单的迭代器对象，就会造成无限循环的初始化。而在其他情况下，我们出于效率的考虑使用这一技巧：传递对象的地址比把对象本身复制为参数更省时间、更省内存：

```
Iterator(const Iterator& iterator);

double getValue() { return m_cellPtr->getValue(); }
void setValue(double value) { m_cellPtr->setValue(value); }
```

hasNext 方法在迭代器尚未到达列表末尾时返回 true，而 next 方法把迭代器向列表末尾方向前移一步，如下例所示：

```
bool hasNext() const { return (m_cellPtr != nullptr); }
void next() { m_cellPtr = m_cellPtr->getNext(); }
```

同样，hasPrevious 方法在迭代器尚未到达列表开头时返回 true，而 previous 方法把迭代器向列表开头方向后退一步：

```
    bool hasPrevious() const { return (m_cellPtr != nullptr); }
    void previous() { m_cellPtr = m_cellPtr->getPrevious(); }

  private:
    Cell *m_cellPtr;
};
```

Iterator.cpp

```
#include "Cell.h"
#include "Iterator.h"

Iterator::Iterator(Cell *cellPtr)
 :m_cellPtr(cellPtr) {
  // Empty.
}

Iterator::Iterator()
 :m_cellPtr(nullptr) {
  // Empty.
}

Iterator::Iterator(const Iterator& iterator)
 :m_cellPtr(iterator.m_cellPtr) {
  // Empty.
}
```

LinkedList 类

LinkedList 类持有查找、加入、插入和移除值的方法，以及比较列表的方法。此外，它还持有读取和写出列表的方法，以及正向和反向遍历列表的方法。这个链表实际上是一个双向链表：我们可以沿着单元格之间的链接双向移动——既可以从头走到尾，也可以从尾走回头。

LinkedList.h

```
class LinkedList {
  public:
    LinkedList();
```

复制构造函数和 assign 方法都会复制给定的列表：

```
LinkedList(const LinkedList& list);
void assign(const LinkedList& list);
```

析构函数释放为链表中各个单元格分配的所有内存：

```
~LinkedList();

int size() const {return m_size;}
bool empty() const {return (m_size == 0);}
```

find 方法搜索某个值。如果找到了该值，就返回 true，并把 findIterator 设置为该值所在的位置：

```
bool find(double value, Iterator& findIterator);
```

equal 和 notEqual 方法把当前链表与给定链表进行比较，分别在相等或不相等时返回 true，如下面的代码片段所示：

```
bool equal(const LinkedList& list) const;
bool notEqual(const LinkedList& list) const;
```

如果我们想把一个值或另一个列表加入现有列表，该怎么办？add 方法把一个值或另一个列表加到当前列表的末尾，而 insert 则把一个值或一个列表插入到迭代器给出的位置：

```
void add(double value);
void add(const LinkedList& list);

void insert(const Iterator& insertPosition, double value);

void insert(const Iterator& insertPosition,
            const LinkedList& list);
```

erase 方法删除给定位置上的值，而 clear 删除列表中的每一个值，如下例所示：

```
void erase(const Iterator& erasePosition);
void clear();
```

remove 方法把从第一个迭代器到最后一个迭代器（含两端）之间的值都移除。第二个参数是默认参数，意思是这个方法既可以用一个参数调用，也可以用两个参数调用。在只用一个参数的情况下，第二个参数取声明中给出的值——这里是 Iterator(nullptr)，它表示列表末尾之后一步的位置。这意味着当只用一个迭代器调用 remove 时，从该迭代器（含）到列表末尾的所有值都会被移除。nullptr 指针实际上是一种特殊的指针，它会被转换成它所指向或所比较的类型——这里是 Cell 指针。通俗地说，如果一个指针的值是 nullptr，我们就说这个指针是空的：

```
void remove(const Iterator& firstPosition,
            const Iterator& lastPosition = Iterator(nullptr));
```

first 和 last 方法返回位于列表第一个值和最后一个值处的迭代器：

```
Iterator first() const { return Iterator(m_firstCellPtr); }
Iterator last() const { return Iterator(m_lastCellPtr); }
```

read 和 write 方法从输入文件流中读取列表的值，并把列表的值写到输出文件流。文件流用来与文件进行通信。注意，我们前面几节用过的 cin 和 cout 对象实际上就是输入流和输出流对象：

```
void read(istream& inStream);
void write(ostream& outStream);
```

与前面一节的队列类似，列表持有指向链表中第一个和最后一个单元格的指针：

```
  private:
    int m_size;
    Cell *m_firstCellPtr, *m_lastCellPtr;
};
```

LinkedList.cpp

```
#include <IOStream>
using namespace std;

#include "Cell.h"
#include "Iterator.h"
#include "List.h"

LinkedList::LinkedList()
 :m_size(0),
  m_firstCellPtr(nullptr),
  m_lastCellPtr(nullptr) {
  // Empty.
}
```

复制构造函数只是调用 assign 来复制列表参数的值：

```
LinkedList::LinkedList(const LinkedList& list) {
  assign(list);
}
```

assign 方法把给定的列表复制到自己的链表中：

```
void LinkedList::assign(const LinkedList& list) {
  m_size = 0;
  m_firstCellPtr = nullptr;
  m_lastCellPtr = nullptr;
  Cell *listCellPtr = list.m_firstCellPtr;
  add(list);
}
```

析构函数只是调用 clear，释放链表中各单元格所占用的所有内存：

```
LinkedList::~LinkedList() {
  clear();
}
```

clear 方法遍历链表并释放每一个单元格：

```
void LinkedList::clear() {
  Cell *currCellPtr = m_firstCellPtr;
```

对链表中的每个单元格，我们必须先把它的地址保存在 deleteCellPtr 中，在链表中向前移动一格，然后释放该单元格。如果直接对 currCellPtr 调用 delete，后面紧接的 getNext 调用就无法工作，因为那样我们就是在对一个已释放的对象调用方法：

```
while (currCellPtr != nullptr) {
  Cell *deleteCellPtr = currCellPtr;
  currCellPtr = currCellPtr->getNext();
  delete deleteCellPtr;
}
```

当列表变空之后，两个单元格指针都为空，大小为零：

```
  m_firstCellPtr = nullptr;
  m_lastCellPtr = nullptr;
  m_size = 0;
}
```

find 方法遍历链表，设置 findIterator，并在找到该值时返回 true。如果没有找到，就返回 false，findIterator 保持不变。为了让这一点成立，findIterator 必须是对 Iterator 对象的引用，而不是 Iterator 对象本身。用指向 Iterator 对象的指针也可以：

```
bool LinkedList::find(double value, Iterator& findIterator) {
  Iterator iterator = first();

  while (iterator.hasNext()) {
    if (value == iterator.getValue()) {
      findIterator = iterator;
      return true;
    }

    iterator.next();
  }

  return false;
}
```

如果两个列表的大小不同，它们就不相等。同样，如果大小相同但值不同，它们也不相等：

```
bool LinkedList::equal(const LinkedList& list) const {
  if (m_size != list.m_size) {
    return false;
  }

  Iterator thisIterator = first(), listIterator = list.first();

  while (thisIterator.hasNext()) {
    if (thisIterator.getValue() != listIterator.getValue()) {
```

```
      return false;
    }

    thisIterator.next();
    listIterator.next();
  }
```

不过，如果两个列表大小相同、值也相同，它们就相等：

```
  return true;
}
```

在判断两个列表是否不相等时，我们只需调用 equal。感叹号（!）是逻辑非运算符，如下例所示：

```
bool LinkedList::notEqual(const LinkedList& list) const {
  return !equal(list);
}
```

向列表加入值时，我们动态分配一个单元格：

```
void LinkedList::add(double value) {
  Cell *newCellPtr = new Cell(value, m_lastCellPtr, nullptr);
```

如果第一个单元格指针为空，由于列表是空的，我们把它设置为指向新单元格：

```
if (m_firstCellPtr == nullptr) {
  m_firstCellPtr = newCellPtr;
}
```

不过，如果第一个单元格指针不为空，说明列表非空，这时我们把最后一个单元格指针的 next 指针设置为指向新单元格：

```
else {
  m_lastCellPtr->setNext(newCellPtr);
}
```

无论哪种情况，我们都把最后一个单元格指针设置为指向新单元格，并让列表的大小加一：

```
  m_lastCellPtr = newCellPtr;
  ++m_size;
}
```

向已有列表加入列表

把整个列表加入列表时，对列表中的每一个值，我们都采取与前面 add 方法加入单个值时相同的做法：动态分配一个新单元格；如果第一个单元格指针为空，就让它指向新单元格；如果不

为空，就把最后一个单元格指针的 next 指针设置为指向新单元格。无论哪种情况，最后都把最后一个单元格指针设置为指向新单元格：

LinkedList.cpp

```
void LinkedList::add(const LinkedList& list) {
  Cell *listCellPtr = list.m_firstCellPtr;
```

while 语句在条件为真期间不断重复执行。这里也就是在我们尚未到达列表末尾期间不断重复：

```
while (listCellPtr != nullptr) {
  double value = listCellPtr->getValue();
  Cell *newCellPtr = new Cell(value, m_lastCellPtr, nullptr);
```

如果 m_firstCellPtr 为空，说明我们的链表还是空的，newCellPtr 指向新链表的第一个单元格。这时我们让 m_firstCellPtr 指向新单元格：

```
if (m_firstCellPtr == nullptr) {
  m_firstCellPtr = newCellPtr;
}
```

如果 m_firstCellPtr 不为空，说明列表非空，此时不能修改 m_firstCellPtr，而是把 m_lastCellPtr 的 next 指针设置为指向新单元格：

```
else {
  m_lastCellPtr->setNext(newCellPtr);
}
```

无论哪种情况，最后一个单元格指针都被设置为新单元格指针：

```
m_lastCellPtr = newCellPtr;
```

最后，列表单元格指针被设置为指向它的下一个单元格。最终这个指针会变成空，while 语句随之结束：

```
    listCellPtr = listCellPtr->getNext();
  }

  m_size += list.m_size;
}
```

在迭代器给出的位置插入值时，我们把它的 previous 指针设置为指向列表中该位置之前的单元格（如果该位置是列表中的第一个，则为空）。然后，我们按照前面 add 方法中相同的方式检查第一个单元格指针是否为空：

```
void LinkedList::insert(const Iterator& insertPosition,
                        double value) {
  Cell *insertCellPtr = insertPosition.m_cellPtr;
```

```
  Cell *newCellPtr =
    new Cell(value, insertCellPtr->getPrevious(), insertCellPtr);
  insertCellPtr->setPrevious(newCellPtr);

  if (insertCellPtr == m_firstCellPtr) {
    m_firstCellPtr = newCellPtr;
  }
  else {
    newCellPtr->getPrevious()->setNext(newCellPtr);
  }

  ++m_size;
}
```

插入整个列表时，我们首先检查该位置是否表示空指针。如果是，说明这个位置已经越过我们列表的末尾，这时直接改用 add：

```
void LinkedList::insert(const Iterator& insertPosition,
                        const LinkedList& list) {
  Cell *insertCellPtr = insertPosition.m_cellPtr;

  if (insertCellPtr == nullptr) {
    add(list);
  }
  else {
    Cell *firstInsertCellPtr = nullptr,
         *lastInsertCellPtr = nullptr,
         *listCellPtr = list.m_firstCellPtr;

    while (listCellPtr != nullptr) {
      Cell *newCellPtr = new Cell(listCellPtr->getValue(),
                                  lastInsertCellPtr, nullptr);

      if (firstInsertCellPtr == nullptr) {
        firstInsertCellPtr = newCellPtr;
      }
      else {
        lastInsertCellPtr->setNext(newCellPtr);
      }

      lastInsertCellPtr = newCellPtr;
      listCellPtr = listCellPtr->getNext();
    }
```

我们把 firstInsertCellPtr 与 nullptr 进行比较，检查待插入的列表是否为空。由于 firstInsert-CellPtr 指向列表的第一个值，它为空就意味着列表为空：

```
    if (firstInsertCellPtr != nullptr) {
      if (insertCellPtr->getPrevious() != nullptr) {
        insertCellPtr->getPrevious()->setNext(firstInsertCellPtr);
        firstInsertCellPtr->
          setPrevious(insertCellPtr->getPrevious());
      }
      else {
        m_firstCellPtr = firstInsertCellPtr;
      }
    }

    if (lastInsertCellPtr != nullptr) {
      lastInsertCellPtr->setNext(insertCellPtr);
      insertCellPtr->setPrevious(lastInsertCellPtr);
    }

    m_size += list.m_size;
  }
}
```

从列表中删除值

erase 方法只是调用 remove，把给定位置同时作为起始和结束位置：

```
void LinkedList::erase(const Iterator& removePosition) {
  remove(removePosition, removePosition);
}
```

从列表中删除值时，我们遍历列表，并为每一个要移除的值释放其单元格：

```
void LinkedList::remove(const Iterator& firstPosition,
          const Iterator& lastPosition /*= Iterator(nullptr)*/) {
  Cell *firstCellPtr = firstPosition.m_cellPtr,
       *lastCellPtr = lastPosition.m_cellPtr;
  lastCellPtr = (lastCellPtr == nullptr)
              ? m_lastCellPtr : lastCellPtr;

  Cell *previousCellPtr = firstCellPtr->getPrevious(),
       *nextCellPtr = lastCellPtr->getNext();

  Cell *currCellPtr = firstCellPtr;
  while (currCellPtr != nextCellPtr) {
    Cell *deleteCellPtr = currCellPtr;
    currCellPtr = currCellPtr->getNext();
    delete deleteCellPtr;
    --m_size;
```

```
    }
```

删除这些单元格时有三种情况要考虑。如果第一个被删除单元格之前的最后一个单元格不为空——意思是删除位置之前还有列表的剩余部分——我们就把它的 next 指针设置为指向删除位置之后的第一个单元格。如果第一个被删除单元格之前的最后一个单元格为空，我们就把第一个单元格指针设置为指向删除位置之后的那个单元格：

```
if (previousCellPtr != nullptr) {
  previousCellPtr->setNext(nextCellPtr);
}
else {
  m_firstCellPtr = nextCellPtr;
}
```

对于最后一个被删除单元格之后的列表剩余位置，我们也做同样的事情。如果删除部分之后还有列表的剩余部分，我们就把它的第一个单元格的 previous 指针设置为指向删除部分之前剩余列表的最后一个单元格：

```
  if (nextCellPtr != nullptr) {
    nextCellPtr->setPrevious(previousCellPtr);
  }
  else {
    m_lastCellPtr = previousCellPtr;
  }
}
```

读取列表时，我们先读取列表的大小，然后再读取各个值：

```
void LinkedList::read(istream& inStream) {
  int size;
  inStream >> size;

  int count = 0;
  while (count < size) {
    double value;
    inStream >> value;
    add(value);
    ++count;
  }
}
```

写出列表时，我们把各个值用逗号分隔、用方括号（"[" 与 "]"）括起来写出：

```
void LinkedList::write(ostream& outStream) {
  outStream << "[";
  bool firstValue = true;
```

```
  Iterator iterator = first();
  while (iterator.hasNext()) {
    outStream << (firstValue ? "" : ",") << iterator.getValue();
    firstValue = false;
    iterator.next();
  }

  outStream << "]";
}
```

我们向列表加入一些值来测试它，然后再正向、反向遍历这些值。

Main.cpp

```
#include <IOStream>
using namespace std;

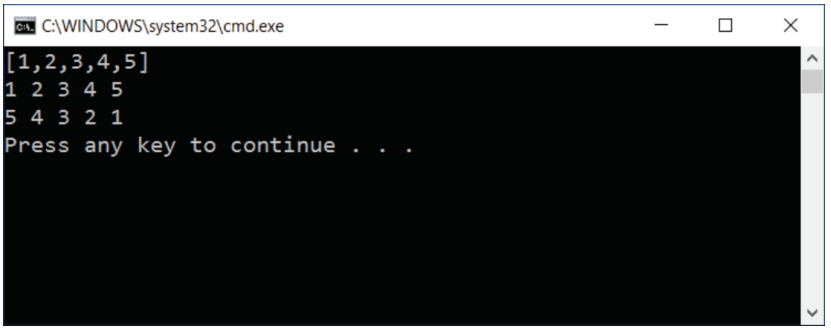
#include "Cell.h"
#include "Iterator.h"
#include "List.h"

void main() {
  LinkedList list;
  list.add(1);
  list.add(2);
  list.add(3);
  list.add(4);
  list.add(5);
  list.write(cout);
  cout << endl;

  { Iterator iterator = list.first();
    while (iterator.hasNext()) {
      cout << iterator.getValue() << " ";
      iterator.next();
    }
    cout << endl;
  }

  { Iterator iterator = list.last();
    while (iterator.hasPrevious()) {
      cout << iterator.getValue() << " ";
      iterator.previous();
    }
    cout << endl;
  }
}
```

执行代码后，输出显示在一个命令窗口中：



集合类

集合是一种没有重复元素的无序结构。Set 类是 LinkedList 的子类。注意这里的继承方式是 private 的，这使得 LinkedList 的所有 public 和 protected 成员在 Set 中都变成 private。

Set.h

```
class Set : private LinkedList {
  public:
    Set();
    Set(double value);
    Set(const Set& set);
    void assign(const Set& set);
    ~Set();
```

equal 方法在集合拥有这些值时返回 true。注意，我们并不关心集合中值的任何顺序：

```
bool equal(const Set& set) const;
bool notEqual(const Set& set) const;
```

exists 方法分别在给定值存在、或给定集合中每个值都存在时返回 true：

```
bool exists(double value) const;
bool exists(const Set& set) const;
```

insert 方法插入给定值，或给定集合中的每个值。它只插入集合中尚不存在的值，因为集合不允许重复：

```
bool insert(double value);
bool insert(const Set& set);
```

remove 方法在存在的情况下移除给定值，或给定集合中的每个值：

```
bool remove(double value);
bool remove(const Set& set);
```

size、empty 和 first 方法只是调用 LinkedList 中的对应方法。由于集合没有顺序，再去覆盖 LinkedList 中的 end 方法就没有意义了：

```
int size() const { return LinkedList::size(); }
bool empty() const { return LinkedList::empty(); }
Iterator first() const { return LinkedList::first(); }
```

自由函数 unionSet、intersection 和 difference 是 Set 的友元，这意味着它们可以访问 Set 的所有 private 和 protected 成员。



我们不能把 unionSet 方法命名为 union，因为 union 是 C++ 的关键字。

注意，当一个类中的方法被标记为友元时，它实际上并不是这个类的方法，而是一个函数：

```
friend Set unionSet(const Set& leftSet, const Set& rightSet);
friend Set intersection(const Set& leftSet,
                        const Set& rightSet);
friend Set difference(const Set& leftSet,
                      const Set& rightSet);
```

read 和 write 方法以与 LinkedList 中对应方法相同的方式读取和写出集合：

```
    void read(istream& inStream);
    void write(ostream& outStream);
};
```

作为 Set 友元的 unionSet、intersection 和 difference 函数在类定义之外声明：

```
Set unionSet(const Set& leftSet, const Set& rightSet);
Set intersection(const Set& leftSet, const Set& rightSet);
Set difference(const Set& leftSet, const Set& rightSet);
```

Set.cpp

```
#include <IOStream>
using namespace std;

#include "..\ListBasic\Cell.h"
#include "..\ListBasic\Iterator.h"
```

```
#include "..\ListBasic\List.h"
#include "Set.h"
```

构造函数调用 LinkedList 中的对应版本。实际上，默认构造函数（不带参数的那个）会隐式调用 LinkedList 的默认构造函数：

```
Set::Set() {
  // Empty.
}

Set::Set(double value) {
  add(value);
}

Set::Set(const Set& set)
 :LinkedList(set) {
  // Empty.
}
```

析构函数隐式调用 LinkedList 中的对应版本，释放与集合中各值相关联的内存。在这里，我们其实可以省略析构函数，LinkedList 的析构函数照样会被调用——代码如下：

```
Set::~Set() {
  // Empty.
}
```

assign 方法只是清空集合，然后加入给定的集合：

```
void Set::assign(const Set& set) {
  clear();
  add(set);
}
```

如果两个集合大小相同，并且其中一个集合的每个值都存在于另一个集合中，那么这两个集合相等。这种情况下，另一个集合的每个值也必然存在于第一个集合中：

```
bool Set::equal(const Set& set) const {
  if (size() != set.size()) {
    return false;
  }

  Iterator iterator = first();
  while (iterator.hasNext()) {
    if (!set.exists(iterator.getValue())) {
      return false;
    }
```

```
    iterator.next();
  }
  return true;
}

bool Set::notEqual(const Set& set) const {
  return !equal(set);
}
```

exists 方法使用 LinkedList 的迭代器遍历集合。如果找到该值，就返回 true：

```
bool Set::exists(double value) const {
  Iterator iterator = first();

  while (iterator.hasNext()) {
    if (value == iterator.getValue()) {
      return true;
    }

    iterator.next();
  }

  return false;
}
```

第二个 exists 方法遍历给定的集合，只要其中有一个值在当前集合中不存在就返回 false；如果所有值都存在，则返回 true：

```
bool Set::exists(const Set& set) const {
  Iterator iterator = set.first();

  while (iterator.hasNext()) {
    if (!exists(iterator.getValue())) {
      return false;
    }

    iterator.next();
  }

  return true;
}
```

第一个 insert 方法在值尚不存在于集合中时把它加入：

```
bool Set::insert(double value) {
  if (!exists(value)) {
    add(value);
```

```
    return true;
  }

  return false;
}
```

第二个 insert 方法遍历给定的集合，通过调用第一个 insert 方法来插入每个值。这样，集合中尚不存在的每个值都会被插入：

```
bool Set::insert(const Set& set) {
  bool inserted = false;
  Iterator iterator = set.first();

  while (iterator.hasNext()) {
    double value = iterator.getValue();

    if (insert(value)) {
      inserted = true;
    }

    iterator.next();
  }

  return inserted;
}
```

第一个 remove 方法在值存在于集合中时将其移除并返回 true；如果值不存在，则返回 false：

```
bool Set::remove(double value) {
  Iterator iterator;
  if (find(value, iterator)) {
    erase(iterator);
    return true;
  }

  return false;
}
```

第二个 remove 方法遍历给定的集合并移除其中每一个值。只要至少移除了一个值，就返回 true：

```
bool Set::remove(const Set& set) {
  bool removed = false;
  Iterator iterator = set.first();

  while (iterator.hasNext()) {
    double value = iterator.getValue();
```

```
    if (remove(value)) {
      removed = true;
    }

    iterator.next();
  }

  return removed;
}
```

并集、交集与差集运算

unionSet 函数创建一个以左侧集合初始化的结果集合，然后把右侧集合加入其中：

```
Set unionSet(const Set& leftSet, const Set& rightSet) {
  Set result(leftSet);
  result.insert(rightSet);
  return result;
}
```

交集方法比并集或差集方法稍微复杂一些。两个集合 A 与 B 的交集可以定义为它们的并集与它们的差集之差：

$$A \cap B = (A \cup B) - ((A - B) - (B - A))$$

```
Set intersection(const Set& leftSet, const Set& rightSet) {
  return difference(difference(unionSet(leftSet, rightSet),
                               difference(leftSet, rightSet)),
                    difference(rightSet, leftSet));
}
```

差集方法用左侧集合创建一个结果集合，然后从中移除右侧集合：

```
Set difference(const Set& leftSet, const Set& rightSet) {
  Set result(leftSet);
  result.remove(rightSet);
  return result;
}
```

read 方法与 LinkedList 中的对应方法类似，不过这里调用的是 insert 而不是 add。这样，集合中就不会插入重复的值：

```
void Set::read(istream& inStream) {
  int size;
  inStream >> size;
```

```
  int count = 0;
  while (count < size) {
    double value;
    inStream >> value;
    insert(value);
    ++count;
  }
}
```

write 方法也与 LinkedList 中的对应方法类似，不过这里集合用花括号（"{ 和 "}）而不是方括号（"[和 "]"）括起来：

```
void Set::write(ostream& outStream) {
  outStream << "{";
  bool firstValue = true;
  Iterator iterator = first();

  while (iterator.hasNext()) {
    outStream << (firstValue ? "" : ",") << iterator.getValue();
    firstValue = false;
    iterator.next();
  }

  outStream << "}";
}
```

我们让用户输入两个集合，然后求出它们的并集、交集与差集，以此来测试集合。

Main.cpp

```
#include <IOStream>
using namespace std;

#include "..\ListBasic\Cell.h"
#include "..\ListBasic\Iterator.h"
#include "..\ListBasic\List.h"
#include "Set.h"

void main() {
  Set s, t;
  s.read(cin);
  t.read(cin);

  cout << endl << "s = ";
  s.write(cout);
  cout << endl;
```

```
  cout << endl << "t = ";
  t.write(cout);
  cout << endl << endl;

  cout << "union: ";
  unionSet(s, t).write(cout);
  cout << endl;

  cout << "intersection: ";
  unionSet(s, t).write(cout);
  cout << endl;

  cout << "difference: ";
  unionSet(s, t).write(cout);
  cout << endl;
}
```

基础搜索与排序

本章我们还将研究一些搜索与排序算法。用线性搜索查找某个值时，我们只是简单地从头到尾扫一遍列表，然后返回该值从零起算的下标；如果没找到，就返回负一。

Search.h

```
int linarySearch(double value, const LinkedList& list);
```

Search.cpp

```
#include <IOStream>
using namespace std;

#include "..\ListBasic\Cell.h"
#include "..\ListBasic\Iterator.h"
#include "..\ListBasic\List.h"
#include "Search.h"

int linarySearch(double value, const LinkedList& list) {
  int index = 0;
```

我们用列表的 first 方法取得遍历列表所用的迭代器；只要列表中还有值，hasNext 就返回 true，而 next 把迭代器在列表中向前移动一步：

```
  Iterator iterator = list.first();
```

```
  while (iterator.hasNext()) {
    if (iterator.getValue() == value) {
      return index;
    }

    ++index;
    iterator.next();
  }

  return -1;
}
```

接下来我们研究选择排序、插入排序和冒泡排序算法。注意，它们接受的是列表的引用，而不是列表本身——只有这样，列表才会被改变。还要注意，在这些场合引用不是常量的；如果是常量的，我们就无法对列表排序了。

Sort.h

```
void selectSort(LinkedList& list);
void insertSort(LinkedList& list);
void bubbleSort(LinkedList& list);
```

Sort.cpp

```
#include <IOStream>
using namespace std;

#include "..\ListBasic\Cell.h"
#include "..\ListBasic\Iterator.h"
#include "..\ListBasic\List.h"
#include "Sort.h"

void insert(double value, LinkedList& list);
void swap(Iterator iterator1, Iterator iterator2);
```

选择排序算法

选择排序算法相当简单：我们反复遍历列表，直到列表变空为止。每一轮都找出最小的值，把它从列表中移除并加入结果列表。这样，结果列表最终会装满与原列表相同的值；由于值是按顺序选出的，结果列表就是有序的。最后，我们把结果列表赋回给原列表：

```
void selectSort(LinkedList& list) {
  LinkedList result;

  while (!list.empty()) {
```

```
    Iterator minIterator = list.first();
    double minValue = minIterator.getValue();

    Iterator iterator = list.first();

    while (iterator.hasNext()) {
      if (iterator.getValue() < minValue) {
        minIterator = iterator;
        minValue = iterator.getValue();
      }

      iterator.next();
    }

    list.erase(minIterator);
    result.add(minValue);
  }

  list.assign(result);
}
```

插入排序算法

插入排序算法会遍历列表，对每一个值，都把它插入到结果列表中合适的位置。然后我们把结果列表赋回给原列表：

```
void insertSort(LinkedList& list) {
  LinkedList result;
  Iterator iterator = list.first();

  while (iterator.hasNext()) {
    insert(iterator.getValue(), result);
    iterator.next();
  }

  list.assign(result);
}
```

insert 函数接受一个列表和一个值，把该值放到列表中正确的位置上。它遍历列表，把该值放在第一个大于它的值之前。如果列表中没有这样的值，就把该值加到列表末尾：

```
void insert(double value, LinkedList& list) {
  Iterator iterator = list.first();

  while (iterator.hasNext()) {
```

```
    if (value < iterator.getValue()) {
      list.insert(iterator, value);
      return;
    }

    iterator.next();
  }

  list.add(value);
}
```

冒泡排序算法

冒泡排序算法两两比较各个值，如果它们顺序不对就交换位置。第一轮之后，我们就知道最大的值已经位于列表末尾，因此第二轮不必再遍历整个列表，可以省掉最后一个值。这样，我们最多遍历列表的次数等于列表中值的个数减一，因为当除第一个值之外的所有值都各就其位时，第一个值自然也就在正确的位置上。不过，列表可能在此之前就已经排好序了。因此，每一轮之后我们都检查是否发生过交换：如果一轮下来没有发生任何交换，说明列表已经排好序，我们就退出算法：

```
void bubbleSort(LinkedList& list) {
  int listSize = list.size();

  if (listSize > 1) {
    int currSize = listSize - 1;
    int outerCount = 0;
    while (outerCount < (listSize - 1)) {
      Iterator currIterator = list.first();
      Iterator nextIterator = currIterator;
      nextIterator.next();
      bool changed = false;

      int innerCount = 0;
      while (innerCount < currSize) {
        if (currIterator.getValue() > nextIterator.getValue()) {
          swap(currIterator, nextIterator);
          changed = true;
        }

        ++innerCount;
        currIterator.next();
        nextIterator.next();
      }

      if (!changed) {
```

```
            break;
          }

          --currSize;
          ++outerCount;
        }
      }
    }
```

swap 函数交换迭代器给出的两个位置上的值：

```
void swap(Iterator iterator1, Iterator iterator2) {
  double tempValue = iterator1.getValue();
  iterator1.setValue(iterator2.getValue());
  iterator2.setValue(tempValue);
}
```

我们向列表加入一些值，然后对列表排序，以此来测试这些算法。

Main.cpp

```
#include <IOStream>
#include <CStdLib>

using namespace std;

#include "..\ListBasic\Cell.h"
#include "..\ListBasic\Iterator.h"
#include "..\ListBasic\List.h"

#include "Search.h"
#include "Sort.h"

void main() {
  cout << "LinkedList" << endl;

  LinkedList list;
  list.add(9);
  list.add(7);
  list.add(5);
  list.add(3);
  list.add(1);

  list.write(cout);
  cout << endl;
```

我们用 Iterator 类遍历列表，并对列表中的每个值调用 linarySearch：

```
Iterator iterator = list.first();
while (iterator.hasNext()) {
  cout << "<" << iterator.getValue() << ","
       << linarySearch(iterator.getValue(), list) << "> ";
  iterator.next();
}
```

我们还对列表中不存在的值测试了搜索算法，它们的下标将是负一：

```
cout << "<0," << linarySearch(0, list) << "> ";
cout << "<6," << linarySearch(6, list) << "> ";
cout << "<10," << linarySearch(10, list) << ">"
     << endl;
```

我们用冒泡排序、选择排序和插入排序算法对列表排序：

```
  cout << "Bubble Sort ";
  bubbleSort(list);
  list.write(cout);
  cout << endl;

  cout << "Select Sort ";
  selectSort(list);
  list.write(cout);
  cout << endl;

  cout << "Insert Sort ";
  insertSort(list);
  list.write(cout);
  cout << endl;
}
```

对搜索和排序算法进行归类的一种方法是使用大 O 记法（big O notation）。粗略地说，这种记法关注的是最坏情形。就插入排序而言，我们对每个值都要遍历一次列表，而对每个值，我们可能还得遍历整个列表才能找到它的正确位置。同样，就选择排序而言，我们对每个值都要遍历一次列表，而对每个值，我们可能也得遍历整个列表。最后，就冒泡排序而言，我们对每个值都要遍历一次列表，而且对每个值都可能要遍历整个列表。在这三种情形下，对长度为 n 的列表，我们都可能要执行 n^2 次操作。因此，插入排序、选择排序和冒泡排序算法的时间效率都是大 O 的 n^2，即 $O(n^2)$。不过在空间效率上，冒泡排序更好，因为它直接在原列表上操作；而插入排序和选择排序则需要一个额外的列表来存放排序结果。

扩展的列表类

本节我们重新审视 LinkedList 类，不过这次我们将从几个方面对它进行扩展：

- Cell 类原本有一组 set 和 get 方法。这次我们将用一对重载的引用方法来替换每一对这样的方法。
- 前面的列表只能存放 double 类型的值。现在我们将把列表定义为模板，使它能够存放任意类型的值。
- 我们将用重载的运算符替换其中一些方法。
- Cell 和 Iterator 原本是独立的类。现在我们将把它们变成内部类，定义在 LinkedList 之内。

List.h

```
class OutOfMemoryException : public exception {
  // Empty.
};
```

前面几节的类中，列表存放的是 double 类型的值。而这些类中，我们不再使用 double，而是使用模板类型 T——一个可以由任意类型实例化的通用类型。本节的 LinkedList 类是一个带通用类型 T 的模板：

```
template <class T>
class LinkedList {
  private:
    class Cell {
      private:
        Cell(const T& value, Cell* previous, Cell* next);
```

value 方法以两个版本重载。第一个版本是常量版本，返回常量值；另一个版本不是常量版本，返回对值的引用。这样就可以对单元格的值进行赋值，如下例所示：

```
public:
  const T value() const { return m_value; }
  T& value() { return m_value; }
```

Cell& 这一构造意味着方法返回的是对指向 Cell 对象的指针的引用。随后可以用这个引用给指针赋一个新值：

```
      const Cell* previous() const { return m_previous; }
      Cell*& previous() { return m_previous; }

      const Cell* next() const { return m_next; }
      Cell*& next() { return m_next; }

      friend class LinkedList;

    private:
      T m_value;
      Cell *m_previous, *m_next;
};
```

```
public:
  class Iterator {
    public:
      Iterator();

    private:
      Iterator(Cell* cellPtr);

    public:
      Iterator(const Iterator& iterator);
      Iterator& operator=(const Iterator& iterator);
```

我们重载相等与不相等运算符来代替 equal 和 notEqual：

```
bool operator==(const Iterator& iterator);
bool operator!=(const Iterator& iterator);
```

我们还用自增（++）和自减（--）运算符替换原来的递增和递减方法。它们各有两个版本——前缀版本和后缀版本。不带参数的是前缀版本（++i 和 --i），带整数参数的是后缀版本（i++ 和 i--）。注意，实际上我们并不给运算符传入整数参数；这个参数只是为了区分两个版本而设，会被编译器忽略：

```
bool operator++();    // prefix: ++i
bool operator++(int); // postfix: i++

bool operator--();    // prefix: --i
bool operator--(int); // postfix: i--
```

我们用两个重载的解引用运算符（*）替换 getValue 和 setValue 方法。它们的工作方式与前面 Cell 类的 value 方法类似：第一个版本是常量版本，返回值；第二个版本不是常量版本，返回对值的引用：

```
    T operator*() const;
    T& operator*();

    friend class LinkedList;

  private:
    Cell *m_cellPtr;
};
```

ReverseIterator 类

为了既能从头遍历到尾，也能从尾遍历到头，我们加入了 ReverseIterator。它与前面使用的 Iterator 几乎完全相同；唯一的区别是自增和自减运算符的移动方向相反：

```
  class ReverseIterator {
    public:
      ReverseIterator();

    private:
      ReverseIterator(Cell* cellPtr);

    public:
      ReverseIterator(const ReverseIterator& iterator);
      const ReverseIterator&
            operator=(const ReverseIterator& iterator);

      bool operator==(const ReverseIterator& iterator);
      bool operator!=(const ReverseIterator& iterator);

      bool operator++();    // prefix: ++i
      bool operator++(int); // postfix: i++

      bool operator--();
      bool operator--(int);

      T operator*() const;
      T& operator*();

      friend class LinkedList;

    private:
      Cell *m_cellPtr;
};

public:
  LinkedList();
  LinkedList(const LinkedList& list);
  LinkedList& operator=(const LinkedList& list);
  ~LinkedList();
  void clear();

  int size() const {return m_size;}
  bool empty() const {return (m_size == 0);}

  bool operator==(const LinkedList& list) const;
```

```
    bool operator!=(const LinkedList& list) const;

    void add(const T& value);
    void add(const LinkedList& list);

    void insert(const Iterator& insertPosition, const T& value);
    void insert(const Iterator& insertPosition,
                const LinkedList& list);

    void erase(const Iterator& erasePosition);
    void remove(const Iterator& firstPosition,
                const Iterator& lastPosition = Iterator(nullptr));
```

前一节只有返回迭代器的 first 和 last 方法。本节中, begin 和 end 方法用于正向遍历, 而 rbegin 和 rend（reverse begin 和 reverse end 的缩写）用于反向遍历:

```
Iterator begin() const { return Iterator(m_firstCellPtr); }
Iterator end() const { return Iterator(nullptr); }
ReverseIterator rbegin() const
  {return ReverseIterator(m_lastCellPtr);}
ReverseIterator rend() const
  { return ReverseIterator(nullptr); }
```

我们用重载的输入和输出流运算符替换 read 和 write 方法。由于它们是函数而不是方法，它们需要自己的模板标记:

```
    template <class U>
    friend istream& operator>>(istream& outStream,
                               LinkedList<U>& list);

    template <class U>
    friend ostream& operator<<(ostream& outStream,
                               const LinkedList<U>& list);

  private:
    int m_size;
    Cell *m_firstCellPtr, *m_lastCellPtr;
};
```

注意，模板类的成员函数是在头文件中实现的。因此，实现模板类时不需要单独的实现文件。

与类定义一样，方法的定义前面也必须带上 template 关键字。注意，类名 LinkedList 后面要跟类型标记 <T>:

```
template <class T>
LinkedList<T>::Cell::Cell(const T& value, Cell* previous,
                          Cell* next)
```

```
 :m_value(value),
  m_previous(previous),
  m_next(next) {
  // Empty.
}

template <class T>
LinkedList<T>::Iterator::Iterator()
 :m_cellPtr(nullptr) {
  // Empty.
}
```

注意，实现内部类的方法时，实现中需要同时写明外部类（LinkedList）和内部类（Cell）的名字：

```
template <class T>
LinkedList<T>::Iterator::Iterator(Cell* cellPtr)
 :m_cellPtr(cellPtr) {
  // Empty.
}

template <class T>
LinkedList<T>::Iterator::Iterator(const Iterator& position)
 :m_cellPtr(position.m_cellPtr) {
  // Empty.
}
```

由于 LinkedList 是模板类，编译器并不知道它的内部类 Iterator 实际上是一个类。在编译器看来，iterator 可能是类型、值，也可能是类。因此，我们需要用 typename 关键字告知编译器：

```
template <class T>
typename LinkedList<T>::Iterator&
LinkedList<T>::Iterator::operator=(const Iterator& iterator) {
  m_cellPtr = iterator.m_cellPtr;
  return *this;
}
```

接下来的运算符版本按与前一版 LinkedList 中对应方法相同的方式实现。也就是说，equal 方法被相等运算符（operator==）取代，notEqual 方法被不相等运算符（operator!=）取代：

```
template <class T>
bool LinkedList<T>::Iterator::operator==(const Iterator& position) {
  return (m_cellPtr == position.m_cellPtr);
}

template <class T>
bool LinkedList<T>::Iterator::operator!=(const Iterator& position) {
```

```
  return !(*this == position);
}
```

自增运算符有前缀和后缀两个版本的 operator++。它们的区别在于：前缀版本不接受任何参数，而后缀版本接受一个整数值作为参数。注意，运算符并不使用这个整数值；它的值是未定义的（不过通常设为零），并且总是被忽略。它的存在只是为了区分前缀和后缀两种情况：

```
template <class T>
bool LinkedList<T>::Iterator::operator++() {
  if (m_cellPtr != nullptr) {
    m_cellPtr = m_cellPtr->next();
    return true;
  }

  return false;
}

template <class T>
bool LinkedList<T>::Iterator::operator++(int) {
  if (m_cellPtr != nullptr) {
    m_cellPtr = m_cellPtr->next();
    return true;
  }

  return false;
}
```

自减运算符同样有前缀和后缀两个版本，工作方式与自增运算符类似：

```
template <class T>
bool LinkedList<T>::Iterator::operator--() {
  if (m_cellPtr != nullptr) {
    m_cellPtr = m_cellPtr->previous();
    return true;
  }

  return false;
}

template <class T>
bool LinkedList<T>::Iterator::operator--(int) {
  if (m_cellPtr != nullptr) {
    m_cellPtr = m_cellPtr->previous();
  }

  return false;
}
```

解引用运算符也有两个版本。第一个版本是常量版本，返回值；第二个版本不是常量版本，返回对值的引用而不是值本身。这样，第一个版本可以在常量对象上调用，此时不允许修改它的值；第二个版本只能在非常量对象上调用，通过给方法返回的值赋一个新值，我们就可以修改原值：

```
template <class T>
T LinkedList<T>::Iterator::operator*() const {
  return m_cellPtr->value();
}

template <class T>
T& LinkedList<T>::Iterator::operator*() {
  return m_cellPtr->value();
}
```

ReverseIterator 类有三个构造函数。第一个是默认构造函数，第二个用 Cell 指针初始化，第三个是复制构造函数——它接受对另一个 ReverseIterator 对象的引用，并初始化 Cell 指针：

```
template <class T>
LinkedList<T>::ReverseIterator::ReverseIterator()
 :m_cellPtr(nullptr) {
  // Empty.
}

template <class T>
LinkedList<T>::ReverseIterator::ReverseIterator(Cell* currCellPtr)
 :m_cellPtr(currCellPtr) {
  // Empty.
}

template <class T>
LinkedList<T>::ReverseIterator::ReverseIterator
                                (const ReverseIterator& position)
 :m_cellPtr(position.m_cellPtr) {
  // Empty.
}
```

赋值运算符用给定 ReverseIterator 对象引用的 Cell 指针来初始化自己的 Cell 指针：

```
template <class T>
const typename LinkedList<T>::ReverseIterator&
LinkedList<T>::ReverseIterator::operator=(const ReverseIterator&
position) {
  m_cellPtr = position.m_cellPtr;
  return *this;
}
```

如果两个反向迭代器的单元格指针指向同一个单元格，它们就相等：

```
template <class T>
bool LinkedList<T>::ReverseIterator::operator==
                            (const ReverseIterator& position) {
  return (m_cellPtr == position.m_cellPtr);
}

template <class T>
bool LinkedList<T>::ReverseIterator::operator!=
                            (const ReverseIterator& position) {
  return !(*this == position);
}
```

Iterator 类与 ReverseIterator 类的自增、自减运算符的区别在于：Iterator 中的自增运算符调用 Cell 的 next，自减运算符调用 previous；而 ReverseIterator 正好相反——自增运算符调用 previous，自减运算符调用 next。正如名字所暗示的：Iterator 正向遍历，而 ReverseIterator 反向遍历：

```
template <class T>
bool LinkedList<T>::ReverseIterator::operator++() {
  if (m_cellPtr != nullptr) {
    m_cellPtr = m_cellPtr->previous();
    return true;
  }

  return false;
}

template <class T>
bool LinkedList<T>::ReverseIterator::operator++(int) {
  if (m_cellPtr != nullptr) {
    m_cellPtr = m_cellPtr->previous();
    return true;
  }

  return false;
}

template <class T>
bool LinkedList<T>::ReverseIterator::operator--() {
  if (m_cellPtr != nullptr) {
    m_cellPtr = m_cellPtr->next();
    return true;
  }
```

```
  return false;
}

template <class T>
bool LinkedList<T>::ReverseIterator::operator--(int) {
  if (m_cellPtr != nullptr) {
    m_cellPtr = m_cellPtr->next();
    return true;
  }

  return false;
}

template <class T>
T LinkedList<T>::ReverseIterator::operator*() const {
  return m_cellPtr->value();
}

template <class T>
T& LinkedList<T>::ReverseIterator::operator*() {
  return m_cellPtr->value();
}
```

LinkedList 的默认构造函数把列表初始化为空，指向第一个和最后一个单元格的指针都设为空：

```
template <class T>
LinkedList<T>::LinkedList()
 :m_size(0),
  m_firstCellPtr(nullptr),
  m_lastCellPtr(nullptr) {
  // Empty.
}

template <class T>
LinkedList<T>::LinkedList(const LinkedList<T>& list) {
  *this = list;
}
```

赋值运算符以与非模板方法相同的方式复制给定列表的值：

```
template <class T>
LinkedList<T>& LinkedList<T>::operator=(const LinkedList<T>& list) {
  m_size = 0;
  m_firstCellPtr = nullptr;
  m_lastCellPtr = nullptr;
```

```
    if (list.m_size > 0) {
      for (Cell *listCellPtr = list.m_firstCellPtr,
                *nextCellPtr = list.m_lastCellPtr->next();
           listCellPtr != nextCellPtr;
           listCellPtr = listCellPtr->next()) {
        Cell *newCellPtr = new Cell(listCellPtr->value(),
                                    m_lastCellPtr, nullptr);
        if (m_firstCellPtr == nullptr) {
          m_firstCellPtr = newCellPtr;
        }
```

注意，这里我们使用的是 next 方法的引用版本，它允许我们对方法的调用结果进行赋值。由于 next 返回的是对单元格 next 指针的引用，我们就可以把 newCellPtr 的值赋给这个指针：

```
        else {
          m_lastCellPtr->next() = newCellPtr;
        }

        m_lastCellPtr = newCellPtr;
        ++m_size;
      }
    }

    return *this;
}
```

析构函数只是调用 clear 方法，后者遍历链表并删除每一个单元格：

```
template <class T>
LinkedList<T>::~LinkedList() {
  clear();
}

template <class T>
void LinkedList<T>::clear() {
  Cell *currCellPtr = m_firstCellPtr;

  while (currCellPtr != nullptr) {
    Cell *deleteCellPtr = currCellPtr;
    currCellPtr = currCellPtr->next();
    delete deleteCellPtr;
  }
```

单元格被删除之后，指向第一个和最后一个单元格的指针都被设为空：

```
  m_size = 0;
  m_firstCellPtr = nullptr;
```

```
  m_lastCellPtr = nullptr;
}
```

如果两个列表大小相同，并且它们的单元格存放着相同的值，它们就相等：

```
template <class T>
bool LinkedList<T>::operator==(const LinkedList<T>& list) const {
  if (m_size != list.m_size) {
    return false;
  }

  for (Iterator thisIterator = begin(),
                listIterator = list.begin();
       thisIterator != end(); ++thisIterator, ++listIterator) {
    if (*thisIterator != *listIterator) {
      return false;
    }
  }

  return true;
}

template <class T>
bool LinkedList<T>::operator!=(const LinkedList<T>& list) const {
  return !(*this == list);
}
```

add 方法在列表末尾加入一个带有新值的单元格，如下例所示：

```
template <class T>
void LinkedList<T>::add(const T& value) {
  Cell *newCellPtr = new Cell(value, m_lastCellPtr, nullptr);

  if (m_lastCellPtr == nullptr) {
    m_firstCellPtr = newCellPtr;
    m_lastCellPtr = newCellPtr;
  }
  else {
    m_lastCellPtr->next() = newCellPtr;
    m_lastCellPtr = newCellPtr;
  }

  ++m_size;
}
```

add 的第二个版本把给定列表加到列表末尾，如下例所示：

```
template <class T>
void LinkedList<T>::add(const LinkedList<T>& list) {
  for (Cell *listCellPtr = list.m_firstCellPtr;
       listCellPtr != nullptr; listCellPtr = listCellPtr->next()) {
    const T& value = listCellPtr->value();
    Cell *newCellPtr = new Cell(value, m_lastCellPtr, nullptr);

    if (m_lastCellPtr == nullptr) {
      m_firstCellPtr = newCellPtr;
    }
    else {
      m_lastCellPtr->next() = newCellPtr;
    }

    m_lastCellPtr = newCellPtr;
  }

  m_size += list.m_size;
}
```

insert 方法在给定位置加入一个值或一个列表:

```
template <class T>
void LinkedList<T>::insert(const Iterator& insertPosition,
                           const T& value) {
  if (insertPosition.m_cellPtr == nullptr) {
    add(value);
  }
  else {
    Cell *insertCellPtr = insertPosition.m_cellPtr;
    Cell *newCellPtr =
      new Cell(value, insertCellPtr->previous(), insertCellPtr);
    insertCellPtr->previous() = newCellPtr;

    if (insertCellPtr == m_firstCellPtr) {
      m_firstCellPtr = newCellPtr;
    }
    else {
      newCellPtr->previous()->next() = newCellPtr;
    }

    ++m_size;
  }
}

template <class T>
void LinkedList<T>::insert(const Iterator& insertPosition,
```

```
                                const LinkedList<T>& list) {
    if (insertPosition.m_cellPtr == nullptr) {
      add(list);
    }
    else {
      Cell *insertCellPtr = insertPosition.m_cellPtr;

      Cell *firstInsertCellPtr = nullptr,
           lastInsertCellPtr = nullptr;

      for (Cell *listCellPtr = list.m_firstCellPtr;
           listCellPtr != nullptr;listCellPtr=listCellPtr->next()) {
        double value = listCellPtr->value();
        Cell *newCellPtr =
          new Cell(value, lastInsertCellPtr, nullptr);

        if (firstInsertCellPtr == nullptr) {
          firstInsertCellPtr = newCellPtr;
        }
        else {
          lastInsertCellPtr->next() = newCellPtr;
        }

        lastInsertCellPtr = newCellPtr;
      }

      if (firstInsertCellPtr != nullptr) {
        if (insertCellPtr->previous() != nullptr) {
          insertCellPtr->previous()->next() = firstInsertCellPtr;
          firstInsertCellPtr->previous() =
            insertCellPtr->previous();
        }
        else {
          m_firstCellPtr = firstInsertCellPtr;
        }
      }

      if (lastInsertCellPtr != nullptr) {
        lastInsertCellPtr->next() = insertCellPtr;
        insertCellPtr->previous() = lastInsertCellPtr;
      }

      m_size += list.m_size;
    }
  }
```

erase 和 remove 方法从列表中移除子列表中的值：

```
template <class T>
void LinkedList<T>::erase(const Iterator& removePosition) {
  remove(removePosition, removePosition);
}

template <class T>
void LinkedList<T>::remove(const Iterator& firstPosition,
           const Iterator& lastPosition /*= Iterator(nullptr)*/) {
  Cell *firstCellPtr = firstPosition.m_cellPtr,
      *lastCellPtr = lastPosition.m_cellPtr;
  lastCellPtr = (lastCellPtr == nullptr)
                ? m_lastCellPtr : lastCellPtr;

  Cell *previousCellPtr = firstCellPtr->previous(),
      *nextCellPtr = lastCellPtr->next();

  Cell *currCellPtr = firstCellPtr;
  while (currCellPtr != nextCellPtr) {
    Cell *deleteCellPtr = currCellPtr;
    currCellPtr = currCellPtr->next();
    delete deleteCellPtr;
    --m_size;
  }

  if (previousCellPtr != nullptr) {
    previousCellPtr->next() = nextCellPtr;
  }
  else {
    m_firstCellPtr = nextCellPtr;
  }

  if (nextCellPtr != nullptr) {
    nextCellPtr->previous() = previousCellPtr;
  }
  else {
    m_lastCellPtr = previousCellPtr;
  }
}
```

输入流运算符先读取列表的大小，然后再读取各个值：

```
template <class T>
istream& operator>>(istream& inStream, LinkedList<T>& list) {
  int size;
  inStream >> size;

  for (int count = 0; count < size; ++count) {
```

```
    T value;
    inStream >> value;
    list.add(value);
  }

  return inStream;
}
```

输出流运算符把列表写到给定的流上，列表用方括号括起来，值之间用逗号分隔：

```
template <class T>
ostream& operator<<(ostream& outStream, const LinkedList<T>& list) {
  outStream << "[";

  bool first = true;
  for (const T& value : list) {
    outStream << (first ? "" : ",") << value;
    first = false;
  }

  outStream << "]";
  return outStream;
}
```

我们让用户输入一个列表，然后用 for 语句自动遍历它，同时用正向和反向迭代器手动遍历它，以此来测试 LinkedList 类。

Main.cpp

```
#include <IOStream>
#include <Exception>
using namespace std;

#include "List.h"

void main() {
  LinkedList<double> list;
  cin >> list;
  cout << list << endl;
```

注意，可以直接对列表使用 for 语句，因为扩展列表带有 begin 方法，它返回的迭代器带有前缀自增（++）和解引用（*）运算符：

```
for (double value : list) {
  cout << value << " ";
}
cout << endl;
```

我们也可以用 Iterator 类的 begin 和 end 方法手动遍历列表：

```
for (LinkedList<double>::Iterator iterator = list.begin();
     iterator != list.end(); ++iterator) {
  cout << *iterator << " ";
}
cout << endl;
```

借助 rbegin 和 rend 方法以及 ReverseIterator 类，我们从列表末尾遍历到列表开头。注意，虽然我们是在反向遍历列表，但使用的仍然是自增（++）而不是自减（--）：

```
  for (LinkedList<double>::ReverseIterator iterator =
       list.rbegin(); iterator != list.rend(); ++iterator) {
    cout << *iterator << " ";
  }
  cout << endl;
}
```

扩展的集合类

与前一节的版本相比，本节的 Set 类在三个方面进行了扩展：

- 集合按有序列表的方式存放，这使得一些方法更高效。
- 这个类是模板；只要类型支持排序比较，它就可以存放任意类型的值。
- 这个类使用了运算符重载，（但愿）这让它的使用更简单、更直观。

在 C++ 中，可以用 typedef 关键字定义我们自己的类型。我们把 Set 的 Iterator 定义为与 LinkedList 中相同的迭代器。前一节中，Iterator 是一个独立类，处理集合时可以直接复用。然而本节中，Iterator 是一个内部类；否则，由于 Set 以 private 方式继承 LinkedList，处理集合时将无法访问 LinkedList。记住：以 private 方式继承时，基类的所有方法和字段在子类中都会变成 private。

Set.h

```
template <class T>
class Set : private LinkedList<T> {
  public:
    typedef LinkedList<T>::Iterator Iterator;

    Set();
    Set(const T& value);
    Set(const Set& set);
    Set& operator=(const Set& set);
    ~Set();
```

我们用重载的比较运算符替换 equal 和 notEqual 方法。这样就可以像比较两个整数那样比较两个集合：

```
bool operator==(const Set& set) const;
bool operator!=(const Set& set) const;

int size() const { return LinkedList<T>::size(); }
bool empty() const { return LinkedList<T>::empty(); }
Iterator begin() const { return LinkedList<T>::begin(); }
```

我们用加法、乘法和减法运算符替换 unionSet、intersection 和 difference 方法：

```
Set operator+(const Set& set) const;
Set operator*(const Set& set) const;
Set operator-(const Set& set) const;
```

merge 函数由集合的各个方法调用，用来高效地合并集合。由于它是函数而不是方法，它必须带有自己的模板标记：

```
private:
  template <class U>
  friend Set<U>
    merge(const Set<U>& leftSet, const Set<U>& rightSet,
          bool addLeft, bool addEqual, bool addRight);

public:
  Set& operator+=(const Set& set);
  Set& operator*=(const Set& set);
  Set& operator-=(const Set& set);
```

与前面的 LinkedList 类类似，我们用重载的流运算符替换 read 和 write 方法。由于它们同样不是方法而是函数，它们同样需要自己的模板标记：

```
    template <class U>
    friend istream& operator>>(istream& inStream, Set<U>& set);

    template <class U>
    friend ostream& operator<<(ostream& outStream,
                               const Set<U>& set);
};
```

与非模板版本相比，这些构造函数看起来几乎一样：

```
template <class T>
Set<T>::Set() {
  // Empty.
}
```

```
template <class T>
Set<T>::Set(const T& value) {
  add(value);
}

template <class T>
Set<T>::Set(const Set& set)
 :LinkedList(set) {
  // Empty.
}

template <class T>
Set<T>::~Set() {
  // Empty.
}

template <class T>
Set<T>& Set<T>::operator=(const Set& set) {
  clear();
  add(set);
  return *this;
}
```

测试两个集合是否相等时，可以直接调用 LinkedList 的相等运算符，因为本节的集合是有序的：

```
template <class T>
bool Set<T>::operator==(const Set& set) const {
  return LinkedList::operator==(set);
}
```

与前面的类类似，我们通过调用相等运算符来测试两个集合是否不相等。不过，在这个类中，我们把当前对象（用 this 指针）与给定集合显式地进行比较：

```
template <class T>
bool Set<T>::operator!=(const Set& set) const {
  return !(*this == set);
}
```

并集、交集与差集

我们用加法、减法和乘法运算符替换 unionSet、intersection 和 difference 方法。它们都会调用 merge，只是 addLeft、addEqual 和 addRight 三个参数的取值不同。就并集而言，三个参数全为 true，意思是：只出现在左侧集合中的值、同时出现在两个集合中的值、以及只出现在

右侧集合中的值，都应加入并集：

```
template <class T>
Set<T> Set<T>::operator+(const Set& set) const {
  return merge(*this, set, true, true, true);
}
```

就交集而言，只有 addEqual 为 true，意思是：同时出现在两个集合中的值应加入交集，而只出现在其中一个集合中的值不应加入。请看下面的例子：

```
template <class T>
Set<T> Set<T>::operator*(const Set& set) const {
  return merge(*this, set, false, true, false);
}
```

就差集而言，只有 addLeft 为 true，意思是：只有出现在左侧集合中、而不同时出现在两个集合中、也不只出现在右侧集合中的值，才应加入差集：

```
template <class T>
Set<T> Set<T>::operator-(const Set& set) const {
  return merge(*this, set, true, false, false);
}
```

merge 方法接受两个集合以及三个布尔值 addLeft、addEqual 和 addRight。如果 addLeft 为 true，只出现在左侧集合中的值会加入结果集合；如果 addEqual 为 true，同时出现在两个集合中的值会加入；如果 rightAdd 为 true，只出现在右侧集合中的值会加入：

```
template <class T>
Set<T> merge(const Set<T>& leftSet, const Set<T>& rightSet,
             bool addLeft, bool addEqual, bool addRight) {
  Set<T> result;
  Set<T>::Iterator leftIterator = leftSet.begin(),
                   rightIterator = rightSet.begin();
```

while 语句在左侧集合和右侧集合都还有值的情况下不断迭代：

```
while ((leftIterator != leftSet.end()) &&
       (rightIterator != rightSet.end())) {
```

如果左侧的值更小，那么当 addLeft 为 true 时它会被加入结果集合。然后左侧集合的迭代器前移一步：

```
if (*leftIterator < *rightIterator) {
  if (addLeft) {
    result.add(*leftIterator);
  }
```

```
    ++leftIterator;
  }
```

如果右侧的值更小，那么当 addRight 为 true 时它会被加入结果集合。然后右侧集合的迭代器前移一步：

```
  else if (*leftIterator > *rightIterator) {
    if (addRight) {
      result.add(*rightIterator);
    }

    ++rightIterator;
  }
```

最后，如果两个值相等，则加入其中一个（但不能两个都加，因为集合不允许重复），并且两个迭代器都前移一步：

```
    else {
      if (addEqual) {
        result.add(*leftIterator);
      }

      ++leftIterator;
      ++rightIterator;
    }
  }
```

如果 addLeft 为 true，左侧集合中剩余的所有值（如果还有的话）都加入结果集合：

```
  if (addLeft) {
    while (leftIterator != leftSet.end()) {
      result.add(*leftIterator);
      ++leftIterator;
    }
  }
```

如果 addRight 为 true，右侧集合中剩余的所有值（如果还有的话）都加入结果集合：

```
  if (addRight) {
    while (rightIterator != rightSet.end()) {
      result.add(*rightIterator);
      ++rightIterator;
    }
  }
```

最后，用下面的语句返回结果集合：

```
  return result;
}
```

对本集合与另一个集合执行并集运算时，我们只需调用加法运算符。注意，我们用 this 指针返回自己的对象：

```
template <class T>
Set<T>& Set<T>::operator+=(const Set& set) {
  *this = *this + set;
  return *this;
}
```

同样，对本集合与另一个集合执行交集和差集运算时，我们调用乘法和减法运算符。请看下面的例子：

```
template <class T>
Set<T>& Set<T>::operator*=(const Set& set) {
  *this = *this * set;
  return *this;
}

template <class T>
Set<T>& Set<T>::operator-=(const Set& set) {
  *this = *this - set;
  return *this;
}
```

读取集合时，先输入集合中值的个数，然后再输入各个值。这个函数与 LinkedList 类中的对应函数非常相似。不过，为了避免出现重复值，这里调用的是复合加法运算符（+=）而不是 add 方法：

```
template <class T>
istream& operator>>(istream& inStream, Set<T>& set) {
  int size;
  inStream >> size;

  for (int count = 0; count < size; ++count) {
    T value;
    inStream >> value;
    set += value;
  }

  return inStream;
}
```

写出集合时，我们用花括号（"{ 和 "}）而不是方括号（"[和 "]"）把值括起来，与列表的情况一样：

```
template <class T>
ostream& operator<<(ostream& outStream, const Set<T>& set) {
  outStream << "{";
  bool first = true;

  for (const T& value : set) {
    outStream << (first ? "" : ",") << value;
    first = false;
  }

  outStream << "}";
  return outStream;
}
```

我们让用户输入两个集合，用迭代器手动遍历它们、用 for 语句自动遍历它们，并求出两个集合的并集、交集与差集，以此来测试集合。

Main.cpp

```
#include <IOStream>
using namespace std;

#include "..\ListAdvanced\List.h"
#include "Set.h"

void main() {
  Set<double> s, t;
  cin >> s >> t;

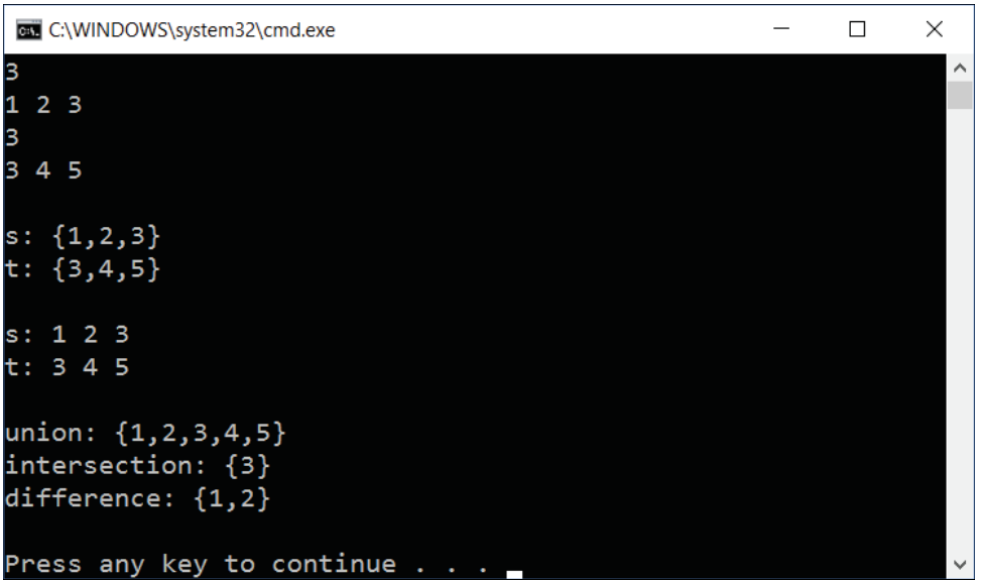
  cout << endl << "s: " << s << endl;
  cout << "t: " << t << endl;

  cout << endl << "s: ";
  for (double value : s) {
    cout << value << " ";
  }

  cout << endl << "t: ";
  for (Set<double>::Iterator iterator = t.begin();
       iterator != t.end(); ++iterator) {
    cout << *iterator << " ";
  }

  cout << endl << endl << "union: " << (s + t) << endl;
  cout << "intersection: " << (s * t) << endl;
  cout << "difference: " << (s - t) << endl << endl;
}
```

执行程序时，输出显示在一个命令窗口中：



高级搜索与排序

前一节我们研究了线性搜索。本节我们来看二分搜索。二分搜索算法先在列表中点处查找该值，然后用列表的一半继续搜索。这样，由于每一次迭代都把列表一分为二，它的时间效率是 $O(\log_2 n)$。

Search.h

```
template <class ListType, class ValueType>
int binarySearch(const ValueType& value, const ListType& list) {
  ListType::Iterator* positionBuffer =
    new ListType::Iterator[list.size()];

  int index = 0;
  for (ListType::Iterator position = list.begin();
       position != list.end(); ++position) {
    positionBuffer[index++] = position;
  }

  int minIndex = 0, maxIndex = list.size() - 1;
```

```
  while (minIndex <= maxIndex) {
    int middleIndex = (maxIndex + minIndex) / 2;
    ListType::Iterator iterator = positionBuffer[middleIndex];
    const ValueType& middleValue = *iterator;

    if (value == middleValue) {
      return middleIndex;
    }
    else if (value < middleValue) {
      maxIndex = middleIndex - 1;
    }
    else {
      minIndex = middleIndex + 1;
    }
  }

  return -1;
}
```

归并排序算法

归并排序算法把列表分成两个大小相等的子列表，通过递归调用对子列表排序（所谓递归调用，就是一个方法或函数调用它自己），然后以类似于前面扩展版 Set 类的 merge 方法的方式把有序的子列表合并起来。

Sort.h

```
template <class ListType, class ValueType>
void mergeSort(ListType& list) {
  int size = list.size();

  if (size > 1) {
    int middle = list.size() / 2;
    ListType::Iterator iterator = list.begin();

    ListType leftList;
    for (int count = 0; count < middle; ++count) {
      leftList.add(*iterator);
      ++iterator;
    }

    ListType rightList;
    for (; iterator != list.end(); ++iterator) {
      rightList.add(*iterator);
    }
```

```
    mergeSort<ListType, ValueType>(leftList);
    mergeSort<ListType, ValueType>(rightList);

    ListType resultList;
    merge<ListType, ValueType>(leftList, rightList, resultList);
    list = resultList;
  }
}
```

本节的 merge 方法复用了本章前面扩展版 Set 类中 merge 的思路：

```
template <class ListType, class ValueType>
void merge(ListType& leftList, ListType& rightList,
           ListType& result) {
  ListType::Iterator leftPosition = leftList.begin();
  ListType::Iterator rightPosition = rightList.begin();

  while ((leftPosition != leftList.end()) &&
         (rightPosition != rightList.end())) {
    if (*leftPosition < *rightPosition) {
      result.add(*leftPosition);
      ++leftPosition;
    }
    else {
      result.add(*rightPosition);
      ++rightPosition;
    }
  }

  while (leftPosition != leftList.end()) {
    result.add(*leftPosition);
    ++leftPosition;
  }

  while (rightPosition != rightList.end()) {
    result.add(*rightPosition);
    ++rightPosition;
  }
}
```

快速排序算法

快速排序算法先选出第一个值（称为枢值，pivot value），然后把所有小于枢值的值放入较小的子列表，把所有大于或等于枢值的值放入较大的子列表。随后通过递归调用对这两个列表排

序，最后只需把它们拼接在一起。我们来看下面的例子：

```
template <class ListType, class ValueType>
void quickSort(ListType& list) {
  if (list.size() > 1) {
    ListType smaller, larger;
    ValueType pivotValue = *list.begin();

    ListType::Iterator position = list.begin();
    ++position;

    for (; position != list.end(); ++position) {
      if (*position < pivotValue) {
        smaller.add(*position);
      }
      else {
        larger.add(*position);
      }
    }

    quickSort<ListType, ValueType>(smaller);
    quickSort<ListType, ValueType>(larger);

    list = smaller;
    list.add(pivotValue);
    list.add(larger);
  }
}
```

归并排序是平衡的：它总是把列表分成相等的两部分并对它们排序。算法必须遍历列表一次才能把列表分成两个子列表，然后再对子列表排序。给定一个值列表，它必须遍历其 n 个值，并把列表分割 $\log_2 n$ 次。因此，归并排序的时间效率为 $O(n \log_2 n)$。

而快速排序算法在最坏情况下（列表已经有序时）并不比插入排序、选择排序或冒泡排序好：同样是 $O(n^2)$。不过，它在平均情况下是很快的。

小结

本章我们为抽象数据类型列表和集合创建了类。列表是一种有头有尾的有序结构，而集合是一种无序结构。

我们先从相当简单的版本入手：列表带有独立的单元格类和迭代器类。接着我们创建了更高级的版本，使用了模板和运算符重载，并把单元格类和迭代器类放进了列表类内部。最后，我们引入了重载的引用方法。

同样地，我们先创建了一个相当简单、低效的集合类版本，然后创建了使用模板和运算符重载的更高级版本；在这个版本中，我们把值按顺序存放，以便更高效地执行并集、交集和差集运算。

此外，我们还实现了线性搜索和二分搜索算法。线性搜索可用于任何无序序列，但效率较低；二分搜索效率更高，但只能用于有序序列。

最后，我们研究了排序算法。我们先从简单但相当低效的插入排序、选择排序和冒泡排序入手，接着继续研究更高级、更高效的归并排序和快速排序算法。

下一章我们将开始构建一个图书馆管理系统。

3
构建图书馆管理系统

本章我们研究一个图书馆管理系统。与前面几章一样，我们继续开发 C++ 类。不过，本章要开发的是一个更接近真实世界的系统。本章的图书馆系统可以供真实的图书馆使用。图书馆由一组图书和一组读者构成。图书记录哪些读者借阅或预约了它们，读者则记录自己借阅和预约了哪些图书。

基本思想是：图书馆保存一组图书和一组读者。每本书都标记为已借出或未借出；如果已借出，就会存放借阅该书的读者的身份编号（identity number）。此外，一本书还可以被一个或多个读者预约，因此每本书还保存一份预约该书的读者编号列表。它必须是列表而不是集合，因为图书要按照读者预约的先后顺序依次借给这些读者。

每位读者保存两个集合，分别存放他们已借阅和已预约图书的编号。这两种情况下我们都使用集合（set）而不是列表（list），因为借阅或预约图书的先后顺序并不重要。

本章将涵盖以下主题：

- 使用由图书和读者构成的类，它们组成一个以整数编号为键的小型数据库。
- 使用标准输入输出流：我们写出图书和读者的信息，并提示用户进行输入。
- 使用文件处理与流。图书和读者通过标准 C++ 文件流写出和读入。
- 最后，我们使用 C++ 标准库中泛型的集合、映射（map）与列表类。

Book 类

我们有三个类：Book、Customer 和 Library：

- `Book` 类记录一本书的信息。每本书都有作者和书名，以及一个唯一的编号。
- `Customer` 类记录一位读者的信息。每位读者都有名字和地址，以及一个唯一的编号。
- `Library` 类负责图书馆的各项操作，比如添加和删除图书与读者、借出、归还和预约图书，以及列出图书和读者。

main 函数只是创建一个 Library 类的对象。

此外，每本书都保存着当前是否被借出的信息；如果被借出，还会存放借阅该书的读者的编号。每本书还保存一份预约列表。同样，每位读者保存着当前借阅和预约图书的集合。

Book 类有两个构造函数。第一个是默认构造函数，用于从文件读取图书；第二个用于向图书馆添加新书，它以作者的名字和书名作为参数。

Book.h

```
class Book {
  public:
    Book(void);
    Book(const string& author, const string& title);
```

author 和 title 方法只是返回图书的作者和书名：

```
const string& author(void) const { return m_author; }
const string& title(void) const { return m_title; }
```

图书馆的图书可以写入文件，也可以从文件读回：

```
void read(ifstream& inStream);
void write(ofstream& outStream) const;
```

一本书可以被借出、预约或归还，预约也可以被取消。注意，图书被借出或预约时，我们需要提供读者的编号；而归还图书时则不需要，因为 Book 类自己记录着当前借阅该书的读者：

```
void borrowBook(int customerId);
int reserveBook(int customerId);
void unreserveBookation(int customerId);
void returnBook();
```

图书被借出时，会存放借阅该书的读者的编号；bookId 返回图书自身的编号：

```
int bookId(void) const { return m_bookId; }
```

borrowed 方法在图书当前被借出时返回 true。此时，customerId 返回借阅该书的读者的编号：

```
bool borrowed(void) const { return m_borrowed; }
int customerId(void) const { return m_customerId; }
```

一本书可以被一列读者预约，reservationList 返回这份预约列表：

```
list<int>& reservationList(void) { return m_reservationList; }
```

MaxBookId 字段是静态的，也就是说它为该类的所有对象所共有：

```
    static int MaxBookId;
```

输出流运算符写出图书的信息：

```
    friend ostream& operator<<(ostream& outStream,
                               const Book& book);
```

图书被借出时，m_borrowed 字段为 true。图书的编号和可能的借阅者存放在 m_bookId 和 m_customerId 中：

```
    private:
      bool m_borrowed = false;
      int m_bookId, m_customerId;
```

图书的作者名和书名存放在 m_author 和 m_title 中：

```
    string m_author, m_title;
```

不止一位读者可以预约同一本书。此时，他们的编号存放在 m_reservationList 中。它是列表而不是集合，因为预约是按先后顺序存放的。当一本书被归还时，按预约顺序排在下一位的读者就会借到这本书：

```
    list<int> m_reservationList;
    };
```

本章我们使用 C++ 标准库中泛型的集合、映射与列表类。它们的声明存放在 Set、Map 和 List 头文件中。集合与列表类保存的集合和列表与上一章我们自己实现的类似。映射是这样一种结构：每个值都由一个唯一的键来标识，以便提供快速访问。

Book.cpp

```
    #include <Set>
    #include <Map>
    #include <List>
    #include <String>
    #include <FStream>
    using namespace std;

    #include "Book.h"
    #include "Customer.h"
    #include "Library.h"
```

由于 MaxBookId 是静态的，我们用双冒号（::）记法来初始化它。每个静态字段都需要在类定义之外初始化：

```
    int Book::MaxBookId = 0;
```

默认构造函数什么也不做。它用于从文件读取时。尽管如此，我们仍然必须有一个默认构造函

数，才能创建 Book 类的对象：

```
Book::Book(void) {
  // Empty.
}
```

创建一本新书时，会为它分配一个唯一的编号。该编号存放在 MaxBookId 中，每创建一个新 Book 对象它就加一：

```
Book::Book(const string& author, const string& title)
 :m_bookId(++MaxBookId),
  m_author(author),
  m_title(title) {
  // Empty.
}
```

写入图书

图书以类似的方式写入流。不过这次不是 read 而是 write。它们的工作方式相似：

```
void Book::write(ofstream& outStream) const {
  outStream.write((char*) &m_bookId, sizeof m_bookId);
```

读取字符串时我们用 getline 而不是流运算符，因为流运算符只能读取一个单词，而 getline 能读取多个单词。不过写入流时，我们就可以用流运算符了，作者名和书名由一个还是多个单词都没有关系：

```
outStream << m_author << endl;
outStream << m_title << endl;

outStream.write((char*) &m_borrowed, sizeof m_borrowed);
outStream.write((char*) &m_customerId, sizeof m_customerId);
```

与读取时一样，我们先写入预约列表中预约的个数，然后再逐个写入预约者的编号：

```
  { int reservationListSize = m_reservationList.size();
    outStream.write((char*) &reservationListSize,
                    sizeof reservationListSize);

    for (int customerId : m_reservationList) {
      outStream.write((char*) &customerId, sizeof customerId);
    }
  }
}
```

读取图书

从文件读取任何类型的值（字符串除外）时，我们都使用 read 方法，它读取固定数目的字节。sizeof 运算符给出 m_bookId 字段以字节为单位的大小。sizeof 运算符也可以用来求某个类型的大小。例如，sizeof(int) 给出 int 类型值以字节为单位的大小。类型必须用圆括号括起来：

```
void Book::read(ifstream& inStream) {
  inStream.read((char*) &m_bookId, sizeof m_bookId);
```

从文件读取字符串值时，我们使用 C++ 标准函数 getline 来读取作者名和书名。如果名字由多个单词组成，使用输入流运算符是行不通的。如果作者名或书名由多个单词组成，只有第一个单词会被读入，其余的单词都不会被读取：

```
getline(inStream, m_author);
getline(inStream, m_title);
```

注意，m_borrowed 字段的值我们也用 read 方法读取，尽管它存放的是 bool 类型而不是 int：

```
inStream.read((char*) &m_borrowed, sizeof m_borrowed);
inStream.read((char*) &m_customerId, sizeof m_customerId);
```

读取预约列表时，我们先读出列表中预约的个数，然后再读出各位预约者的编号：

```
  { int reservationListSize;
    inStream.read((char*) &reservationListSize,
                  sizeof reservationListSize);

    for (int count = 0; count < reservationListSize; ++count) {
      int customerId;
      inStream.read((char*) &customerId, sizeof customerId);
      m_reservationList.push_back(customerId);
    }
  }
}
```

图书的借出与预约

图书被借出时，m_borrowed 变为 true，m_customerId 被设置为借阅该书的读者的编号：

```
void Book::borrowBook(int customerId) {
  m_borrowed = true;
  m_customerId = customerId;
}
```

图书被预约时的情况稍有不同。虽然一本书一次只能被一位读者借阅，却可以被多位读者预约。

读者的编号被加入 m_reservationList。列表的大小会被返回，让调用者知道自己在预约列表中的位置：

```
int Book::reserveBook(int customerId) {
  m_reservationList.push_back(customerId);
  return m_reservationList.size();
}
```

图书被归还时，我们只是把 m_borrowed 设为 false。不需要把 m_customerId 设置成任何特定的值，只要图书未被借出，它就无关紧要：

```
void Book::returnBook() {
  m_borrowed = false;
}
```

读者可以把自己从预约列表中移除。此时，我们对 m_reservationList 调用 remove：

```
void Book::unreserveBookation(int customerId) {
  m_reservationList.remove(customerId);
}
```

显示图书

输出流运算符写出图书的书名和作者。如果图书被借出，就写出读者的名字；如果预约列表非空，就写出各位预约读者的名字：

```
ostream& operator<<(ostream& outStream, const Book& book) {
  outStream << "\"" << book.m_title << "\" by " << book.m_author;
```

访问静态字段（比如 Library 中的 s_customerMap）时，我们使用双冒号（::）记法：

```
  if (book.m_borrowed) {
    outStream << endl << "  Borrowed by: "
              << Library::s_customerMap[book.m_customerId].name()
              << ".";
  }

  if (!book.m_reservationList.empty()) {
    outStream << endl << "  Reserved by: ";
    bool first = true;

    for (int customerId : book.m_reservationList) {
      outStream << (first ? "" : ",")
                << Library::s_customerMap[customerId].name();
      first = false;
    }
```

```
      outStream << ".";
    }

    return outStream;
  }
```

Customer 类

Customer 类记录一位读者的信息。它保存着该读者当前借阅和预约图书的集合。

Customer.h

```
class Customer {
  public:
    Customer(void);
    Customer(const string& name, const string& address);

    void read(ifstream& inStream);
    void write(ofstream& outStream) const;

    void borrowBook(int bookId);
    void reserveBook(int bookId);
    void returnBook(int bookId);
    void unreserveBook(int bookId);
```

hasBorrowed 方法在该读者当前至少借有一本书时返回 true。在下一节的 Library 类中，当前借有图书的读者是无法被删除的：

```
bool hasBorrowed(void) const { return !m_loanSet.empty(); }

const string& name(void) const {return m_name;}
const string& address(void) const {return m_address;}
int id(void) const {return m_customerId;}
```

与前面用过的 Book 类一样，我们使用静态字段 MaxCustomerId 来统计读者的编号。我们还使用输出流运算符写出读者的信息：

```
static int MaxCustomerId;
friend ostream& operator<<(ostream& outStream,
                           const Customer& customer);
```

每位读者都有名字、地址和一个唯一的编号。集合 m_loanSet 和 m_reservationSet 存放着该读者当前借阅和预约的图书的编号。注意，我们使用集合而不是列表，因为借阅和预约图书的先后顺序并不重要：

```
  private:
    int m_customerId;
    string m_name, m_address;
    set<int> m_loanSet, m_reservationSet;
};
```

Customer.cpp

```
#include <Set>
#include <Map>
#include <List>
#include <String>
#include <FStream>
using namespace std;

#include "Book.h"
#include "Customer.h"
#include "Library.h"
```

由于 MaxCustomerId 是静态字段，它需要在类之外定义：

```
int Customer::MaxCustomerId;
```

默认构造函数只在从文件加载对象时使用，因此不需要对字段进行初始化：

```
Customer::Customer(void) {
  // Empty.
}
```

第二个构造函数用于创建新的读者对象。我们用 MaxCustomerId 字段来初始化读者的编号，同时还初始化读者的名字和地址：

```
Customer::Customer(const string& name, const string& address)
 :m_customerId(++MaxCustomerId),
  m_name(name),
  m_address(address) {
  // Empty.
}
```

从文件读取读者

read 方法从文件流中读取一位读者的信息：

```
void Customer::read(ifstream& inStream) {
  inStream.read((char*) &m_customerId, sizeof m_customerId);
```

与 Book 类的 read 方法一样，我们必须使用 getline 函数而不是输入流运算符，因为输入流运

算符只会读入一个单词：

```
    getline(inStream, m_name);
    getline(inStream, m_address);

    { int loanSetSize;
      inStream.read((char*) &loanSetSize, sizeof loanSetSize);

      for (int count = 0; count < loanSetSize; ++count) {
        int bookId;
        inStream.read((char*) &bookId, sizeof bookId);
        m_loanSet.insert(bookId);
      }
    }

    { int reservationListSize;
      inStream.read((char*) &reservationListSize,
                    sizeof reservationListSize);

      for (int count = 0; count < reservationListSize; ++count) {
        int bookId;
        inStream.read((char*) &bookId, sizeof bookId);
        m_loanSet.insert(bookId);
      }
    }
  }
```

将读者写入文件

write 方法以与前面 Book 类相同的方式把读者的信息写入流：

```
  void Customer::write(ofstream& outStream) const {
    outStream.write((char*) &m_customerId, sizeof m_customerId);

    outStream << m_name << endl;
    outStream << m_address << endl;
```

写入集合时，我们先写入集合的大小，然后再逐个写入集合中的各个值：

```
    { int loanSetSize = m_loanSet.size();
      outStream.write((char*) &loanSetSize, sizeof loanSetSize);

      for (int bookId : m_loanSet) {
        outStream.write((char*) &bookId, sizeof bookId);
      }
    }
```

```
  { int reservationListSize = m_reservationSet.size();
    outStream.write((char*) &reservationListSize,
                    sizeof reservationListSize);

    for (int bookId : m_reservationSet) {
      outStream.write((char*) &bookId, sizeof bookId);
    }
  }
}
```

读者借阅与预约图书

读者借阅一本书时，这本书被插入该读者的借阅集合：

```
void Customer::borrowBook(int bookId) {
  m_loanSet.insert(bookId);
}
```

同样，读者预约一本书时，这本书被插入该读者的预约集合：

```
void Customer::reserveBook(int bookId) {
  m_reservationSet.insert(bookId);
}
```

读者归还或取消预约一本书时，这本书就从借阅集合或预约集合中移除：

```
void Customer::returnBook(int bookId) {
  m_loanSet.erase(bookId);
}

void Customer::unreserveBook(int bookId) {
  m_reservationSet.erase(bookId);
}
```

显示读者

输出流运算符写出读者的名字和地址。如果读者借阅或预约了图书，这些图书也会被写出：

```
ostream& operator<<(ostream& outStream, const Customer& customer){
  outStream << customer.m_customerId << ". " << customer.m_name
            << ", " << customer.m_address << ".";

  if (!customer.m_loanSet.empty()) {
    outStream << endl << "  Borrowed books: ";
```

```
      bool first = true;
      for (int bookId : customer.m_loanSet) {
        outStream << (first ? "" : ",")
                  << Library::s_bookMap[bookId].author();
        first = false;
      }
    }

    if (!customer.m_reservationSet.empty()) {
      outStream << endl << "  Reserved books: ";

      bool first = true;
      for (int bookId : customer.m_reservationSet) {
        outStream << (first ? "" : ",")
                  << Library::s_bookMap[bookId].title();
        first = false;
      }
    }

    return outStream;
  }
```

Library 类

最后，Library 类负责处理图书馆本身。它围绕借出和归还图书执行一系列任务。

Library.h

```
  class Library {
    public:
      Library();
    private:
      static string s_binaryPath;
```

lookupBook 方法按作者和书名查找一本书。如果找到，它返回 true；此时该书的信息（一个 Book 类的对象）会被复制到 bookPtr 所指的对象中：

```
  bool lookupBook(const string& author, const string& title,
                  Book* bookPtr = nullptr);
```

同样，lookupCustomer 按名字和地址查找一位读者。如果找到该读者，就返回 true，并把信息复制到 customerPtr 所指的对象中：

```
    bool lookupCustomer(const string& name, const string& address,
                        Customer* customerPtr = nullptr);
```

本章的应用程序围绕下面这些方法展开。它们执行图书馆系统的各项任务。每个方法都会提示用户进行输入，然后执行一项任务，比如借出或归还一本书。

下面的方法各自执行一项任务，即查找图书或读者的信息、添加或删除图书、列出图书、在图书馆中添加和删除图书，以及借出、预约和归还图书：

```
    void addBook(void);
    void deleteBook(void);
    void listBooks(void);
    void addCustomer(void);
    void deleteCustomer(void);
    void listCustomers(void);
    void borrowBook(void);
    void reserveBook(void);
    void returnBook(void);
```

load 和 save 方法分别在执行的开始和结束时被调用：

```
    void load();
    void save();
```

有两个映射分别保存图书馆的图书和读者。如前所述，映射是这样一种结构：每个值都由一个唯一的键来标识，以便提供快速访问。图书和读者的唯一编号就是键：

```
  public:
    static map<int,Book> s_bookMap;
    static map<int,Customer> s_customerMap;
};
```

Library.cpp

```
#include <Set>
#include <Map>
#include <List>
#include <String>
#include <FStream>
#include <IOStream>
#include <Algorithm>
using namespace std;

#include "Book.h"
#include "Customer.h"
#include "Library.h"
```

```
map<int,Book> Library::s_bookMap;
map<int,Customer> Library::s_customerMap;
```

两次执行之间，图书馆的信息存放在硬盘上的 Library.bin 文件中。注意，字符串中我们用两个反斜线表示一个反斜线：第一个反斜线表明后面的字符是特殊字符，第二个反斜线则表明它就是一个反斜线：

```
string Library::s_binaryPath("Library.bin");
```

构造函数加载图书馆的信息，显示一个菜单，然后不断循环，直到用户退出。在执行结束之前，图书馆的信息会被保存：

```
Library::Library(void) {
```

在显示菜单之前，先从文件加载图书馆的信息（图书、读者、借阅和预约）：

```
load();
```

只要 quit 不为 `true`，while 语句就会一直循环下去。直到用户从菜单中选择 Quit 选项，它才会变成 `true`：

```
bool quit = false;
while (!quit) {
  cout << "1. Add Book" << endl
       << "2. Delete Book" << endl
       << "3. List Books" << endl
       << "4. Add Customer" << endl
       << "5. Delete Customer" << endl
       << "6. List Customers" << endl
       << "7. Borrow Book" << endl
       << "8. Reserve Book" << endl
       << "9. Return Book" << endl
       << "0. Quit" << endl
       << ": ";
```

用户从控制台输入流（`cin`）输入一个整数值，它存放在 `choice` 中：

```
int choice;
cin >> choice;
```

我们用 switch 语句来执行所请求的任务：

```
  switch (choice) {
    case 1:
      addBook();
      break;

    case 2:
```

```
            deleteBook();
            break;

        case 3:
            listBooks();
            break;

        case 4:
            addCustomer();
            break;

        case 5:
            deleteCustomer();
            break;

        case 6:
            listCustomers();
            break;

        case 7:
            borrowBook();
            break;

        case 8:
            reserveBook();
            break;

        case 9:
            returnBook();
            break;

        case 0:
            quit = true;
            break;
    }

    cout << endl;
}
```

在程序结束之前，图书馆的信息会被保存：

```
    save();
}
```

查找图书与读者

lookupBook 方法遍历图书映射。如果存在指定作者和书名的图书，它返回 true。如果该书存在，只要指针不为空，就把它的信息复制到 bookPtr 参数所指的对象中并返回 true。如果该书不存在，就返回 false，并且不向对象复制任何信息：

```
bool Library::lookupBook(const string& author,
        const string& title, Book* bookPtr /* = nullptr*/) {
  for (const pair<int,Book>& entry : s_bookMap) {
    const Book& book = entry.second;
```

注意，bookPtr 可能是 nullptr。此时只返回 true，不向 bookPtr 所指的对象写入任何信息：

```
      if ((book.author() == author) && (book.title() == title)) {
        if (bookPtr != nullptr) {
          *bookPtr = book;
        }

        return true;
      }
    }

    return false;
}
```

同样，lookupCustomer 遍历读者映射；如果存在指定名字的读者，它返回 true，并把读者信息复制到一个 Customer 对象中：

```
bool Library::lookupCustomer(const string& name,
       const string& address, Customer* customerPtr /*=nullptr*/){
  for (const pair<int,Customer>& entry : s_customerMap) {
    const Customer& customer = entry.second;
```

同样，在这种情况下，customerPtr 也可能是 nullptr。此时只返回 true。添加新读者时，我们想知道是否已经存在同名同地址的读者：

```
      if ((customer.name() == name) &&
          (customer.address() == address)) {
        if (customerPtr != nullptr) {
          *customerPtr = customer;
        }

        return true;
      }
    }

    return false;
```

```
}
```

添加图书

addBook 方法提示用户输入新书的作者名和书名：

```
void Library::addBook(void) {
  string author;
  cout << "Author: ";
  cin >> author;

  string title;
  cout << "Title: ";
  cin >> title;
```

如果指定作者和书名的图书已经存在，就显示一条错误消息：

```
if (lookupBook(author, title)) {
  cout << endl << "The book \"" << title << "\" by "
       << author << " already exists." << endl;
  return;
}
```

如果该书尚不存在，我们就创建一个新的 Book 对象，并把它加入图书映射：

```
  Book book(author, title);
  s_bookMap[book.bookId()] = book;
  cout << endl << "Added: " << book << endl;
}
```

删除图书

deleteBook 方法提示用户输入图书的作者名和书名，如果该书存在就删除它：

```
void Library::deleteBook() {
  string author;
  cout << "Author: ";
  cin >> author;

  string title;
  cout << "Title: ";
  cin >> title;
```

如果该书不存在，就显示一条错误消息：

```
    Book book;
    if (!lookupBook(author, title, &book)) {
      cout << endl << "There is no book \"" << title << "\" by "
           << "author " << author << "." << endl;
      return;
    }
```

删除一本书时，我们遍历所有读者，并对每位读者归还并取消预约这本书。我们对每一本书都这样做，以防该书已被读者借出或预约。在下一章中，我们将使用指针，让我们能够以更高效的方式归还和取消预约图书。

注意，当我们遍历映射并获取每个 `Customer` 对象后，需要在修改其字段值之后把它放回映射中：

```
    for (pair<int,Customer> entry : s_customerMap) {
      Customer& customer = entry.second;
      customer.returnBook(book.bookId());
      customer.unreserveBook(book.bookId());
      s_customerMap[customer.id()] = customer;
    }
```

最后，在确认图书存在、并且已经归还和取消预约之后，我们把它从图书映射中移除：

```
      s_bookMap.erase(book.bookId());
      cout << endl << "Deleted." << endl;
    }
```

列出图书

`listBooks` 方法相当简单。首先，我们检查图书映射是否为空。如果为空，就写出「No books.」；如果不为空，就遍历该映射，对每一本书都把它的信息写到控制台输出流（`cout`）：

```
    void Library::listBooks(void) {
      if (s_bookMap.empty()) {
        cout << "No books." << endl;
        return;
      }

      for (const pair<int,Book>& entry : s_bookMap) {
        const Book& book = entry.second;
        cout << book << endl;
      }
    }
```

添加读者

addCustomer 方法提示用户输入新读者的名字和地址：

```
void Library::addCustomer(void) {
  string name;
  cout << "Name: ";
  cin >> name;

  string address;
  cout << "Address: ";
  cin >> address;
```

如果同名同地址的读者已经存在，就显示一条错误消息：

```
if (lookupCustomer(name, address)) {
  cout << endl << "A customer with name " << name
       << " and address " << address << " already exists."
       << endl;
  return;
}
```

最后，我们创建一个新的 Customer 对象，并把它加入读者映射：

```
  Customer customer(name, address);
  s_customerMap[customer.id()] = customer;
  cout << endl << "Added." << endl;
}
```

删除读者

deleteCustomer 方法删除指定的读者（如果存在）：

```
void Library::deleteCustomer(void) {
  string name;
  cout << "Name: ";
  cin >> name;

  string address;
  cout << "Address: ";
  cin >> address;

  Customer customer;
  if (!lookupCustomer(name, address, &customer)) {
    cout << endl << "There is no customer with name " << name
         << " and address " << address << "." << endl;
```

```
    return;
  }
```

如果该读者至少借有一本书，必须先归还，才能删除该读者：

```
  if (customer.hasBorrowed()) {
    cout << "Customer " << name << " has borrowed at least "
         << "one book and cannot be deleted." << endl;
    return;
  }
```

不过，如果该读者预约了图书，我们在删除读者之前只需取消这些预约：

```
    for (pair<int,Book> entry : s_bookMap) {
      Book& book = entry.second;
      book.unreserveBookation(customer.id());
      s_bookMap[book.bookId()] = book;
    }

    cout << endl << "Deleted." << endl;
    s_customerMap.erase(customer.id());
  }
```

列出读者

`listCustomers` 方法的工作方式与 `listBooks` 类似。如果没有读者，就写出「No customers.」；如果有读者，就把他们写到控制台输出流（`cout`）：

```
  void Library::listCustomers(void) {
    if (s_customerMap.empty()) {
      cout << "No customers." << endl;
      return;
    }

    for (const pair<int,Customer>& entry : s_customerMap) {
      const Customer& customer = entry.second;
      cout << customer << endl;
    }
  }
```

借出图书

`borrowBook` 方法提示用户输入图书的作者名和书名：

```
void Library::borrowBook(void) {
  string author;
  cout << "Author: ";
  cin >> author;

  string title;
  cout << "Title: ";
  cin >> title;
```

如果不存在指定作者和书名的图书，就显示一条错误消息：

```
Book book;
if (!lookupBook(author, title, &book)) {
  cout << endl << "There is no book \"" << title << "\" by "
       << "author " << author << "." << endl;
  return;
}
```

此外，如果该书已经被借出，也显示一条错误消息：

```
if (book.borrowed()) {
  cout << endl << "The book \"" << title << "\" by " << author
       << " has already been borrowed." << endl;
  return;
}
```

然后我们提示用户输入读者的名字和地址：

```
string name;
cout << "Customer name: ";
cin >> name;

string address;
cout << "Adddress: ";
cin >> address;
```

如果不存在指定名字和地址的读者，就显示一条错误消息：

```
Customer customer;
if (!lookupCustomer(name, address, &customer)) {
  cout << endl << "There is no customer with name " << name
       << " and address " << address << "." << endl;
  return;
}
```

不过，如果图书存在且尚未被借出，而读者也存在，我们就把这本书加入该读者的借阅集合，并把该书标记为由该读者借出：

```
    book.borrowBook(customer.id());
    customer.borrowBook(book.bookId());
```

注意，修改 Book 和 Customer 对象之后，我们必须把它们放回各自的映射中。在下一章中，我们将使用一种更直接的指针处理方式：

```
      s_bookMap[book.bookId()] = book;
      s_customerMap[customer.id()] = customer;
      cout << endl << "Borrowed." << endl;
    }
```

预约图书

reserveBook 方法的工作方式与 borrowBook 相同。它提示用户输入图书的作者名和书名：

```
    void Library::reserveBook(void) {
      string author;
      cout << "Author: ";
      cin >> author;

      string title;
      cout << "Title: ";
      cin >> title;
```

与 borrowBook 的情况类似，我们检查指定作者和书名的图书是否存在：

```
    Book book;
    if (!lookupBook(author, title, &book)) {
      cout << endl << "There is no book \"" << title << "\" by "
           << "author " << author << "." << endl;
      return;
    }
```

不过，与 borrowBook 相比有一个不同之处：图书必须已经被借出，才能被预约。如果尚未借出，预约它毫无意义。在这种情况下，用户应该直接借阅这本书：

```
    if (!book.borrowed()) {
      cout << endl << "The book with author " << author
           << " and title \"" << title << "\" has not been "
           << "borrowed. Please borrow the book instead." << endl;
      return;
    }
```

如果图书存在且已被借出，我们就提示用户输入读者的名字和地址：

```
string name;
cout << "Customer name: ";
cin >> name;

string address;
cout << "Address: ";
cin >> address;
```

如果该读者不存在，就显示一条错误消息：

```
Customer customer;
if (!lookupCustomer(name, address, &customer)) {
  cout << endl << "No customer with name " << name
       << " and address " << address << " exists." << endl;
  return;
}
```

此外，如果这本书已经由该读者借出，我们就显示一条错误消息：

```
if (book.customerId() == customer.id()) {
  cout << endl << "The book has already been borrowed by "
       << name << "." << endl;
  return;
}
```

如果图书存在且已被借出、但借出者不是该读者，我们就把该读者加入这本书的预约列表，并把这本书加入该读者的预约集合：

```
customer.reserveBook(book.bookId());
int position = book.reserveBook(customer.id());
```

同样，在这种情况下，我们也必须把 Book 和 Customer 对象放回各自的映射中：

```
s_bookMap[book.bookId()] = book;
s_customerMap[customer.id()] = customer;
```

最后，我们写出该读者在预约列表中的位置：

```
  cout << endl << position << "nd reserve." << endl;
}
```

归还图书

returnBook 方法提示用户输入图书的作者名和书名：

```
void Library::returnBook(void) {
  string author;
```

```
  cout << "Author: ";
  cin >> author;

  string title;
  cout << "Title: ";
  cin >> title;
```

如果该书不存在，就显示一条错误消息：

```
Book book;
if (!lookupBook(author, title, &book)) {
  cout << endl << "No book \"" << title
       << "\" by " << author << " exists." << endl;
  return;
}
```

如果该书尚未被借出，就显示一条错误消息：

```
if (!book.borrowed()) {
  cout << endl << "The book \"" << title
       << "\" by " << author
       << "\" has not been borrowed." << endl;
  return;
}
```

与前面描述的方法不同，这种情况我们不需要询问读者。我们直接归还这本书，然后在预约列表中查找下一位读者：

```
book.returnBook();
cout << endl << "Returned." << endl;

Customer customer = s_customerMap[book.customerId()];
customer.returnBook(book.bookId());
s_customerMap[customer.id()] = customer;
```

如果这本书已被预约，我们就找出预约列表中的第一位读者，把他们从预约列表中移除，并让他们借阅这本书：

```
  list<int>& reservationList = book.reservationList();

  if (!reservationList.empty()) {
    int newCustomerId = reservationList.front();
    reservationList.erase(reservationList.begin());
    book.borrowBook(newCustomerId);

    Customer newCustomer = s_customerMap[newCustomerId];
    newCustomer.borrowBook(book.bookId());
```

```
      s_customerMap[newCustomerId] = newCustomer;
      cout << endl << "Borrowed by " << newCustomer.name() << endl;
    }

    s_bookMap[book.bookId()] = book;
  }
```

将图书馆信息保存到文件

保存图书馆的信息时，我们首先打开文件：

```
void Library::save() {
  ofstream outStream(s_binaryPath);
```

如果文件被正确打开，我们先写出图书的数目，然后通过对 Book 对象调用 write 来写出每本书的信息：

```
  if (outStream) {
    int numberOfBooks = s_bookMap.size();
    outStream.write((char*) &numberOfBooks, sizeof numberOfBooks);

    for (const pair<int,Book>& entry : s_bookMap) {
      const Book& book = entry.second;
      book.write(outStream);
    }
```

同样，我们写出读者的数目，然后通过调用 write 写出每位读者的信息：

```
      int numberOfCustomers = s_customerMap.size();
      outStream.write((char*) &numberOfCustomers,
                      sizeof numberOfCustomers);

      for (const pair<int,Customer>& entry : s_customerMap) {
        const Customer& customer = entry.second;
        customer.write(outStream);
      }
    }
  }
```

从文件加载图书馆信息

从文件加载图书馆信息时，我们对 read 的使用方式也与前面相同。我们首先打开文件：

```
void Library::load() {
  ifstream inStream(s_binaryPath);
```

我们读出图书的数目，然后读出每本书的信息：

```
if (inStream) {
  int numberOfBooks;
  inStream.read((char*) &numberOfBooks, sizeof numberOfBooks);
```

对每本书，我们创建一个新的 Book 对象，通过调用 read 读出它的信息，并把它加入图书映射。我们还计算静态字段 MaxBookId 的新值，即把它设置为它自身与该书编号两者的最大值：

```
for (int count = 0; count < numberOfBooks; ++count) {
  Book book;
  book.read(inStream);
  s_bookMap[book.bookId()] = book;
  Book::MaxBookId = max(Book::MaxBookId, book.bookId());
}
```

同样，我们读出读者的数目，然后通过调用 read 读出每位读者的信息：

```
int numberOfCustomers;
inStream.read((char*) &numberOfCustomers,
            sizeof numberOfCustomers);
```

对每位读者，我们创建一个 Customer 对象，从文件中读出它的信息，把它加入读者映射，并计算静态字段 MaxCustomerId 的新值：

```
    for (int count = 0; count < numberOfCustomers; ++count) {
      Customer customer;
      customer.read(inStream);
      s_customerMap[customer.id()] = customer;
      Customer::MaxCustomerId =
        max(Customer::MaxCustomerId, customer.id());
    }
  }
}
```

main 函数

最后，我们来编写执行这个图书馆的 main 函数。它非常简单；唯一要做的事情就是实例化一个 Library 类的对象。然后构造函数会显示主菜单：

Main.cpp

```
#include <Set>
#include <Map>
#include <List>
#include <String>
#include <FStream>
#include <IOStream>
using namespace std;

#include "Book.h"
#include "Customer.h"
#include "Library.h"

void main(void) {
  Library();
}
```

小结

本章我们构建了一个由 Book、Customer 和 Library 类组成的图书馆管理系统。

Book 类保存一本书的信息。每个 Book 对象都持有一个唯一的编号。它还记录着借阅者（如果该书已被借出）以及一份预约列表。同样，Customer 类保存一位读者的信息。与图书一样，每位读者持有一个唯一的编号。每个 Customer 对象还保存着已借阅和已预约图书的集合。最后，Library 类提供一系列服务，比如添加和删除图书与读者、借出、归还和预约图书，以及列出图书和读者。

本章中，每本书和每位读者都有一个唯一的编号。下一章我们将再次研究这个图书馆系统，不过我们会省去编号，改用指针来处理。

4
基于指针的图书馆管理系统

本章我们将继续研究图书馆管理系统。与第 3 章「构建图书馆管理系统」一样，我们同样有三个类：Book、Customer 和 Library。不过，这一次有一个重大区别：我们不再使用编号，而是使用指针。每个 Book 对象持有一个指针，指向借走这本书的顾客（Customer 类的对象），同时还持有一份指针列表，指向预约这本书的各位顾客。同样，每位顾客也持有多组指针，指向自己借阅和预约的图书（Book 类的对象）。

不过，这种做法也带来了一个问题：我们无法把指针的值直接存入文件。因此，保存文件时需要把指针转换成图书列表和顾客列表中的索引，而加载文件时则需要把索引再转换回指针。这一过程称为 marshmallowing（直译为「棉花糖化」）。

本章将深入讨论以下主题：

- 与第 3 章「构建图书馆管理系统」一样，我们仍将使用构成一个小型数据库的图书类和顾客类。不过本章将直接使用指针，而不再使用整数编号。
- 由于使用指针而不是整数编号，文件处理变得更加复杂。我们需要执行一个称为 marshmallowing 的过程。
- 最后，我们将使用 C++ 标准库中的泛型类 set 和 list。不过本章中，它们存放的是指向图书对象和顾客对象的指针，而不是对象本身。

Book 类

与上一章的系统类似，我们同样有三个类：Book、Customer 和 Library。Book 类负责记录一本书的信息，每本书都有作者和书名。Customer 类负责记录一位顾客的信息，每位顾客都有姓名和地址。Library 类负责图书馆的各项操作，如借阅、归还和预约。最后，main 函数只是创建一个 Library 类的对象。

Book 类与第 3 章「构建图书馆管理系统」中的 Book 类相似。唯一实质的区别是：这里没有编

号，只有指针。

Book.h

```
class Customer;

class Book {
  public:
    Book();
    Book(const string& author, const string& title);
    const string& author() const { return m_author; }
    const string& title() const { return m_title; }
    void read(ifstream& inStream);
    void write(ofstream& outStream) const;
    int reserveBook(Customer* customerPtr);
    void removeReservation(Customer* customerPtr);
    void returnBook();
```

我们没有返回图书编号的方法，因为本章的图书不使用编号。

borrowerPtr 方法返回借走这本书的顾客的地址；如果这本书当前无人借阅，则返回 nullptr。它有两个版本：第一个版本返回对顾客对象指针的引用，这样我们就可以为该顾客指针赋予新值；第二个版本是常量版本，意味着它可以在常量对象上调用：

```
Customer*& borrowerPtr() { return m_borrowerPtr; }
const Customer* borrowerPtr() const { return m_borrowerPtr; }
```

注意，本章没有 borrowed 方法。我们不需要它，因为如果这本书当前无人借阅，borrowerPtr 就会返回 nullptr。

本章中，reservationPtrList 返回的是顾客指针列表，而不是整数值。它同样有两个版本：第一个版本返回对列表的引用，这样我们就可以在列表中添加和移除指针；第二个版本是常量版本，返回常量列表，这意味着它可以在常量 Book 对象上调用，并且返回的列表不可修改：

```
list<Customer*>& reservationPtrList()
    { return m_reservationPtrList; }
const list<Customer*> reservationPtrList() const
    { return m_reservationPtrList; }
```

输出流运算符的用法与第 3 章「构建图书馆管理系统」相同：

```
friend ostream& operator<<(ostream& outStream,
    const Book& book);
```

m_author 和 m_title 字段是字符串，与第 3 章「构建图书馆管理系统」类似：

```
private:
  string m_author, m_title;
```

不过，我们省去了 m_bookId 字段，因为本章不使用编号。我们还将 m_borrowedId 和 m_customerId 字段替换为 m_borrowerPtr，它被初始化为 nullptr，因为这本书从一开始并未被借阅：

```
    Customer* m_borrowerPtr = nullptr;
```

m_reservationPtrList 字段存放的是指向预约这本书的各位顾客的指针列表，而不是第 3 章「构建图书馆管理系统」中的整数编号列表：

```
        list<Customer*> m_reservationPtrList;
    };
```

Book.cpp

```
    #include <Set>
    #include <Map>
    #include <String>
    #include <FStream>
    #include <Algorithm>
    using namespace std;

    #include "Book.h"
    #include "Customer.h"
    #include "Library.h"
```

默认构造函数与第 3 章「构建图书馆管理系统」中的构造函数类似：

```
    Book::Book() {
      // Empty.
    }
```

第二个构造函数也与第 3 章「构建图书馆管理系统」中的构造函数类似，不过这里没有需要初始化的 m_bookId 字段：

```
    Book::Book(const string& author, const string& title)
      :m_author(author),
       m_title(title) {
      // Empty.
    }
```

读取与写入图书

read 和 write 方法在本章做了简化：它们只读取和写入书的作者与书名。可能存在的借阅者与预约列表，则由 Library 类的 save 和 load 方法来读取和写入：

```
void Book::read(ifstream& inStream) {
  getline(inStream, m_author);
  getline(inStream, m_title);
}

void Book::write(ofstream& outStream) const {
  outStream << m_author << endl;
  outStream << m_title << endl;
}
```

借阅与预约图书

当顾客预约一本书时，指向该 Customer 对象的指针会被加入这本书的预约指针列表，并返回列表的大小，以便通知顾客其在预约列表中的位置：

```
int Book::reserveBook(Customer* borrowerPtr) {
  m_reservationPtrList.push_back(borrowerPtr);
  return m_reservationPtrList.size();
}
```

当顾客归还一本书时，我们只需把 m_borrowerPtr 设为 nullptr，表示这本书已不再被借阅：

```
void Book::returnBook() {
  m_borrowerPtr = nullptr;
}
```

removeReservation 方法只是把该顾客指针从预约列表中移除：

```
void Book::removeReservation(Customer* customerPtr) {
  m_reservationPtrList.remove(customerPtr);
}
```

显示图书

输出流运算符写出书名和作者、借走这本书的顾客（如果有），以及预约这本书的各位顾客（如果有）：

```
ostream& operator<<(ostream& outStream, const Book& book) {
  outStream << "\"" << book.m_title << "\" by "
            << book.m_author;
```

如果这本书已被借阅，我们就把借阅者写入流：

```
if (book.m_borrowerPtr != nullptr) {
  outStream << endl << "  Borrowed by: "
```

```
            << book.m_borrowerPtr->name() << ".";
  }
```

如果这本书的预约列表非空，我们就遍历该列表，并为每一项预约写出对应的顾客：

```
  if (!book.m_reservationPtrList.empty()) {
    outStream << endl << "  Reserved by: ";
    bool first = true;
    for (Customer* customerPtr : book.m_reservationPtrList) {
      outStream << (first ? "" : ",") << customerPtr->name();
      first = false;
    }
    outStream << ".";
  }
  return outStream;
```

Customer 类

本章的 Customer 类与第 3 章「构建图书馆管理系统」中的 Customer 类相似。同样，这里的区别在于我们使用的是指针，而不是整数编号。

Customer.h

```
  class Customer {
    public:
      Customer();
      Customer(const string& name, const string& address);
      const string& name() const { return m_name; }
      const string& address() const { return m_address; }
      void read(ifstream& inStream);
      void write(ofstream& outStream) const;
```

borrowBook、returnBook、reserveBook 和 unreserveBook 方法都以指向 Book 对象的指针作为参数：

```
  void borrowBook(Book* bookPtr);
  void returnBook(Book* bookPtr);
  void reserveBook(Book* bookPtr);
  void unreserveBook(Book* bookPtr);
```

loanPtrSet 和 reservationPtrSet 方法返回的是 Book 指针的集合，而不是整数编号的集合：

```
  set<Book*>& loanPtrSet() { return m_loanPtrSet; }
  const set<Book*> loanPtrSet() const { return m_loanPtrSet; }
  set<Book*>& reservationPtrSet()
```

```
        { return m_reservationPtrSet; }
    const set<Book*> reservationPtrSet() const
        { return m_reservationPtrSet; }
```

与第 3 章「构建图书馆管理系统」相比，输出流运算符没有变化：

```
    friend ostream& operator<<(ostream& outStream,
        const Customer& customer);
```

m_name 和 m_address 字段存放顾客的姓名和地址，与第 3 章「构建图书馆管理系统」一样：

```
    private:
      string m_name, m_address;
```

m_loanPtrSet 和 m_reservationPtrSet 字段存放的是指向 Book 对象的指针，而不是整数编号：

```
        set<Book*> m_loanPtrSet, m_reservationPtrSet;
    };
```

Customer.cpp

```
    #include <Set>
    #include <Map>
    #include <String>
    #include <FStream>
    using namespace std;

    #include "Book.h"
    #include "Customer.h"
    #include "Library.h"
```

构造函数与第 3 章「构建图书馆管理系统」中的构造函数类似。第一个构造函数什么也不做，在从文件加载顾客列表时被调用：

```
    Customer::Customer() {
      // Empty.
    }
```

第二个构造函数初始化顾客的姓名和地址。不过，与第 3 章「构建图书馆管理系统」的构造函数相比，这里没有需要初始化的 m_customerId 字段：

```
    Customer::Customer(const string& name, const string& address)
      :m_name(name),
       m_address(address) {
      // Empty.
    }
```

读取与写入顾客

与前面 Book 的情况类似，read 和 write 方法也做了简化：它们只读取和写入姓名与地址。借阅集合与预约集合的读取和写入则放在 Library 类中完成，如下所示：

```
void Customer::read(ifstream& inStream) {
  getline(inStream, m_name);
  getline(inStream, m_address);
}

void Customer::write(ofstream& outStream) const {
  outStream << m_name << endl;
  outStream << m_address << endl;
}
```

借阅与预约图书

borrowBook 方法把图书指针加入借阅集合，如果这本书此前被预约过，则同时把它从预约集合中移除：

```
void Customer::borrowBook(Book* bookPtr) {
  m_loanPtrSet.insert(bookPtr);
  m_reservationPtrSet.erase(bookPtr);
}
```

reserveBook 方法只是把图书指针加入预约集合；而 returnBook 和 unreserveBook 则分别把图书指针从借阅集合和预约集合中移除：

```
void Customer::reserveBook(Book* bookPtr) {
  m_reservationPtrSet.insert(bookPtr);
}

void Customer::returnBook(Book* bookPtr) {
  m_loanPtrSet.erase(bookPtr);
}

void Customer::unreserveBook(Book* bookPtr) {
  m_reservationPtrSet.erase(bookPtr);
}
```

显示顾客

输出流运算符的用法与第 3 章「构建图书馆管理系统」相同：它写出顾客的姓名和地址，以及借阅和预约的图书集合（如果有）：

```
ostream& operator<<(ostream& outStream,
                    const Customer& customer) {
  outStream << customer.m_name << ", "
            << customer.m_address << ".";
```

如果该顾客的借阅集合非空，我们就遍历它，并为每一次借阅写出对应的图书：

```
if (!customer.m_loanPtrSet.empty()) {
  outStream << endl << "  Borrowed books: ";
  bool first = true;
  for (const Book* bookPtr : customer.m_loanPtrSet) {
    outStream << (first ? "" : ", ") << bookPtr->author();
    first = false;
  }
}
```

同样，如果该顾客的预约集合非空，我们就遍历它，并为每一项预约写出对应的图书：

```
if (!customer.m_reservationPtrSet.empty()) {
  outStream << endl << "  Reserved books: ";
  bool first = true;
  for (Book* bookPtr : customer.m_reservationPtrSet) {
    outStream << (first ? "" : ", ") << bookPtr->author();
    first = false;
  }
}
return outStream;
```

Library 类

Library 类与第 3 章「构建图书馆管理系统」中的 Library 类十分相似。不过，我们新增了一些查找方法，用于在把图书馆信息保存到文件以及从文件加载时，在指针与列表索引之间进行转换：

Library.h

```
class Library {
  public:
    Library();
```

析构函数释放本应用程序中所有动态分配的内存：

```
    ~Library();
private:
  static string s_binaryPath;
```

lookupBook 和 lookupCustomer 方法返回指向 Book 和 Customer 对象的指针。如果图书或顾客不存在，则返回 nullptr：

```
    Book* lookupBook(const string& author,
                     const string& title);
    Customer* lookupCustomer(const string& name,
                             const string& address);
    void addBook();
    void deleteBook();
    void listBooks();
    void addCustomer();
    void deleteCustomer();
    void listCustomers();
    void borrowBook();
    void reserveBook();
    void returnBook();
```

lookupBookIndex 和 lookupCustomerIndex 方法接受一个指针，在图书列表和顾客列表中查找该指针所指的对象，并返回它在列表中的索引：

```
    int lookupBookIndex(const Book* bookPtr);
    int lookupCustomerIndex(const Customer* customerPtr);
```

lookupBookPtr 和 lookupCustomerPtr 方法接受一个索引，返回图书列表和顾客列表中相应位置上的对象的指针：

```
    Book* lookupBookPtr(int bookIndex);
    Customer* lookupCustomerPtr(int customerIndex);
```

save 和 load 方法负责把图书馆信息保存到文件以及从文件加载。不过，它们比第 3 章「构建图书馆管理系统」中的对应方法要复杂得多：

```
    void save();
    void load();
```

m_bookPtrList 和 m_customerPtrList 字段存放的是指向 Book 和 Customer 对象的指针，而不是像第 3 章「构建图书馆管理系统」那样直接存放对象本身：

```
        list<Book*> m_bookPtrList;
        list<Customer*> m_customerPtrList;
    };
```

Library.cpp

```
    #include <Set>
    #include <Map>
    #include <List>
```

```
#include <String>
#include <FStream>
#include <IOStream>
#include <CAssert>
using namespace std;

#include "Book.h"
#include "Customer.h"
#include "Library.h"

string Library::s_binaryPath("C:\\Users\\Stefan\\Library.binary");
```

构造函数与第 3 章「构建图书馆管理系统」中的构造函数完全相同：

```
Library::Library() {
  load();
  bool quit = false;
  while (!quit) {
    cout << "1. Add Book" << endl
         << "2. Delete Book" << endl
         << "3. List Books" << endl
         << "4. Add Customer" << endl
         << "5. Delete Customer" << endl
         << "6. List Customers" << endl
         << "7. Borrow Book" << endl
         << "8. Reserve Book" << endl
         << "9. Return Book" << endl
         << "0. Quit" << endl
         << ": ";
    int choice;
    cin >> choice;
    cout << endl;
    switch (choice) {
      case 1:
        addBook();
        break;
      case 2:
        deleteBook();
        break;
      case 3:
        listBooks();
        break;
      case 4:
        addCustomer();
        break;
      case 5:
        deleteCustomer();
```

```
        break;
      case 6:
        listCustomers();
        break;
      case 7:
        borrowBook();
        break;
      case 8:
        reserveBook();
        break;
      case 9:
        returnBook();
        break;
      case 0:
        quit = true;
        break;
    }
    cout << endl;
  }
  save();
}
```

查找图书与顾客

本章的 lookupBook 方法按作者和书名查找 Book 对象，方式与第 3 章「构建图书馆管理系统」类似。不过，如果找到作者与书名都匹配的 Book 对象，它不会把信息复制到给定对象中，而是直接返回指向该对象的指针。如果没有找到 Book 对象，则返回 nullptr：

```
Book* Library::lookupBook(const string& author,
                          const string& title) {
  for (Book* bookPtr : m_bookPtrList) {
    if ((bookPtr->author() == author) &&
        (bookPtr->title() == title)) {
      return bookPtr;
    }
  }
  return nullptr;
}
```

同样，lookupCustomer 尝试查找姓名和地址都匹配的 Customer 对象。如果找到该对象，就返回它的指针；如果没有找到，则返回 nullptr：

```
Customer* Library::lookupCustomer(const string& name,
                                  const string& address) {
  for (Customer* customerPtr : m_customerPtrList) {
```

```
      if ((customerPtr->name() == name) &&
          (customerPtr->address() == address)) {
        return customerPtr;
      }
    }
    return nullptr;
  }
```

新增图书

addBook 方法提示用户输入作者和书名：

```
  void Library::addBook() {
    string author;
    cout << "Author: ";
    cin >> author;
    string title;
    cout << "Title: ";
    cin >> title;
```

在检查这本书是否已经存在时，我们调用 lookupBook。如果书已存在，就返回指向该 Book 对象的指针；如果不存在，则返回 nullptr。因此，我们测试返回值是否不等于 nullptr。如果不等于 nullptr，说明这本书已经存在，于是显示一条错误消息：

```
  if (lookupBook(author, title) != nullptr) {
    cout << endl << "The book \"" << title << "\" by "
         << author << " already exists." << endl;
    return;
  }
```

在添加这本书时，我们用 new 运算符动态创建一个新的 Book 对象。我们用 C++ 标准的 assert 宏检查图书指针是否为空。如果指针为空，程序将带着错误信息中止执行：

```
    Book* bookPtr = new Book(author, title);
    assert(bookPtr != nullptr);
    m_bookPtrList.push_back(bookPtr);
    cout << endl << "Added." << endl;
  }
```

删除图书

deleteBook 方法通过提示用户输入书的作者和书名来从图书馆中删除一本书。如果这本书存在，我们就先办理归还、再取消预约，最后删除它：

```
void Library::deleteBook() {
  string author;
  cout << "Author: ";
  cin >> author;
  string title;
  cout << "Title: ";
  cin >> title;
```

我们通过调用 lookupBook 获得指向该 Book 对象的指针：

```
Book* bookPtr = lookupBook(author, title);
```

如果指针为 nullptr，说明这本书不存在，于是显示一条错误消息：

```
if (bookPtr == nullptr) {
  cout << endl << "The book \"" << title << "\" by "
       << author << " does not exist." << endl;
  return;
}
```

我们通过查找借阅者来检查这本书是否已被借出：

```
Customer* borrowerPtr = bookPtr->borrowerPtr();
```

如果 borrowerPtr 返回的指针不是 nullptr，我们就调用借阅者的 returnBook 方法归还这本书。这样，这本书就不再登记为该顾客借阅的了：

```
if (borrowerPtr != nullptr) {
  borrowerPtr->returnBook(bookPtr);
}
```

此外，我们还需要检查这本书是否被其他顾客预约过。为此，我们取得这本书的预约列表，并为列表中的每位顾客取消对这本书的预约：

```
list<Customer*> reservationPtrList =
    bookPtr->reservationPtrList();
```

注意，我们并不检查该顾客是否真的预约了这本书，而只是简单地取消预约。还要注意，我们不需要把任何对象放回列表，因为我们操作的是指向对象的指针，并不复制对象：

```
for (Customer* reserverPtr : reservationPtrList) {
  reserverPtr->unreserveBook(bookPtr);
}
```

在删除这本书时，我们先从图书指针列表中移除该图书指针，然后释放 Book 对象。先显示消息、再删除图书指针，看起来也许有些奇怪，但顺序必须如此：对象一旦被删除，我们就再也不能对它做任何事情了。如果先删除对象再输出它，会导致内存错误：

```
    m_bookPtrList.remove(bookPtr);
    cout << endl << "Deleted: " << bookPtr << endl;
    delete bookPtr;
  }
```

列出图书

在列出图书时，我们首先检查列表是否为空。如果为空，就只输出 `No books.`：

```
  void Library::listBooks() {
    if (m_bookPtrList.empty()) {
      cout << "No books." << endl;
      return;
    }
```

如果列表不为空，我们就遍历图书指针列表，对每个图书指针解引用并输出相应的信息：

```
    for (const Book* bookPtr : m_bookPtrList) {
      cout << (*bookPtr) << endl;
    }
  }
```

新增顾客

`addCustomer` 方法提示用户输入顾客的姓名和地址：

```
  void Library::addCustomer() {
    string name;
    cout << "Name: ";
    cin >> name;
    string address;
    cout << "Address: ";
    cin >> address;
```

如果已有同名同地址的顾客，则显示一条错误消息：

```
  if (lookupCustomer(name, address) != nullptr) {
    cout << endl << "A customer with name " << name
         << " and address " << address << " already exists."
         << endl;
    return;
  }
```

在添加顾客时，我们动态创建一个新的 `Customer` 对象，并把它加入顾客对象指针列表：

```
  Customer* customerPtr = new Customer(name, address);
  assert(customerPtr != nullptr);
  m_customerPtrList.push_back(customerPtr);
  cout << endl << "Added." << endl;
}
```

删除顾客

在删除顾客时，我们先查找该顾客，如果不存在则显示一条错误消息：

```
void Library::deleteCustomer() {
  string name;
  cout << "Customer name: ";
  cin >> name;
  string address;
  cout << "Address: ";
  cin >> address;
  Customer* customerPtr = lookupCustomer(name, address);
```

如果给定姓名和地址的顾客不存在，则显示一条错误消息。请看下面的代码：

```
if (customerPtr == nullptr) {
  cout << endl << "Customer " << name
       << " does not exists." << endl;
  return;
}
```

如果该顾客至少借阅了一本书，就不能删除该顾客，并显示如下所示的错误消息：

```
if (!customerPtr->loanPtrSet().empty()) {
  cout << "The customer " << customerPtr->name()
       << " has borrowed books and cannot be deleted."
       << endl;
  return;
}
```

不过，如果该顾客没有借阅任何图书，就先把他从图书馆中每本书的预约列表中移除，如下面的代码所示：

```
for (Book* bookPtr : m_bookPtrList) {
  bookPtr->removeReservation(customerPtr);
}
```

然后把该顾客从顾客列表中移除，并用 `delete` 运算符释放 `Customer` 对象。再次提醒注意：必须先输出顾客信息，再删除其对象；反过来是行不通的，因为我们无法查看一个已被删除的对象，那样会导致内存错误：

```
  m_customerPtrList.remove(customerPtr);
  cout << endl << "Deleted." << (*customerPtr) << endl;
  delete customerPtr;
}
```

列出顾客

在列出顾客时，我们遍历顾客列表，并对每个 Customer 对象指针解引用，输出对象的信息：

```
void Library::listCustomers() {
  if (m_customerPtrList.empty()) {
    cout << "No customers." << endl;
    return;
  }
  for (const Customer* customerPtr: m_customerPtrList) {
    cout << (*customerPtr) << endl;
  }
}
```

借阅图书

在借阅一本书时，我们先提示用户输入作者和书名，如下面的代码片段所示：

```
void Library::borrowBook() {
  string author;
  cout << "Author: ";
  cin >> author;
  string title;
  cout << "Title: ";
  cin >> title;
```

我们查找这本书，如果不存在，则显示一条错误消息，如下面的代码所示：

```
Book* bookPtr = lookupBook(author, title);
if (bookPtr == nullptr) {
  cout << endl << "There is no book \"" << title
       << "\" by " << author << "." << endl;
  return;
}
```

如果这本书已被其他顾客借走，就不能再次借阅：

```
if (bookPtr->borrowerPtr() != nullptr) {
  cout << endl << "The book \"" << title << "\" by "
       << author << " has already been borrowed." << endl;
```

```
    return;
  }
```

我们提示用户输入顾客的姓名和地址：

```
  string name;
  cout << "Customer name: ";
  cin >> name;
  string address;
  cout << "Address: ";
  cin >> address;
  Customer* customerPtr = lookupCustomer(name, address);
```

如果不存在给定姓名和地址的顾客，则显示一条错误消息：

```
  if (customerPtr == nullptr) {
    cout << endl << "No customer with name " << name
         << " and address " << address << " exists."
         << endl;
    return;
  }
```

最后，我们把这本书加入该顾客的借阅集合，并把该顾客登记为这本书的借阅者：

```
    bookPtr->borrowerPtr() = customerPtr;
    customerPtr->borrowBook(bookPtr);
    cout << endl << "Borrowed." << endl;
  }
```

预约图书

预约流程与前面的借阅流程类似。我们提示用户输入书的作者和书名，以及顾客的姓名和地址，如下所示：

```
  void Library::reserveBook() {
    string author;
    cout << "Author: ";
    cin >> author;
    string title;
    cout << "Title: ";
    cin >> title;
    Book* bookPtr = lookupBook(author, title);
```

如果这本书不存在，则显示一条错误消息：

```
if (bookPtr == nullptr) {
  cout << endl << "There is no book \"" << title
       << "\" by " << author << "." << endl;
  return;
}
```

如果这本书尚未被借出，就不能预约它。这时我们建议用户直接借阅这本书：

```
if (bookPtr->borrowerPtr() == nullptr) {
  cout << endl << "The book \"" << title << "\" by "
       << author << " has not been not borrowed. "
       << "Please borrow the book instead of reserving it."
       << endl;
  return;
}
```

我们提示用户输入顾客的姓名和地址：

```
string name;
cout << "Customer name: ";
cin >> name;
string address;
cout << "Address: ";
cin >> address;
Customer* customerPtr = lookupCustomer(name, address);
```

如果该顾客不存在，则显示一条错误消息：

```
if (customerPtr == nullptr) {
  cout << endl << "There is no customer with name "
       << name << " and address " << address << "." << endl;
  return;
}
```

如果该顾客已经借阅了这本书，就不能再预约它：

```
if (bookPtr->borrowerPtr() == customerPtr) {
  cout << endl << "The book has already been borrowed by "
       << name << "." << endl;
  return;
}
```

最后，我们把该顾客加入这本书的预约列表，并把这本书加入该顾客的预约集合。注意，一本书对应的是一个预约顾客列表，而一位顾客对应的是一个预约图书集合。原因在于：当一本书被归还时，预约列表中的第一位顾客将借走这本书；而顾客的预约集合则没有这样的限制：

```
int position = bookPtr->reserveBook(customerPtr);
customerPtr->reserveBook(bookPtr);
```

我们把该顾客在预约列表中的位置通知给这位顾客：

```
    cout << endl << position << "nd reserve." << endl;
}
```

归还图书

在归还一本书时，我们提示用户输入作者和书名，但不需要询问是哪位顾客借走了这本书——这一信息已经存放在 Book 对象中：

```
void Library::returnBook() {
  string author;
  cout << "Author: ";
  cin >> author;
  string title;
  cout << "Title: ";
  cin >> title;
  Book* bookPtr = lookupBook(author, title);
```

如果给定作者和书名的这本书不存在，则显示一条错误消息：

```
if (bookPtr == nullptr) {
  cout << endl << "There is no book \"" << title << "\" by "
       << author << "." << endl;
  return;
}
Customer* customerPtr = bookPtr->borrowerPtr();
```

如果这本书当前无人借阅，则显示一条错误消息：

```
if (customerPtr == nullptr) {
  cout << endl << "The book \"" << title << "\" by "
       << author << " has not been borrowed." << endl;
  return;
}
bookPtr->returnBook();
customerPtr->returnBook(bookPtr);
cout << endl << "Returned." << endl;
```

归还这本书之后，我们需要查明是否有顾客预约了它：

```
list<Customer*>& reservationPtrList =
    bookPtr->reservationPtrList();
```

如果这本书的预约列表中至少有一位顾客，我们就取出排在最前面的顾客，把他从这本书的预约列表中移除，把他登记为这本书的借阅者，并把这本书加入该顾客的借阅集合：

```
    if (!reservationPtrList.empty()) {
      Customer* newCustomerPtr = reservationPtrList.front();
      reservationPtrList.erase(reservationPtrList.begin());
      bookPtr->borrowBook(newCustomerPtr);
      newCustomerPtr->borrowBook(bookPtr);
      cout << endl << "Borrowed by "
           << newCustomerPtr->name() << endl;
    }
  }
```

查找图书与顾客（索引与指针）

在把图书馆信息保存到文件以及从文件加载时，我们需要在指向 Book 和 Customer 对象的指针与图书列表和顾客列表中的索引之间进行转换。lookupBookIndex 方法接受一个指向 Book 对象的指针，并返回它在图书列表中的索引：

```
int Library::lookupBookIndex(const Book* bookPtr) {
  int index = 0;
  for (Book* testPtr : m_bookPtrList) {
    if (bookPtr == testPtr) {
      return index;
    }
    ++index;
  }
```

如果执行到了这里，assert 宏将带着错误信息中止程序的执行。不过我们本不该到达这一点，因为该 Book 指针应当就在图书指针列表中：

```
  assert(false);
  return -1;
}
```

lookupBookPtr 方法执行相反的任务：它在图书指针列表中找出 bookIndex 所指位置上的 Book 对象指针。如果索引超出了列表的范围，assert 宏将带着错误信息中止程序的执行。不过这不应该发生，因为所有索引都应当落在列表范围之内：

```
Book* Library::lookupBookPtr(int bookIndex) {
  assert((bookIndex >= 0) &&
         (bookIndex < ((int) m_bookPtrList.size())));
  auto iterator = m_bookPtrList.begin();
  for (int count = 0; count < bookIndex; ++count) {
    ++iterator;
  }
  return *iterator;
}
```

lookupCustomerIndex 方法以与前面 lookupBookIndex 方法相同的方式，给出 Customer 指针在顾客指针列表中的索引：

```
int Library::lookupCustomerIndex(const Customer* customerPtr) {
  int index = 0;
  for (Customer* testPtr : m_customerPtrList) {
    if (customerPtr == testPtr) {
      return index;
    }
    ++index;
  }
  assert(false);
  return -1;
}
```

lookupCustomerPtr 方法按照与前面 lookupBookPtr 方法相同的方式，在顾客指针列表中查找给定索引处的 Customer 指针：

```
Customer* Library::lookupCustomerPtr(int customerIndex) {
  assert((customerIndex >= 0) &&
         (customerIndex < ((int) m_customerPtrList.size())));
  auto iterator = m_customerPtrList.begin();
  for (int count = 0; count < customerIndex; ++count) {
    ++iterator;
  }
  return *iterator;
}
```

Marshmallowing（棉花糖化）

本章 Library 类的 save 和 load 方法比第 3 章「构建图书馆管理系统」中的对应方法要复杂一些。原因在于，我们无法直接保存指针，因为指针存放的内存地址在每次运行之间都会变化。我们必须改为把索引保存到文件中。这种把指针转换成索引、再把索引转换回指针的过程称为 marshmallowing。保存图书馆信息时，我们把整个保存过程分成若干步骤：

- **保存图书清单**：此时只保存作者与书名。
- **保存顾客清单**：此时只保存姓名与地址。
- **对每本图书**：保存其借阅者（如果已被借出）以及（可能为空的）预约列表。我们保存的是顾客列表的索引，而不是指向顾客的指针。
- **对每位顾客**：保存其借阅与预约的图书集合。我们保存的是图书列表的索引，而不是指向图书的指针。

将图书馆信息保存到文件

save 方法打开文件，如果文件成功打开，就写出图书馆中的图书和顾客：

```
void Library::save() {
  ofstream outStream(s_binaryPath);
```

写入图书对象

我们保存图书对象。这里只通过对每个 Book 对象调用 write 来保存书的作者与书名，暂不保存可能存在的借阅者和预约列表。

我们首先把列表中的图书数量写入文件：

```
if (outStream) {
  int bookPtrListSize = m_bookPtrList.size();
  outStream.write((char*) &bookPtrListSize,
                  sizeof bookPtrListSize);
```

然后，通过对每个 Book 对象指针调用 write，把每本书的信息写入文件：

```
  for (const Book* bookPtr : m_bookPtrList) {
    bookPtr->write(outStream);
  }
}
```

写入顾客对象

我们保存顾客对象。与前面的图书情况类似，这里只通过对每个 Customer 对象调用 write 来保存顾客的姓名与地址，暂不保存借阅与预约的图书集合。

与前面的图书情况相同，我们首先把列表中的顾客数量写入文件：

```
int customerPtrListSize = m_customerPtrList.size();
outStream.write((char*) &customerPtrListSize,
                sizeof customerPtrListSize);
```

然后，通过对每个 Customer 对象指针调用 write 方法，把每位顾客的信息写入文件：

```
for (const Customer* customerPtr : m_customerPtrList) {
  customerPtr->write(outStream);
}
```

写入借阅者索引

对每个 Book 对象，如果这本书已被借出，我们就查找并保存相应 Customer 的索引，而不是保存指向该对象的指针：

```
for (const Book* bookPtr : m_bookPtrList) {
  const Customer* borrowerPtr = bookPtr->borrowerPtr();
```

对每本书，我们首先检查它是否已被借出。如果已被借出，就向文件写入值 true，表示这本书处于被借出状态：

```
if (borrowerPtr != nullptr) {
  bool borrowed = true;
  outStream.write((char*) &borrowed, sizeof borrowed);
```

然后，我们在顾客指针列表中查找借走这本书的顾客的索引，并把该索引写入文件：

```
  int loanIndex = lookupCustomerIndex(borrowerPtr);
  outStream.write((char*) &loanIndex, sizeof loanIndex);
}
```

如果这本书未被借出，我们只需向文件写入值 false，表示这本书没有被借阅：

```
  else {
    bool borrowed = false;
    outStream.write((char*) &borrowed, sizeof borrowed);
  }
}
```

写入预约索引

由于一本书可以同时被多位顾客预约，我们遍历预约列表，并保存列表中每位顾客的索引：

```
const list<Customer*>& reservationPtrList =
    bookPtr->reservationPtrList();
```

对每本书，我们首先把该书的预约数量写入文件：

```
int reserveSetSize = reservationPtrList.size();
outStream.write((char*) &reserveSetSize,
                sizeof reserveSetSize);
```

然后遍历预约列表，对每一项预约，查找并写出预约这本书的每位顾客的索引：

```
  for (const Customer* customerPtr : reservationPtrList) {
    int customerIndex = lookupCustomerIndex(customerPtr);
```

```
      outStream.write((char*) &customerIndex,
                      sizeof customerIndex);
    }
  }
```

写入借阅图书索引

对每位顾客，我们保存其借阅的各本书的索引：先保存借阅列表的大小，再保存各本书的索引：

```
  for (const Customer* customerPtr : m_customerPtrList) {
    const set<Book*>& loanPtrSet = customerPtr->loanPtrSet();
```

对每位顾客，我们首先把借阅数量写入文件：

```
  int loanPtrSetSize = loanPtrSet.size();
  outStream.write((char*) &loanPtrSetSize,
                  sizeof loanPtrSetSize);
```

然后遍历借阅集合，对每一次借阅，查找并把每本书的索引写入文件：

```
    for (const Book* bookPtr : loanPtrSet) {
      int bookIndex = lookupBookIndex(bookPtr);
      outStream.write((char*) &bookIndex,
                      sizeof bookIndex);
    }
  }
```

写入预约图书索引

同样，对每位顾客，我们保存其预约的各本书的索引：先保存预约列表的大小，再保存其所预约各本书的索引：

```
  const set<Book*>& reservationPtrSet =
      customerPtr->reservationPtrSet();
```

对每位顾客，我们首先把预约的图书数量写入文件：

```
  int reservationPtrSetSize = reservationPtrSet.size();
  outStream.write((char*) &reservationPtrSetSize,
                  sizeof reservationPtrSetSize);
```

然后遍历预约集合，对每一项预约，查找并把每本书的索引写入文件：

```
      for (const Book* bookPtr : reservationPtrSet) {
        int bookIndex = lookupBookIndex(bookPtr);
```

```
        outStream.write((char*) &bookIndex,
                        sizeof bookIndex);
      }
    }
  }
```

从文件加载图书馆信息

加载文件时，我们按照与保存文件时相同的方式来进行：

```
void Library::load() {
  ifstream inStream(s_binaryPath);
```

读取图书对象

我们先读取图书列表的大小，再读取各本书本身。记住，到目前为止我们只读取了书的作者与书名：

```
  if (inStream) {
    int bookPtrListSize;
```

我们首先读取图书数量：

```
  inStream.read((char*) &bookPtrListSize,
                sizeof bookPtrListSize);
```

然后读取各本书本身。对每一本书，我们动态分配一个 `Book` 对象，通过对该指针调用 `read` 来读取其信息，并把该指针加入图书指针列表：

```
    for (int count = 0; count < bookPtrListSize; ++count) {
      Book *bookPtr = new Book();
      assert(bookPtr != nullptr);
      bookPtr->read(inStream);
      m_bookPtrList.push_back(bookPtr);
    }
  }
```

读取顾客对象

同样，我们先读取顾客列表的大小，再读取各位顾客本身。到目前为止，我们只读取了顾客的姓名与地址：

```
  int customerPtrListSize;
```

我们首先读取顾客数量：

```
inStream.read((char*) &customerPtrListSize,
              sizeof customerPtrListSize);
```

然后读取各位顾客本身。对每一位顾客，我们动态分配一个 Customer 对象，通过对该指针调用 read 来读取其信息，并把该指针加入顾客指针列表：

```
for (int count = 0; count < customerPtrListSize; ++count) {
  Customer *customerPtr = new Customer();
  assert(customerPtr != nullptr);
  customerPtr->read(inStream);
  m_customerPtrList.push_back(customerPtr);
}
```

读取借阅者索引

对每本书，我们读取借阅过它的顾客（如果有的话），以及预约这本书的顾客列表：

```
for (Book* bookPtr : m_bookPtrList) {
  bool borrowed;
  inStream.read((char*) &borrowed, sizeof borrowed);
```

如果 borrowed 为 true，说明这本书已被借出。此时我们读取该顾客的索引，然后查找相应 Customer 对象的指针，并把它登记为这本书的借阅者：

```
if (borrowed) {
  int loanIndex;
  inStream.read((char*) &loanIndex, sizeof loanIndex);
  bookPtr->borrowerPtr() = lookupCustomerPtr(loanIndex);
}
```

如果 borrowed 为 false，说明这本书没有被借出。此时我们把指向借阅者顾客的指针设为 nullptr：

```
  else {
    bookPtr->borrowerPtr() = nullptr;
  }
}
```

读取预约索引

对每本书，我们还要读取预约列表：先读取列表的大小，再读取各条顾客索引：

```
    list<Customer*>& reservationPtrList =
        bookPtr->reservationPtrList();
    int reservationPtrListSize;
```

我们首先读取这本书的预约数量：

```
    inStream.read((char*) &reservationPtrListSize,
                  sizeof reservationPtrListSize);
```

对每一项预约，我们读取顾客的索引，并调用 `lookupCustomerPtr` 获得相应 `Customer` 对象的指针，然后把它加入这本书的预约指针列表：

```
      for (int count = 0; count < reservationPtrListSize;
           ++count) {
        int customerIndex;
        inStream.read((char*) &customerIndex,
                      sizeof customerIndex);
        Customer* customerPtr =
            lookupCustomerPtr(customerIndex);
        reservationPtrList.push_back(customerPtr);
      }
    }
```

读取借阅图书索引

对每位顾客，我们读取其借阅的图书集合：

```
    for (Customer* customerPtr : m_customerPtrList) {
      set<Book*>& loanPtrSet = customerPtr->loanPtrSet();
      int loanPtrSetSize = loanPtrSet.size();
```

我们首先读取借阅列表的大小：

```
    inStream.read((char*) &loanPtrSetSize,
                  sizeof loanPtrSetSize);
```

对每一次借阅，我们读取该书的索引，并调用 `lookupBookPtr` 获得相应 `Book` 对象的指针，然后把它加入借阅指针集合：

```
      for (int count = 0; count < loanPtrSetSize; ++count) {
        int bookIndex;
        inStream.read((char*) &bookIndex, sizeof bookIndex);
        Book* bookPtr = lookupBookPtr(bookIndex);
        loanPtrSet.insert(bookPtr);
      }
    }
```

读取预约图书索引

同样，对每位顾客，我们读取其预约的图书集合：

```
set<Book*>& reservationPtrSet =
    customerPtr->reservationPtrSet();
```

我们首先读取预约列表的大小：

```
int reservationPtrSetSize = reservationPtrSet.size();
inStream.read((char*) &reservationPtrSetSize,
              sizeof reservationPtrSetSize);
```

对每一项预约，我们读取该书的索引，并调用 `lookupBookPtr` 获得相应 `Book` 对象的指针，然后把它加入预约指针集合：

```
    for (int count = 0; count < reservationPtrSetSize;
         ++count) {
      int bookIndex;
      inStream.read((char*) &bookIndex, sizeof bookIndex);
      Book* bookPtr = lookupBookPtr(bookIndex);
      reservationPtrSet.insert(bookPtr);
    }
  }
}
```

释放内存

由于我们向列表中添加的是动态分配的 `Book` 和 `Customer` 对象，因此需要在程序执行结束时释放它们。析构函数遍历图书指针列表和顾客指针列表，释放所有图书指针和顾客指针：

```
Library::~Library() {
  for (const Book* bookPtr : m_bookPtrList) {
    delete bookPtr;
  }
  for (const Customer* customerPtr : m_customerPtrList) {
    delete customerPtr;
  }
}
```

main 函数

与第 3 章「构建图书馆管理系统」类似，main 函数只是创建一个 `Library` 对象：

Main.cpp

```
#include <Set>
#include <Map>
#include <String>
#include <FStream>
#include <IOStream>
using namespace std;

#include "Book.h"
#include "Customer.h"
#include "Library.h"

void main() {
  Library();
}
```

小结

本章我们构建了一个与第 3 章「构建图书馆管理系统」类似的图书馆管理系统。不过，我们省去了所有整数编号，改用指针。这样做的好处是可以更直接地登记借阅与预约，但代价是把它们保存到文件以及从文件加载变得更加困难。

在第 5 章「Qt 图形应用程序」中，我们将研究图形应用程序。

5
Qt 图形应用

本章介绍如何用 Qt 开发图形应用。在第 4 章「基于指针的图书馆管理系统」中，我们开发了抽象数据类型和一个图书馆管理系统，不过那些应用程序都是文本模式的。本章我们将研究三个用 Qt 图形库开发的图形应用：

- 时钟（clock）：我们开发一个模拟时钟，带有时针、分针和秒针，并用刻度线标记小时、分钟和秒。
- 画图程序（drawing program）：一个用不同颜色绘制直线、矩形和椭圆的程序。
- 编辑器（editor）：用户可以输入和编辑文本的程序。

我们还将学习 Qt 库的以下内容：

- 窗口与部件；
- 菜单与工具栏；
- 在窗口中绘制图形与书写文本；
- 如何捕获鼠标和键盘事件。

创建时钟应用程序

本章以及下一章我们都将使用 Qt——一个面向对象的图形应用类库；同时我们使用的开发环境是 Qt Creator 而不是 Visual Studio，后者是一个集成开发环境。

环境搭建

在 Qt Creator 中创建新的图形工程时，我们在 File 菜单中选择 New File or Project 菜单项，此时会显示出 New File or Project 对话框窗口。我们选择 Qt Widgets Application，然后点击

Choose 按钮。

接着会显示 Introduction and Project Location 对话框。我们把工程命名为 Clock，放在合适的位置，然后点击 Next 按钮。在 Kit Selection 对话框中，我们选择最新版本的 Qt 库，然后点击 Next。在 Class Information 对话框中，我们把应用的基础类命名为 clock。通常，图形应用的窗口继承一个窗口类；不过这里要处理的是一个相对简单的应用，所以我们继承 Qt 的 QWidget 类——尽管部件（widget）通常指嵌在窗口中的较小图形对象。在 Qt Creator 中可以添加表单（form），但本章我们不使用该特性，因此取消勾选 Generate form 选项。



Qt 中所有类名都以字母 Q 开头。

最后，在 Project Management 对话框中，我们直接接受默认值并点击 Finish，生成包含 Clock.h 和 Clock.cpp 文件的工程。

Clock 类

工程由 Clock.h、Clock.cpp 和 Main.cpp 文件组成。类定义与前面几章的类略有不同：我们用包含保护（include guard）把类定义包起来。也就是说，必须用预处理指令 `ifndef`、`define` 和 `endif` 把类定义包起来。预处理器执行文本替换。

`ifndef` 和 `endif` 指令的作用类似于 C++ 中的 `if` 语句。如果条件不成立，指令之间的代码就会被省略。这里，只有当 `CLOCK_H` 宏尚未定义时才会包含这段代码；如果包含，下一行的 `define` 指令就会定义这个宏。这样，类定义在整个工程中只会被包含一次。此外，我们还在 Clock.h 头文件（而不是 Clock.cpp 定义文件）中包含系统头文件 QWidget 和 QTimer。

Clock.h

```
#ifndef CLOCK_H
#define CLOCK_H

#include <QWidget>
#include <QTimer>
```

由于 Clock 是 Qt QWidget 类的子类，必须包含 Q_OBJECT 宏，它会引入 Qt 库中的某些代码。我们需要它来使用这里所示的 SIGNAL 和 SLOT 宏：

```
class Clock : public QWidget {
  Q_OBJECT
```

构造函数接受一个指向父部件的指针，默认值为 `nullptr`：

```
  public:
    Clock(QWidget* parentWidgetPtr = nullptr);
```

每当窗口需要重绘时，框架就会调用 paintEvent 方法。它接受一个指向 QPaintEvent 对象的指针作为参数，该参数可用于确定重绘应如何进行：

```
  void paintEvent(QPaintEvent *eventPtr);
```

QTimer 是一个处理定时器的 Qt 系统类。我们将用它来移动时钟的指针：

```
  private:
    QTimer m_timer;
  };

#endif // CLOCK_H
```

定义文件主要由处理时钟绘制的 paintEvent 方法组成。

Clock.cpp

```
#include <QtWidgets>
#include "Clock.h"
```

在构造函数中，我们调用基类 QWidget 的构造函数并传入 parentWidgetPtr 参数（它可以是 nullptr）：

```
Clock::Clock(QWidget* parentWidgetPtr /* = nullptr */)
 :QWidget(parentWidgetPtr) {
```

我们把窗口标题设置为 Clock。在 Qt 中，字面文本总是用 tr 函数处理，它进而调用 Qt QCoreApplication 类中的 translate 方法，确保文本被转换为适合显示的形式。我们还将窗口大小调整为 1000 × 500 像素，这对大多数屏幕都合适：

```
  setWindowTitle(tr("Clock"));
  resize(1000, 500);
```

我们需要一种把定时器与时钟部件连接起来的方法：定时器倒计时结束时，时钟应当被更新。为此，Qt 向我们提供了信号（signal）/槽（slot）机制。当定时器到达计时期限时，它会调用自身的 timeout 方法。我们用 connect 方法配合 SIGNAL 和 SLOT 宏，把对 timeout 的调用与 Qt QWidget 类中 update 方法的调用连接起来，后者更新时钟的绘制。SIGNAL 宏登记 timeout 调用将发出一个信号，SLOT 宏登记信号发出时应调用 update 方法，connect 方法则把信号与槽连接起来。这样，我们就在定时器的 timeout 与时钟的更新之间建立了一个连接：

```
  m_timer.setParent(this);
  connect(&m_timer, SIGNAL(timeout()), this, SLOT(update()));
  m_timer.start(100);
}
```

每当窗口需要重绘时就会调用 paintEvent 方法。原因可能是外部因素，比如用户改变了窗口大小；也可能是对 QMainWindow 类 update 方法的调用，它最终会调用 paintEvent。

本例中我们不需要事件的任何信息，所以把 eventPtr 参数用注释括起来。width 和 height 方法给出窗口可绘制部分的宽度和高度（单位为像素）。我们调用 qMin 方法来决定窗口较短的那条边，并调用 QTime 类的 currentTime 方法来获取时钟的当前时间：

```
void Clock::paintEvent(QPaintEvent* /* eventPtr */) {
  int side = qMin(width(), height());
  QTime time = QTime::currentTime();
```

QPainter 类可以看作一块绘图画布。我们首先把它初始化为合适的抗锯齿设置，然后调用 translate 和 scale 方法，把以像素为单位的物理尺寸变换为 200 × 200 单位的逻辑尺寸：

```
QPainter painter(this);
painter.setRenderHint(QPainter::Antialiasing);
painter.setRenderHint(QPainter::TextAntialiasing);
painter.translate(width() / 2, height() / 2);
painter.scale(side / 200.0, side / 200.0);
```

我们为分钟绘制 60 条刻度线。每第五条线要稍长一些，用来标记小时。对每一分钟，我们先画一条线，然后调用 Qt 的 rotate 方法把绘图旋转 6 度。这样，我们以 6 度为单位旋转 60 次，总共转过 360 度，即一整圈：

```
for (int second = 0; second <= 60; ++second) {
  if ((second % 5) == 0) {
    painter.drawLine(QPoint(0, 81), QPoint(0, 98));
  }
  else {
    painter.drawLine(QPoint(0, 90), QPoint(0, 98));
  }
```

一整圈是 360 度。每条线旋转 6 度，因为 360 除以 60 等于 6 度。旋转完毕后，把绘图恢复为初始设置：

```
  painter.rotate(6);
}
```

我们从 QTime 对象获取当前的时、分、秒和毫秒：

```
double hours = time.hour(), minutes = time.minute(),
       seconds = time.second(), milliseconds = time.msec();
```

我们把画笔颜色设为黑色，把背景色设为灰色：

```
painter.setPen(Qt::black);
painter.setBrush(Qt::gray);
```

我们定义时针的端点。时针比分针和秒针稍粗稍短。我们定义构成时针端点的三个点。时针底座长 16 个单位，距原点 8 个单位，因此我们把底座两点的 x 坐标设为 8 和 −8，y 坐标设为 8。最后，把时针长度定义为 60 个单位。该值为负，以便与当前旋转方向一致：

```
{ static const QPoint hourHand[3] =
      {QPoint(8, 8), QPoint(-8, 8), QPoint(0, -60)};
```

save 方法保存 QPainter 对象的当前设置，之后由 restore 方法恢复：

```
painter.save();
```

通过计算小时、分、秒和毫秒，我们求出时针的精确角度，然后旋转以设置时针。每小时对应 30 度，因为一圈有 12 个小时，360 度除以 12 等于 30 度：

```
double hour = hours + (minutes / 60.0) + (seconds / 3600.0) +
              (milliseconds / 3600000.0);
painter.rotate(30.0 * hour);
```

我们用时针的三个点调用 drawConvexPolygon 方法：

```
  painter.drawConvexPolygon(hourHand, 3);
  painter.restore();
}
```

我们以同样的方式绘制分针。它比时针稍细稍长。另一个区别是：小时有 12 个，分钟却有 60 个，因此每分钟对应 6 度——360 度除以 60 等于 6 度：

```
{ static const QPoint minuteHand[3] =
      {QPoint(6, 8), QPoint(-6, 8), QPoint(0, -70)};
  painter.save();
```

计算当前分针角度时，我们使用分钟、秒和毫秒：

```
  double minute = minutes + (seconds / 60.0) +
                  (milliseconds / 60000.0);
  painter.rotate(6.0 * minute);
  painter.drawConvexPolygon(minuteHand, 3);
  painter.restore();
}
```

秒针的绘制与前面的分针几乎完全相同，唯一的区别是计算秒针角度时只使用秒和毫秒：

```
  { static const QPoint secondHand[3] =
        {QPoint(4, 8), QPoint(-4, 8), QPoint(0, -80)};
    painter.save();
    double second = seconds + (milliseconds / 1000);
    painter.rotate(6.0 * second);
    painter.drawConvexPolygon(secondHand, 3);
```

```
      painter.restore();
    }
  }
```

main 函数

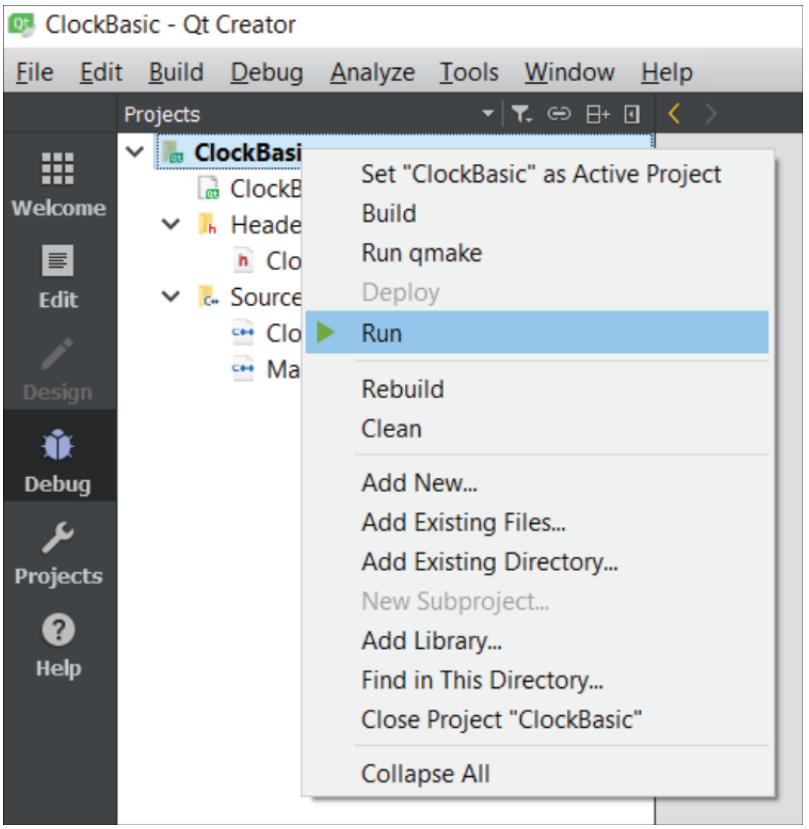
在 main 函数中，我们初始化并启动 Qt 应用程序。main 函数可以接受参数 argc 和 argv，它们保存应用程序的命令行参数：argc 保存参数的个数，argv 数组保存参数本身。argv 的第一个元素总是可执行文件的路径，最后一个元素总是 `nullptr`。QApplication 类接受 argc 和 argv 并初始化 Qt 应用程序。我们创建一个 Clock 类的对象，并调用 show 使其可见。最后，调用 QApplication 对象的 exec。

Main.cpp

```
#include <QApplication>
#include "Clock.h"

int main(int argc, char *argv[]) {
  QApplication application(argc, argv);
  Clock Clock;
  Clock.show();
  return application.exec();
}
```

要执行应用程序，我们在项目上选择 Run 选项：



程序的执行会一直持续到用户点击右上角的关闭按钮、关闭 Clock 窗口为止：



为窗口和部件搭建可复用的类

图形应用中有窗口和部件。窗口通常是一个带边框的完整窗口，边框上有标题、菜单栏以及关闭窗口和调整窗口大小的按钮；而部件通常是嵌在窗口中的较小图形对象。在 Clock 工程中，我们只用了继承 QWidget 类的一个部件类。不过本节我们将离开 Clock 工程，研究更高级的同时包含窗口和部件的应用：窗口持有带菜单栏和工具栏的边框，而部件位于窗口中并负责图形内容。

本章接下来的各节中，我们将研究画图程序和编辑器。这些应用是典型的文档应用：我们打开和保存文档，还可以对文档元素进行剪切、复制、粘贴和删除。为了给窗口添加菜单和工具栏，我们需要继承 Qt 的两个类 QMainWindow 和 QWidget：QMainWindow 用于给窗口边框添加菜单和工具栏，QWidget 用于在窗口区域绘制图形。

为了在本章剩余部分以及下一章介绍的应用中复用文档代码，本节我们定义 MainWindow 和 DocumentWidget 类。这些类随后将被本章后面各节中的画图程序和编辑器使用。MainWindow 搭建带 File 和 Edit 菜单及工具栏的窗口，而 DocumentWidget 提供一个框架，为 New、Open、Save、SaveAs、Cut、Copy、Paste、Delete 和 Exit 菜单项搭建骨架代码。

本节我们不新建 Qt 工程，只编写 MainWindow 和 DocumentWidget 类——它们将作为本章后面画图程序和编辑器的基类，还有用于设置菜单和工具栏项的 LISTENER 宏。

添加监听器

监听器（listener）是用户选择菜单项或工具栏项时被调用的方法。LISTENER 宏向类添加一个监听器。

Listener.h

```
#ifndef LISTENER_H
#define LISTENER_H

#include <QObject>
```

由于 Qt 对菜单和工具栏的规定，Qt 框架响应用户操作而调用的监听器必须是函数而不是方法。



方法属于类，而函数是自由独立的。

DEFINE_LISTENER 宏同时定义一个友元函数和一个方法。Qt 框架调用该函数，函数再调用方法：

```
#define DEFINE_LISTENER(BaseClass, Listener)
  friend bool Listener(QObject* baseObjectPtr) {
     return ((BaseClass*) baseObjectPtr)->Listener();
  }
  bool Listener()
```

LISTENER 宏定义为指向方法的指针：

```
#define LISTENER(Listener) (&::Listener)
```

监听器方法接受一个 QObject 指针作为参数并返回布尔值：

```
typedef bool (*Listener)(QObject*);
#endif // LISTENER_H
```

基窗口类

MainWindow 类搭建一个带 File 和 Edit 菜单及工具栏的文档窗口。它还提供 addAction 方法，供子类添加应用专属的菜单和工具栏。

MainWindow.h

```
#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QMainWindow>
#include <QActionGroup>
#include <QPair>
#include <QMap>

#include "Listener.h"
#include "DocumentWidget.h"

class MainWindow : public QMainWindow {
  Q_OBJECT

  public:
    MainWindow(QWidget* parentWidgetPtr = nullptr);
    ~MainWindow();

  protected:
    void addFileMenu();
    void addEditMenu();
```

addAction 方法添加一个菜单项，可带快捷键、工具栏图标，以及用于给菜单项标注复选框或单选按钮的监听器：

```
protected:
  void addAction(QMenu* menuPtr, QString text,
                 const char* onSelectPtr,
                 QKeySequence acceleratorKey = 0,
                 QString iconName = QString(),
                 QToolBar* toolBarPtr = nullptr,
                 QString statusTip = QString(),
                 Listener enableListener = nullptr,
                 Listener checkListener = nullptr,
                 QActionGroup* groupPtr = nullptr);
```

我们用 DEFINE_LISTENER 宏添加监听器，用于决定菜单项是否应处于启用状态。监听器在菜单项应启用时返回 true。DocumentWidget 是 Qt 类 QWidget 的子类，我们将在下一小节定义它。借助 DEFINE_LISTENER 宏，我们给 MainWindow 类添加 isSaveEnabled、isCutEnabled、isCopyEnabled、isPasteEnabled 和 isDeleteEnabled 方法。当用户选择菜单项时会调用它们：

```
DEFINE_LISTENER(DocumentWidget, isSaveEnabled);
DEFINE_LISTENER(DocumentWidget, isCutEnabled);
DEFINE_LISTENER(DocumentWidget, isCopyEnabled);
DEFINE_LISTENER(DocumentWidget, isPasteEnabled);
DEFINE_LISTENER(DocumentWidget, isDeleteEnabled);
```

onMenuShow 方法在菜单变为可见之前被调用；它调用菜单中各菜单项的监听器，以决定菜单项应被禁用，还是标注复选框或单选按钮。框架也会调用它来禁用工具栏图标：

```
  public slots:
    void onMenuShow();
```

m_enableMap 和 m_checkMap 字段保存菜单项监听器的映射。前面的 onMenuShow 方法用它们来决定禁用菜单项，还是标注复选框或单选按钮：

```
    private:
      QMap<QAction*,QPair<QObject*,Listener>> m_enableMap,
                                              m_checkMap;
    };

  #endif // MAINWINDOW_H
```

MainWindow.cpp

```
  #include "MainWindow.h"
  #include <QtWidgets>
```

构造函数调用 Qt QMainWindow 类的构造函数，参数是父部件指针：

```
  MainWindow::MainWindow(QWidget* parentWidgetPtr /*= nullptr*/)
   :QMainWindow(parentWidgetPtr) {
  }
```

菜单项添加时会连接到一个动作（action）。析构函数释放菜单栏的所有动作：

```
  MainWindow::~MainWindow() {
    for (QAction* actionPtr : menuBar()->actions()) {
      delete actionPtr;
    }
  }
```

addFileMenu 方法把标准 File 菜单添加到菜单栏；menuBar 是一个 Qt 方法，返回窗口菜单栏的指针：

```
  void MainWindow::addFileMenu() {
    QMenu* fileMenuPtr = menuBar()->addMenu(tr("&File"));
```

下面的代码片段与 connect 方法类似，把菜单项与 onMenuShow 方法连接起来。Qt 宏 SIGNAL 和 SLOT 确保在菜单变为可见之前调用 onMenuShow。onMenuShow 方法在菜单可见之前设置菜单中每一项的启用、复选框和单选按钮状态；它还设置工具栏图标的启用状态。aboutToShow 方法在每个菜单可见之前被调用，以便启用或禁用菜单项，并可能给它们标注复选框或单选按钮：

```
connect(fileMenuPtr, SIGNAL(aboutToShow()), this,
        SLOT(onMenuShow()));
```

Qt 的 addToolBar 方法向窗口边框添加一个工具栏。在这里调用 addAction 时，菜单项会被添加到菜单中；如果存在对应的工具栏，也会添加到工具栏：

```
QToolBar *fileToolBarPtr = addToolBar(tr("File"));
```

addAction 方法添加 New、Open、Save、SaveAs 和 Exit 菜单项。它接受以下参数：

- 菜单项所属菜单的指针。
- 菜单项文本。文本前的和号（&）（如 &New）表示紧随其后的字母（N）将带下划线，用户可按 Alt-N 选择该项。
- 快捷键信息。QKeySequence 是保存快捷键组合的 Qt 枚举。QKeySequence::New 表示用户可按 Ctrl-N 选择该项，菜单项文本后还会附加上 Ctrl+N。
- 图标文件的名称（new）。该文件的图标显示在菜单项文本左侧和工具栏上。图标文件本身在 Qt Creator 中添加到工程。
- 工具栏的指针；如果菜单项不连接工具栏，则为 nullptr。
- 用户把鼠标悬停在工具栏项上时显示的文本。如果菜单项不连接工具栏，则被忽略。
- 监听器（默认 nullptr），在菜单和工具栏变为可见之前调用，决定该项是启用，还是标注复选框或单选按钮。

```
addAction(fileMenuPtr, tr("&New"), SLOT(onNew()),
          QKeySequence::New, tr("new"), fileToolBarPtr,
          tr("Create a new file"));

addAction(fileMenuPtr, tr("&Open"), SLOT(onOpen()),
          QKeySequence::Open, tr("open"), fileToolBarPtr,
          tr("Open an existing file"));
```

当文档自上次保存后没有更改时，文档无需保存，Save 菜单项应禁用。因此我们添加一个额外的参数，指明应调用 isSaveEnabled 方法来启用或禁用菜单项和工具栏图标：

```
addAction(fileMenuPtr, tr("&Save"), SLOT(onSave()),
          QKeySequence::Save, tr("save"), fileToolBarPtr,
          tr("Save the document to disk"),
          LISTENER(isSaveEnabled));
```

SaveAs 菜单项没有按键序列，也没有工具栏项。因此图标文件名和工具栏文本是默认的 QString 对象，工具栏指针为 nullptr：

```
addAction(fileMenuPtr, tr("Save &As"), SLOT(onSaveAs()),
          0, QString(), nullptr, QString(),
          LISTENER(isSaveEnabled));
```

addSeparator 方法在两个菜单项之间添加一条水平分隔线：

```
    fileMenuPtr->addSeparator();
    addAction(fileMenuPtr, tr("E&xit"),
              SLOT(onExit()), QKeySequence::Quit);
}
```

addEditMenu 方法以与前面 File 菜单相同的方式把 Edit 菜单添加到窗口菜单栏:

```
void MainWindow::addEditMenu() {
    QMenu* editMenuPtr = menuBar()->addMenu(tr("&Edit"));
    QToolBar* editToolBarPtr = addToolBar(tr("Edit"));
    connect(editMenuPtr, SIGNAL(aboutToShow()),
            this, SLOT(onMenuShow()));

    addAction(editMenuPtr, tr("&Cut"), SLOT(onCut()),
              QKeySequence::Cut, tr("cut"), editToolBarPtr,
          tr("Cut the current selection's contents to the clipboard"),
              LISTENER(isCutEnabled));

    addAction(editMenuPtr, tr("&Copy"), SLOT(onCopy()),
              QKeySequence::Copy, tr("copy"), editToolBarPtr,
           tr("Copy the current selection's contents to the clipboard"),
              LISTENER(isCopyEnabled));

    addAction(editMenuPtr, tr("&Paste"),
              SLOT(onPaste()),
              QKeySequence::Paste, tr("paste"), editToolBarPtr,
          tr("Paste the current selection's contents to the clipboard"),
              LISTENER(isPasteEnabled));

    editMenuPtr->addSeparator();
    addAction(editMenuPtr, tr("&Delete"), SLOT(onDelete()),
              QKeySequence::Delete, tr("delete"), editToolBarPtr,
              tr("Delete the current selection"),
              LISTENER(isDeleteEnabled));
}
```

addAction 方法向菜单栏添加菜单项并向工具栏添加工具栏图标，还把菜单项与用户选择该项时被调用的 onSelectPtr 方法，以及启用该项、标注复选框或单选按钮的方法连接起来。除非快捷键为零，否则会为动作添加快捷键。groupPtr 参数定义菜单项是否属于一个组。如果 checkListener 不是 `nullptr`，那么当 groupPtr 为 `nullptr` 时菜单项标注复选框，否则标注单选按钮。对于单选按钮，同一时刻每组只有一个会被选中:

```
void MainWindow::addAction(QMenu* menuPtr, QString itemText,
                           const char* onSelectPtr,
                           QKeySequence acceleratorKey /* = 0 */,
                           QString iconName /*= QString()*/,
```

```
                                QToolBar* toolBarPtr /*= nullptr*/,
                                QString statusTip /*= QString()*/,
                                Listener enableListener /*= nullptr*/,
                                Listener checkListener /*= nullptr*/,
                                QActionGroup* groupPtr /*= nullptr*/) {
  QAction* actionPtr;
```

如果 iconName 非空，我们从工程资源中的文件加载图标，然后创建带图标的新 QAction 对象：

```
  if (!iconName.isEmpty()) {
    const QIcon icon = QIcon::fromTheme("document-" + iconName,
                       QIcon(":/images/" + iconName + ".png"));
    actionPtr = new QAction(icon, itemText, this);
  }
```

如果 iconName 为空，我们创建不带图标的新 QAction 对象：

```
  else {
    actionPtr = new QAction(itemText, this);
  }
```

我们把菜单项连接到选择方法。当用户选择菜单项或点击工具栏图标时，会调用 onSelectPtr：

```
  connect(actionPtr, SIGNAL(triggered()),
          centralWidget(), onSelectPtr);
```

如果快捷键不为零，我们把它加到动作指针上：

```
  if (acceleratorKey != 0) {
    actionPtr->setShortcut(acceleratorKey);
  }
```

最后，我们把动作指针加到菜单指针上，以便菜单处理用户的选择：

```
  menuPtr->addAction(actionPtr);
```

如果 toolBarPtr 不是 `nullptr`，我们把动作添加到窗口的工具栏：

```
  if (toolBarPtr != nullptr) {
    toolBarPtr->addAction(actionPtr);
  }
```

如果状态提示非空，我们把它添加到工具栏的工具提示（tooltip）和状态提示中：

```
  if (!statusTip.isEmpty()) {
      actionPtr->setToolTip(statusTip);
      actionPtr->setStatusTip(statusTip);
```

```
}
```

如果启用监听器不为空，我们向 m_enableMap 添加一个由窗口中央部件指针和监听器组成的对，并调用该监听器来初始化菜单项和工具栏图标的启用状态：

```
if (enableListener != nullptr) {
  QWidget* widgetPtr = centralWidget();
  m_enableMap[actionPtr] =
    QPair<QObject*,Listener>(widgetPtr, enableListener);
  actionPtr->setEnabled(enableListener(widgetPtr));
}
```

同样，如果复选监听器不为空，我们向 m_checkMap 添加窗口中央部件指针和监听器。m_enableMap 和 m_checkMap 都由 onMenuShow 使用，方式如下。我们还调用该监听器来初始化菜单项的复选状态（工具栏图标不复选）：

```
if (checkListener != nullptr) {
  actionPtr->setCheckable(true);
  QWidget* widgetPtr = centralWidget();
  m_checkMap[actionPtr] =
    QPair<QObject*,Listener>(widgetPtr, checkListener);
  actionPtr->setChecked(checkListener(widgetPtr));
}
```

最后，如果组指针不为空，我们把动作加入该组。这样，菜单项将标注单选按钮而不是复选框。框架还会跟踪各组，确保同一时刻每组只有一个单选按钮被选中：

```
  if (groupPtr != nullptr) {
    groupPtr->addAction(actionPtr);
  }
}
```

onMenuShow 方法在菜单或工具栏图标变为可见之前被调用。它确保每个菜单项被启用或禁用，并确保菜单项标注复选框或单选按钮。

我们首先遍历启用映射。对映射中的每一项，我们查找部件和启用函数，调用该函数得到 true 或 false，并用结果通过在动作对象指针上调用 setEnabled 来启用或禁用菜单项：

```
void MainWindow::onMenuShow() {
  for (QMap<QAction*,QPair<QObject*,Listener>>::iterator i =
       m_enableMap.begin(); i != m_enableMap.end(); ++i) {
    QAction* actionPtr = i.key();
    QPair<QObject*,Listener> pair = i.value();
    QObject* baseObjectPtr = pair.first;
    Listener enableFunction = pair.second;
    actionPtr->setEnabled(enableFunction(baseObjectPtr));
  }
```

同样，我们遍历复选映射。对每一项，我们查找部件和复选函数，调用该函数并用结果通过在动作对象指针上调用 setCheckable 和 setChecked 来复选菜单项。如果菜单项属于某个组，Qt 框架确保它标注单选按钮；否则标注复选框：

```
    for (QMap<QAction*,QPair<QObject*,Listener>>::iterator i =
         m_checkMap.begin(); i != m_checkMap.end(); ++i) {
      QAction* actionPtr = i.key();
      QPair<QObject*,Listener> pair = i.value();
      QObject* baseObjectPtr = pair.first;
      Listener checkFunction = pair.second;
      actionPtr->setCheckable(true);
      actionPtr->setChecked(checkFunction(baseObjectPtr));
    }
  }
```

基部件类

DocumentWidget 是处理文档的应用的骨架框架。它处理文档的加载与保存，并提供供子类重写的方法，对应 Cut、Copy、Paste 和 Delete 菜单项。

前面的 MainWindow 类处理窗口边框及其菜单和工具栏，而 DocumentWidget 类负责绘制窗口的内容。思路是：MainWindow 的子类创建一个 DocumentWidget 子类的对象，并把它放到窗口中央。参见后面各节中 DrawingWindow 和 EditorWindow 的构造函数。

DocumentWidget.h

```
#ifndef DOCUMENTWIDGET_H
#define DOCUMENTWIDGET_H

#include "Listener.h"
#include <QWidget>
#include <QtWidgets>
#include <FStream>
using namespace std;

class DocumentWidget : public QWidget {
  Q_OBJECT
```

构造函数接受应用名称（显示在窗口顶部横幅上）、用标准文件对话框加载和保存文档时使用的文件名掩码，以及指向潜在父部件（通常是外层主窗口）的指针：

```
public:
  DocumentWidget(const QString& name, const QString& fileMask,
                 QWidget* parentWidgetPtr);
  ~DocumentWidget();
```

setFilePath 方法设置当前文档的路径。路径显示在窗口顶部横幅上，并作为标准加载和保存对话框中的默认路径：

```
protected:
  void setFilePath(QString filePath);
```

文档更改后，modified 标志（有时称为 dirty 标志）被置位。这会使窗口顶部横幅上的文件路径旁出现一个星号（*），并启用 Save 和 SaveAs 菜单项：

```
public:
  void setModifiedFlag(bool flag);
```

setMainWindowTitle 方法是一个辅助方法，用于拼出窗口的标题。它由文件路径以及一个可能的星号（*）组成，星号表示 modified 标志已置位：

```
private:
  void setMainWindowTitle();
```

closeEvent 方法从 QWidget 重写而来，在用户关闭窗口时被调用。通过设置 eventPtr 参数的字段，可以阻止关闭。例如，如果文档尚未保存，可以询问用户是要保存文档，还是取消关闭窗口：

```
public:
  virtual void closeEvent(QCloseEvent* eventPtr);
```

isClearOk 方法是一个辅助方法，当用户试图在未保存文档的情况下关闭窗口或退出应用时，它会显示一个消息框：

```
private:
  bool isClearOk(QString title);
```

用户选择菜单项或点击工具栏图标时，框架会调用下列方法。为此，我们把方法标记为槽（slot），这是 connect 调用中 SLOT 宏所必需的：

```
public slots:
  virtual void onNew();
  virtual void onOpen();
  virtual bool onSave();
  virtual bool onSaveAs();
  virtual void onExit();
```

文档未更改时无需保存。此时应禁用 Save 和 SaveAs 菜单项及工具栏图标。onMenuShow 在 File 菜单可见之前调用 isSaveEnabled 方法。只有当文档已更改且需要保存时，它才返回 true：

```
virtual bool isSaveEnabled();
```

tryWriteFile 方法是一个辅助方法，它尝试写文件。如果失败，消息框会显示一条错误消息：

```
private:
    bool tryWriteFile(QString filePath);
```

下面的方法是打算被子类重写的虚方法，在用户选择 New、Save、SaveAs 和 Open 菜单项时被调用：

```
protected:
  virtual void newDocument() = 0;
  virtual bool writeFile(const QString& filePath) = 0;
  virtual bool readFile(const QString& filePath) = 0;
```

下面的方法在 Edit 菜单可见之前被调用，它们决定 Cut、Copy、Paste 和 Delete 菜单项是否应启用：

```
public:
  virtual bool isCutEnabled();
  virtual bool isCopyEnabled();
  virtual bool isPasteEnabled();
  virtual bool isDeleteEnabled();
```

下面的方法在用户选择 Cut、Copy、Paste 和 Delete 菜单项或工具栏图标时被调用：

```
public slots:
  virtual void onCut();
  virtual void onCopy();
  virtual void onPaste();
  virtual void onDelete();
```

m_applicationName 字段保存应用的名称，而不是文档的名称。后面各节中名称将是 Drawing 和 Editor。m_fileMask 字段保存用标准对话框加载和保存文档时使用的掩码。举例来说，假设文档以 .abc 结尾，那么掩码可以是 Abc files (*.abc)。m_filePath 字段保存当前文档的路径。当文档是新的且尚未保存时，该字段保存空字符串。最后，m_modifiedFlag 在文档已修改、需要在应用退出前保存时为 true：

```
private:
  QString m_applicationName, m_fileMask, m_filePath;
  bool m_modifiedFlag = false;
};
```

最后还有一些重载的辅助运算符。加法和减法运算符把一个点与一个尺寸相加减，把一个矩形与一个尺寸相加减：

```
QPoint& operator+=(QPoint& point, const QSize& size);
QPoint& operator-=(QPoint& point, const QSize& size);

QRect& operator+=(QRect& rect, int size);
QRect& operator-=(QRect& rect, int size);
```

writePoint 和 readPoint 方法从输入流写入和读取一个点：

```
void writePoint(ofstream& outStream, const QPoint& point);
void readPoint(ifstream& inStream, QPoint& point);
```

writeColor 和 readColor 方法从输入流写入和读取一个颜色：

```
void writeColor(ofstream& outStream, const QColor& color);
void readColor(ifstream& inStream, QColor& color);
```

makeRect 方法创建一个以 point 为中心、以 size 为大小的矩形：

```
QRect makeRect(const QPoint& centerPoint, int halfSide);
#endif // DOCUMENTWIDGET_H
```

DocumentWidget.cpp

```
#include <QtWidgets>
#include <QMessageBox>

#include "MainWindow.h"
#include "DocumentWidget.h"
```

构造函数设置应用名称、保存和加载标准对话框的文件掩码，以及指向外层父部件（通常是外层主窗口）的指针：

```
DocumentWidget::DocumentWidget(const QString& name,
                               const QString& fileMask, QWidget* parentWidgetPtr)
 :m_applicationName(name),
  m_fileMask(fileMask),
  QWidget(parentWidgetPtr) {
  setMainWindowTitle();
}
```

析构函数什么也不做，包含它只是为了完整起见：

```
DocumentWidget::~DocumentWidget() {
  // Empty.
}
```

setFilePath 方法调用 setMainWindowTitle 来更新窗口顶部横幅上的文本：

```
void DocumentWidget::setFilePath(QString filePath) {
  m_filePath = filePath;
  setMainWindowTitle();
}
```

setModifiedFlag 方法同样调用 setMainWindowTitle 来更新窗口顶部横幅上的文本。此外，它

还调用父部件的 onMenuShow 来更新工具栏的图标：

```
void DocumentWidget::setModifiedFlag(bool modifiedFlag) {
  m_modifiedFlag = modifiedFlag;
  setMainWindowTitle();
  ((MainWindow*) parentWidget())->onMenuShow();
}
```

工具栏顶部横幅上显示的标题是应用名称、文档文件路径（如果非空），以及文档已修改但尚未保存时的一个星号：

```
void DocumentWidget::setMainWindowTitle() {
  QString title= m_applicationName +
          (m_filePath.isEmpty() ? "" : (" [" + m_filePath + "]"))+
          (m_modifiedFlag ? " *" : "");
  this->parentWidget()->setWindowTitle(title);
}
```

isClearOk 方法在文档已修改但尚未保存时显示一个消息框。用户可以选择以下按钮之一：

- **Yes**：保存文档，应用退出。不过如果保存失败，会显示一条错误消息，应用不会退出。
- **No**：应用不保存文档直接退出。
- **Cancel**：取消应用的关闭。文档不保存。

```
bool DocumentWidget::isClearOk(QString title) {
  if (m_modifiedFlag) {
    QMessageBox messageBox(QMessageBox::Warning,
                           title, QString());
    messageBox.setText(tr("The document has been modified."));
    messageBox.setInformativeText(
              tr("Do you want to save your changes?"));
    messageBox.setStandardButtons(QMessageBox::Yes |
                          QMessageBox::No | QMessageBox::Cancel);
    messageBox.setDefaultButton(QMessageBox::Yes);

    switch (messageBox.exec()) {
      case QMessageBox::Yes:
        return onSave();

      case QMessageBox::No:
        return true;

      case QMessageBox::Cancel:
        return false;
    }
  }
```

```
    return true;
  }
```

如果文档被清除，则调用 newDocument——该方法打算被子类重写以执行应用专属的初始化。此外，modified 标志和文件路径被清除。最后，调用 Qt 的 update 方法强制重绘窗口的内容：

```
  void DocumentWidget::onNew() {
    if (isClearOk(tr("New File"))) {
      newDocument();
      setModifiedFlag(false);
      setFilePath(QString());
      update();
    }
  }
```

如果文档被清除，onOpen 用标准打开对话框获得文档的文件路径：

```
  void DocumentWidget::onOpen() {
    if (isClearOk(tr("Open File"))) {
      QString file =
        QFileDialog::getOpenFileName(this, tr("Open File"),
                    tr("C:\Users\Stefan\Documents\"
                       "A A_Cpp_By_Example\Draw"),
                m_fileMask + tr(";;Text files (*.txt)"));
```

如果文件成功读取，则清除 modified 标志、设置文件路径，并调用 update 强制重绘窗口：

```
    if (!file.isEmpty()) {
      if (readFile(file)) {
        setModifiedFlag(false);
        setFilePath(file);
        update();
      }
```

如果读取不成功，则显示一条带错误消息的消息框：

```
      else {
        QMessageBox messageBox;
        messageBox.setIcon(QMessageBox::Critical);
        messageBox.setText(tr("Read File"));
        messageBox.setInformativeText(tr("Could not read \"") +
                                      m_filePath  + tr("\""));
        messageBox.setStandardButtons(QMessageBox::Ok);
        messageBox.setDefaultButton(QMessageBox::Ok);
        messageBox.exec();
      }
    }
```

```
    }
  }
```

isSaveEnabled 方法只返回 m_modifiedFlag 的值。不过，我们需要这个方法监听器才能工作：

```
bool DocumentWidget::isSaveEnabled() {
  return m_modifiedFlag;
}
```

用户选择 Save 或 SaveAs 菜单项或工具栏图标时调用 onSave 方法。如果文档已经命名，我们就直接尝试写文件；如果尚未命名，则调用 onSaveAs，为用户显示标准的保存对话框：

```
bool DocumentWidget::onSave() {
  if (!m_filePath.isEmpty()) {
    return tryWriteFile(m_filePath);
  }
  else {
    return onSaveAs();
  }
}
```

用户选择 SaveAs 菜单项（该项没有工具栏图标）时调用 onSaveAs 方法。它打开标准打开对话框并尝试写文件。如果写入不成功，则返回 false。原因是：只有写入成功时 isClearOk 才会关闭窗口：

```
bool DocumentWidget::onSaveAs() {
  QString filePath =
          QFileDialog::getSaveFileName(this, tr("Save File"),
               tr("C:\Users\Stefan\Documents\"
                  "A A_Cpp_By_Example\Draw"),
            m_fileMask + tr(";;Text files (*.txt)"));

  if (!filePath.isEmpty()) {
    return tryWriteFile(filePath);
  }
  else {
    return false;
  }
}
```

tryWriteFile 方法通过调用 write——该方法打算被子类重写——尝试写文件。如果成功，设置 modified 标志和文件路径。如果文件写入不成功，则显示一条带错误消息的消息框：

```
bool DocumentWidget::tryWriteFile(QString filePath) {
  if (writeFile(filePath)) {
    setModifiedFlag(false);
    setFilePath(filePath);
```

```
      return true;
    }
    else {
      QMessageBox messageBox;
      messageBox.setIcon(QMessageBox::Critical);
      messageBox.setText(tr("Write File"));
      messageBox.setInformativeText(tr("Could not write \"") +
                                    filePath  + tr("\""));
      messageBox.setStandardButtons(QMessageBox::Ok);
      messageBox.setDefaultButton(QMessageBox::Ok);
      messageBox.exec();
      return false;
    }
  }
```

用户选择 Exit 菜单项时调用 onExit 方法。它检查是否可以关闭窗口，如果可以则退出应用：

```
  void DocumentWidget::onExit() {
    if (isClearOk(tr("Exit"))) {
      qApp->exit(0);
    }
  }
```

isCutEnabled 和 isDeleteEnabled 的默认行为是调用 isCopyEnabled，因为它们通常在相同的条件下启用：

```
  bool DocumentWidget::isCutEnabled() {
    return isCopyEnabled();
  }

  bool DocumentWidget::isDeleteEnabled() {
    return isCopyEnabled();
  }
```

onCut 的默认行为是直接调用 onCopy 和 onDelete：

```
  void DocumentWidget::onCut() {
    onCopy();
    onDelete();
  }
```

剪切和复制方法组中其余方法的默认行为是返回 false 并什么也不做，这会让菜单项保持禁用状态，除非子类重写这些方法：

```
  bool DocumentWidget::isCopyEnabled() {
    return false;
  }
```

```
void DocumentWidget::onCopy() {
  // Empty.
}

bool DocumentWidget::isPasteEnabled() {
  return false;
}

void DocumentWidget::onPaste() {
  // Empty.
}

void DocumentWidget::onDelete() {
  // Empty.
}
```

最后，用户试图关闭窗口时调用 closeEvent。如果窗口可以清除，就在 eventPtr 上调用 accept——这会关闭窗口——并对全局对象 qApp 调用 exit，使应用退出：

```
void DocumentWidget::closeEvent(QCloseEvent* eventPtr) {
  if (isClearOk(tr("Close Window"))) {
    eventPtr->accept();
    qApp->exit(0);
  }
```

如果窗口不能清除，就在 eventPtr 上调用 ignore，窗口保持打开（应用继续运行）：

```
  else {
    eventPtr->ignore();
  }
}
```

此外，还有一组用于处理点、尺寸、矩形和颜色的辅助函数。下面的运算符把一个点与一个尺寸相加减，并返回得到的点：

```
QPoint& operator+=(QPoint& point, const QSize& size) {
  point.setX(point.x() + size.width());
  point.setY(point.y() + size.height());
  return point;
}

QPoint& operator-=(QPoint& point, const QSize& size) {
  point.setX(point.x() - size.width());
  point.setY(point.y() - size.height());
  return point;
}
```

下面的运算符把一个整数与矩形相加减，并返回得到的矩形。加法运算符把矩形的大小向每个方向扩大，减法运算符把矩形向每个方向缩小：

```
QRect& operator+=(QRect& rect, int size) {
  rect.setLeft(rect.left() - size);
  rect.setTop(rect.top() - size);
  rect.setWidth(rect.width() + size);
  rect.setHeight(rect.height() + size);
  return rect;
}

QRect& operator-=(QRect& rect, int size) {
  rect.setLeft(rect.left() + size);
  rect.setTop(rect.top() + size);
  rect.setWidth(rect.width() - size);
  rect.setHeight(rect.height() - size);
  return rect;
}
```

writePoint 和 readPoint 函数从文件写入和读取一个点。它们分别写入和读取 x 和 y 坐标：

```
void writePoint(ofstream& outStream, const QPoint& point) {
  int x = point.x(), y = point.y();
  outStream.write((char*) &x, sizeof x);
  outStream.write((char*) &y, sizeof y);
}

void readPoint(ifstream& inStream, QPoint& point) {
  int x, y;
  inStream.read((char*) &x, sizeof x);
  inStream.read((char*) &y, sizeof y);
  point = QPoint(x, y);
}
```

writeColor 和 readColor 函数从文件写入和读取一个颜色。一个颜色由红、绿、蓝三个分量组成，每个分量是 0 到 255（含两端）之间的整数值。这两个函数从文件流写入和读取各分量：

```
void writeColor(ofstream& outStream, const QColor& color) {
  int red = color.red(), green = color.green(),
  blue = color.blue();
  outStream.write((char*) &red, sizeof red);
  outStream.write((char*) &green, sizeof green);
  outStream.write((char*) &blue, sizeof blue);
}

void readColor(ifstream& inStream, QColor& color) {
  int red, green, blue;
```

```
    inStream.read((char*) &red, sizeof red);
    inStream.read((char*) &green, sizeof green);
    inStream.read((char*) &blue, sizeof blue);
```

读取完各分量后，我们创建一个 QColor 对象并把它赋给 color 参数：

```
    color = QColor(red, green, blue);
  }
```

makeRect 函数创建一个以该点为中心的矩形：

```
  QRect makeRect(const QPoint& centerPoint, int halfSide) {
    return QRect(centerPoint.x() - halfSide,
                 centerPoint.y() - halfSide,
                 2 * halfSide, 2 * halfSide);
  }
```

构建画图程序

现在我们开始一个新工程，利用上一节的主窗口类和文档部件类——也就是画图程序。本章我们先实现一个基础版本，下一章再继续构建更高级的版本。用本章的画图程序，我们可以用不同颜色绘制直线、矩形和椭圆，还可以保存和加载我们的绘图。注意，本工程中的窗口类和部件类继承自上一节的 MainWindow 和 DocumentWidget 类。

Figure 基类

应用程序的图形构成一个类层次结构，其中 Figure 是基类，它的子类是 Line、RectangleX 和 EllipseX，稍后介绍。我们不能把类命名为 Rectangle 和 Ellipse，因为会与同名的 Qt 方法冲突。我选择简单地给名字加一个「X」。

Figure 类是抽象的，也就是说我们不能创建该类的对象，只能把它用作基类供子类继承。

Figure.h

```
  #ifndef FIGURE_H
  #define FIGURE_H

  enum FigureId {LineId, RectangleId, EllipseId};

  #include <QtWidgets>
  #include <FStream>
  using namespace std;

  class Figure {
```

```
public:
  Figure();
```

下面的方法是纯虚的，也就是说它们不需要在这里定义。含有一个纯虚方法的类就成为抽象类。子类必须定义其所有基类的全部纯虚方法，否则自己也会成为抽象类。这样就保证了所有非抽象类的方法都有定义。每个子类定义 getId 并返回其类的标识枚举：

```
virtual FigureId getId() const = 0;
```

每个图形有一个首点和末点，具体如何定义由各子类自行决定：

```
virtual void initializePoints(QPoint point) = 0;
virtual void setLastPoint(QPoint point) = 0;
```

isClick 方法在图形被该点命中时返回 true：

```
virtual bool isClick(QPoint mousePoint) = 0;
```

move 方法把图形移动一定的距离：

```
virtual void move(QSize distance) = 0;
```

draw 方法把图形绘制到画笔区域上：

```
virtual void draw(QPainter &painter) const = 0;
```

write 和 read 方法从文件写入和读取图形；write 是常量方法，因为它不改变图形：

```
virtual bool write(ofstream& outStream) const;
virtual bool read(ifstream& inStream);
```

color 方法返回图形的颜色。它有两个版本：第一个是常量版本，返回对常量 QColor 对象的引用；第二个是非常量版本，返回对非常量对象的引用：

```
const QColor& color() const {return m_color;}
QColor& color() {return m_color;}
```

filled 方法只适用于二维图形（矩形和椭圆）。如果图形被填充，它们返回 true。注意，第二个版本返回对 m_filled 字段的引用，这允许方法调用者修改 m_filled 的值：

```
virtual bool filled() const {return m_filled;}
virtual bool& filled() {return m_filled;}
```

图形被标记时，会在其四角绘制小方块。方块的边长由静态字段 Tolerance 定义：

```
static const int Tolerance;
```

writeColor 和 readColor 方法是读写颜色的辅助方法。它们是静态的，因为它们由 Figure 类层次结构之外的方法调用：

```
  static void writeColor(ofstream& outStream,
                         const QColor& color);
  static void readColor(ifstream& inStream, QColor& color);
```

每个图形都有一个颜色，并且可以被标记或填充：

```
  private:
    QColor m_color;
    bool m_marked = false, m_filled = false;
  };

#endif
```

Figure.cpp 文件存放 Figure 类的定义。它定义 Tolerance 字段以及 write 和 read 方法。

Figure.cpp

```
#include "..\MainWindow\DocumentWidget.h"
#include "Figure.h"
```

Tolerance 是静态字段，必须在全局空间中定义并初始化。我们把标记方块的尺寸定义为 6 像素：

```
const int Figure::Tolerance(6);
```

默认构造函数只在从文件读取图形时被调用：

```
Figure::Figure() {
  // Empty.
}
```

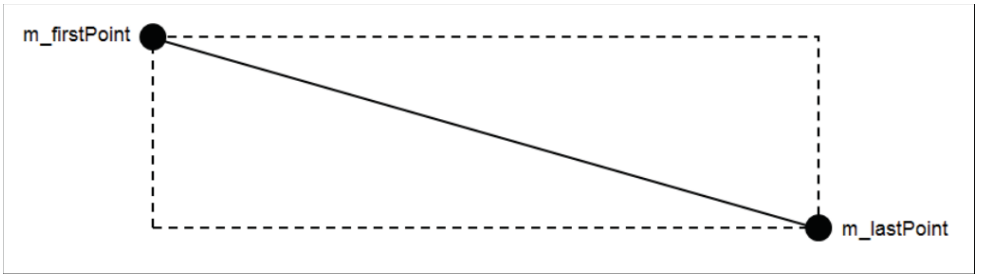
write 和 read 方法写入和读取图形的颜色以及图形是否被填充：

```
bool Figure::write(ofstream& outStream) const {
  writeColor(outStream, m_color);
  outStream.write((char*) &m_filled, sizeof m_filled);
  return ((bool) outStream);
}

bool Figure::read(ifstream& inStream) {
  readColor(inStream, m_color);
  inStream.read((char*) &m_filled, sizeof m_filled);
  return ((bool) inStream);
}
```

Line 子类

Line 类是 Figure 的子类。它通过定义 Figure 的每个纯虚方法而成为非抽象类。直线画在两个端点之间，端点由 Line 中的 m_firstPoint 和 m_lastPoint 字段表示：



Line.h

```
#ifndef LINE_H
#define LINE_H

#include <FStream>
using namespace std;

#include "Figure.h"

class Line : public Figure {
  public:
```

默认构造函数只在从文件读取 Line 对象时被调用；getId 只返回直线的标识枚举：

```
Line();
FigureId getId() const {return LineId;}
```

直线有两个端点。两个点都在直线创建时设置，之后当用户拖动第二个点时修改它：

```
void initializePoints(QPoint point);
void setLastPoint(QPoint point);
```

isClick 方法在鼠标点击位于直线上（有一定容差）时返回 true：

```
bool isClick(QPoint mousePoint);
```

move 方法把直线（包括两个端点）移动给定的距离：

```
void move(QSize distance);
```

draw 方法把直线画到 QPainter 对象上：

```
    void draw(QPainter& painter) const;
```

write 和 read 方法从文件流写入和读取直线的端点：

```
    bool write(ofstream& outStream) const;
    bool read(ifstream& inStream);
```

直线的首点和末点保存在 Line 对象中：

```
      private:
        QPoint m_firstPoint, m_lastPoint;
      };

    #endif
```

Line.cpp 文件定义 Line 类的方法。

Line.cpp

```
    #include "..\MainWindow\DocumentWidget.h"
    #include "Line.h"

    Line::Line() {
      // Empty.
    }
```

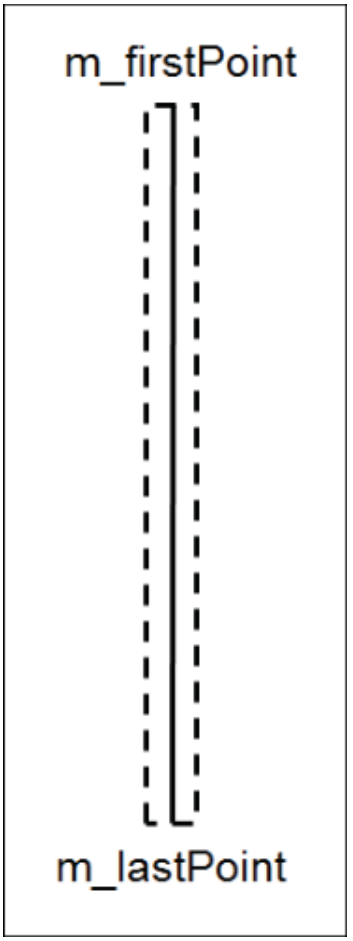
initializePoints 方法在用户向绘图添加一条新直线时被调用，它设置直线的两个端点：

```
    void Line::initializePoints(QPoint point) {
      m_firstPoint = point;
      m_lastPoint = point;
    }
```

setLastPoint 方法在用户添加直线并修改其形状时被调用，它设置末点：

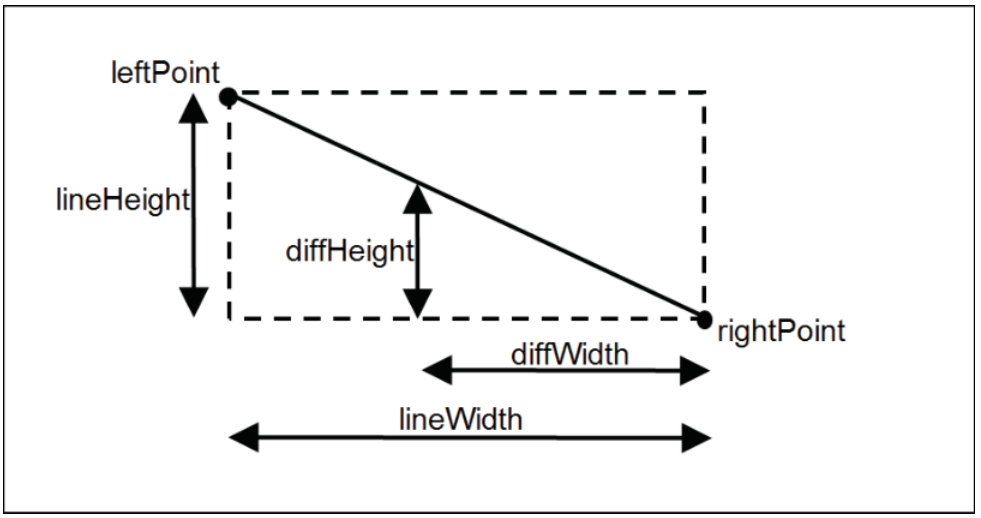
```
    void Line::setLastPoint(QPoint point) {
      m_lastPoint = point;
    }
```

isClick 方法测试用户是否用鼠标点击了直线。有两种情形需要考虑。第一种是直线完全竖直的特殊情况，即两个端点的 x 坐标相等。我们用 Qt 的 QRect 类创建一个包围直线的矩形，并测试该点是否被矩形包围：




```
bool Line::isClick(QPoint mousePoint) {
  if (m_firstPoint.x() == m_lastPoint.x()) {
    QRect lineRect(m_firstPoint, m_lastPoint);
    lineRect.normalized();
    lineRect += Tolerance;
    return lineRect.contains(mousePoint);
  }
```

在直线不是竖直的一般情形中，我们首先创建一个包围矩形，并测试鼠标点是否在其中。如果在，我们把 leftPoint 设为 firstPoint 和 lastPoint 中最靠左的点，把 rightPoint 设为最靠右的点。然后计算包围矩形的宽度（lineWidth）和高度（lineHeight），以及 rightPoint 与 mousePoint 在 x 和 y 方向上的距离（diffWidth 和 diffHeight）：



根据相似性，如果鼠标指针命中直线，则下面的等式成立：

$$\frac{diffWidth}{diffHeight} = \frac{lineWidth}{lineHeight}$$

$$diffHeight - \frac{lineHeight}{lineWidth} diffWidth = 0$$

不过，要让左边表达式的值恰好为零，用户必须精确地点在直线上。因此，我们允许一个小的容差，使用 Tolerance 字段：

$$\left| diffHeight - \frac{lineHeight}{lineWidth} diffWidth \right| \leq Tolerance$$

```
else {
  QPoint leftPoint = (m_firstPoint.x() < m_lastPoint.x())
                   ? m_firstPoint : m_lastPoint,
         rightPoint = (m_firstPoint.x() < m_lastPoint.x())
                    ? m_lastPoint : m_firstPoint;

  if ((leftPoint.x() <= mousePoint.x()) &&
      (mousePoint.x() <= rightPoint.x())) {
    int lineWidth = rightPoint.x() - leftPoint.x(),
```

```
            lineHeight = rightPoint.y() - leftPoint.y();

        int diffWidth = mousePoint.x() - leftPoint.x(),
            diffHeight = mousePoint.y() - leftPoint.y();
```

我们必须把 lineHeight 转换为 double 才能进行非整数除法：

```
      return (fabs(diffHeight - (((double) lineHeight) /
                  lineWidth) * diffWidth) <= Tolerance);
    }
```

如果鼠标点位于包围直线的矩形之外，我们直接返回 false：

```
          return false;
        }
    }
```

move 方法简单地移动直线的两个端点：

```
    void Line::move(QSize distance) {
      m_firstPoint += distance;
      m_lastPoint += distance;
    }
```

绘制直线时，我们设置画笔颜色并画出直线。Figure 类的 color 方法返回直线的颜色：

```
    void Line::draw(QPainter& painter) const {
      painter.setPen(color());
      painter.drawLine(m_firstPoint, m_lastPoint);
    }
```

写入直线时，我们首先调用 Figure 中的 write 来写入图形的颜色，然后写入直线的端点。最后返回输出流的布尔值——写入成功时为 true：

```
    bool Line::write(ofstream& outStream) const {
      Figure::write(outStream);
      writePoint(outStream, m_firstPoint);
      writePoint(outStream, m_lastPoint);
      return ((bool) outStream);
    }
```

同样，读取直线时，我们首先调用 Figure 中的 read 来读取直线的颜色，然后读取直线的端点，并返回输入流的布尔值：

```
    bool Line::read(ifstream& inStream) {
      Figure::read(inStream);
      readPoint(inStream, m_firstPoint);
      readPoint(inStream, m_lastPoint);
```

```
    return ((bool) inStream);
  }
```

Rectangle 子类

RectangleX 是 Figure 的子类，处理矩形。与 Line 类似，它也持有两个点，表示矩形相对的两个角：

Rectangle.h

```
  #ifndef RECTANGLE_H
  #define RECTANGLE_H

  #include <FStream>
  using namespace std;

  #include "Figure.h"

  class RectangleX : public Figure {
    public:
```

与前面的 Line 类类似，RectangleX 有一个从文件读取对象时使用的默认构造函数：

```
  RectangleX();
  virtual FigureId getId() const {return RectangleId;}

  RectangleX(const RectangleX& rectangle);

  virtual void initializePoints(QPoint point);
  virtual void setLastPoint(QPoint point);
  virtual bool isClick(QPoint mousePoint);
  virtual void move(QSize distance);
  virtual void draw(QPainter& painter) const;
  virtual bool write(ofstream& outStream) const;
  virtual bool read(ifstream& inStream);

    protected:
      QPoint m_topLeft, m_bottomRight;
    };

  #endif
```

Rectangle.cpp

```
#include "..\MainWindow\DocumentWidget.h"
#include "Rectangle.h"

RectangleX::RectangleX() {
  // Empty.
}
```

initializePoints 和 setLastPoint 方法的工作方式与 Line 中的对应方法类似：initializePoints 设置两个角点，而 setLastPoint 设置最后一个角点：

```
void RectangleX::initializePoints(QPoint point) {
  m_topLeft = point;
  m_bottomRight = point;
}

void RectangleX::setLastPoint(QPoint point) {
  m_bottomRight = point;
}
```

isClick 方法比 Line 中的对应方法简单：

```
bool RectangleX::isClick(QPoint mousePoint) {
  QRect areaRect(m_topLeft, m_bottomRight);
```

如果矩形被填充，我们直接调用 QRect 的 contains 检查鼠标点击是否命中矩形：

```
if (filled()) {
  return areaRect.contains(mousePoint);
}
```

如果矩形未被填充，我们需要检查鼠标是否点击在矩形的边框上。为此，我们创建两个稍小和稍大的矩形。如果鼠标点击命中较大的矩形而未命中较小的矩形，我们就认为点击命中了矩形边框：

```
  else {
    QRect largeAreaRect(areaRect), smallAreaRect(areaRect);
    largeAreaRect += Tolerance;
    smallAreaRect -= Tolerance;
    return largeAreaRect.contains(mousePoint) &&
           !smallAreaRect.contains(mousePoint);
  }

  return false;
}
```

移动矩形时，我们简单地移动首末两个角点：

```
void RectangleX::move(QSize distance) {
  addSizeToPoint(m_topLeft, distance);
  addSizeToPoint(m_bottomRight, distance);
}
```

绘制矩形时，我们首先通过调用 Figure 中的 color 设置画笔颜色：

```
void RectangleX::draw(QPainter& painter) const {
  painter.setPen(color());
```

如果矩形被填充，我们直接调用 QPainter 对象的 fillRect：

```
if (filled()) {
  painter.fillRect(QRect(m_topLeft, m_bottomRight), color());
}
```

如果矩形未被填充，我们禁用画刷使矩形成为空心，然后调用 QPainter 对象的 drawRect 画出矩形边框：

```
  else {
    painter.setBrush(Qt::NoBrush);
    painter.drawRect(QRect(m_topLeft, m_bottomRight));
  }
}
```

write 方法首先调用 Figure 中的 write，然后写入矩形的首末角点：

```
bool RectangleX::write(ofstream& outStream) const {
  Figure::write(outStream);
  writePoint(outStream, m_topLeft);
  writePoint(outStream, m_bottomRight);
  return ((bool) outStream);
}
```

同样，read 首先调用 Figure 中的 read，然后读取矩形的首末角点：

```
bool RectangleX::read (ifstream& inStream) {
  Figure::read(inStream);
  readPoint(inStream, m_topLeft);
  readPoint(inStream, m_bottomRight);
  return ((bool) inStream);
}
```

Ellipse 子类

EllipseX 是 RectangleX 的子类，处理椭圆。RectangleX 的部分功能在 EllipseX 中得到复用。更具体地说，initializePoints、setLastPoint、move、write 和 read 都是从 RectangleX 重写而来的。

Ellipse.h

```
#ifndef ELLIPSE_H
#define ELLIPSE_H

#include "Rectangle.h"

class EllipseX : public RectangleX {
  public:
    EllipseX();
    FigureId getId() const {return EllipseId;}

    EllipseX(const EllipseX& ellipse);

    bool isClick(QPoint mousePoint);
    void draw(QPainter& painter) const;
  };

#endif
```

Ellipse.cpp

```
#include "..\MainWindow\DocumentWidget.h"
#include "Ellipse.h"

EllipseX::EllipseX() {
  // Empty.
}
```

EllipseX 的 isClick 方法与 RectangleX 中的对应方法类似。我们用 Qt 的 QRegion 类创建椭圆区域，并把它们与鼠标点击进行比较：

```
bool EllipseX::isClick(QPoint mousePoint) {
  QRect normalRect(m_topLeft, m_bottomRight);
  normalRect.normalized();
```

如果椭圆被填充，我们创建一个椭圆区域并测试鼠标点击是否命中该区域：

```
if (filled()) {
  QRegion normalEllipse(normalRect, QRegion::Ellipse);
  return normalEllipse.contains(mousePoint);
```

```
  }
```

如果椭圆未被填充，我们创建稍小和稍大的椭圆区域。如果鼠标点击命中较大的区域而未命中较小的区域，我们就认为点击命中了椭圆边框：

```
    else {
      QRect largeRect(normalRect), smallRect(normalRect);
      largeRect += Tolerance;
      smallRect -= Tolerance;

      QRegion largeEllipse(largeRect, QRegion::Ellipse),
              smallEllipse(smallRect, QRegion::Ellipse);

      return (largeEllipse.contains(mousePoint) &&
              !smallEllipse.contains(mousePoint));
    }
  }
```

绘制椭圆时，我们首先通过调用 Figure 中的 color 设置画笔颜色：

```
  void EllipseX::draw(QPainter& painter) const {
    painter.setPen(color());
```

如果椭圆被填充，我们设置画刷并绘制椭圆：

```
  if (filled()) {
    painter.setBrush(color());
    painter.drawEllipse(QRect(m_topLeft, m_bottomRight));
  }
```

如果椭圆未被填充，我们把画刷设为空心并绘制椭圆边框：

```
    else {
      painter.setBrush(Qt::NoBrush);
      painter.drawEllipse(QRect(m_topLeft, m_bottomRight));
    }
  }
```

绘制窗口

DrawingWindow 类是上一节 MainWindow 类的子类。

DrawingWindow.h

```
  #ifndef DRAWINGWINDOW_H
  #define DRAWINGWINDOW_H
```

```
#include <QMainWindow>
#include <QActionGroup>

#include "..\MainWindow\MainWindow.h"
#include "DrawingWidget.h"

class DrawingWindow : public MainWindow {
  Q_OBJECT

  public:
    DrawingWindow(QWidget* parentWidgetPtr = nullptr);
    ~DrawingWindow();

  public:
    void closeEvent(QCloseEvent *eventPtr)
         { m_drawingWidgetPtr->closeEvent(eventPtr); }

  private:
    DrawingWidget* m_drawingWidgetPtr;
    QActionGroup* m_figureGroupPtr;
  };

#endif // DRAWINGWINDOW_H
```

DrawingWindow.cpp

```
#include "..\MainWindow\DocumentWidget.h"
#include "DrawingWindow.h"
```

构造函数把窗口大小设为 1000 × 500 像素：

```
DrawingWindow::DrawingWindow(QWidget* parentWidgetPtr
                             /* = nullptr */)
 :MainWindow(parentWidgetPtr) {
  resize(1000, 500);
```

m_drawingWidgetPtr 字段初始化为指向一个 DrawingWidget 类的对象，该对象随后被设置为窗口的中央部分：

```
m_drawingWidgetPtr = new DrawingWidget(this);
setCentralWidget(m_drawingWidgetPtr);
```

标准 File 菜单被添加到窗口菜单栏：

```
addFileMenu();
```

然后我们添加应用专属的 Format 菜单。它与上一节 DocumentWidget 类的 onMenuShow 方

法连接：

```
{ QMenu* formatMenuPtr = menuBar()->addMenu(tr("F&ormat"));
  connect(formatMenuPtr, SIGNAL(aboutToShow()),
          this, SLOT(onMenuShow()));
```

Format 菜单包含颜色和填充菜单项：

```
addAction(formatMenuPtr, tr("&Color"),
          SLOT(onColor()), QKeySequence(Qt::ALT + Qt::Key_C),
          QString(), nullptr, tr("Figure Color"));
```

当画图程序的下一个图形是二维图形（矩形或椭圆）时，Fill 菜单项才会启用：

```
  addAction(formatMenuPtr, tr("&Fill"),
            SLOT(onFill()), QKeySequence(Qt::CTRL + Qt::Key_F),
            QString(), nullptr, tr("Figure Fill"),
            LISTENER(isFillEnabled));
}
```

对于图形菜单，我们为直线、矩形和椭圆菜单项创建一个新的动作组。同一时刻它们之中只能有一个被选中：

```
{ m_figureGroupPtr = new QActionGroup(this);

  QMenu* figureMenuPtr = menuBar()->addMenu(tr("F&igure"));
  connect(figureMenuPtr, SIGNAL(aboutToShow()),
          this, SLOT(onMenuShow()));
```

当前选中的菜单项应标注单选按钮：

```
    addAction(figureMenuPtr, tr("&Line"),
              SLOT(onLine()), QKeySequence(Qt::CTRL + Qt::Key_L),
              QString(), nullptr, tr("Line Figure"), nullptr,
              LISTENER(isLineChecked), m_figureGroupPtr);
    addAction(figureMenuPtr, tr("&Rectangle"),
              SLOT(onRectangle()),
              QKeySequence(Qt::CTRL + Qt::Key_R),
              QString(), nullptr, tr("Rectangle Figure"), nullptr,
              LISTENER(isRectangleChecked), m_figureGroupPtr);
    addAction(figureMenuPtr, tr("&Ellipse"),
              SLOT(onEllipse()),
              QKeySequence(Qt::CTRL + Qt::Key_E),
              QString(), nullptr, tr("Ellipse Figure"), nullptr,
              LISTENER(isEllipseChecked), m_figureGroupPtr);
  }
}
```

析构函数释放构造函数中动态分配的图形组：

```
DrawingWindow::~DrawingWindow() {
  delete m_figureGroupPtr;
}
```

绘制部件

DrawingWidget 是上一节 DocumentWidget 的子类。它处理鼠标输入、图形的绘制以及绘图的保存和加载，还提供决定菜单项何时应被标记和启用的方法。

DrawingWidget.h

```
#ifndef DRAWINGWIDGET_H
#define DRAWINGWIDGET_H

#include "..\MainWindow\MainWindow.h"
#include "..\MainWindow\DocumentWidget.h"
#include "Figure.h"

class DrawingWidget : public DocumentWidget {
  Q_OBJECT

  public:
    DrawingWidget(QWidget* parentWidgetPtr);
    ~DrawingWidget();
```

mousePressEvent、mouseReleaseEvent 和 mouseMoveEvent 是重写的方法，在用户按下或松开某个鼠标键或移动鼠标时被调用：

```
public:
  void mousePressEvent(QMouseEvent *eventPtr);
  void mouseReleaseEvent(QMouseEvent *eventPtr);
  void mouseMoveEvent(QMouseEvent *eventPtr);
```

paintEvent 方法在窗口需要重绘时被调用。原因可能有多种，例如用户可以改变窗口大小。重绘也可以由对 update 方法的调用强制触发，该调用最终会导致 paintEvent 被调用：

```
void paintEvent(QPaintEvent *eventPtr);
```

用户选择 New 菜单项时调用 newDocument 方法,选择 save 或 save as 菜单项时调用 writeFile，选择 open 菜单项时调用 readFile：

```
private:
  void newDocument() override;
  bool writeFile(const QString& filePath);
```

```
    bool readFile(const QString& filePath);
    Figure* createFigure(FigureId figureId);
```

用户选择颜色和填充菜单项时调用 onColor 和 onFill 方法：

```
public slots:
    void onColor();
    void onFill();
```

isFillEnabled 方法在用户选择 Format 菜单之前被调用。如果它返回 true，Fill 菜单项就会被启用：

```
DEFINE_LISTENER(DrawingWidget, isFillEnabled);
```

isLineChecked、isRectangleChecked 和 isEllipseChecked 方法也在 Figure 菜单可见之前被调用。如果这些方法返回 true，相应的菜单项就会标注单选按钮：

```
DEFINE_LISTENER(DrawingWidget, isLineChecked);
DEFINE_LISTENER(DrawingWidget, isRectangleChecked);
DEFINE_LISTENER(DrawingWidget, isEllipseChecked);
```

用户选择直线、矩形和椭圆菜单项时调用 onLine、onRectangle 和 onEllipse 方法：

```
void onLine();
void onRectangle();
void onEllipse();
```

运行时，应用可以处于 Idle、Create 或 Move 三种模式：

- **Idle**：应用等待用户输入。
- **Create**：用户正在向绘图添加新图形。发生在用户按下鼠标左键且未命中任何图形时。此时会添加一个新图形，并修改其端点，直到用户松开鼠标键。
- **Move**：用户正在移动一个图形。发生在用户按下鼠标左键并命中某个图形时。该图形被移动，直到用户松开鼠标键。

```
private:
    enum ApplicationMode {Idle, Create, Move};
    ApplicationMode m_applicationMode = Idle;
    void setApplicationMode(ApplicationMode mode);
```

m_currColor 字段保存用户接下来要添加的图形的颜色；m_currFilled 决定下一个图形（如果是矩形或椭圆）是否被填充。m_addFigureId 字段保存用户接下来要添加的图形类型（直线、矩形或椭圆）的标识整数：

```
QColor m_currColor = Qt::black;
bool m_currFilled = false;
FigureId m_addFigureId = LineId;
```

用户按下鼠标键并移动图形时，我们需要保存之前的鼠标点，以便计算自上次鼠标事件以来图形移动的距离：

```
QPoint m_mousePoint;
```

最后，m_figurePtrList 保存指向绘图中各图形的指针。绘图中最上层的图形位于列表末尾：

```
    QList<Figure*> m_figurePtrList;
  };

#endif // DRAWINGWIDGET_H
```

DrawingWidget.cpp

```
#include "..\MainWindow\DocumentWidget.h"
#include "DrawingWidget.h"

#include "Line.h"
#include "Rectangle.h"
#include "Ellipse.h"
```

构造函数调用基类 DocumentWidget 的构造函数，标题为 Drawing；它还把保存和加载掩码设为 Drawing files (*.drw)，也就是说标准保存和加载对话框默认选择的文件以后缀 drw 结尾：

```
DrawingWidget::DrawingWidget(QWidget* parentWidgetPtr)
 :DocumentWidget(tr("Drawing"), tr("Drawing files (*.drw)"),
                 parentWidgetPtr) {
  // Empty.
}
```

析构函数释放图形指针列表中的图形指针：

```
DrawingWidget::~DrawingWidget() {
  for (Figure* figurePtr : m_figurePtrList) {
    delete figurePtr;
  }
}
```

setApplicationMode 方法设置应用模式，并调用主窗口的 onMenuShow，使工具栏图标被正确启用：

```
void DrawingWidget::setApplicationMode(ApplicationMode mode) {
  m_applicationMode = mode;
  ((MainWindow*) parent())->onMenuShow();
}
```

用户选择 New 菜单项时调用 newDocument。图形指针列表中的图形被释放，列表本身也被清空。用户接下来要添加的图形是一条黑色直线，填充状态为 false：

```
void DrawingWidget::newDocument() {
  for (Figure* figurePtr : m_figurePtrList) {
    delete figurePtr;
  }
  m_figurePtrList.clear();

  m_currColor = Qt::black;
  m_addFigureId = LineId;
  m_currFilled = false;
}
```

用户选择 save 或 save as 菜单项时调用 writeFile 方法：

```
bool DrawingWidget::writeFile(const QString& filePath) {
  ofstream outStream(filePath.toStdString());
```

我们首先写入当前的颜色和填充状态，然后写入图形指针列表的大小以及各图形本身：

```
if (outStream) {
  writeColor(outStream, m_currColor);
  outStream.write((char*) &m_currFilled, sizeof m_currFilled);

  int size = m_figurePtrList.size();
  outStream.write((char*) &size, sizeof size);
```

对每个图形，我们先写入它的标识号，再写入图形本身：

```
  for (Figure* figurePtr : m_figurePtrList) {
    FigureId figureId = figurePtr->getId();
    outStream.write((char*) &figureId, sizeof figureId);
    figurePtr->write(outStream);
  }

  return ((bool) outStream);
}
```

如果文件无法打开，则返回 false：

```
  return false;
}
```

用户选择 open 菜单项时调用 readFile 方法。与前面的 writeFile 一样，我们读取颜色和填充状态、图形指针列表的大小以及各图形本身：

```
bool DrawingWidget::readFile(const QString& filePath) {
  ifstream inStream(filePath.toStdString());

  if (inStream) {
```

```
      readColor(inStream, m_currColor);
      inStream.read((char*) &m_currFilled, sizeof m_currFilled);

      int size;
      inStream.read((char*) &size, sizeof size);
```

读取图形时，我们先读取它的标识号，然后调用 createFigure 创建一个与该图形标识号对应的类的对象。接着调用图形指针上的 read 读取图形的字段。注意，我们其实并不知道（也不关心）它是哪种图形。我们只是简单地调用图形指针的 read，而这个指针实际上指向 Line、RectangleX 或 EllipseX 的对象：

```
      for (int count = 0; count < size; ++count) {
        FigureId figureId = (FigureId) 0;
        inStream.read((char*) &figureId, sizeof figureId);
        Figure* figurePtr = createFigure(figureId);
        figurePtr->read(inStream);
        m_figurePtrList.push_back(figurePtr);
      }

      return ((bool) inStream);
    }

    return false;
  }
```

createFigure 方法根据 figureId 参数的值动态创建 Line、RectangleX 或 EllipseX 类的对象：

```
  Figure* DrawingWidget::createFigure(FigureId figureId) {
    Figure* figurePtr = nullptr;

    switch (figureId) {
      case LineId:
        figurePtr = new Line();
        break;

      case RectangleId:
        figurePtr = new RectangleX();
        break;

      case EllipseId:
        figurePtr = new EllipseX();
        break;
    }

    return figurePtr;
  }
```

用户选择颜色菜单项时调用 onColor 方法。它设置用户接下来要添加的图形的颜色：

```
void DrawingWidget::onColor() {
  QColor newColor = QColorDialog::getColor(m_currColor, this);

  if (newColor.isValid() && (m_currColor != newColor)) {
    m_currColor = newColor;
    setModifiedFlag(true);
  }
}
```

isFillEnabled 方法在 Format 菜单可见之前被调用，如果用户接下来要添加的图形是矩形或椭圆，它返回 true：

```
bool DrawingWidget::isFillEnabled() {
  return (m_addFigureId == RectangleId) ||
         (m_addFigureId == EllipseId);
}
```

用户选择填充菜单项时调用 onFill 方法。它把 m_currFilled 字段取反。由于文档已受影响，它还设置 modified 标志：

```
void DrawingWidget::onFill() {
  m_currFilled = !m_currFilled;
  setModifiedFlag(true);
}
```

isLineChecked、isRectangleChecked 和 isEllipseChecked 方法在 Figure 菜单可见之前被调用。如果用户接下来要添加的图形正是相应图形，它们返回 true，菜单项就会标注单选按钮：

```
bool DrawingWidget::isLineChecked() {
  return (m_addFigureId == LineId);
}

bool DrawingWidget::isRectangleChecked() {
  return (m_addFigureId == RectangleId);
}

bool DrawingWidget::isEllipseChecked() {
  return (m_addFigureId == EllipseId);
}
```

用户选择 Figure 菜单中的菜单项时调用 onLine、onRectangle 和 onEllipse 方法。它们把用户接下来要添加的图形设置为相应的图形：

```
void DrawingWidget::onLine() {
  m_addFigureId = LineId;
```

```
}

void DrawingWidget::onRectangle() {
  m_addFigureId = RectangleId;
}

void DrawingWidget::onEllipse() {
  m_addFigureId = EllipseId;
}
```

用户每次按下某个鼠标键时都会调用 mousePressEvent 方法。首先，我们需要检查用户按下的是否是鼠标左键：

```
void DrawingWidget::mousePressEvent(QMouseEvent* eventPtr) {
  if (eventPtr->buttons() == Qt::LeftButton) {
```

在下面代码片段中对 mouseMoveEvent 的调用中，我们需要记录最新的鼠标点，以便计算鼠标移动之间的距离。因此，我们把 m_mousePoint 设为该鼠标点：

```
m_mousePoint = eventPtr->pos();
```

我们遍历图形指针列表，对每个图形调用 isClick，检查该图形是否被鼠标点击命中。我们需要以一种比较别扭的方式反向遍历，以便先找到最上层的图形。我们使用 reverse_iterator 类以及 rbegin 和 rend 方法来反向遍历：

```
for (QList<Figure*>::reverse_iterator iterator =
     m_figurePtrList.rbegin();
     iterator != m_figurePtrList.rend(); ++iterator) {
```

我们用解引用运算符（*）取得列表中的图形指针：

```
Figure* figurePtr = *iterator;
```

如果该图形被鼠标点击命中，我们把应用模式设为 Move，并通过调用列表的 removeOne 和 push_back 把该图形移到列表末尾，使它在绘图中显示为最上层。最后，我们中断循环，因为已经找到了要找的图形：

```
    if (figurePtr->isClick(m_mousePoint)) {
      setApplicationMode(Move);
      m_figurePtrList.removeOne(figurePtr);
      m_figurePtrList.push_back(figurePtr);
      break;
    }
  }
```

如果应用模式仍是 Idle（未发生移动），说明没有图形被鼠标点击命中。此时我们把应用模式设为 Create，并调用 createFigure 找一个图形来复制。然后设置该图形的颜色、填充状态以及

各点。最后，我们调用 push_back 把图形指针加入图形指针列表（它被加到列表末尾，以便显示在绘图最上层），并把 modified 标志设为 true，因为绘图已被修改：

```
        if (m_applicationMode == Idle) {
          setApplicationMode(Create);
          Figure* newFigurePtr = createFigure(m_addFigureId);
          newFigurePtr->color() = m_currColor;
          newFigurePtr->filled() = m_currFilled;
          newFigurePtr->initializePoints(m_mousePoint);
          m_figurePtrList.push_back(newFigurePtr);
          setModifiedFlag(true);
        }
      }
  }
```

用户每次移动鼠标时都会调用 mouseMoveEvent。首先，我们需要检查用户移动鼠标时按下的是否是鼠标左键：

```
  void DrawingWidget::mouseMoveEvent(QMouseEvent* eventPtr) {
    if (eventPtr->buttons() == Qt::LeftButton) {
      QPoint newMousePoint = eventPtr->pos();
```

然后检查应用模式。如果我们正在向绘图添加新图形，就修改它的末点：

```
  switch (m_applicationMode) {
    case Create:
      m_figurePtrList.back()->setLastPoint(m_mousePoint);
      break;
```

如果我们正在移动一个图形，我们计算自上次鼠标事件以来的距离，并移动位于图形指针列表末尾的图形。记住，在前面 mousePressEvent 中被鼠标点击命中的图形已被移到图形指针列表的末尾：

```
    case Move: {
        QSize distance(newMousePoint.x() - m_mousePoint.x(),
                       newMousePoint.y() - m_mousePoint.y());
        m_figurePtrList.back()->move(distance);
        setModifiedFlag(true);
      }
      break;
  }
```

最后，我们更新当前鼠标点，供下一次调用 mouseMoveEvent 使用。我们还调用 update 方法强制重绘窗口：

```
      m_mousePoint = newMousePoint;
      update();
```

```
  }
}
```

用户松开某个鼠标键时调用 mouseReleaseEvent 方法。我们把应用模式设为 Idle：

```
void DrawingWidget::mouseReleaseEvent(QMouseEvent* eventPtr) {
  if (eventPtr->buttons() == Qt::LeftButton) {
    setApplicationMode(Idle);
  }
}
```

每当窗口需要重绘时就会调用 paintEvent 方法。原因可能有多种：例如用户可能改变了窗口大小；也可能是 Qt QWidget 类中 update 调用的结果——它强制重绘窗口，并最终调用 paintEvent。

我们首先创建一个 QPainter 对象——它可以看作一块画布——并设置合适的渲染方式。然后遍历图形指针列表，绘制每个图形。这样，列表中的最后一个图形绘制在绘图的最上层：

```
void DrawingWidget::paintEvent(QPaintEvent* /* eventPtr */) {
  QPainter painter(this);
  painter.setRenderHint(QPainter::Antialiasing);
  painter.setRenderHint(QPainter::TextAntialiasing);

  for (Figure* figurePtr : m_figurePtrList) {
    figurePtr->draw(painter);
  }
}
```

main 函数

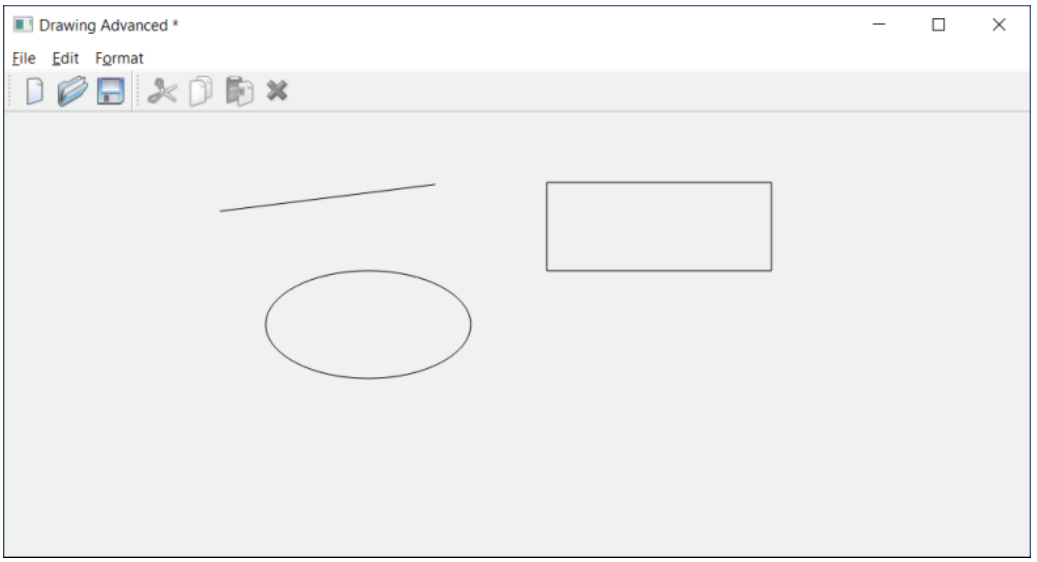
最后，我们在 main 函数中启动应用程序：创建应用对象，显示主窗口，并执行应用。

Main.cpp

```
#include "DrawingWindow.h"
#include <QApplication>

int main(int argc, char *argv[]) {
  QApplication application(argc, argv);
  DrawingWindow drawingWindow;
  drawingWindow.show();
  return application.exec();
}
```

得到如下输出：



构建编辑器

下一个应用是编辑器，用户可以在其中输入和编辑文本。当前的输入位置由插入符（caret）指示。可以用方向键移动插入符，也可以用鼠标点击移动。

Caret 类

Caret 类处理插入符，也就是标记下一个输入字符位置的闪烁竖线。

Caret.h

```
#ifndef CARET_H
#define CARET_H

#include <QObject>
#include <QWidget>
#include <QTimer>

class Caret : public QObject {
  Q_OBJECT

  public:
    Caret(QWidget* parentWidgetPtr = nullptr);
```

show 和 hide 方法显示和隐藏插入符。本应用中插入符从不隐藏；不过下一章的高级版本中，插入符在某些情况下会被隐藏：

```
void show();
void hide();
```

set 方法设置插入符的当前大小和位置，paint 则把插入符画到 QPainter 对象上：

```
void set(QRect rect);
void paint(QPainter& painter);
```

插入符每次闪烁时都会调用 onTimer 方法：

```
public slots:
  void onTimer(void);

private:
  QWidget* m_parentWidgetPtr;
```

插入符可见时 m_visible 字段为 true：

```
bool m_visible, m_blink;
```

m_rect 字段保存插入符的矩形区域；让插入符闪烁的定时器由 m_timer 字段处理：

```
    QRect m_rect;
    QTimer m_timer;
  };

#endif // CARET_H
```

Caret.cpp 文件存放 Caret 类方法的定义。

Caret.cpp

```
#include "Caret.h"
#include <QPainter>
```

构造函数把定时器信号连接到 onTimer，其效果是每次超时都会调用 onTimer。定时器随后被初始化为 500 毫秒。也就是说，onTimer 每 500 毫秒被调用一次，插入符每 500 毫秒显示和隐藏一次：

```
Caret::Caret(QWidget* parentWidgetPtr)
  :m_parentWidgetPtr(parentWidgetPtr) {
  m_timer.setParent(this);
  connect(&m_timer, SIGNAL(timeout()), this, SLOT(onTimer()));
  m_timer.start(500);
}
```

show 和 hide 方法设置 m_visible 字段，并通过在父窗口上调用 update 强制重绘插入符区域：

```
void Caret::show() {
  m_visible = true;
  m_parentWidgetPtr->update(m_rect);
}

void Caret::hide() {
  m_visible = false;
  m_parentWidgetPtr->update(m_rect);
}
```

set 方法设置插入符的大小和位置。不过，插入符的宽度总是设为 1，这使它看起来像一条细竖线：

```
void Caret::set(QRect rect) {
  m_rect = rect;
  m_rect.setWidth(1);
  m_parentWidgetPtr->update(m_rect);
}
```

onTimer 方法每 500 毫秒被调用一次。它把 m_blink 取反并强制重绘插入符。结果是插入符以一秒的间隔闪烁：

```
void Caret::onTimer(void) {
  m_blink = !m_blink;
  m_parentWidgetPtr->update(m_rect);
}
```

每当插入符需要重绘时就会调用 paint 方法。只有 m_visible 和 m_blink 都为 true 时才绘制插入符——即插入符被设为可见且正处于闪烁的可见区间。调用 paint 之前插入符区域会被清空，因此如果没有绘制发生，插入符就被清除：

```
void Caret::paint(QPainter& painter) {
  if (m_visible && m_blink) {
    painter.save();
    painter.setPen(Qt::NoPen);
    painter.setBrush(Qt::black);
    painter.drawRect(m_rect);
    painter.restore();
  }
}
```

绘制编辑器窗口

EditorWindow 是上一节 MainWindow 类的子类。它处理窗口的关闭，还处理按键事件。

EditorWindow.h

```
#ifndef EDITORWINDOW_H
#define EDITORWINDOW_H

#include <QMainWindow>
#include <QActionGroup>
#include <QPair>
#include <QMap>

#include "..\MainWindow\MainWindow.h"
#include "EditorWidget.h"

class EditorWindow : public MainWindow {
  Q_OBJECT

  public:
    EditorWindow(QWidget* parentWidgetPtr = nullptr);
    ~EditorWindow();
```

用户每次按下按键都会调用 keyPressEvent 方法；用户试图关闭窗口时调用 closeEvent 方法：

```
  protected:
    void keyPressEvent(QKeyEvent* eventPtr);
    void closeEvent(QCloseEvent* eventPtr);

  private:
    EditorWidget* m_editorWidgetPtr;
  };

#endif // EDITORWINDOW_H
```

EditorWindow 类实际上相当小。它只定义了构造函数和析构函数，以及 keyPressEvent 和 closeEvent 方法。

EditorWindow.cpp

```
#include "EditorWindow.h"
#include <QtWidgets>
```

构造函数把窗口大小设为 1000 × 500 像素，并把标准 File 菜单添加到菜单栏：

```
EditorWindow::EditorWindow(QWidget* parentWidgetPtr /*= nullptr*/)
 :MainWindow(parentWidgetPtr) {
  resize(1000, 500);
  m_editorWidgetPtr = new EditorWidget(this);
  setCentralWidget(m_editorWidgetPtr);
```

```
  addFileMenu();
}

EditorWindow::~EditorWindow() {
  // Empty.
}
```

keyPressEvent 和 closeEvent 方法只是把消息转发给位于窗口中央的编辑器部件中的对应方法：

```
void EditorWindow::keyPressEvent(QKeyEvent* eventPtr) {
  m_editorWidgetPtr->keyPressEvent(eventPtr);
}

void EditorWindow::closeEvent(QCloseEvent* eventPtr) {
  m_editorWidgetPtr->closeEvent(eventPtr);
}
```

绘制编辑器部件

EditorWidget 类是上一节 DocumentWidget 的子类。它捕获按键、鼠标、窗口大小改变和关闭事件，还重写了保存和加载文档的方法。

EditorWidget.h

```
#ifndef EDITORWIDGET_H
#define EDITORWIDGET_H

#include <QWidget>
#include <QMap>
#include <QMenu>
#include <QToolBar>
#include <QPair>

#include "Caret.h"
#include "..\MainWindow\DocumentWidget.h"

class EditorWidget : public DocumentWidget {
  Q_OBJECT

  public:
    EditorWidget(QWidget* parentWidgetPtr);
```

用户按下按键时调用 keyPressEvent，用户点击鼠标时调用 mousePressEvent：

```
    void keyPressEvent(QKeyEvent* eventPtr);
    void mousePressEvent(QMouseEvent* eventPtr);
```

mouseToIndex 是一个辅助方法，计算用户用鼠标点击的字符的索引：

```
    private:
      int mouseToIndex(QPoint point);
```

窗口需要重绘时调用 paintEvent 方法，用户改变窗口大小时调用 resizeEvent 方法。本应用中我们捕获窗口大小改变事件，是因为要重新计算每行能容纳的字符数：

```
    public:
      void paintEvent(QPaintEvent* eventPtr);
      void resizeEvent(QResizeEvent* eventPtr);
```

与上一节的画图程序类似，用户选择 New 菜单项时调用 newDocument，选择 save 或 save as 菜单项时调用 writeFile，选择 open 菜单项时调用 readFile：

```
    private:
      void newDocument(void);
      bool writeFile(const QString& filePath);
      bool readFile(const QString& filePath);
```

响应用户输入或鼠标点击而设置插入符时调用 setCaret 方法：

```
    private:
      void setCaret();
```

用户上移或下移插入符时，我们需要找到插入符上方或下方字符的索引。最简单的办法是模拟一次鼠标点击：

```
    void simulateMouseClick(int x, int y);
```

calculate 是一个辅助方法，计算行数以及每行中每个字符的位置：

```
    private:
      void calculate();
```

m_editIndex 字段保存用户输入文本的位置的索引。该位置也是插入符可见的位置：

```
    int m_editIndex = 0;
```

m_caret 字段保存应用的插入符：

```
    Caret m_caret;
```

编辑器的文本保存在 m_editorText 中：

```
    QString m_editorText;
```

编辑器的文本可能分布在多行上；m_lineList 记录每一行的首末索引：

```
QList<QPair<int,int>> m_lineList;
```

前面提到的 calculate 方法计算编辑器文本中每个字符的矩形，并把它们放入 m_rectList：

```
QList<QRect> m_rectList;
```

本章的应用中所有字符使用同一字体，保存在 TextFont 中：

```
static const QFont TextFont;
```

FontWidth 和 FontHeight 保存 TextFont 中一个字符的宽度和高度：

```
    int FontWidth, FontHeight;
};

#endif // EDITORWIDGET_H
```

EditorWidget 类相当庞大，它定义编辑器的功能。

EditorWidget.cpp

```
#include "EditorWidget.h"
#include <QtWidgets>
using namespace std;
```

我们把文本字体初始化为 12 磅的 Courier New：

```
const QFont EditorWidget::TextFont("Courier New", 12);
```

构造函数把标题设为 Editor，并把标准 Load 和 Save 对话框的文件后缀设为 edi。文本字体中一个字符的高度和平均宽度（像素）用 Qt 的 QFontMetrics 类取得。计算每个字符的矩形，并把插入符设到文本的第一个字符上：

```
EditorWidget::EditorWidget(QWidget* parentWidgetPtr)
 :DocumentWidget(tr("Editor"), tr("Editor files (*.edi)"),
                 parentWidgetPtr),
  m_caret(this),
  m_editorText(tr("Hello World")) {
  QFontMetrics metrics(TextFont);
  FontHeight = metrics.height();
  FontWidth = metrics.averageCharWidth();
  calculate();
  setCaret();
  m_caret.show();
}
```

用户选择 New 菜单项时调用 newDocument 方法。它清空文本、设置插入符并重新计算字符矩形：

```
void EditorWidget::newDocument(void) {
  m_editIndex = 0;
  m_editorText.clear();
  calculate();
  setCaret();
}
```

用户选择 save 或 save as 菜单项时调用 writeFile 方法。它简单地写出编辑器当前文本：

```
bool EditorWidget::writeFile(const QString& filePath) {
  QFile file(filePath);
  if (file.open(QIODevice::WriteOnly | QIODevice::Text)) {
    QTextStream outStream(&file);
    outStream << m_editorText;
```

我们用输出流的 Ok 字段判断写入是否成功：

```
    return ((bool) outStream.Ok);
}
```

如果文件无法以写模式打开，则返回 false：

```
    return false;
}
```

用户选择 load 菜单项时调用 readFile 方法。它通过调用输入流上的 readAll 读取编辑器的全部文本：

```
bool EditorWidget::readFile(const QString& filePath) {
  QFile file(filePath);
  if (file.open(QIODevice::ReadOnly | QIODevice::Text)) {
    QTextStream inStream(&file);
    m_editorText = inStream.readAll();
```

文本读取完毕后，计算字符矩形并设置插入符：

```
calculate();
setCaret();
```

我们用输入流的 Ok 字段判断读取是否成功：

```
    return ((bool) inStream.Ok);
}
```

如果文件无法以读模式打开，则返回 false：

```
    return false;
  }
```

用户按下某个鼠标键时调用 mousePressEvent。如果用户按下左键，我们调用 mouseToIndex 计算被点击字符的索引，并把插入符设到该索引处：

```
  void EditorWidget::mousePressEvent(QMouseEvent* eventPtr) {
    if (eventPtr->buttons() == Qt::LeftButton) {
      m_editIndex = mouseToIndex(eventPtr->pos());
      setCaret();
    }
  }
```

用户按下按键时调用 keyPressEvent。首先，我们检查它是否是方向键、Delete、退格或回车键；如果不是，我们就把该字符插入到插入符指示的位置：

```
  void EditorWidget::keyPressEvent(QKeyEvent* eventPtr) {
    switch (eventPtr->key()) {
```

如果按键是左方向键，且编辑插入符尚未位于文本开头，我们就减小编辑索引：

```
  case Qt::Key_Left:
    if (m_editIndex > 0) {
      --m_editIndex;
    }
    break;
```

如果按键是右方向键，且编辑插入符尚未位于文本末尾，我们就增大编辑索引：

```
  case Qt::Key_Right:
    if (m_editIndex < m_editorText.size()) {
      ++m_editIndex;
    }
    break;
```

如果按键是上方向键，且编辑插入符尚未位于编辑器顶部，我们调用 simulateMouseClick 模拟用户在当前索引上方一点的鼠标点击。这样，新的编辑索引将位于当前行的上一行：

```
  case Qt::Key_Up: {
      QRect charRect = m_rectList[m_editIndex];

      if (charRect.top() > 0) {
        int x = charRect.left() + (charRect.width() / 2),
            y = charRect.top() - 1;
        simulateMouseClick(x, y);
      }
    }
```

```
    break;
```

如果按键是下方向键，我们调用 simulateMouseClick 模拟用户在当前索引下方一点的鼠标点击。这样，编辑插入符将位于当前字符正下方的字符处。注意，如果索引已经在最后一行，则什么也不发生：

```
case Qt::Key_Down: {
    QRect charRect = m_rectList[m_editIndex];
    int x = charRect.left() + (charRect.width() / 2),
        y = charRect.bottom() + 1;
    simulateMouseClick(x, y);
  }
  break;
```

用户按下 Delete 键且编辑索引尚未越过文本末尾时，删除当前字符：

```
case Qt::Key_Delete:
  if (m_editIndex < m_editorText.size()) {
    m_editorText.remove(m_editIndex, 1);
    setModifiedFlag(true);
  }
  break;
```

用户按下退格键且编辑索引尚未位于文本开头时，删除当前字符之前的字符：

```
case Qt::Key_Backspace:
  if (m_editIndex > 0) {
    m_editorText.remove(--m_editIndex, 1);
    setModifiedFlag(true);
  }
  break;
```

用户按下回车键时，插入换行符（\n）：

```
case Qt::Key_Return:
  m_editorText.insert(m_editIndex++, '\n');
  setModifiedFlag(true);
  break;
```

用户按下可读字符时，该字符由 text 方法给出，我们把它的第一个字符插入编辑索引处：

```
  default: {
      QString text = eventPtr->text();

      if (!text.isEmpty()) {
        m_editorText.insert(m_editIndex++, text[0]);
        setModifiedFlag(true);
      }
```

```
      }
      break;
  }
```

文本被修改后，我们需要计算字符矩形、设置插入符，并通过调用 update 强制重绘：

```
    calculate();
    setCaret();
    update();
  }
```

simulateMouseClick 方法通过以给定的点调用 mousePressEvent 和 mouseReleaseEvent 来模拟一次鼠标点击：

```
  void EditorWidget::simulateMouseClick(int x, int y) {
    QMouseEvent pressEvent(QEvent::MouseButtonPress, QPointF(x, y),
                           Qt::LeftButton, Qt::NoButton, Qt::NoModifier);
    mousePressEvent(&pressEvent);

    QMouseEvent releaseEvent(QEvent::MouseButtonRelease,
                             QPointF(x, y), Qt::LeftButton,
                             Qt::NoButton, Qt::NoModifier);
    mousePressEvent(&releaseEvent);
  }
```

setCaret 方法创建一个保存插入符大小和位置的矩形，然后隐藏、设置并显示插入符：

```
  void EditorWidget::setCaret() {
    QRect charRect = m_rectList[m_editIndex];
    QRect caretRect(charRect.left(), charRect.top(),
                    1, charRect.height());
    m_caret.hide();
    m_caret.set(caretRect);
    m_caret.show();
  }
```

mouseToIndex 方法计算给定鼠标点的编辑索引：

```
  int EditorWidget::mouseToIndex(QPoint mousePoint) {
    int x = mousePoint.x(), y = mousePoint.y();
```

首先，以防点击位置在文本之下，我们把 y 坐标限制到文本范围内：

```
  if (y > (FontHeight * m_lineList.size())) {
    y = ((FontHeight * m_lineList.size()) - 1);
  }
```

我们计算鼠标点所在的行：

```
int lineIndex = y / FontHeight;
QPair<int,int> lineInfo = m_lineList[lineIndex];
int firstIndex = lineInfo.first, lastIndex = lineInfo.second;
```

我们找出该行上的索引：

```
  if (x > ((lastIndex - firstIndex + 1) * FontWidth)) {
    return (lineIndex == (m_lineList.size() - 1))
           ? (lineInfo.second + 1) : lineInfo.second;
  }
  else {
    return firstIndex + (x / FontWidth);
  }

  return 0;
}
```

用户改变窗口大小时调用 resizeEvent 方法。字符矩形需要重新计算，因为各行可能变短或变长：

```
void EditorWidget::resizeEvent(QResizeEvent* eventPtr) {
  calculate();
  DocumentWidget::resizeEvent(eventPtr);
}
```

文本发生变化或窗口大小改变时都会调用 calculate 方法。它遍历文本，计算每个字符的矩形：

```
void EditorWidget::calculate() {
  m_lineList.clear();
  m_rectList.clear();
  int windowWidth = width();
```

首先，我们需要把文本划分为行。每行的延伸直到放不进窗口、遇到换行符或文本结束为止：

```
{ int firstIndex = 0, lineWidth = 0;
  for (int charIndex = 0; charIndex < m_editorText.size();
       ++charIndex) {
    QChar c = m_editorText[charIndex];

    if (c == '\n') {
      m_lineList.push_back
                 (QPair<int,int>(firstIndex, charIndex));
      firstIndex = charIndex + 1;
      lineWidth = 0;
    }
    else {
      if ((lineWidth + FontWidth) > windowWidth) {
```

```
          if (firstIndex == charIndex) {
            m_lineList.push_back
                       (QPair<int,int>(firstIndex, charIndex));
            firstIndex = charIndex + 1;
          }
          else {
            m_lineList.push_back(QPair<int,int>(firstIndex,
                                                charIndex - 1));
            firstIndex = charIndex;
          }

          lineWidth = 0;
        }
        else {
          lineWidth += FontWidth;
        }
      }
    }

    m_lineList.push_back(QPair<int,int>(firstIndex,
                                        m_editorText.size() - 1));
  }
```

然后我们遍历各行，对每行计算其中每个字符的矩形：

```
    { int top = 0;
      for (int lineIndex = 0; lineIndex < m_lineList.size();
           ++lineIndex) {
        QPair<int,int> lineInfo = m_lineList[lineIndex];
        int firstIndex = lineInfo.first,
            lastIndex = lineInfo.second, left = 0;

        for (int charIndex = firstIndex;
             charIndex <= lastIndex; ++charIndex){
          QRect charRect(left, top, FontWidth, FontHeight);
          m_rectList.push_back(charRect);
          left += FontWidth;
        }

        if (lastIndex == (m_editorText.size() - 1)) {
          QRect lastRect(left, top, 1, FontHeight);
          m_rectList.push_back(lastRect);
        }

        top += FontHeight;
      }
```

```
    }
  }
```

窗口需要重绘时调用 paintEvent 方法：

```
void EditorWidget::paintEvent(QPaintEvent* /*eventPtr*/) {
  QPainter painter(this);
  painter.setRenderHint(QPainter::Antialiasing);
  painter.setRenderHint(QPainter::TextAntialiasing);
  painter.setFont(TextFont);
  painter.setPen(Qt::black);
  painter.setBrush(Qt::white);
```

我们遍历编辑器的文本，除换行符以外的每个字符都写到其相应位置：

```
  for (int index = 0; index < m_editorText.length(); ++index) {
    QChar c = m_editorText[index];

    if (c != '\n') {
      QRect rect = m_rectList[index];
      painter.drawText(rect, c);
    }
  }

  m_caret.paint(painter);
}
```

main 函数

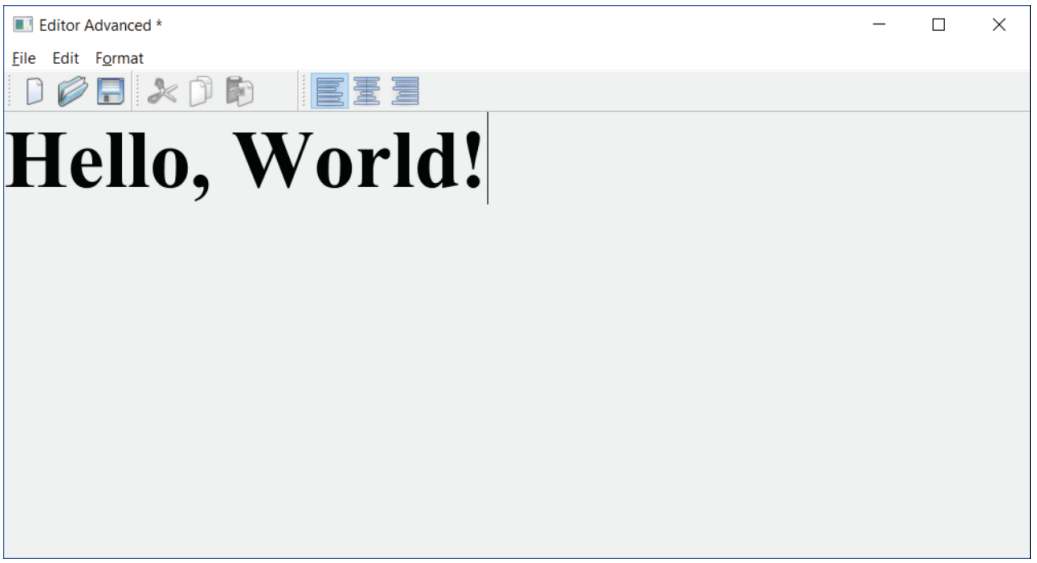
最后，main 函数的工作方式与本章前面的应用类似：创建应用，创建编辑器窗口，然后执行应用。

Main.cpp

```
#include "EditorWindow.h"
#include <QApplication>

int main(int argc, char *argv[]) {
  QApplication application(argc, argv);
  EditorWindow editorWindow;
  editorWindow.show();
  return application.exec();
}
```

得到如下输出：



小结

本章我们用 Qt 库开发了三个图形应用——一个模拟时钟、一个画图程序和一个编辑器。时钟显示当前的时、分、秒；画图程序中我们可以绘制直线、矩形和椭圆；编辑器中我们可以输入和编辑文本。

下一章我们将继续完善这些应用，开发更高级的版本。

6
增强 Qt 图形应用

本章继续完善第 5 章「Qt 图形应用」中开发的三个图形 Qt 应用——模拟时钟、画图程序和编辑器。不过，我们将在本章做如下改进：

- 时钟：我们给钟面添加数字
- 画图程序：增加移动和修改图形、剪切粘贴（cut-and-paste）图形以及标记一个或多个图形的能力
- 编辑器：增加更改字体与文本对齐方式以及标记文本块的能力

本章还将继续使用 Qt 库的以下内容：

- 窗口与部件；
- 菜单与工具栏；
- 鼠标与键盘事件。

改进时钟

本章中，我们将把原来的钟面刻度线替换成数字。

Clock 类

Clock 类的定义与第 5 章「Qt 图形应用」中的相似。定时器每秒更新窗口 10 次。构造函数初始化时钟，每当窗口需要重绘时就会调用 paintEvent 方法。

Clock.h

```
#ifndef CLOCK_H
#define CLOCK_H

#include <QWidget>
#include <QTimer>

class Clock : public QWidget {
  Q_OBJECT

  public:
    Clock(QWidget *parentWidget = nullptr);
    void paintEvent(QPaintEvent *eventPtr);

  private:
    QTimer m_timer;
};

#endif // CLOCK_H
```

Clock.cpp

```
#include <QtWidgets>
#include "Clock.h"
```

与第 5 章「Qt 图形应用」一样，构造函数把窗口标题设置为 Clock Advanced，把窗口尺寸设置为 1000×500 像素，把定时器初始化为每 100 毫秒发送一次 timeout 消息，并把 timeout 消息连接到 update 方法——后者强制窗口在每次超时时重绘：

```
Clock::Clock(QWidget *parentWidget /*= nullptr*/)
 :QWidget(parentWidget) {
  setWindowTitle(tr("Clock Advanced"));
  resize(1000, 500);

  m_timer.setParent(this);
  connect(&m_timer, SIGNAL(timeout()), this, SLOT(update()));
  m_timer.start(100);
}
```

每当窗口需要重绘时就会调用 paintEvent 方法。我们首先计算时钟的边长，并获取当前时间：

```
void Clock::paintEvent(QPaintEvent* /*event*/) {
  int side = qMin(width(), height());
  QTime time = QTime::currentTime();
```

接着，我们创建并初始化一个 QPainter 对象。我们调用 translate 和 scale，把物理尺寸（像素）匹配到 200×200 单位的逻辑尺寸：

```
QPainter painter(this);
painter.setRenderHint(QPainter::Antialiasing);
painter.setRenderHint(QPainter::TextAntialiasing);
painter.translate(width() / 2, height() / 2);
painter.scale(side / 200.0, side / 200.0);
```

由于本版的时钟要在钟面上书写数字，我们为画笔添加 12 磅的 Times New Roman 字体：

```
painter.setFont(QFont(tr("Times New Roman"), 12));
```

我们把时钟的数字 1 到 12 写在钟面上，如下面的代码所示：

```
for (int hour = 1; hour <= 12; ++hour) {
  QString text;
  text.setNum(hour);
```

一整圈是 360°，相邻两个数字之间的夹角是 30°，因为 360 除以 12 等于 30：

```
double angle = (30.0 * hour) - 90;
double radius = 90.0;
```

数字的 x 和 y 坐标由正弦函数和余弦函数计算。不过，我们首先要把角度换算成弧度，因为正弦和余弦只接受弧度。如下面的代码所示：

```
double x = radius * qCos(qDegreesToRadians(angle)),
       y = radius * qSin(qDegreesToRadians(angle));
```

drawText 方法写出数字，如下所示：

```
QRect rect(x - 100, y - 100, 200, 200);
painter.drawText(rect, Qt::AlignHCenter |
                       Qt::AlignVCenter, text);
}
```

数字写完之后，我们按照与第 5 章「Qt 图形应用」相同的方式绘制时针、分针和秒针：

```
  double hours = time.hour(), minutes = time.minute(),
    seconds = time.second(), milliseconds = time.msec();

  painter.setPen(Qt::black);
  painter.setBrush(Qt::gray);

  { static const QPoint hourHand[3] =
     {QPoint(8, 8), QPoint(-8, 8), QPoint(0, -60)};

    painter.save();
    double hour = hours + (minutes / 60.0) + (seconds / 3600.0) +
                  (milliseconds / 3600000.0);
```

```
    painter.rotate(30.0 * hour);
    painter.drawConvexPolygon(hourHand, 3);
    painter.restore();
  }

  { static const QPoint minuteHand[3] =
    {QPoint(6, 8), QPoint(-6, 8), QPoint(0, -70)};

    painter.save();
    double minute = minutes + (seconds / 60.0) +
                    (milliseconds / 60000.0);
    painter.rotate(6.0 * minute);
    painter.drawConvexPolygon(minuteHand, 3);
    painter.restore();
  }

  { static const QPoint secondHand[3] =
     {QPoint(4, 8), QPoint(-4, 8), QPoint(0, -80)};

    painter.save();
    double second = seconds + (milliseconds / 1000);
    painter.rotate(6.0 * second);
    painter.drawConvexPolygon(secondHand, 3);
    painter.restore();
  }
}
```

main 函数

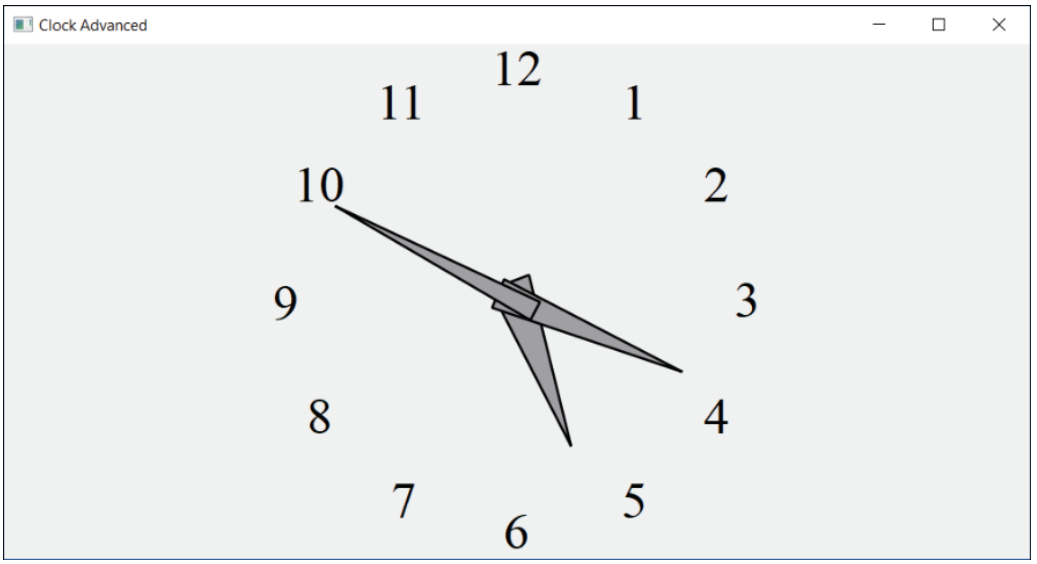
main 函数与第 5 章「Qt 图形应用」中的相似。它创建应用对象，初始化时钟，然后执行应用。

Main.cpp

```
#include <QApplication>
#include "Clock.h"

int main(int argc, char *argv[]) {
  QApplication application(argc, argv);
  Clock Clock;
  Clock.show();
  return application.exec();
}
```

程序的输出如下面的截图所示：



改进画图程序

本章的画图程序是第 5 章「Qt 图形应用」中画图程序的高级版本。在这一版本中，我们可以修改图形，可以圈选一个或多个图形然后更改它们的颜色，还可以剪切粘贴图形。

Figure 类

Figure 类与第 5 章「Qt 图形应用」中的相当相似，不过新增了 isInside、doubleClick、modify 和 marked。

Figure.h

```
#ifndef FIGURE_H
#define FIGURE_H

enum FigureId {LineId, RectangleId, EllipseId};

#include <QtWidgets>
#include <FStream>
using namespace std;

class Figure {
  public:
```

```
    Figure();
```

本版新增了纯虚函数 clone，这是为了支持剪切粘贴。粘贴图形时，我们想创建它的副本，而实际上并不知道该对象属于哪个类。仅靠复制构造函数无法做到这一点。这正是本节的核心：如何使用纯虚方法，以及如何利用动态绑定。我们需要 clone，它调用所属类的复制构造函数，返回指向新对象的指针：

```
virtual Figure* clone() const = 0;

virtual FigureId getId() const = 0;
virtual void initializePoints(QPoint point) = 0;
```

在本版的画图程序中，onClick 会设置一些字段，指明图形应被修改还是被移动。如果用户抓取了图形的某个抓取点（不同种类的图形各不相同），就修改图形；否则就移动图形。当用户抓取图形的某个角时调用 modify 方法。此时图形将被修改而不是移动：

```
virtual bool isClick(QPoint mousePoint) = 0;
virtual void modify(QSize distance) = 0;
```

如果图形完全被圈在区域内，isInside 方法就返回 true。当用户用鼠标圈选图形时会调用它：

```
virtual bool isInside(QRect area) = 0;
```

当用户在图形上双击时调用 doubleClick 方法，每种图形会执行某个合适的动作：

```
virtual void doubleClick(QPoint mousePoint) = 0;
virtual void move(QSize distance) = 0;
virtual void draw(QPainter &painter) const = 0;
virtual bool write(ofstream& outStream) const;
virtual bool read(ifstream& inStream);
```

marked 方法返回并设置 `m_marked` 字段。图形被标记之后，会用一些小方块标注出来：

```
  bool marked() const {return m_marked;}
  bool& marked() {return m_marked;}

  const QColor& color() const {return m_color;}
  QColor& color() {return m_color;}

  virtual bool filled() const {return m_filled;}
  virtual bool& filled() {return m_filled;}

  static const int Tolerance;

  private:
    QColor m_color;
    bool m_marked = false, m_filled = false;
```

```
};

#endif
```

Figure.cpp

```
#include "..\MainWindow\DocumentWidget.h"
#include "Figure.h"

const int Figure::Tolerance(6);

Figure::Figure() {
   // Empty.
}
```

write 和 read 方法写出并读取图形的颜色以及是否填充。不过，它们不写出也不读取标记状态。图形在写出或读取时总是处于未标记状态：

```
bool Figure::write(ofstream& outStream) const {
  writeColor(outStream, m_color);
  outStream.write((char*) &m_filled, sizeof m_filled);
  return ((bool) outStream);
}

bool Figure::read(ifstream& inStream) {
  readColor(inStream, m_color);
  inStream.read((char*) &m_filled, sizeof m_filled);
  return ((bool) inStream);
}
```

Line 类

Line 类是 Figure 的子类。

Line.h

```
#ifndef LINE_H
#define LINE_H

#include <FStream>
using namespace std;

#include "Figure.h"

class Line : public Figure {
  public:
```

```
    Line();
    FigureId getId() const {return LineId;}
    Line(const Line& line);
    Figure* clone() const;
    void initializePoints(QPoint point);
```

正如前面 Figure 节提到的，isClick 决定直线应被修改还是被移动。如果用户抓取了直线的某个端点，就只移动该端点。如果用户抓取了两个端点之间的直线，就移动整条直线。也就是说，直线的两个端点都要移动：

```
bool isClick(QPoint mousePoint);
```

isInside 方法检查直线是否完全被区域圈住：

```
bool isInside(QRect area);
```

doubleClick 方法在 Line 类中什么也不做。不过，我们仍然需要定义它，因为它在 Figure 中是纯虚的。如果不定义它，Line 就会变成抽象类：

```
void doubleClick(QPoint /* mousePoint */) {/* Empty. */}
```

modify 方法按照前面的 isClick 所做的设置来修改直线。如果用户抓取了某个端点，就移动该端点；否则移动整条直线（两个端点都移动）：

```
void modify(QSize distance);
void move(QSize distance);
```

如果直线被标记，area 方法就返回一个稍微扩大了的区域，以便把标记方块包含进来：

```
QRect area() const;
void draw(QPainter& painter) const;
bool write(ofstream& outStream) const;
bool read(ifstream& inStream);
```

m_lineMode 字段记录直线的移动或修改状态。直线创建时，m_lineMode 被设置为 LastPoint。当用户抓取直线的第一个或最后一个端点时，m_lineMode 被设置为 FirstPoint 或 LastPoint。当用户抓取两个端点之间的直线时，m_lineMode 被设置为 MoveLine：

```
private:
    enum {FirstPoint, LastPoint, MoveLine} m_lineMode;
    QPoint m_firstPoint, m_lastPoint;
```

isPointInLine 方法判断用户是否在一定的容差范围内点击了直线：

```
  static bool isPointInLine(QPoint m_firstPoint,
                            QPoint m_lastPoint, QPoint point);
};
```

```
#endif
```

Line.cpp

```
#include "..\MainWindow\DocumentWidget.h"
#include "Line.h"
```

直线创建时，直线模式被设置为最后一个端点。也就是说，用户移动鼠标时，改变的是直线的最后一个端点：

```
Line::Line()
:m_lineMode(LastPoint) {
  // Empty.
}
```

粘贴直线时会调用 clone 方法。这里调用 Figure 的复制构造函数来设置图形的颜色。注意，我们以一个 Line 对象为参数调用 Figure 构造函数，尽管它接受的是一个 Figure 对象的引用。这样做是允许的，因为 Line 是 Figure 的子类，调用期间 Line 对象会被转换成 Figure 对象。此外，直线的第一个和最后一个端点也被复制。注意，我们不需要复制 m_lineMode 的值，因为它只在用户创建、修改或移动直线时才会被设置：

```
Line::Line(const Line& line)
 :Figure(line),
   m_firstPoint(line.m_firstPoint),
   m_lastPoint(line.m_lastPoint) {
  // Empty.
 }
```

clone 方法用复制构造函数创建一个新对象，然后把它返回：

```
Figure* Line::clone() const {
  Line* linePtr = new Line(*this);
  return linePtr;
}
```

直线创建之后不久会调用 initializePoints 方法。之所以有这一步，是因为我们并不直接创建 Line 对象，而是通过调用 clone 间接创建直线。因此，我们需要调用 initializePoints 来初始化两个端点：

```
void Line::initializePoints(QPoint point) {
  m_firstPoint = point;
  m_lastPoint = point;
}
```

用户用鼠标点击时会调用 isClick 方法。我们首先检查用户是否点击了第一个端点。我们利用 Tolerance 字段创建一个小方块，以第一个端点为中心。如果用户点击了这个方块，就把 m_lineMode 设置为 FirstPoint 并返回 true：

```
bool Line::isClick(QPoint mousePoint) {
  QRect firstSquare(makeRect(m_firstPoint, Tolerance));

  if (firstSquare.contains(mousePoint)) {
   m_lineMode = FirstPoint;
   return true;
 }
```

以同样的方式，我们创建一个以最后一个端点为中心的小方块。如果用户点击了这个方块，就把 m_lineMode 设置为 LastPoint 并返回 true：

```
QRect lastSquare(makeRect(m_lastPoint, Tolerance));

if (lastSquare.contains(mousePoint)) {
  m_lineMode = LastPoint;
  return true;
}
```

如果用户没有点击任何一个端点，我们就检查用户是否点击了直线本身。如果是，就把 m_lineMode 设置为 MoveLine 并返回 true：

```
if (isPointInLine(m_firstPoint, m_lastPoint, mousePoint)) {
  m_lineMode = MoveLine;
  return true;
}
```

最后，如果用户既没有点击端点也没有点击直线本身，就说明完全没点到这条直线，返回 false：

```
  return false;
}
```

如果直线完全被区域圈住，isInside 方法就返回 true。这很容易：我们只需检查两个端点是否都位于区域内：

```
bool Line::isInside(QRect area) {
 return area.contains(m_firstPoint) &&
   area.contains(m_lastPoint);
}
```

isPointInLine 方法与第 5 章「Qt 图形应用」版本中的 isClick 完全相同：

```
bool Line::isPointInLine(QPoint m_firstPoint, QPoint m_lastPoint,
                         QPoint point) {
  if (m_firstPoint.x() == m_lastPoint.x()) {
    QRect lineRect(m_firstPoint, m_lastPoint);
    lineRect.normalized();
    lineRect += Tolerance;
    return lineRect.contains(point);
```

```
    }
    else {
      QPoint leftPoint = (m_firstPoint.x() < m_lastPoint.x())
                       ? m_firstPoint : m_lastPoint,
             rightPoint = (m_firstPoint.x() < m_lastPoint.x())
                        ? m_lastPoint : m_firstPoint;

      if ((leftPoint.x() <= point.x()) &&
          (point.x() <= rightPoint.x())) {
        int lineWidth = rightPoint.x() - leftPoint.x(),
            lineHeight = rightPoint.y() - leftPoint.y();

        int diffWidth = point.x() - leftPoint.x(),
            diffHeight = point.y() - leftPoint.y();

        double delta = fabs(diffHeight -
                 (diffWidth * ((double) lineHeight) / lineWidth));

        return (delta <= Tolerance);
      }

      return false;
    }
  }
```

modify 方法根据前面的 isClick 方法中对 m_lineMode 的设置，移动第一个端点、最后一个端点，或者两个端点：

```
void Line::modify(QSize distance) {
  switch (m_lineMode) {
    case FirstPoint:
    m_firstPoint += distance;
    break;

    case LastPoint:
    m_lastPoint += distance;
    break;

    case MoveLine:
    move(distance);
    break;
  }
}
```

move 方法只是把直线的两个端点都移动一下：

```
void Line::move(QSize distance) {
  m_firstPoint += distance;
  m_lastPoint += distance;
}
```

draw 方法绘制直线。与第 5 章「Qt 图形应用」的版本相比，不同之处在于：如果直线被标记，还会在直线两端绘制方块：

```
void Line::draw(QPainter& painter) const {
  painter.setPen(color());
  painter.drawLine(m_firstPoint, m_lastPoint);

  if (marked()) {
    painter.fillRect(makeRect(m_firstPoint, Tolerance),
                     Qt::black);
    painter.fillRect(makeRect(m_lastPoint, Tolerance),
                     Qt::black);
  }
}
```

area 方法返回覆盖直线的区域。如果直线被标记，区域会稍微扩大，以便覆盖标记端点的方块：

```
QRect Line::area() const {
  QRect lineArea(m_firstPoint, m_lastPoint);
  lineArea.normalized();

  if (marked()) {
     lineArea += Tolerance;
  }

  return lineArea;
}
```

与第 5 章「Qt 图形应用」的版本一样，write 和 read 先调用 Figure 中的对应方法，然后写出并读取直线的两个端点：

```
bool Line::write(ofstream& outStream) const {
  Figure::write(outStream);
  writePoint(outStream, m_firstPoint);
  writePoint(outStream, m_lastPoint);
  return ((bool) outStream);
}

bool Line::read(ifstream& inStream) {
  Figure::read(inStream);
  readPoint(inStream, m_firstPoint);
  readPoint(inStream, m_lastPoint);
```

```
    return ((bool) inStream);
}
```

Rectangle 类

RectangleX 是 Figure 的子类，是第 5 章「Qt 图形应用」版本的扩展版本。其中 isClick 方法经过了修改，还新增了 doubleClick 和 modify。

Rectangle.h

```
#ifndef RECTANGLE_H
#define RECTANGLE_H

#include <FStream>
using namespace std;

#include "Figure.h"

class RectangleX : public Figure {
  public:
  RectangleX();
  virtual FigureId getId() const {return RectangleId;}

  RectangleX(const RectangleX& rectangle);
  Figure* clone() const;

  virtual void initializePoints(QPoint point);

  virtual bool isClick(QPoint mousePoint);
  virtual void modify(QSize distance);
  virtual bool isInside(QRect area);
  virtual void doubleClick(QPoint mousePoint);

  virtual void move(QSize distance);
  virtual QRect area() const;
  virtual void draw(QPainter& painter) const;

  virtual bool write(ofstream& outStream) const;
  virtual bool read(ifstream& inStream);

  private:
    enum {TopLeftPoint, TopRightPoint, BottomRightPoint,
        BottomLeftPoint, MoveRectangle} m_rectangleMode;

  protected:
```

```
    QPoint m_topLeft, m_bottomRight;
};

#endif
```

Rectangle.cpp

```
#include <CAssert>
#include "..\MainWindow\DocumentWidget.h"
#include "Rectangle.h"
```

用户添加矩形时，其模式为 BottomRightPoint。也就是说，用户移动鼠标时，改变的是矩形的右下角：

```
RectangleX::RectangleX()
:m_rectangleMode(BottomRightPoint) {
  // Empty.
}
```

复制构造函数复制矩形。具体来说，它先调用 Figure 类的复制构造函数，然后复制左上角和右下角。注意，它不复制 `m_rectangleMode` 字段，因为这个字段只在用户移动鼠标时才会用到：

```
RectangleX::RectangleX(const RectangleX& rectangle)
:Figure(rectangle),
  m_topLeft(rectangle.m_topLeft),
  m_bottomRight(rectangle.m_bottomRight) {
  // Empty.
}
```

clone 方法调用复制构造函数来创建一个新对象，并返回指向它的指针：

```
Figure* RectangleX::clone() const {
  RectangleX* rectanglePtr = new RectangleX(*this);
  return rectanglePtr;
}

void RectangleX::initializePoints(QPoint point) {
  m_topLeft = point;
  m_bottomRight = point;
}
```

用户用鼠标点击时会调用 isClick 方法。与前面的 Line 类似，我们首先检查用户是否点击了任意一个角。如果没有，再根据矩形是否为填充状态，检查用户点击的是矩形边框还是矩形内部。

我们先定义一个覆盖左上角的小方块。如果用户点击了它，就把 `m_rectangleMode` 字段设置为 TopLeftPoint 并返回 true：

```
bool RectangleX::isClick(QPoint mousePoint) {
  QRect topLeftRect(makeRect(m_topLeft, Tolerance));

  if (topLeftRect.contains(mousePoint)) {
    m_rectangleMode = TopLeftPoint;
    return true;
  }
```

接着定义一个覆盖右上角的小方块。如果用户点击了它，就把 m_rectangleMode 设置为 TopRightPoint 并返回 true：

```
QPoint topRightPoint(m_bottomRight.x(), m_topLeft.y());
QRect topRectRight(makeRect(topRightPoint, Tolerance));

if (topRectRight.contains(mousePoint)) {
  m_rectangleMode = TopRightPoint;
  return true;
}
```

如果用户点击了覆盖右下角的小方块，就把 m_rectangleMode 设置为 BottomRightPoint 并返回 true：

```
QRect m_bottomRightRect(makeRect(m_bottomRight, Tolerance));
if (m_bottomRightRect.contains(mousePoint)) {
  m_rectangleMode = BottomRightPoint;
  return true;
}
```

如果用户点击了覆盖左下角的小方块，就把 m_rectangleMode 设置为 BottomLeftPoint 并返回 true：

```
QPoint bottomLeftPoint(m_topLeft.x(), m_bottomRight.y());
QRect bottomLeftRect(makeRect(bottomLeftPoint, Tolerance));

if (bottomLeftRect.contains(mousePoint)) {
  m_rectangleMode = BottomLeftPoint;
  return true;
}
```

如果用户没有点击矩形的任何一个角，我们就检查矩形本身。如果矩形是填充状态的，我们就检查鼠标指针是否位于矩形内部。如果是，就把 m_rectangleMode 设置为 MoveRectangle 并返回 true：

```
QRect areaRect(m_topLeft, m_bottomRight);

if (filled()) {
  if (areaRect.contains(mousePoint)) {
```

```
      m_rectangleMode = MoveRectangle;
      return true;
    }
  }
```

如果矩形不是填充状态的,我们就定义一个稍微大一点和一个稍微小一点的矩形。如果鼠标点击位于较大的矩形之内、而不在较小的矩形之内, 就把 m_rectangleMode 设置为 MoveRectangle 并返回 true:

```
else {
  QRect largeAreaRect(areaRect), smallAreaRect(areaRect);

  largeAreaRect += Tolerance;
  smallAreaRect -= Tolerance;

  if (largeAreaRect.contains(mousePoint) &&
      !smallAreaRect.contains(mousePoint)) {
    m_rectangleMode = MoveRectangle;
    return true;
  }
}
```

最后，如果用户既没有点击任何一个角，也没有点击矩形本身，就说明没有点到这个矩形，我们返回 false:

```
  return false;
}
```

isInside 方法相当简单。我们只需检查矩形的左上角和右下角是否都位于该矩形区域内:

```
bool RectangleX::isInside(QRect area) {
   return area.contains(m_topLeft) &&
     area.contains(m_bottomRight);
 }
```

用户用鼠标双击时会调用 doubleClick 方法。如果对 isClick 的调用返回 true，就会调用 doubleClick。对于矩形来说，填充状态会因此改变——已填充的矩形变为未填充，未填充的矩形变为已填充:

```
void RectangleX::doubleClick(QPoint mousePoint) {
  if (isClick(mousePoint)) {
    filled() = !filled();
  }
}
```

第一次调用 filled 时，调用的是返回 m_filled 字段引用的那个版本，这使我们能够改变返回的值。

modify 方法按照前面的 isClick 所设置的 m_rectangleMode 字段来修改矩形。如果该字段设置为四个角之一，我们就修改相应的角；否则移动整个矩形：

```
void RectangleX::modify(QSize distance) {
  switch (m_rectangleMode) {
    case TopLeftPoint:
    m_topLeft += distance;
    break;

    case TopRightPoint:
    m_topLeft.setY(m_topLeft.y() + distance.height());
    m_bottomRight.setX(m_bottomRight.x() + distance.width());
    break;

    case BottomRightPoint:
    m_bottomRight += distance;
    break;

    case BottomLeftPoint:
    m_topLeft.setX(m_topLeft.x() + distance.width());
    m_bottomRight.setY(m_bottomRight.y() + distance.height());
    break;

    case MoveRectangle:
    move(distance);
    break;
  }
}
```

move 方法相当简单。它只是同时改变左上角和右下角：

```
void RectangleX::move(QSize distance) {
  m_topLeft += distance;
  m_bottomRight += distance;
}
```

area 方法返回覆盖矩形的区域。如果矩形被标记，我们就把区域稍微扩大，以便覆盖标记方块：

```
QRect RectangleX::area() const {
  QRect areaRect(m_topLeft, m_bottomRight);
  areaRect.normalized();

  if (marked()) {
    areaRect += Tolerance;
  }

  return areaRect;
```

```
}
```

draw 方法绘制矩形：如果矩形是填充状态的，就用画刷填充；如果是未填充的，就用空心画刷绘制：

```
void RectangleX::draw(QPainter& painter) const {
  painter.setPen(color());

  if (filled()) {
    painter.fillRect(QRect(m_topLeft, m_bottomRight), color());
  }
  else {
    painter.setBrush(Qt::NoBrush);
    painter.drawRect(QRect(m_topLeft, m_bottomRight));
  }
```

如果矩形被标记，还要绘制覆盖矩形四个角的方块：

```
  if (marked()) {
    painter.fillRect(makeRect(m_topLeft, Tolerance), Qt::black);
    QPoint topRight(m_bottomRight.x(), m_topLeft.y());
    painter.fillRect(makeRect(topRight, Tolerance), Qt::black);
    painter.fillRect(makeRect(m_bottomRight, Tolerance),
                     Qt::black);
    QPoint bottomLeft(m_topLeft.x(), m_bottomRight.y());
    painter.fillRect(makeRect(bottomLeft, Tolerance), Qt::black);
  }
}
```

write 和 read 方法先调用 Figure 中的对应方法，写出并读取矩形的颜色，然后写出并读取左上角和右下角：

```
bool RectangleX::write(ofstream& outStream) const {
  Figure::write(outStream);
  writePoint(outStream, m_topLeft);
  writePoint(outStream, m_bottomRight);
  return ((bool) outStream);
}

bool RectangleX::read (ifstream& inStream) {
  Figure::read(inStream);
  readPoint(inStream, m_topLeft);
  readPoint(inStream, m_bottomRight);
  return ((bool) inStream);
}
```

Ellipse 类

EllipseX 是 RectangleX 的直接子类，也是 Figure 的间接子类，它绘制填充的或未填充的椭圆：

EllipseX.h

```
#ifndef ELLIPSE_H
#define ELLIPSE_H

#include "Rectangle.h"

class EllipseX : public RectangleX {
  public:
  EllipseX();
  FigureId getId() const {return EllipseId;}

  EllipseX(const EllipseX& ellipse);
  Figure* clone() const;
```

与前面的矩形类似，isClick 检查用户是否抓取了椭圆的某个角，或者是否应移动椭圆本身：

```
bool isClick(QPoint mousePoint);
```

modify 方法按照前面 isClick 中对 `m_ellipseMode` 的设置来修改椭圆：

```
void modify(QSize distance);
void draw(QPainter& painter) const;
```

前面的矩形可以通过四个角来抓取，而椭圆则是通过左、上、右、下四个点来抓取。因此，我们需要增加 CreateEllipse 枚举值，用它来修改覆盖椭圆的区域右下角：

```
  private:
    enum {CreateEllipse, LeftPoint, TopPoint, RightPoint,
        BottomPoint, MoveEllipse} m_ellipseMode;
};

#endif
```

EllipseX.cpp

```
#include <CAssert>
#include "..\MainWindow\DocumentWidget.h"
#include "Ellipse.h"
```

与前面直线和矩形的情况不同，我们把 `m_ellipseMode` 字段设置为 CreateEllipse，它只在椭圆被创建时有效：

```
EllipseX::EllipseX()
  :m_ellipseMode(CreateEllipse) {
  // Empty.
}
```

复制构造函数不需要设置 m_topLeft 和 m_bottomRight 字段，因为这项工作由 RectangleX 的复制构造函数完成，而 EllipseX 的复制构造函数会调用它：

```
EllipseX::EllipseX(const EllipseX& ellipse)
:RectangleX(ellipse) {
  // Empty.
}

Figure* EllipseX::clone() const {
  EllipseX* ellipsePtr = new EllipseX(*this);
  return ellipsePtr;
}
```

与前面的矩形类似，isClick 检查用户是否通过椭圆的四个抓取点之一抓取椭圆。不过，对于椭圆来说，我们不再检查矩形的四个角，而是检查椭圆的左、上、右、下四个位置。我们为每个位置创建一个小方块，并检查用户是否点击了它们。如果是，就把 m_ellipseMode 字段设置为相应的值并返回 true：

```
bool EllipseX::isClick(QPoint mousePoint) {
  QPoint leftPoint(m_topLeft.x(),
                   (m_topLeft.y() + m_bottomRight.y()) / 2);
  QRect leftRect(makeRect(leftPoint, Tolerance));

  if (leftRect.contains(mousePoint)) {
    m_ellipseMode = LeftPoint;
    return true;
  }

  QPoint topPoint((m_topLeft.x() + m_bottomRight.x()) / 2,
                  m_topLeft.y());
  QRect topRect(makeRect(topPoint, Tolerance));
  if (topRect.contains(mousePoint)) {
    m_ellipseMode = TopPoint;
    return true;
  }

  QPoint rightPoint(m_bottomRight.x(),
                    (m_topLeft.y() + m_bottomRight.y()) / 2);
  QRect rightRect(makeRect(rightPoint, Tolerance));

  if (rightRect.contains(mousePoint)) {
```

```
    m_ellipseMode = RightPoint;
    return true;
  }

  QPoint bottomPoint((m_topLeft.x() + m_bottomRight.x()) / 2,
                     m_bottomRight.y());
  QRect bottomRect(makeRect(bottomPoint, Tolerance));

  if (bottomRect.contains(mousePoint)) {
    m_ellipseMode = BottomPoint;
    return true;
  }
```

如果用户没有点击这四个位置中的任何一个，我们就检查用户是否点击了椭圆本身。如果椭圆是填充状态的，我们就用 Qt 的 QRegion 类创建一个椭圆区域，并检查鼠标点是否位于该区域内：

```
QRect normalRect(m_topLeft, m_bottomRight);
normalRect.normalized();

if (filled()) {
  QRegion normalEllipse(normalRect, QRegion::Ellipse);

  if (normalEllipse.contains(mousePoint)) {
    m_ellipseMode = MoveEllipse;
    return true;
  }
}
```

如果椭圆是未填充的，我们就创建一个稍微大一点和一个稍微小一点的椭圆区域，然后检查鼠标点是否位于较大的区域内、且不在较小的区域内：

```
else {
  QRect largeRect(normalRect), smallRect(normalRect);
  largeRect += Tolerance;
  smallRect -= Tolerance;

  QRegion largeEllipse(largeRect, QRegion::Ellipse),
       smallEllipse(smallRect, QRegion::Ellipse);

  if (largeEllipse.contains(mousePoint) &&
      !smallEllipse.contains(mousePoint)) {
    m_ellipseMode = MoveEllipse;
    return true;
  }
}
```

最后，如果用户既没有点击任何一个抓取点，也没有点击椭圆本身，我们就返回 false：

```
  return false;
}
```

modify 方法按照 onClick 中对 m_ellipseMode 的设置来修改椭圆：

```
void EllipseX::modify(QSize distance) {
  switch (m_ellipseMode) {
    case CreateEllipse:
    m_bottomRight += distance;
    break;

    case LeftPoint:
    m_topLeft.setX(m_topLeft.x() + distance.width());
    break;

    case RightPoint:
    m_bottomRight.setX(m_bottomRight.x() + distance.width());
    break;

    case TopPoint:
    m_topLeft.setY(m_topLeft.y() + distance.height());
    break;

    case BottomPoint:
    m_bottomRight.setY(m_bottomRight.y() + distance.height());
    break;

    case MoveEllipse:
    move(distance);
    break;
  }
}
```

draw 方法绘制椭圆：如果是填充状态的，就用实心画刷；如果是未填充的，就用空心画刷：

```
void EllipseX::draw(QPainter& painter) const {
  painter.setPen(color());
  if (filled()) {
    painter.setBrush(color());
    painter.drawEllipse(QRect(m_topLeft, m_bottomRight));
  }
  else {
    painter.setBrush(Qt::NoBrush);
    painter.drawEllipse(QRect(m_topLeft, m_bottomRight));
  }
```

如果椭圆被标记，还要绘制覆盖椭圆上、左、右、下四个点的方块：

```
  if (marked()) {
    QPoint leftPoint(m_topLeft.x(),
                     (m_topLeft.y() + m_bottomRight.y())/2);
    painter.fillRect(makeRect(leftPoint, Tolerance), Qt::black);

    QPoint topPoint((m_topLeft.x() + m_bottomRight.x()) / 2,
                    m_topLeft.y());
    painter.fillRect(makeRect(topPoint, Tolerance), Qt::black);

    QPoint rightPoint(m_bottomRight.x(),
                      (m_topLeft.y() + m_bottomRight.y()) / 2);
    painter.fillRect(makeRect(rightPoint, Tolerance), Qt::black);

    QPoint bottomPoint((m_topLeft.x() + m_bottomRight.x()) / 2,
                       m_bottomRight.y());
    painter.fillRect(makeRect(bottomPoint, Tolerance), Qt::black);
  }
}
```

DrawingWindow 类

DrawingWindow 类与上一章的版本相似。它重写了 closeEvent 方法。

DrawingWindow.h

```
#ifndef DRAWINGWINDOW_H
#define DRAWINGWINDOW_H

#include <QMainWindow>
#include <QActionGroup>

#include "..\MainWindow\MainWindow.h"
#include "DrawingWidget.h"

class DrawingWindow : public MainWindow {
  Q_OBJECT

  public:
    DrawingWindow(QWidget *parentWidget = nullptr);
    ~DrawingWindow();

    public:
    void closeEvent(QCloseEvent *eventPtr)
```

```
        { m_drawingWidgetPtr->closeEvent(eventPtr); }

    private:
     DrawingWidget* m_drawingWidgetPtr;
     QActionGroup* m_figureGroupPtr;
};

#endif // DRAWINGWINDOW_H
```

DrawingWindow.cpp

```
#include "..\MainWindow\DocumentWidget.h"
#include "DrawingWindow.h"
```

构造函数把窗口尺寸设置为 1000×500 像素，把画图部件放在窗口中间，添加标准的 File 和 Edit 菜单，并添加应用专属的 Format 和 Figure 菜单：

```
DrawingWindow::DrawingWindow(QWidget *parentWidget /*= nullptr*/)
 :MainWindow(parentWidget) {
  resize(1000, 500);

  m_drawingWidgetPtr = new DrawingWidget(this);
  setCentralWidget(m_drawingWidgetPtr);
  addFileMenu();
  addEditMenu();
```

Format 菜单包含 Color、Fill 和 Modify 三个菜单项，以及 Figure 子菜单：

```
{ QMenu* formatMenuPtr = menuBar()->addMenu(tr("F&ormat"));
  connect(formatMenuPtr, SIGNAL(aboutToShow()),
       this, SLOT(onMenuShow()));

  addAction(formatMenuPtr, tr("&Color"),
        SLOT(onColor()), QKeySequence(Qt::ALT + Qt::Key_C),
        QString(), nullptr, tr("Figure Color"));

  addAction(formatMenuPtr, tr("&Fill"),
        SLOT(onFill()), QKeySequence(Qt::CTRL + Qt::Key_F),
        QString(), nullptr, tr("Figure Fill"),
        LISTENER(isFillEnabled));
```

用户想标记或修改已有图形而不是添加新图形时，会选择 Modify 菜单项：

```
m_figureGroupPtr = new QActionGroup(this);
addAction(formatMenuPtr, tr("&Modify"),
          SLOT(onModify()),
          QKeySequence(Qt::CTRL + Qt::Key_M),
```

```
            QString(), nullptr, tr("Modify Figure"), nullptr,
            LISTENER(isModifyChecked), m_figureGroupPtr);
```

Figure 菜单是一个包含 Line、Rectangle 和 Ellipse 三个菜单项的子菜单。把它加到 Format 菜单之后，它就成了一个子菜单：

```
  { QMenu* figureMenuPtr =
              formatMenuPtr->addMenu(tr("&Figure"));
    connect(figureMenuPtr, SIGNAL(aboutToShow()),
            this, SLOT(onMenuShow()));

    addAction(figureMenuPtr, tr("&Line"),
              SLOT(onLine()),
              QKeySequence(Qt::CTRL + Qt::Key_L),
              QString(), nullptr, tr("Line Figure"), nullptr,
              LISTENER(isLineChecked), m_figureGroupPtr);

    addAction(figureMenuPtr, tr("&Rectangle"),
              SLOT(onRectangle()),
              QKeySequence(Qt::CTRL + Qt::Key_R),
              QString(), nullptr, tr("Rectangle Figure"),
              nullptr, LISTENER(isRectangleChecked),
              m_figureGroupPtr);

    addAction(figureMenuPtr, tr("&Ellipse"),
              SLOT(onEllipse()),
              QKeySequence(Qt::CTRL + Qt::Key_E),
              QString(), nullptr, tr("Ellipse Figure"), nullptr,
              LISTENER(isEllipseChecked), m_figureGroupPtr);
  }
}

DrawingWindow::~DrawingWindow() {
  delete m_figureGroupPtr;
}
```

DrawingWidget 类

DrawingWidget 类是画图程序的主类。它捕获鼠标事件和绘制事件，还捕获 File、Edit 和 Figure 菜单的菜单项选择。

DrawingWidget.h

```
#ifndef DRAWINGWIDGET_H
#define DRAWINGWIDGET_H
```

```
#include "..\MainWindow\MainWindow.h"
#include "..\MainWindow\DocumentWidget.h"
#include "Figure.h"

class DrawingWidget : public DocumentWidget {
  Q_OBJECT

  public:
    DrawingWidget(QWidget* parentWidget);
    ~DrawingWidget();

  public:
    void mousePressEvent(QMouseEvent *eventPtr);
    void mouseMoveEvent(QMouseEvent *eventPtr);
    void mouseReleaseEvent(QMouseEvent *eventPtr);
    void mouseDoubleClickEvent(QMouseEvent *eventPtr);
    void paintEvent(QPaintEvent *eventPtr);

    private:
    void newDocument(void);
    bool writeFile(const QString& filePath);
    bool readFile(const QString& filePath);
    Figure* createFigure(FigureId figureId);
```

与第 5 章「Qt 图形应用」的版本不同，本版重写了剪切和复制的事件方法：

```
public slots:
  bool isCopyEnabled();
  void onCopy(void);
  bool isPasteEnabled();
  void onPaste(void);
  void onDelete(void);
  void onColor(void);

  DEFINE_LISTENER(DrawingWidget, isFillEnabled);
  void onFill(void);

  DEFINE_LISTENER(DrawingWidget, isModifyChecked);
  void onModify(void);

  DEFINE_LISTENER(DrawingWidget, isLineChecked);
  void onLine(void);

  DEFINE_LISTENER(DrawingWidget, isRectangleChecked);
  void onRectangle(void);
```

```
    DEFINE_LISTENER(DrawingWidget, isEllipseChecked);
    void onEllipse(void);
```

m_applicationMode 字段可取 Idle、ModifySingle 或 ModifyRectangle 三个值。Idle 模式在用户没有按下鼠标时处于活动状态。当用户抓取一个图形并修改或移动它时（取决于用户抓取了图形的哪个部位），ModifySingle 模式变为活动状态。最后，当用户在窗口上点击而没有命中任何图形时，ModifyRectangle 模式变为活动状态。此时窗口中会显示一个矩形，用户松开鼠标按钮时，被这个矩形圈住的每个图形都会被标记。用户可以删除被标记的图形、剪切粘贴它们，或者更改它们的颜色和填充状态。用户松开鼠标按钮后，应用模式重新变为 Idle：

```
  private:
    enum ApplicationMode {Idle, ModifySingle, ModifyRectangle};
    ApplicationMode m_applicationMode = Idle;
    void setApplicationMode(ApplicationMode mode);
```

m_actionMode 字段可取 Modify 或 Add 两个值。在 Modify 模式下，用户用鼠标点击时，m_applicationMode 会根据是否命中图形而被设置为 ModifySingle 或 ModifyRectangle。在 Add 模式下，无论用户是否命中图形，都会添加一个新图形。要添加的图形种类由 m_addFigureId 设置，它可取 LineId、RectangleId 或 EllipseId：

```
  enum ActionMode {Modify, Add};
  ActionMode m_actionMode = Add;
  FigureId m_addFigureId = LineId;
```

接下来要添加到画图中的图形颜色初始化为黑色，填充状态初始化为 false（未填充）。两者之后都可以由用户更改：

```
  QColor m_nextColor = Qt::black;
  bool m_nextFilled = false;
```

我们需要保存最近的鼠标点，以便计算两次鼠标移动之间的距离：

```
  QPoint m_mousePoint;
```

指向画图中各图形的指针存放在 m_figurePtrList 中。最上面的图形存放在列表末尾。当用户剪切或复制一个或多个图形时，会把图形复制一份，指向副本的指针存放在 m_copyPtrList 中：

```
  QList<Figure*> m_figurePtrList, m_copyPtrList;
```

当 m_actionMode 为 Modify 而用户按下鼠标按钮却没有命中图形时，窗口中会出现一个矩形。这个矩形存放在 m_insideRectangle 中：

```
    QRect m_insideRectangle;
  };
```

```
#endif // DRAWINGWIDGET_H
```

DrawingWidget.cpp

```
#include <CAssert>
#include "..\MainWindow\DocumentWidget.h"
#include "DrawingWidget.h"
#include "Line.h"
#include "Rectangle.h"
#include "Ellipse.h"
```

构造函数调用基类 DocumentWidget 的构造函数，把窗口标题设置为 Drawing Advanced，把画图文件的后缀设置为 drw：

```
DrawingWidget::DrawingWidget(QWidget* parentWidget)
:DocumentWidget(tr("Drawing Advanced"),
                tr("Drawing files (*.drw)"),
                parentWidget) {
   // Empty.
}
```

析构函数什么也不做，只是为了完整性才包含了它：

```
DrawingWidget::~DrawingWidget() {
   // Empty.
}
```

setApplicationMode 方法设置应用模式，并调用主窗口的 onMenuShow，以便正确启用工具栏图标：

```
void DrawingWidget::setApplicationMode(ApplicationMode mode) {
  m_applicationMode = mode;
  ((MainWindow*) parent())->onMenuShow();
}
```

用户选择 New 菜单项时会调用 newDocument 方法。我们先把图形指针列表和副本指针列表中的每个图形都释放掉，然后清空列表本身：

```
void DrawingWidget::newDocument(void) {
  for (Figure* figurePtr : m_figurePtrList) {
    delete figurePtr;
  }

  for (Figure* copyPtr : m_copyPtrList) {
    delete copyPtr;
  }
```

```
    m_figurePtrList.clear();
    m_copyPtrList.clear();
```

当前颜色和填充状态被设置为黑色和 false（未填充）。动作模式被设置为 Add，要添加的图形标识被设置为 LineId。也就是说，用户按下鼠标按钮时，会往画图中添加一条黑线：

```
    m_nextColor = Qt::black;
    m_nextFilled = false;
    m_actionMode = Add;
    m_addFigureId = LineId;
  }
```

用户选择 Save 或 Save As 菜单项时会调用 writeFile 方法：

```
bool DrawingWidget::writeFile(const QString& filePath) {
  ofstream outStream(filePath.toStdString());

  if (outStream) {
    writeColor(outStream, m_nextColor);
    outStream.write((char*) &m_nextFilled, sizeof m_nextFilled);
```

如果文件成功打开，我们首先写出下一个颜色和填充状态，然后写出画图中图形的个数，再写出各个图形本身：

```
int size = m_figurePtrList.size();
outStream.write((char*) &size, sizeof size);
```

对于每个图形，我们先写出它的标识值，然后通过调用其指针的 write 来写出图形本身。注意，我们并不知道图形指针指向的是哪个类。我们不需要知道这一点，因为 write 是基类 Figure 中的纯虚方法：

```
for (Figure* figurePtr : m_figurePtrList) {
  FigureId figureId = figurePtr->getId();
  outStream.write((char*) &figureId, sizeof figureId);
  figurePtr->write(outStream);
}
```

我们返回转换为 bool 的输出流——写出成功时为 true：

```
return ((bool) outStream);
}
```

如果文件未能成功打开，我们返回 false：

```
    return false;
}
```

用户选择 Open 菜单项时会调用 readFile 方法。我们按照前面 writeFile 中写出文件的相同顺

序读取文件的各个部分：

```
bool DrawingWidget::readFile(const QString& filePath) {
  ifstream inStream(filePath.toStdString());

  if (inStream) {
    readColor(inStream, m_nextColor);
    inStream.read((char*) &m_nextFilled, sizeof m_nextFilled);
```

如果文件成功打开，我们首先读取下一个颜色和填充状态，然后读取画图中图形的个数，再读取各个图形本身：

```
int size;
inStream.read((char*) &size, sizeof size);
```

对于每个图形，我们先读取它的标识值，然后通过调用 createFigure 来创建由该标识值指定类的图形。最后，我们通过调用其指针的 write 来读取图形本身：

```
for (int count = 0; count < size; ++count) {
  FigureId figureId = (FigureId) 0;
  inStream.read((char*) &figureId, sizeof figureId);
  Figure* figurePtr = createFigure(figureId);
  figurePtr->read(inStream);
  m_figurePtrList.push_back(figurePtr);
}
```

我们返回转换为 bool 的输入流——读取成功时为 true：

```
return ((bool) inStream);
}
```

如果文件未能成功打开，我们返回 false：

```
  return false;
}
```

createFigure 方法根据 figureId 参数的值，动态创建 Line、RectangleX 或 EllipseX 类的对象：

```
Figure* DrawingWidget::createFigure(FigureId figureId) {
  Figure* figurePtr = nullptr;

  switch (figureId) {
    case LineId:
     figurePtr = new Line();
     break;

    case RectangleId:
     figurePtr = new RectangleX();
```

```
        break;

      case EllipseId:
        figurePtr = new EllipseX();
        break;
    }

    return figurePtr;
}
```

Edit 菜单变为可见之前会调用 isCopyEnabled 方法，用于启用 Copy 菜单项。框架也会调用它来启用 Copy 工具栏图标。如果至少有一个图形被标记，它就返回 true——此时图形已可复制。返回 true 时，Copy 菜单项和工具栏图标就会被启用：

```
bool DrawingWidget::isCopyEnabled() {
  for (Figure* figurePtr : m_figurePtrList) {
    if (figurePtr->marked()) {
      return true;
    }
  }

  return false;
}
```

用户选择 Copy 菜单项时会调用 onCopy 方法。首先，它释放副本指针列表中的每个图形，并清空列表本身：

```
void DrawingWidget::onCopy(void) {
  for (Figure* copyPtr : m_copyPtrList) {
     delete copyPtr;
  }
  m_copyPtrList.clear();
```

然后，我们遍历图形指针列表，把指向每个被标记图形的副本的指针加入副本指针列表。我们调用每个图形指针的 clone 来获得副本：

```
  for (Figure* figurePtr : m_figurePtrList) {
    if (figurePtr->marked()) {
      m_copyPtrList.push_back(figurePtr->clone());
    }
  }
}
```

Edit 菜单变为可见之前会调用 isPasteEnabled 方法，用于启用 Paste 菜单项。框架也会调用它来启用 Paste 工具栏图标。如果副本指针列表非空，它就返回 true，从而启用 Paste 菜单项和图标。也就是说，只要有图形已复制、可粘贴，它就返回 true：

```
bool DrawingWidget::isPasteEnabled() {
  return !m_copyPtrList.isEmpty();
}
```

用户选择 Edit 菜单中的 Paste 菜单项，或编辑工具栏中的 Paste 图标时会调用 onPaste 方法。我们遍历副本指针列表，把每个图形的副本（通过调用 clone 获得）加入图形指针列表，加入之前先把它向下和向右移动 10 个像素：

```
void DrawingWidget::onPaste(void) {
  for (Figure* copyPtr : m_copyPtrList) {
    Figure* pastePtr = copyPtr->clone();
    pastePtr->move(QSize(10, 10));
    m_figurePtrList.push_back(pastePtr);
  }
```

最后，图形全部加入列表之后，我们调用 update 来强制在合适时机调用 paintEvent：

```
  update();
}
```

用户每次选择 Delete 菜单项或 Delete 工具栏图标时都会调用 onDelete 方法。我们遍历图形指针列表，移除每个被标记的图形：

```
void DrawingWidget::onDelete(void) {
  for (Figure* figurePtr : m_figurePtrList) {
    if (figurePtr->marked()) {
      m_figurePtrList.removeOne(figurePtr);
      delete figurePtr;
    }
  }
```

同样，图形删除完之后，我们也调用 update 方法来强制在合适时机调用 paintEvent：

```
  update();
}
```

用户每次选择 Format 菜单中的 Color 菜单项时都会调用 onColor 方法。我们首先调用 Qt QColorDialog 类的静态方法 getColor 来获得新颜色：

```
void DrawingWidget::onColor(void) {
  QColor newColor = QColorDialog::getColor(m_nextColor, this);
```

如果颜色有效——即用户是通过按下 Ok 按钮而不是 Cancel 按钮关闭对话框，并且选择了一个新颜色——我们就把下一个颜色设置为新颜色，并设置已修改标记。我们还遍历图形指针列表，把每个被标记图形的颜色设置为该颜色：

```
  if (newColor.isValid() && (m_nextColor != newColor)) {
    m_nextColor = newColor;
    setModifiedFlag(true);

    for (Figure* figurePtr : m_figurePtrList) {
      if (figurePtr->marked()) {
        figurePtr->color() = m_nextColor;

        update();
      }
    }
  }
}
```

只要至少有一个图形被标记，我们就调用 update 来强制在合适时机调用 paintEvent。

Format 菜单中的 Fill 菜单项变为可见之前会调用 isFillEnabled 方法：

```
bool DrawingWidget::isFillEnabled(void) {
  switch (m_actionMode) {
```

在 Modify 模式下，我们遍历图形指针列表。如果至少有一个矩形或椭圆被标记，我们就返回 true，菜单项随即被启用：

```
case Modify:
for (Figure* figurePtr : m_figurePtrList) {
  if (figurePtr->marked() &&
      ((figurePtr->getId() == RectangleId) ||
       (figurePtr->getId() == EllipseId))) {
    return true;
  }
}
```

如果没有矩形或椭圆被标记，我们就返回 false，菜单项随即被禁用：

```
return false;
```

在 Add 模式下，如果用户接下来要添加的图形是矩形或椭圆，我们就返回 true：

```
  case Add:
  return (m_addFigureId == RectangleId) ||
         (m_addFigureId == EllipseId);
}
```

按理说程序不应走到这一步。assert 宏调用只是调试之用。不过，方法末尾仍然必须返回一个值：

```
    assert(false);
    return true;
  }
```

用户选择 Format 菜单中的 Fill 菜单项时会调用 onFill 方法：

```
  void DrawingWidget::onFill(void) {
    switch (m_actionMode) {
```

在 Modify 模式下，我们遍历图形指针列表，反转所有被标记图形的填充状态。只要有一个图形发生了变化，我们就调用 update 来强制在合适时机调用 paintEvent：

```
  case Modify:
  for (Figure* figurePtr : m_figurePtrList) {
    if (figurePtr->marked()) {
      figurePtr->filled() = !figurePtr->filled();
      update();
    }
  }
```

我们还反转下一个要添加的图形的填充状态：

```
  m_nextFilled = !m_nextFilled;
  break;
```

在 Add 模式下，我们反转用户下一个要添加的图形的填充状态：

```
      case Add:
       m_nextFilled = !m_nextFilled;
       break;
    }
  }
```

Format 菜单中的 Modify 菜单项变为可见之前会调用 isModifyChecked 方法。在 Modify 模式下它返回 true，并启用该菜单项：

```
  bool DrawingWidget::isModifyChecked(void) {
    return (m_actionMode == Modify);
  }
```

用户选择 Format 菜单中的 Modify 菜单项时会调用 onModify 方法。它把动作模式设置为 Modify：

```
  void DrawingWidget::onModify(void) {
    m_actionMode = Modify;
  }
```

Add 子菜单中的 Line 菜单项变为可见之前会调用 isLineChecked 方法。在 Add 动作模式下

且下一个要添加的图形是直线时，它返回 true，该菜单项就会被勾选（带一个单选按钮，因为该菜单项属于一个组）：

```
bool DrawingWidget::isLineChecked(void) {
  return (m_actionMode == Add) && (m_addFigureId == LineId);
}
```

用户选择 Add 子菜单中的 Line 菜单项时会调用 onLine 方法。它把动作模式设置为 Add，把用户下一个要添加的图形设置为直线：

```
void DrawingWidget::onLine(void) {
  m_actionMode = Add;
  m_addFigureId = LineId;
}
```

Add 子菜单中的 Rectangle 菜单项变为可见之前会调用 isRectangleChecked 方法。在 Add 动作模式下且下一个要添加的图形是矩形时，它返回 true：

```
bool DrawingWidget::isRectangleChecked(void) {
  return (m_actionMode == Add) && (m_addFigureId == RectangleId);
}
```

用户选择 Rectangle 菜单项时会调用 onRectangle 方法。它把动作模式设置为 Add，把用户下一个要添加的图形设置为矩形：

```
void DrawingWidget::onRectangle(void) {
  m_actionMode = Add;
  m_addFigureId = RectangleId;
}
```

Add 子菜单中的 Ellipse 菜单项变为可见之前会调用 isEllipseChecked 方法。在 Add 动作模式下且下一个要添加的图形是椭圆时，它返回 true：

```
bool DrawingWidget::isEllipseEnabled(void) {
  return !isEllipseChecked();
}
```

用户选择 Ellipse 菜单项时会调用 onEllipse 方法。它把动作模式设置为 Add，把用户下一个要添加的图形设置为椭圆：

```
void DrawingWidget::onEllipse(void) {
  m_actionMode = Add;
  m_addFigureId = EllipseId;
}
```

用户按下某个鼠标按钮时会调用 mousePressEvent 方法。我们把鼠标点保存在 `m_mousePoint` 中，供后续的 mouseMoveEvent 使用：

```
void DrawingWidget::mousePressEvent(QMouseEvent* eventPtr) {
   if (eventPtr->buttons() == Qt::LeftButton) {
     m_mousePoint = eventPtr->pos();
```

在 Modify 模式下，我们首先遍历图形指针列表，取消每个图形的标记：

```
switch (m_actionMode) {
  case Modify: {
    for (Figure* figurePtr : m_figurePtrList) {
      figurePtr->marked() = false;
    }
```

然后我们再遍历一次列表，检查用户是否命中了某个图形。由于最上面的图形放在列表末尾，我们需要从后往前遍历列表。我们利用 Qt QList 类的 reverse_iterator 类型来实现：

```
m_clickedFigurePtr = nullptr;
for (QList<Figure*>::reverse_iterator iterator =
    m_figurePtrList.rbegin();
iterator != m_figurePtrList.rend(); ++iterator) {
  Figure* figurePtr = *iterator;
```

如果我们发现（通过在图形上调用 isClick）某个图形被用户的鼠标点击命中了，就把应用模式设置为 ModifySingle 并标记该图形。我们还把它从列表中移除，再加到列表末尾，使它在画图中处于最上层。最后，我们跳出循环，因为已经找到了图形：

```
  if (figurePtr->isClick(m_mousePoint)) {
    setApplicationMode(ModifySingle);
    m_clickedFigurePtr = figurePtr;
    figurePtr->marked() = true;
    m_figurePtrList.removeOne(figurePtr);
    m_figurePtrList.push_back(figurePtr);
    break;
  }
}
```

如果没有找到图形，我们就把应用模式设置为 ModifyRectangle，并把圈选矩形的左上角和右下角都初始化为鼠标点：

```
  if (m_clickedFigurePtr == nullptr) {
    setApplicationMode(ModifyRectangle);
    m_insideRectangle = QRect(m_mousePoint, m_mousePoint);
  }
}
break;
```

在 Add 动作模式下，我们调用 createFigure 并传入用户下一个要添加图形的标识值来创建新图形。然后设置新图形的颜色和填充状态，并初始化它的端点：

```
    case Add: {
      Figure* newFigurePtr = createFigure(m_addFigureId);
      newFigurePtr->color() = m_nextColor;
      newFigurePtr->filled() = m_nextFilled;
      newFigurePtr->initializePoints(m_mousePoint);
```

新图形创建并初始化完毕后,我们把它加到图形指针列表的末尾,并把应用模式设置为 ModifySingle——因为 mouseMoveEvent 方法接下来会继续修改列表末尾的图形，就像用户在 Modify 模式下命中了一个图形一样。我们还设置已修改标记，因为我们已经往画图中添加了一个图形：

```
      m_figurePtrList.push_back(newFigurePtr);
      setApplicationMode(ModifySingle);
      setModifiedFlag(true);
    }
    break;
```

最后，我们调用 update 来强制在合适时机调用 paintEvent：

```
      update();
  }
}
```

用户移动鼠标时会调用 mouseMoveEvent 方法。如果用户同时按下了鼠标左键，我们就保存鼠标点，供后续调用 mouseMoveEvent 时使用，并计算自上次调用 mousePressEvent 或 mouseMoveEvent 以来的距离：

```
void DrawingWidget::mouseMoveEvent(QMouseEvent* eventPtr) {
  if (eventPtr->buttons() == Qt::LeftButton) {
    QPoint newMousePoint = eventPtr->pos();
    QSize distance(newMousePoint.x() - m_mousePoint.x(),
                   newMousePoint.y() - m_mousePoint.y());
    m_mousePoint = newMousePoint;
```

在 Modify 模式下，我们通过调用 modify 来修改当前图形（即放在图形指针列表末尾的图形）。请记住，图形既可能被修改也可能被移动，取决于前面 onMousePress 中对 isClick 调用的设置。我们还设置已修改标记，因为图形已经被改变了：

```
    switch (m_applicationMode) {
      case ModifySingle:
        m_figurePtrList.back()->modify(distance);
        setModifiedFlag(true);
        break;
```

对于圈选矩形，我们只更新它的右下角。注意，我们不设置已修改标记，因为还没有任何图形被改变：

```
      case ModifyRectangle:
        m_insideRectangle.setBottomRight(m_mousePoint);
        break;
    }
```

最后，我们调用 update 来强制在合适时机调用 paintEvent：

```
      update();
    }
  }
```

用户松开某个鼠标按钮时会调用 mouseReleaseEvent 方法。如果松开的是鼠标左键，我们就检查应用模式。真正关心的只有圈选矩形模式：

```
void DrawingWidget::mouseReleaseEvent(QMouseEvent* eventPtr) {
  if (eventPtr->buttons() == Qt::LeftButton) {
    switch (m_applicationMode) {
      case ModifyRectangle: {
        QList<Figure*> insidePtrList;
```

我们遍历图形指针列表，对每个图形调用 isInside。完全被矩形圈住的每个图形都会被标记，从列表中移除，并加入 insidePtrList，稍后再添加到图形指针列表的末尾：

```
for (Figure* figurePtr : m_figurePtrList) {
  if (figurePtr->isInside(m_insideRectangle)) {
    figurePtr->marked() = true;
    m_figurePtrList.removeOne(figurePtr);
    insidePtrList.push_back(figurePtr);
  }
}
```

完全被矩形圈住的每个图形都从图形指针列表中移除了：

```
for (Figure* figurePtr : insidePtrList) {
  m_figurePtrList.removeOne(figurePtr);
}
```

最后，所有被圈住的图形都加到列表末尾，以便在画图中处于最上层：

```
    m_figurePtrList.append(insidePtrList);
  }
  break;
```

用户松开鼠标按钮后，应用模式被设置为 Idle，我们调用 update 来强制在合适时机调用 paintEvent：

```
        setApplicationMode(Idle);
        update();
      }
    }
}
```

用户双击某个鼠标按钮时会调用 mouseDoubleClick 方法。不过，mouseClickEvent 总是在 mouseDoubleClickEvent 之前被调用。如果前面对 mouseClickEvent 的调用已经让 `m_clickedFigurePtr` 指向被点击的图形，我们就对该图形调用 doubleClick。这可能会使图形发生一些变化，具体取决于图形的种类：

```
void DrawingWidget::mouseDoubleClickEvent(QMouseEvent
   *eventPtr) {
  if ((eventPtr->buttons() == Qt::LeftButton) &&
      (m_clickedFigurePtr != nullptr)) {
    m_clickedFigurePtr->doubleClick(eventPtr->pos());
    update();
  }
}
```

最后，每当窗口内容需要重绘时就会调用 paintEvent。在调用之前，框架会先清空窗口：

```
void DrawingWidget::paintEvent(QPaintEvent* /*
   eventPtr */) {
  QPainter painter(this);
  painter.setRenderHint(QPainter::Antialiasing);
  painter.setRenderHint(QPainter::TextAntialiasing);
```

我们遍历图形指针列表，绘制每个图形。列表末尾的图形放在列表末尾，以便出现在画图的最上层：

```
for (Figure* figurePtr : m_figurePtrList) {
  figurePtr->draw(painter);
}
```

在圈选矩形模式下，我们绘制一个空心矩形，边框为浅灰色：

```
  if (m_applicationMode == ModifyRectangle) {
    painter.setPen(Qt::lightGray);
    painter.setBrush(Qt::NoBrush);
    painter.drawRect(m_insideRectangle);
  }
}
```

main 函数

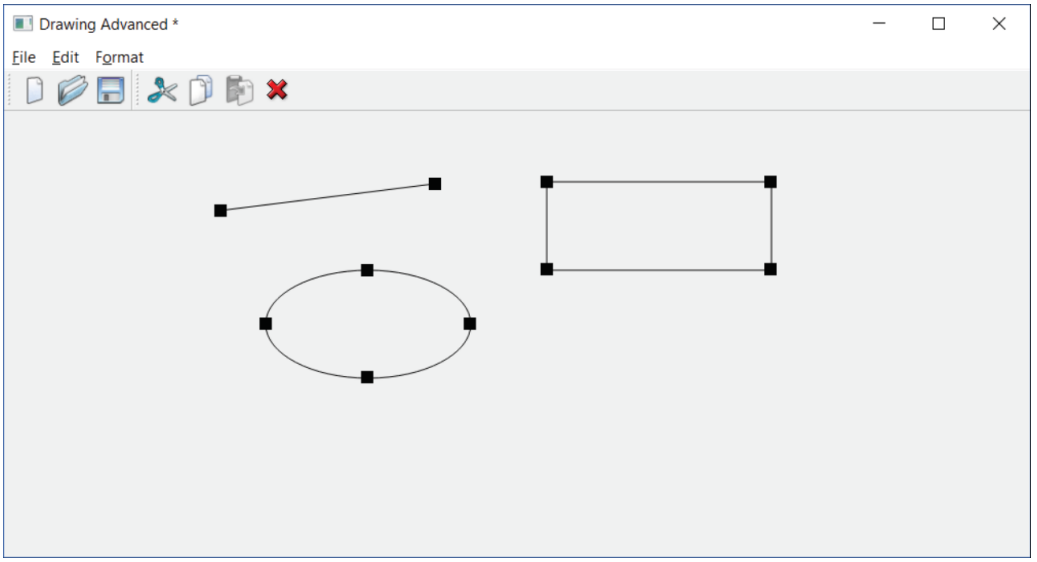
main 函数与前面各应用的 main 函数相似：它创建应用，显示画图窗口，然后开始执行应用。

Main.cpp

```
#include "DrawingWindow.h"
#include <QApplication>

int main(int argc, char *argv[]) {
  QApplication application(argc, argv);
  DrawingWindow drawingWindow;
  drawingWindow.show();
  return application.exec();
}
```

程序的输出如下面的截图所示：



改进编辑器

本章的编辑器是第 5 章「Qt 图形应用」中编辑器的更高级版本。在这一版本中，我们可以更改文本的字体和对齐方式，可以标记文本，还可以剪切粘贴文本。

EditorWindow 类

本章的 EditorWindow 类与第 5 章「Qt 图形应用」中的类相似。它捕获按键事件和窗口关闭事件。

EditorWindow.h

```
#ifndef EDITORWINDOW_H
#define EDITORWINDOW_H

#include <QMainWindow>
#include <QActionGroup>
#include <QPair>
#include <QMap>

#include "..\MainWindow\MainWindow.h"
#include "EditorWidget.h"

class EditorWindow : public MainWindow {
  Q_OBJECT

  public:
    EditorWindow(QWidget *parentWidgetPtr = nullptr);
    ~EditorWindow();

  protected:
    void keyPressEvent(QKeyEvent* eventPtr);
    void closeEvent(QCloseEvent* eventPtr);

  private:
    EditorWidget* m_editorWidgetPtr;
    QActionGroup* m_alignmentGroupPtr;
};

#endif // EDITORWINDOW_H
```

EditorWindow.cpp

```
#include "EditorWindow.h"
#include <QtWidgets>
```

构造函数初始化编辑器窗口。它把窗口尺寸设置为 1000×500 像素，动态创建一个编辑器部件，并添加标准的 File 和 Edit 菜单：

```
EditorWindow::EditorWindow(QWidget *parentWidgetPtr /*= nullptr*/)
 :MainWindow(parentWidgetPtr) {
  resize(1000, 500);
```

```
    m_editorWidgetPtr = new EditorWidget(this);
    setCentralWidget(m_editorWidgetPtr);
    addFileMenu();
    addEditMenu();
```

与第 5 章「Qt 图形应用」相比，Format 菜单有所不同。我们添加 Font 菜单项和 Alignment 子菜单，再往子菜单里加入左对齐、居中、右对齐三个菜单项：

```
{ QMenu* formatMenuPtr = menuBar()->addMenu(tr("F&ormat"));
  connect(formatMenuPtr, SIGNAL(aboutToShow()), this,
        SLOT(onMenuShow()));

  addAction(formatMenuPtr, tr("&Font"), SLOT(onFont()),
        0, QString(), nullptr, QString(),
        LISTENER(isFontEnabled));

  { QMenu* alignmentMenuPtr =
         formatMenuPtr->addMenu(tr("&Alignment"));
    connect(alignmentMenuPtr, SIGNAL(aboutToShow()),
          this, SLOT(onMenuShow()));
```

我们还为 Alignment 菜单添加一个工具栏：

```
    QToolBar* alignmentToolBarPtr = addToolBar(tr("Alignment"));
    m_alignmentGroupPtr = new QActionGroup(this);

    addAction(alignmentMenuPtr, tr("&Left"), SLOT(onLeft()),
            QKeySequence(Qt::ALT + Qt::Key_L), tr("left"),
            alignmentToolBarPtr, tr("Left-aligned text"),
            nullptr, LISTENER(isLeftChecked));

    addAction(alignmentMenuPtr, tr("&Center"),
            SLOT(onCenter()),
            QKeySequence(Qt::ALT + Qt::Key_C),
            tr("center"), alignmentToolBarPtr,
            tr("Center-aligned text"), nullptr,
            LISTENER(isCenterChecked));

    addAction(alignmentMenuPtr, tr("&Right"),
            SLOT(onRight()),
            QKeySequence(Qt::ALT + Qt::Key_R),
            tr("right"), alignmentToolBarPtr,
            tr("Right-aligned text"), nullptr,
            LISTENER(isRightChecked));
  }
}
```

```
  m_editorWidgetPtr->setModifiedFlag(false);
}

EditorWindow::~EditorWindow() {
 delete m_alignmentGroupPtr;
}
```

按键事件和窗口关闭事件都会传递给编辑器部件：

```
void EditorWindow::keyPressEvent(QKeyEvent* eventPtr) {
  m_editorWidgetPtr->keyPressEvent(eventPtr);
}

void EditorWindow::closeEvent(QCloseEvent* eventPtr) {
  m_editorWidgetPtr->closeEvent(eventPtr);
}
```

EditorWidget 类

EditorWidget 类与第 5 章「Qt 图形应用」的版本相似，不过新增了处理字体和对齐的方法与监听器。

EditorWidget.h

```
#ifndef EDITORWIDGET_H
#define EDITORWIDGET_H

#include <QWidget>
#include <QMap>
#include <QMenu>
#include <QToolBar>
#include <QPair>
#include "Caret.h"

#include "..\MainWindow\Listener.h"
#include "..\MainWindow\DocumentWidget.h"

class EditorWidget : public DocumentWidget {
  Q_OBJECT

  public:
    EditorWidget(QWidget* parentWidgetPtr);
    void keyPressEvent(QKeyEvent* eventPtr);
```

```
  private:
  void keyEditPressEvent(QKeyEvent* eventPtr);
  void keyMarkPressEvent(QKeyEvent* eventPtr);
```

用户按下鼠标按钮、移动鼠标和松开鼠标按钮时会分别调用 mousePressEvent、mouseMoveEvent 和 mouseReleaseEvent：

```
public:
  void mousePressEvent(QMouseEvent* eventPtr);
  void mouseMoveEvent(QMouseEvent* eventPtr);
  void mouseReleaseEvent(QMouseEvent* eventPtr);

  private:
  int mouseToIndex(QPoint point);

  public:
  void paintEvent(QPaintEvent* eventPtr);
  void resizeEvent(QResizeEvent* eventPtr);
```

用户选择 New 菜单项时会调用 newDocument 方法；用户选择 Save 或 Save As 时会调用 writeFile；用户选择 Open 菜单项时会调用 readFile：

```
private:
  void newDocument(void);
  bool writeFile(const QString& filePath);
  bool readFile(const QString& filePath);

public slots:
  bool isCopyEnabled();
  void onCopy(void);
  bool isPasteEnabled();
  void onPaste(void);
  void onDelete(void);

  DEFINE_LISTENER(EditorWidget, isFontEnabled);
  void onFont(void);
```

Alignment 子菜单变为可见之前会调用 isLeftChecked、isCenterChecked 和 isRightChecked 方法。它们随后给当前选中的对齐方式标注一个单选按钮：

```
DEFINE_LISTENER(EditorWidget, isLeftChecked);
DEFINE_LISTENER(EditorWidget, isCenterChecked);
DEFINE_LISTENER(EditorWidget, isRightChecked);
```

用户选择 Alignment 子菜单中的某个菜单项时会调用 onLeft、onCenter 和 onRight 方法：

```
void onLeft(void);
void onCenter(void);
void onRight(void);

private:
 void setCaret();
 void simulateMouseClick(int x, int y);
```

在本版的编辑器中，我们有两种模式：edit（编辑）和 mark（标记）。用户输入文本或用方向键移动光标时，edit 模式处于活动状态；用户用鼠标标记了一段文本时，mark 模式处于活动状态。光标在 edit 模式下可见，在 mark 模式下不可见：

```
private:
  enum Mode {Edit, Mark} m_mode;
```

文本可以在左对齐、居中和右对齐三个方向上对齐：

```
enum Alignment {Left, Center, Right} m_alignment;
```

在 edit 模式下，`m_editIndex` 存放下一个将由用户输入的字符要放置的索引，它同时也是光标的位置。在 mark 模式下，`m_firstIndex` 和 `m_lastIndex` 存放第一个和最后一个被标记字符的索引：

```
int m_editIndex, m_firstIndex, m_lastIndex;
```

`m_caret` 对象持有编辑器的光标。光标在 edit 模式下可见，在 mark 模式下不可见：

```
Caret m_caret;
```

`m_editorText` 字段存放编辑器的文本，`m_copyText` 存放用户剪切或复制的文本：

```
QString m_editorText, m_copyText;
```

编辑器的文本按行划分；每行第一个和最后一个字符的索引存放在 `m_lineList` 中：

```
QList<QPair<int,int>> m_lineList;
```

文本当前字体存放在 `m_textFont` 中。当前字体下一个字符的高度（以像素为单位）存放在 `m_fontHeight` 中：

```
QFont m_textFont;
int m_fontHeight;
```

mousePressEvent 和 mouseMoveEvent 方法保存最近的鼠标点，以便计算鼠标事件之间的距离：

```
Qt::MouseButton m_button;
```

与第 5 章「Qt 图形应用」中的方法类似，calculate 是一个辅助方法，用于计算文本中每个字符的外接矩形。不过，本章的版本更复杂，因为它必须考虑文本是左对齐、居中对齐还是右对齐：

```
    void calculate();
```

外接矩形存放在 m_rectList 中，随后供光标和 paintEvent 使用：

```
      QList<QRect> m_rectList;
    };

#endif // EDITORWIDGET_H
```

EditorWidget.cpp

```
#include "EditorWidget.h"
#include <QtWidgets>
#include <CAssert>
using namespace std;
```

构造函数把窗口标题设置为 Editor Advanced，把文件后缀设置为 edi：

```
EditorWidget::EditorWidget(QWidget* parentWidgetPtr)
 :DocumentWidget(tr("Editor Advanced"),
     tr("Editor files (*.edi)"), parentWidgetPtr),
```

文本字体初始化为 12 磅的 Times New Roman。应用模式设置为 edit，下一个将由用户输入的字符的索引设置为零，文本从一开始就是左对齐的：

```
m_textFont(tr("Times New Roman"), 12),
m_mode(Edit),
m_editIndex(0),
m_alignment(Left),
m_caret(this) {
```

字符的外接矩形由 calculate 计算；光标被初始化并显示，因为应用从一开始就处于 edit 模式：

```
  calculate();
  setCaret();
  m_caret.show();
}
```

用户选择 New 菜单项时会调用 newDocument 方法。我们首先把应用模式设置为 edit，把编辑索引设为零。文本字体设置为 12 磅的 Times New Roman。编辑器的文本被清空，字符的外接矩形由 calculate 计算，光标被设置：

```
void EditorWidget::newDocument(void) {
  m_mode = Edit;
```

```
    m_editIndex = 0;
    m_textFont = QFont(tr("Times New Roman"), 12);
    m_editorText.clear();
    calculate();
    setCaret();
}
```

用户选择 Save 或 Save As 菜单项时会调用 writeFile 方法。文件格式相当简单：我们在第一行写出字体，随后几行写出编辑器的文本：

```
bool EditorWidget::writeFile(const QString& filePath) {
  QFile file(filePath);
  if (file.open(QIODevice::WriteOnly | QIODevice::Text)) {
    QTextStream outStream(&file);
    outStream << m_textFont.toString() << endl << m_editorText;
```

我们利用输入流的 Ok 字段判断写出是否成功：

```
    return ((bool) outStream.Ok);
}
```

如果文件未能为写出而打开，我们返回 false：

```
    return false;
}
```

用户选择 Open 菜单项时会调用 readFile 方法。与前面的 writeFile 类似，我们先读取第一行并用它初始化文本字体，然后读取编辑器文本：

```
bool EditorWidget::readFile(const QString& filePath) {
  QFile file(filePath);
  if (file.open(QIODevice::ReadOnly | QIODevice::Text)) {
    QTextStream inStream(&file);
    m_textFont.fromString(inStream.readLine());
    m_editorText = inStream.readAll();
```

文本读取完毕后，我们调用 calculate 来计算文本中各字符的外接矩形，然后设置光标，并返回 true——因为读取成功了：

```
calculate();
setCaret();
```

我们利用输入流的 Ok 字段判断读取是否成功：

```
    return ((bool) inStream.Ok);
}
```

如果文件未能为读取而打开，我们返回 false：

```
    return false;
}
```

Edit 菜单变为可见之前会调用 isCopyEnabled 方法。框架也会调用它来决定复制工具栏图标是否应被启用。如果应用处于 mark 模式，它就返回 true（菜单项随即被启用）——这意味着用户已经标记了一部分文本，可以复制了：

```
bool EditorWidget::isCopyEnabled() {
  return (m_mode == Mark);
}
```

用户选择 Copy 菜单项时会调用 onCopy 方法。我们把被标记的文本复制到 `m_editorText` 中：

```
void EditorWidget::onCopy(void) {
  int minIndex = qMin(m_firstIndex, m_lastIndex),
  maxIndex = qMax(m_firstIndex, m_lastIndex);

  m_copyText =
    m_editorText.mid(minIndex, maxIndex - minIndex + 1);
}
```

Edit 菜单变为可见之前也会调用 isPasteEnabled 方法。如果复制文本非空，它就返回 true（菜单项随即可见）。也就是说，只要有一块已复制、待粘贴的文本，它就返回 true：

```
bool EditorWidget::isPasteEnabled() {
  return !m_copyText.isEmpty();
}
```

用户选择 Paste 菜单项时会调用 onPaste 方法。在 mark 模式下，我们调用 onDelete，被标记的文本随即被删除：

```
void EditorWidget::onPaste(void) {
  if (m_mode == Mark) {
     onDelete();
  }
```

然后，我们把复制的文本插入编辑器文本。我们还更新 `m_editIndex`，因为文本插入之后的编辑索引应位于被插入文本之后的位置：

```
m_editorText.insert(m_editIndex, m_copyText);
m_editIndex += m_copyText.size();
```

最后，我们计算文本中各字符的外接矩形，把光标设置到新索引处，设置已修改标记——因为文本已被改变——并调用 update 来强制在合适时机调用 paintEvent，以显示新文本：

```
  calculate();
  setCaret();
```

```
    setModifiedFlag(true);
    update();
  }
```

用户选择 Delete 菜单项或 Delete 工具栏图标时会调用 onDelete 方法。其效果与用户按下 Delete 键的事件类似。因此，我们准备一个带 Delete 键的按键事件，把它作为参数用在 keyPressEvent 的调用中。

注意，没有 isDeleteEnabled 方法，因为用户总是可以使用 Delete 菜单项。在 edit 模式下，删除下一个字符；在 mark 模式下，删除被标记的文本：

```
void EditorWidget::onDelete(void) {
  QKeyEvent event(QEvent::KeyPress, Qt::Key_Delete,
                  Qt::NoModifier);
  keyPressEvent(&event);
}
```

Format 菜单变为可见之前会调用 isCopyEnabled。在 edit 模式下它返回 true——因为当只有一部分字符被标记时，更改所有字符的字体是不合逻辑的：

```
bool EditorWidget::isFontEnabled() {
  return (m_mode == Edit);
}
```

用户选择 Font 菜单项时会调用 onFont 方法。我们让用户通过 Qt 的 QFontDialog 类选择新字体：

```
void EditorWidget::onFont(void) {
   bool pressedOkButton;
   QFont newFont =
     QFontDialog::getFont(&pressedOkButton, m_textFont, this);
```

如果用户按下 Ok 按钮关闭对话框，我们就设置编辑器的字体字段（m_textFont）和已修改标记：

```
if (pressedOkButton) {
  m_textFont = newFont;
  setModifiedFlag(true);
```

我们通过调用 calculate 计算新的外接矩形，设置光标，并调用 update 来强制在合适时机调用 paintEvent：

```
     calculate();
     m_caret.set(m_rectList[m_editIndex]);
     update();
   }
}
```

Alignment 子菜单变为可见之前会调用 isLeftChecked、isCenterChecked 和 isRightChecked 方法。它们针对当前的对齐方式返回 true：

```
bool EditorWidget::isLeftChecked(void) {
  return (m_alignment == Left);
}

bool EditorWidget::isCenterChecked(void) {
  return (m_alignment == Center);
}

bool EditorWidget::isRightChecked(void) {
  return (m_alignment == Right);
}
```

用户选择 Left、Center 和 Right 菜单项时会调用 onLeft、onCenter 和 onRight 方法。它们设置对齐方式和已修改标记，计算新的外接矩形，设置光标，并调用 update 来强制在合适时机调用 paintEvent：

```
void EditorWidget::onLeft(void) {
  m_alignment = Left;
  setModifiedFlag(true);
  calculate();
  setCaret();
  update();
}

void EditorWidget::onCenter(void) {
  m_alignment = Center;
  setModifiedFlag(true);
  calculate();
  setCaret();
  update();
}

void EditorWidget::onRight(void) {
  m_alignment = Right;
  setModifiedFlag(true);
  calculate();
  setCaret();
  update();
}
```

用户按下某个鼠标按钮时会调用 mousePressEvent 方法。我们调用 mouseToIndex 来找到用户点击的字符索引。眼下，第一个和最后一个标记索引都被设置为鼠标索引。最后一个索引随后可能被下一小段中的 mouseMoveEvent 调用改变。最后，模式被设置为 mark，光标被隐藏：

```
void EditorWidget::mousePressEvent(QMouseEvent* eventPtr) {
  if (eventPtr->buttons() == Qt::LeftButton) {
     m_firstIndex = m_lastIndex = mouseToIndex(eventPtr->pos());
     m_mode = Mark;
     m_caret.hide();
   }
}
```

用户移动鼠标时会调用 mouseMoveEvent 方法。我们把最后一个标记索引设置为鼠标索引，并调用 update 来强制在合适时机调用 paintEvent：

```
void EditorWidget::mouseMoveEvent(QMouseEvent* eventPtr) {
  if (eventPtr->buttons() == Qt::LeftButton) {
     m_lastIndex = mouseToIndex(eventPtr->pos());
     update();
  }
}
```

用户松开鼠标按钮时会调用 mouseReleaseEvent 方法。如果用户把鼠标移回了本次鼠标移动的起始位置，就没有任何需要标记的内容，我们把应用设置为 edit 模式。此时，我们把编辑索引设置为第一个标记索引，设置并显示光标（因为在 edit 模式下光标应当可见）。最后，我们调用 update 来强制在合适时机调用 paintEvent：

```
void EditorWidget::mouseReleaseEvent(QMouseEvent* eventPtr) {
  if (eventPtr->buttons() == Qt::LeftButton) {
    if (m_firstIndex == m_lastIndex) {
      m_mode = Edit;
      m_editIndex = m_firstIndex;
      setCaret();
      m_caret.show();
      update();
    }
  }
}
```

用户按下键盘上的某个键时会调用 keyPressEvent。根据应用模式（edit 或 mark），我们调用 keyEditPressEvent 或后面的 keyMarkPressEvent 来进一步处理按键事件：

```
void EditorWidget::keyPressEvent(QKeyEvent* eventPtr) {
   switch (m_mode) {
     case Edit:
     keyEditPressEvent(eventPtr);
     break;

     case Mark:
     keyMarkPressEvent(eventPtr);
```

```
        break;
      }
    }
```

keyEditPressEvent 处理 edit 模式下的按键。我们首先检查按键是否为方向键、上翻页键、下翻页键、Delete 键、Backspace 键或回车键：

```
void EditorWidget::keyEditPressEvent(QKeyEvent* eventPtr) {
  switch (eventPtr->key()) {
```

对于左方向键，我们把编辑索引向后移动一步，除非它已经位于文本的开头：

```
case Qt::Key_Left:
  if (m_editIndex > 0) {
     --m_editIndex;
  }
  break;
```

对于右方向键，我们把编辑索引向前移动一步，除非它已经位于文本的末尾：

```
case Qt::Key_Right:
 if (m_editIndex < m_editorText.size()) {
   ++m_editIndex;
 }
 break;
```

对于上方向键，我们计算上一行相应字符的 x 和 y 位置，除非它已经位于文本的顶端。然后我们调用 simulateMouseClick，它的效果如同用户点击了上一行中的字符：

```
case Qt::Key_Up: {
  QRect charRect = m_rectList[m_editIndex];

  if (charRect.top() > 0) {
    int x = charRect.left() + (charRect.width() / 2),
        y = charRect.top() - 1;
    simulateMouseClick(x, y);
  }
}
break;
```

同样，对于下方向键，我们把编辑索引向下移动一行，除非它已经位于文本的底部。我们计算下一行相应字符的 x 和 y 位置，并调用 simulateMouseClick，它的效果如同用户点击了该位置的字符：

```
case Qt::Key_Down: {
  QRect charRect = m_rectList[m_editIndex];
  int x = charRect.left() + (charRect.width() / 2),
```

```
        y = charRect.bottom() + 1;
      simulateMouseClick(x, y);
    }
    break;
```

对于 Delete 键，我们删除当前字符，除非我们位于文本的末尾。也就是说，除非我们正好在最后一个字符之后一步的位置：

```
    case Qt::Key_Delete:
     if (m_editIndex < m_editorText.size()) {
       m_editorText.remove(m_editIndex, 1);
       setModifiedFlag(true);
     }
     break;
```

对于退格键，我们把编辑索引向后移动一步——除非它已经位于文本的开头——然后调用 onDelete。这样，我们就删除了前一个字符，并把编辑索引向后移动了一步：

```
    case Qt::Key_Backspace:
    if (m_editIndex > 0) {
      --m_editIndex;
      onDelete();
    }
    break;
```

对于回车键，我们简单地在文本中插入换行符：

```
    case Qt::Key_Return:
      m_editorText.insert(m_editIndex++, 'n');
      setModifiedFlag(true);
      break;
```

如果按键不是特殊键，我们就调用按键事件指针的 text 来检查它是否为常规字符。如果文本非空，就把它的第一个字符加入文本：

```
     default: {
       QString text = eventPtr->text();

       if (!text.isEmpty()) {
         m_editorText.insert(m_editIndex++, text[0]);
         setModifiedFlag(true);
       }
     }
     break;
    }
```

最后，我们计算外接矩形，设置光标，并调用 update 来强制在合适时机调用 paintEvent：

```
    calculate();
    setCaret();
    update();
  }
```

用户在 mark 模式下按键时会调用 keyMarkPressEvent：

```
  void EditorWidget::keyMarkPressEvent(QKeyEvent* eventPtr) {
    switch (eventPtr->key()) {
```

对于左方向键，我们把应用设置为 edit 模式，并把编辑索引设置为第一个和最后一个标记索引中的较小者。不过，如果较小索引位于文本的开头，我们就什么都不做：

```
    case Qt::Key_Left: {
      int minIndex = qMin(m_firstIndex, m_lastIndex);

      if (minIndex > 0) {
        m_mode = Edit;
        m_caret.show();
        m_editIndex = minIndex;
      }
    }
    break;
```

反过来，对于右方向键，我们把应用设置为 edit 模式，并把编辑索引设置为第一个和最后一个标记索引中的较大者。不过，如果较大索引位于文本的末尾，我们就什么都不做：

```
    case Qt::Key_Right: {
      int maxIndex = qMax(m_firstIndex, m_lastIndex);

      if (maxIndex < m_editorText.size()) {
        m_mode = Edit;
        m_caret.show();
        m_editIndex = maxIndex;
      }
    }
    break;
```

对于上、下方向键，我们在当前行的上一行或下一行模拟一次鼠标点击，就像前面的 edit 模式那样：

```
    case Qt::Key_Up: {
      QRect charRect = m_rectList[m_editIndex];

      if (charRect.top() > 0) {
        int x = charRect.left() + (charRect.width() / 2),
            y = charRect.top() - 1;
```

```
      simulateMouseClick(x, y);
    }
  }
  break;

  case Qt::Key_Down: {
    QRect charRect = m_rectList[m_editIndex];
    int x = charRect.left() + (charRect.width() / 2),
        y = charRect.bottom() + 1;
    simulateMouseClick(x, y);
  }
  break;
```

在 mark 模式下，Delete 键和退格键执行同样的任务——删除被标记的文本：

```
  case Qt::Key_Delete:
  case Qt::Key_Backspace: {
    int minIndex = qMin(m_firstIndex, m_lastIndex),
        maxIndex = qMax(m_firstIndex, m_lastIndex);
```

我们把被标记的文本从编辑文本中移除，设置已修改标记，把应用设置为 edit 模式，把编辑索引设置为第一个和最后一个标记索引中的较小者，并显示光标：

```
    m_editorText.remove(minIndex, maxIndex - minIndex);
    setModifiedFlag(true);
    m_mode = Edit;
    m_editIndex = minIndex;
    m_caret.show();
  }
  break;
```

回车键的情况与前面 edit 模式的情况类似，不同之处在于我们先删除被标记的文本，然后给编辑器文本添加一个新行：

```
  case Qt::Key_Return:
    onDelete();
    m_editorText.insert(m_editIndex++, 'n');
    setModifiedFlag(true);
    break;
```

如果按键不是特殊键，我们就调用按键事件指针的 text 来检查它是否为常规字符。如果文本非空，说明用户输入了一个常规字符，我们把该字符插入编辑器文本：

```
  default: {
    QString text = eventPtr->text();

    if (!text.isEmpty()) {
```

```
            onDelete();
            m_editorText.insert(m_editIndex++, text[0]);
            setModifiedFlag(true);
          }
        }
        break;
      }
```

最后,我们计算新的字符外接矩形,设置光标,并调用 update 来强制在合适时机调用 paintEvent:

```
        calculate();
        setCaret();
        update();
      }
```

用户上下移动光标时会调用 simulateMouseClick 方法。它通过调用 mousePressEvent 和 mouseReleaseEvent，并使用事先准备好的事件对象，来模拟一次鼠标点击:

```
void EditorWidget::simulateMouseClick(int x, int y) {
  QMouseEvent pressEvent(QEvent::MouseButtonPress, QPointF(x, y),
      Qt::LeftButton, Qt::NoButton, Qt::NoModifier);
  mousePressEvent(&pressEvent);
  QMouseEvent releaseEvent(QEvent::MouseButtonRelease,
                QPointF(x, y), Qt::LeftButton,
                Qt::NoButton, Qt::NoModifier);
  mousePressEvent(&releaseEvent);
}
```

setCaret 方法在 edit 模式下把光标设置到合适的大小和位置。首先，我们利用 `m_editIndex` 找到正确字符的矩形。然后，我们创建一个宽度仅为一个像素的新矩形，以便光标看起来像一条细竖线:

```
void EditorWidget::setCaret() {
  QRect charRect = m_rectList[m_editIndex];
  QRect caretRect(charRect.left(), charRect.top(),
                1, charRect.height());
  m_caret.set(caretRect);
}
```

mouseToIndex 方法接受一个鼠标点，返回该点所在字符的索引。与第 5 章「Qt 图形应用」的版本不同，我们需要考虑文本可能居中或右对齐的情况:

```
int EditorWidget::mouseToIndex(QPoint point) {
   int x = point.x(), y = point.y();
```

如果鼠标点位于编辑器文本的下方，就返回最后一个字符的索引:

```
if (y > (m_fontHeight * m_lineList.size())) {
  return m_editorText.size();
}
```

否则，我们首先找到鼠标点所在的行，并获取该行第一个和最后一个字符的索引：

```
else {
  int lineIndex = y / m_fontHeight;
  QPair<int,int> lineInfo = m_lineList[lineIndex];
  int firstIndex = lineInfo.first, lastIndex = lineInfo.second;
```

如果鼠标点位于该行第一个字符的左侧（文本居中或右对齐时可能如此），我们返回该行第一个字符的索引：

```
 if (x < m_rectList[firstIndex].left()) {
   return firstIndex;
}
```

反过来，如果鼠标点位于该行的右侧，我们返回该行最后一个字符下一个字符的索引：

```
else if (x >= m_rectList[lastIndex].right()) {
  return (lastIndex + 1);
}
```

否则，我们遍历该行中的字符，对每个字符检查鼠标点是否位于该字符的外接矩形之内：

```
else {
  for (int charIndex = firstIndex + 1;
      charIndex <= lastIndex; ++charIndex){
    int left = m_rectList[charIndex].left();
```

如果鼠标点位于矩形之内，我们再检查它更靠近矩形的左边框还是右边框。如果更靠近左边框，我们返回该字符的索引；如果更靠近右边框，我们则返回下一个字符的索引：

```
      if (x < left) {
        int last = m_rectList[charIndex - 1].left();
        int leftSize = x - last, rightSize = left - x;
        return (leftSize < rightSize) ? (charIndex - 1)
                                      : charIndex;
      }
    }
  }
}
```

按理说程序不应走到这一步。加入 assert 宏只是调试之用：

```
  assert(false);
  return 0;
```

```
}
```

用户调整窗口大小时会调用 resizeEvent 方法。我们重新计算字符的外接矩形，因为窗口宽度可能发生了变化，这会使每行容纳的字符数变少或变多：

```
void EditorWidget::resizeEvent(QResizeEvent* eventPtr) {
  calculate();
  DocumentWidget::resizeEvent(eventPtr);
}
```

calculate 方法把文本划分为若干行，并计算文本中每个字符的外接矩形。每行第一个和最后一个字符的索引存放在 m_lineList 中，外接矩形存放在 m_rectList 中：

```
void EditorWidget::calculate() {
  m_lineList.clear();
  m_rectList.clear();
```

我们利用 Qt 的 QFontMetrics 类来获得编辑器字体下一个字符的高度。高度存放在 m_fontHeight 中。width 方法给出窗口内容区宽度（以像素为单位）：

```
QFontMetrics metrics(m_textFont);
m_fontHeight = metrics.height();
QList<int> charWidthList, lineWidthList;
int windowWidth = width();
```

我们首先遍历编辑器文本，把文本划分为若干行：

```
{ int firstIndex = 0, lineWidth = 0;
  for (int charIndex = 0; charIndex < m_editorText.size();
     ++charIndex) {
    QChar c = m_editorText[charIndex];
```

遇到换行符时，我们把当前行的第一个和最后一个索引加入 m_lineList：

```
if (c == 'n') {
  charWidthList.push_back(1);
  lineWidthList.push_back(lineWidth);
  m_lineList.push_back
             (QPair<int,int>(firstIndex, charIndex));
  firstIndex = charIndex + 1;
  lineWidth = 0;
}
```

否则，我们调用 Qt QMetrics 对象的 width 方法来获得该字符的宽度（以像素为单位）：

```
else {
  int charWidth = metrics.width(c);
  charWidthList.push_back(charWidth);
```

如果该字符会使行宽超过窗口内容区宽度，我们就把第一个和最后一个索引加入 m_lineList，并另起一行。

不过，有两种情况需要考虑。如果当前字符是本行的第一个字符，就出现了该字符宽度超过窗口内容区宽度的情况（虽然不大可能）。此时，我们把该字符的索引同时作为第一行和最后一个索引加入 m_lineList。下一行的第一个索引就是该字符的下一个字符：

```
if ((lineWidth + charWidth) > windowWidth) {
   if (firstIndex == charIndex) {
     lineWidthList.push_back(windowWidth);
     m_lineList.push_back
          (QPair<int,int>(firstIndex, charIndex));
     firstIndex = charIndex + 1;
   }
```

如果当前字符不是本行的第一个字符，我们就把第一个字符和当前字符前一个字符的索引加入 m_lineList。下一行的索引就变成当前字符的索引：

```
   else {
      lineWidthList.push_back(lineWidth);
      m_lineList.push_back(QPair<int,int>(firstIndex,
          charIndex - 1));
      firstIndex = charIndex;
   }
   lineWidth = 0;
}
```

如果该字符没有使行宽超过窗口内容区宽度，我们就简单地把该字符的宽度加到行宽上：

```
  else {
    lineWidth += charWidth;
  }
  }
}
```

最后，我们需要把最后一行加入 m_lineList：

```
  m_lineList.push_back(QPair<int,int>(firstIndex,
       m_editorText.size() - 1));
  lineWidthList.push_back(lineWidth);
}
```

把文本划分为若干行之后，我们继续计算各个字符的外接矩形。我们先把 top 设为零，因为它存放的是行的顶部位置。每处理一行，它就增加一个行高：

```
{ int top = 0, left;
  for (int lineIndex = 0; lineIndex < m_lineList.size();
```

```
         ++lineIndex) {
        QPair<int,int> lineInfo = m_lineList[lineIndex];
        int lineWidth = lineWidthList[lineIndex];
        int firstIndex = lineInfo.first,
           lastIndex = lineInfo.second;
```

根据文本的对齐方式，我们需要决定行从哪里开始。左对齐的情况下，我们把行的左端位置设为零：

```
switch (m_alignment) {
  case Left:
    left = 0;
    break;
```

居中对齐的情况下，我们把左端位置设为窗口内容区宽度与行宽之差的一半。这样，该行就会出现在窗口的中央：

```
case Center:
  left = (windowWidth - lineWidth) / 2;
  break;
```

右对齐的情况下，我们把左端位置设为窗口内容区宽度与行宽之差。这样，该行就会出现在窗口的右侧：

```
  case Right:
    left = windowWidth - lineWidth;
    break;
}
```

确定了行的起始左端位置以及文本中每个字符的宽度之后，我们遍历该行，计算每个字符的外接矩形：

```
for (int charIndex = firstIndex;
    charIndex <= lastIndex;++charIndex){
 int charWidth = charWidthList[charIndex];
 QRect charRect(left, top, charWidth, m_fontHeight);
 m_rectList.push_back(charRect);
 left += charWidth;
}
```

对于文本的最后一行，我们还要添加一个存放最后一个字符之后位置的矩形：

```
if (lastIndex == (m_editorText.size() - 1)) {
  QRect lastRect(left, top, 1, m_fontHeight);
  m_rectList.push_back(lastRect);
}
```

每处理一个新行，top 字段就增加一个行高：

```
      top += m_fontHeight;
    }
  }
}
```

每当窗口需要重绘时框架就会调用 paintEvent 方法，或者我们通过调用 update 强制重绘时它也会被调用。在调用 paintEvent 之前，框架会先清空窗口内容。

我们首先创建一个随后用来书写的 QPainter 对象，并设置一些渲染提示和文本字体：

```
void EditorWidget::paintEvent(QPaintEvent* /* eventPtr */) {
   QPainter painter(this);
   painter.setRenderHint(QPainter::Antialiasing);
   painter.setRenderHint(QPainter::TextAntialiasing);
   painter.setFont(m_textFont);
```

我们计算被标记文本的最小和最大索引（尽管我们还没确定应用是否处于 mark 模式）：

```
int minIndex = qMin(m_firstIndex, m_lastIndex),
maxIndex = qMax(m_firstIndex, m_lastIndex);
```

我们遍历编辑器的文本，写出每个字符，换行符除外：

```
for (int index = 0; index < m_editorText.length(); ++index) {
  QChar c = m_editorText[index];
```

如果字符被标记，我们就以白字黑底写出它：

```
if (c != 'n') {
  if ((m_mode == Mark) &&
      (index >= minIndex) && (index < maxIndex)) {
    painter.setPen(Qt::white);
    painter.setBackground(Qt::black);
  }
```

如果字符未被标记，我们就以黑字白底写出它：

```
else {
  painter.setPen(Qt::black);
  painter.setBrush(Qt::white);
}
```

文本和背景的颜色都设置好之后，我们查找该字符的外接矩形，写出字符本身：

```
    QRect rect = m_rectList[index];
    painter.drawText(rect, c);
  }
}
```

最后，我们还绘制光标：

```
    m_caret.paint(&painter);
}
```

main 函数

main 函数与前面各应用的 main 函数相似：它创建应用，显示编辑器窗口，然后开始执行应用。

Main.cpp

```
#include "EditorWindow.h"
#include <QApplication>

int main(int argc, char *argv[]) {
  QApplication application(argc, argv);
  EditorWindow editorWindow;
  editorWindow.show();
  return application.exec();
}
```

程序的输出如下面的截图所示：



小结

本章我们开发了更高级版本的模拟时钟、画图程序和编辑器。时钟显示当前的时、分、秒；画图程序允许用户绘制直线、矩形和椭圆；编辑器允许用户输入和编辑文本。钟面用数字替代了刻度线；画图程序中我们可以标记、修改和剪切粘贴图形；编辑器中我们可以更改字体和对齐方式，并标记文本块。

第 7 章将开始开发游戏奥赛罗（Othello）和圈叉棋（井字棋）。

7
游戏

在第 6 章「增强 Qt 图形应用」中，我们用 Qt 图形库开发了一个模拟时钟、一个画图程序和一个编辑器。本章我们继续用 Qt 库开发奥赛罗棋（Othello，即黑白棋）和圈叉棋（Noughts and Crosses，即井字棋）这两个游戏。你可以在本篇引言之后找到对这两个游戏的介绍。本章我们从基本版本入手，由两名玩家互相对弈。在第 8 章「计算机对弈」中，我们将改进这两个游戏，让计算机与人类对弈。

本章将涵盖以下主题：

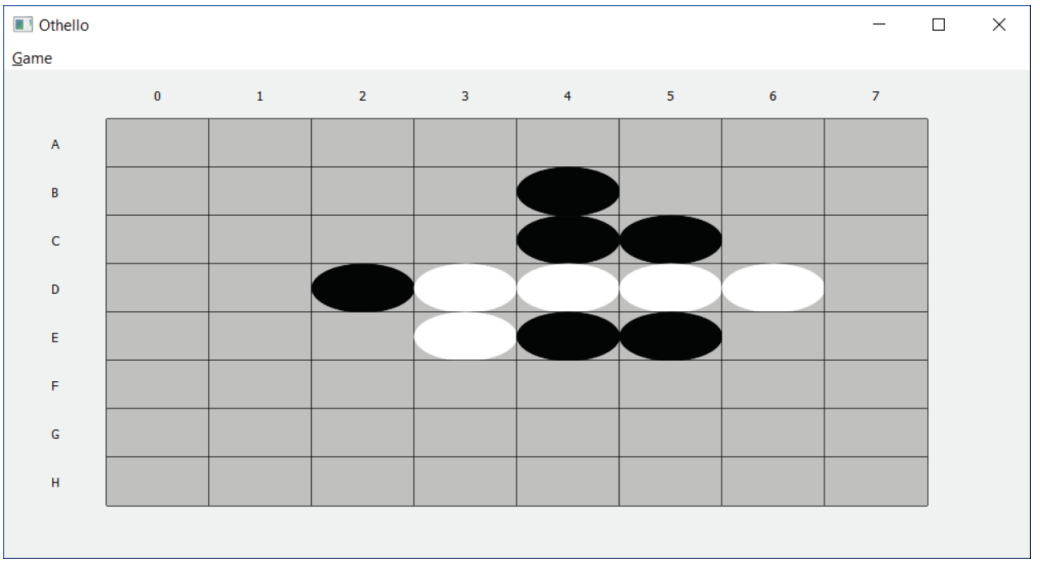
- 博弈论简介。我们开发一个游戏棋盘（game grid），玩家在其中轮流把自己的棋子加入棋盘。
- 我们宣布获胜者。在奥赛罗棋中，每走一步之后，我们计算有多少对方的棋子可以被翻转。当棋盘的每个位置都被占据之后，我们宣布获胜者或平局。
- 在圈叉棋中，我们统计一行中棋子的数量。如果一行中有五枚棋子，我们就宣布获胜者。
- 我们继续使用类、字段和方法等 C++ 特性，也继续使用窗口和部件等 Qt 特性。

奥赛罗棋

在奥赛罗棋中，游戏开始时棋盘是空的。对局过程中，两名玩家轮流往棋盘上添加黑、白两色的棋子。每当一名玩家添加一枚棋子，我们就查看其他棋子，看看这枚新棋子是否把对方的某些棋子围住了。如果是，我们就把被围住的对方棋子换成己方的颜色。

例如，如果黑方玩家在一处位置添加一枚黑棋子，而该位置左边的三枚棋子是白色的、第四枚是黑色的，那么这三枚白色棋子就被这两枚黑色棋子围住了，它们会被换成黑色棋子。当棋盘上的每个位置都被黑色和白色棋子占满之后，我们统计棋子数量，拥有最多棋子的玩家获胜。如果黑色和白色棋子的数量相等，就是平局。

下面就是我们这个游戏应有的样子：



游戏部件

首先，我们需要一个游戏棋盘。GameWidget 类是本章以及第 8 章「计算机对弈」中所有应用程序共有的。在第 5 章「Qt 图形应用」和第 6 章「增强 Qt 图形应用」中，我们开发的是 DocumentWidget 类，因为那时处理的是基于文档的应用程序。而在本章和第 8 章「计算机对弈」中，我们转而开发 GameWidget 类。

前两章的 DocumentWidget 类与本章及下一章的 GameWidget 类既有相似之处，也有不同之处。它们都是 Qt 类 QWidget 的子类，都打算嵌入到窗口中。不过，DocumentWidget 用来容纳文档，而 GameWidget 用来容纳游戏棋盘。它负责绘制棋盘，并捕获棋盘各个位置上的鼠标点击。GameWidget 是一个抽象类，让它的子类定义当用户点击鼠标时、或棋盘某个位置需要重绘时被调用的方法。

不过，我们复用前几章的 MainWindow 类来存放应用程序的主窗口及其菜单栏。

GameWidget.h

```
#ifndef GAMEWIDGET_H
#define GAMEWIDGET_H

#include <QPainter>
#include <QMouseEvent>
#include <QMessageBox>

#include "..\MainWindow\MainWindow.h"
```

```
class GameWidget : public QWidget {
  Q_OBJECT
```

构造函数初始化棋盘的行数和列数：

```
public:
  GameWidget(int rows, int columns, QWidget* parentWidget);
```

clearGrid 方法把棋盘中的每个位置都设为零，零被假定表示空位置。因此，每个继承 GameWidget 的类都应让零表示空位置：

```
void clearGrid();
```

当用户改变窗口大小时会调用 resizeEvent 方法。由于行数和列数是常数，每个位置的宽和高会按照窗口的新尺寸相应改变：

```
void resizeEvent(QResizeEvent *eventPtr);
```

当用户按下某个鼠标按钮时会调用 mousePressEvent，当窗口需要重绘时会调用 paintEvent，当用户点击窗口右上角的关闭框时会调用 closeEvent：

```
void mousePressEvent(QMouseEvent *eventPtr);
void paintEvent(QPaintEvent *eventPtr);
void closeEvent(QCloseEvent *eventPtr);
```

mouseMark 和 drawMark 方法是打算被子类重写的纯虚方法：当用户点击棋盘中的某个位置时会调用 mouseMark，当某个位置需要重绘时会调用 drawMark。它们是纯虚方法，GameWidget 因此是抽象的，这意味着 GameWidget 只能用作基类。GameWidget 的子类必须重写这些方法才能成为非抽象类：

```
virtual void mouseMark(int row, int column) = 0;
virtual void drawMark(QPainter& painter,
                      const QRect& markRect, int mark) = 0;
```

isQuitOk 方法显示一个消息框，询问用户是否真的想退出对局：

```
private:
  bool isQuitOk();
```

在 Game 菜单变为可见之前会调用 isQuitEnabled 方法。当对局正在进行时，Quit 菜单项才被启用：

```
public slots:
  DEFINE_LISTENER(GameWidget, isQuitEnabled);
```

当用户选择 Quit 或 Exit 菜单项时会调用 onQuit 和 onExit 方法：

```
    void onQuit();
    void onExit();
```

isGameInProgress 和 setGameInProgress 方法返回并设置 m_gameInProgress 字段的值：

```
  protected:
    bool isGameInProgress() const {return m_gameInProgress;}
    void setGameInProgress(bool active)
                           {m_gameInProgress = active;}
```

get 和 set 方法读取并设置棋盘中某个位置上的值。值是一个整数；记住，空位置被假定保存值零：

```
  protected:
    int get(int row, int column) const;
    void set(int row, int column, int value);
```

只要对局正在进行，m_gameInProgress 字段就为 true。m_rows 和 m_columns 字段保存棋盘的行数和列数；m_rowHeight 和 m_columnWidth 保存棋盘中每个位置以像素为单位的高度和宽度。最后，m_gameGrid 是指向一个缓冲区的指针，缓冲区中保存着棋盘各个位置上的值：

```
    private:
      bool m_gameInProgress = false;
      int m_rows, m_columns;
      int m_rowHeight, m_columnWidth;
      int* m_gameGrid;
};

#endif // GAMEWIDGET_H
```

GameWidget.cpp 文件存放 GameWidget 类各个方法的定义、鼠标事件方法、菜单方法，以及棋子的绘制和设置。

GameWidget.cpp

```
#include "GameWidget.h"
#include <QApplication>
#include <CAssert>
```

构造函数初始化棋盘的行数和列数，动态分配内存，并调用 clearGrid 来清空棋盘：

```
GameWidget::GameWidget(int rows, int columns,
                       QWidget* parentWidget)
 :QWidget(parentWidget),
  m_rows(rows),
  m_columns(columns),
  m_gameGrid(new int[rows * columns]) {
```

```
  assert(rows > 0);
  assert(columns > 0);
  clearGrid();
}
```

get 方法返回行和列所指位置上的值，set 设置该值。保存值的缓冲区按行组织。也就是说，缓冲区的第一部分存放第一行，然后是第二行，依此类推：

```
int GameWidget::get(int row, int column) const {
  return m_gameGrid[(row * m_columns) + column];
}

void GameWidget::set(int row, int column, int value) {
  m_gameGrid[(row * m_columns) + column] = value;
}
```

clearGrid 方法把每个位置都设为零，因为零被假定表示空位置：

```
void GameWidget::clearGrid() {
  for (int row = 0; row < m_rows; ++row) {
    for (int column = 0; column < m_columns; ++column) {
      set(row, column, 0);
    }
  }
}
```

只要对局正在进行，Quit 菜单项就处于启用状态：

```
bool GameWidget::isQuitEnabled() {
  return m_gameInProgress;
}
```

当用户选择退出对局时，如果对局正在进行，就会显示一个带有确认问题的消息框：

```
bool GameWidget::isQuitOk() {
  if (m_gameInProgress) {
    QMessageBox messageBox(QMessageBox::Warning,
                           tr("Quit"), QString());
    messageBox.setText(tr("Quit the Game."));
    messageBox.setInformativeText
                  (tr("Do you really want to quit the game?"));
    messageBox.setStandardButtons(QMessageBox::Yes |
                                  QMessageBox::No);
    messageBox.setDefaultButton(QMessageBox::No);
```

如果用户按下 Yes 按钮，就返回 true：

```
  return (messageBox.exec() == QMessageBox::Yes);
}
```

如果没有对局正在进行，则不需要任何确认：

```
  return true;
}
```

当用户选择 Quit 菜单项时会调用 onQuit 方法。如果对 isQuitOk 的调用返回 true，就把 `m_gameInProgress` 设为 false，并调用 update，这最终会强制重绘窗口，从而清空棋盘。

```
void GameWidget::onQuit() {
  if (isQuitOk()) {
    m_gameInProgress = false;
    update();
  }
}
```

当用户选择 Exit 菜单项时会调用 onExit 方法。如果对 isQuitOk 的调用返回 true，应用程序就退出。如下面的代码所示：

```
void GameWidget::onExit() {
  if (isQuitOk()) {
    qApp->exit(0);
  }
}
```

当用户调整窗口大小时会调用 resizeEvent 方法。由于无论窗口大小如何，行数和列数都是常数，因此需要重新计算行高和列宽。我们把窗口的高度和宽度除以行数和列数加二，因为我们添加了额外的行和列作为边距。考虑下面的代码：

```
void GameWidget::resizeEvent(QResizeEvent* eventPtr) {
  m_rowHeight = height() / (m_rows + 2);
  m_columnWidth = width() / (m_columns + 2);
  QWidget::resizeEvent(eventPtr);
  update();
}
```

当用户点击窗口时会调用 mousePressEvent 方法：

```
void GameWidget::mousePressEvent(QMouseEvent* eventPtr) {
  if (m_gameInProgress &&
        (eventPtr->button() == Qt::LeftButton)) {
    QPoint mousePoint = eventPtr->pos();
```

由于棋盘被边距包围，需要从鼠标点中减去列宽和行高：

```
    mousePoint.setX(mousePoint.x() - m_columnWidth);
    mousePoint.setY(mousePoint.y() - m_rowHeight);
```

如果鼠标点位于棋盘的某个位置之内，且该位置为空（零），就调用纯虚方法 mouseMark，由它负责鼠标点击的实际动作。在下一节中，会往棋盘添加黑色和白色棋子；在稍后的圈叉棋应用程序中，则会往棋盘添加圈和叉：

```
    int row = mousePoint.y() / m_rowHeight,
       column = mousePoint.x() / m_columnWidth;
```

如果所点击的行和列位于棋盘之内（而不是棋盘外的边距中），且该位置为空（零），我们就以该行和列为参数调用纯虚方法 mouseMark：

```
      if ((row < m_rows) && (column < m_columns) &&
          (get(row, column) == 0)) {
        mouseMark(row, column);
        update();
      }
    }
  }
```

当窗口需要重绘时会调用 paintEvent 方法。如果对局正在进行（m_gameInProgress 为 true），就写出各行和各列，然后对棋盘中的每个位置调用纯虚方法 drawMark，由它负责每个位置的实际绘制：

```
  void GameWidget::paintEvent(QPaintEvent* /*eventPtr*/) {
    if (m_gameInProgress) {
      QPainter painter(this);
      painter.setRenderHint(QPainter::Antialiasing);
      painter.setRenderHint(QPainter::TextAntialiasing);
```

首先，我们遍历各行，对每一行写出一个从 A 到 Z 的字母。字母表共有 26 个字母，我们假定行数不超过 26：

```
      for (int row = 0; row < m_rows; ++row) {
        QString text;
        text.sprintf("%c", (char) (((int) 'A') + row));
        QRect charRect(0, (row + 1) * m_rowHeight,
                       m_columnWidth, m_rowHeight);
```



```
        painter.drawText(charRect, Qt::AlignCenter |
                         Qt::AlignHCenter, text);
  }
```

然后我们遍历各列，对每一列写出它的编号：

```
for (int column = 0; column < m_columns; ++column) {
  QString text;
  text.sprintf("%i", column);
  QRect charRect((column + 1) * m_columnWidth, 0,
                 m_columnWidth, m_rowHeight);
  painter.drawText(charRect, Qt::AlignCenter |
                   Qt::AlignHCenter, text);
}

painter.save();
painter.translate(m_columnWidth, m_rowHeight);
```



纯虚方法是不打算在类本身中定义、而只打算在其子类中定义的方法。持有至少一个纯虚方法的类将成为抽象类，这意味着无法创建该类的对象。这样的类只能用作类层次结构中的基类。继承抽象类的类必须定义基类的每一个纯虚方法，否则它自己也会成为抽象类。

最后，我们遍历棋盘，对每个位置，以该位置的矩形和它当前的棋子为参数调用纯虚方法 drawMark：

```
    for (int row = 0; row < m_rows; ++row) {
      for (int column = 0; column < m_columns; ++column) {
        QRect markRect(column * m_columnWidth, row * m_rowHeight,
                       m_columnWidth, m_rowHeight);
        painter.setPen(Qt::black);
        painter.drawRect(markRect);
        painter.fillRect(markRect, Qt::lightGray);
        drawMark(painter, markRect, get(row, column));
      }
    }

    painter.restore();
  }
}
```

当用户点击窗口右上角的关闭框时会调用 closeEvent 方法。如果对 isQuitOk 的调用返回 true，窗口就关闭，应用程序退出：

```
void GameWidget::closeEvent(QCloseEvent* eventPtr) {
  if (isQuitOk()) {
    eventPtr->accept();
    qApp->exit(0);
  }
  else {
```

```
        eventPtr->ignore();
      }
    }
```

OthelloWindow 类

OthelloWindow 类继承自第 6 章「增强 Qt 图形应用」的 MainWindow。它向窗口添加菜单，并把 OthelloWidget 类——GameWidget 的一个子类——设为窗口的中心部件。

OthelloWindow.h

```
#ifndef OTHELLOWINDOW_H
#define OTHELLOWINDOW_H

#include "..\MainWindow\MainWindow.h"
#include "OthelloWidget.h"

class OthelloWindow : public MainWindow {
  Q_OBJECT

  public:
    OthelloWindow(QWidget *parentWidget = nullptr);
    ~OthelloWindow();

    void closeEvent(QCloseEvent *eventPtr)
      {m_othelloWidgetPtr->closeEvent(eventPtr);}
```

`m_othelloWidgetPtr` 字段保存指向窗口中心部件的指针。它指向一个 OthelloWidget 类的对象：

```
  private:
    OthelloWidget* m_othelloWidgetPtr;
};

#endif // OTHELLOWINDOW_H
```

OthelloWindow.cpp 文件存放 OthelloWindow 类各个方法的定义。

OthelloWindow.cpp

```
#include "OthelloWidget.h"
#include "OthelloWindow.h"
#include <QtWidgets>
```

构造函数把窗口的标题设为 Othello，并把大小设为 1000 × 500 像素：

```
OthelloWindow::OthelloWindow(QWidget *parentWidget /*= nullptr*/)
 :MainWindow(parentWidget) {
  setWindowTitle(tr("Othello"));
  resize(1000, 500);
```

一个 OthelloWidget 对象被动态创建，并放在窗口的中心：

```
m_othelloWidgetPtr = new OthelloWidget(this);
setCentralWidget(m_othelloWidgetPtr);
```

我们向菜单栏添加 Game 菜单，并把 onMenuShow 方法连接到该菜单，使得它在菜单变为可见之前被调用：

```
{ QMenu* gameMenuPtr = menuBar()->addMenu(tr("&Game"));
  connect(gameMenuPtr, SIGNAL(aboutToShow()),
          this, SLOT(onMenuShow()));
```

用户可以选择黑色或白色来先走。在菜单项变为可见之前会调用 isBlackStartsEnabled 和 isWhiteStartsEnabled 方法。当对局正在进行时，菜单项被禁用：

```
addAction(gameMenuPtr, tr("&Black Starts"),
          SLOT(onBlackStarts()), 0,
          tr("Black Starts"), nullptr,tr("Black Starts"),
          LISTENER(isBlackStartsEnabled));

addAction(gameMenuPtr, tr("&White Starts"),
          SLOT(onWhiteStarts()), 0,
          tr("White Starts"), nullptr, tr("White Starts"),
          LISTENER(isWhiteStartsEnabled));

gameMenuPtr->addSeparator();
```

当对局正在进行时，用户可以退出对局。当没有对局正在进行时，该菜单项被禁用：

```
addAction(gameMenuPtr, tr("&Quit the Game"),
          SLOT(onQuit()),
          QKeySequence(Qt::CTRL + Qt::Key_Q),
          tr("Quit Game"), nullptr, tr("Quit the Game"),
          LISTENER(isQuitEnabled));
```

用户可以随时退出应用程序：

```
    addAction(gameMenuPtr, tr("E&xit"),
              SLOT(onExit()), QKeySequence::Quit);
  }
}
```

析构函数释放窗口中心的奥赛罗部件：

```
OthelloWindow::~OthelloWindow() {
  delete m_othelloWidgetPtr;
}
```

OthelloWidget 类

OthelloWidget 是本章开头定义的 GameWidget 类的子类。它通过重写 mouseMark 和 drawMark 成为非抽象类：当用户点击棋盘中的某个位置时会调用 mouseMark，当某个位置需要重绘时会调用 drawMark。

OthelloWidget.h

```
#ifndef OTHELLOWIDGET_H
#define OTHELLOWIDGET_H

#include "..\MainWindow\GameWidget.h"

#define ROWS    8
#define COLUMNS 8
```

奥赛罗棋中的棋子可以是黑色或白色。我们使用 Mark 枚举来保存棋盘上的值。Empty 项的值为零，而 GameWidget 假定零表示空位置：

```
enum Mark {Empty = 0, Black, White};

class OthelloWidget : public GameWidget {

Q_OBJECT

public:
  OthelloWidget(QWidget* parentWidget);

  void mouseMark(int row, int column);
  void drawMark(QPainter& painter,
                const QRect& markRect, int mark);
```

在 BlackStarts 和 WhiteStarts 菜单项变为可见之前会调用 isBlackStartsEnabled 和 isWhiteStartsEnabled 监听器，以决定是否启用它们。注意，监听器和方法必须标记为 public slots，菜单框架才会把它们当作监听器：

```
public slots:
  DEFINE_LISTENER(OthelloWidget, isBlackStartsEnabled);
  DEFINE_LISTENER(OthelloWidget, isWhiteStartsEnabled);
```

当用户选择 BlackStarts 和 WhiteStarts 菜单项时会调用 onBlackStarts 和 onWhiteStarts 方法：

```
  void onBlackStarts();
  void onWhiteStarts();
```

checkWinner 方法检查棋盘上的每个位置是否都已被黑色或白色棋子占据。如果是，就统计棋子，并宣布获胜者，除非是平局：

```
private:
  void checkWinner();
```

turn 方法在其中一名玩家走了一步之后被调用。它计算这次走法将要翻转的位置：

```
  void turn(int row, int column, Mark mark);
```

calculateMark 方法计算如果玩家把棋子放在行和列所指定的位置，需要翻转的棋子集合：

```
  void calculateMark(int row, int column, Mark mark,
                     QSet<QPair<int,int>>& resultSet);
```

m_nextMark 字段交替取前述 Mark 枚举中的 Black 和 White 值，取决于接下来要由哪位玩家走棋。它由 onBlackStarts 或 onWhiteStarts 初始化，如前面的代码所示：

```
  Mark m_nextMark;
};

#endif // OTHELLOWIDGET_H
```

OthelloWidget 类持有对局的功能。它允许玩家往棋盘添加黑色和白色棋子、翻转棋子，并宣布获胜者。

OthelloWidget.cpp

```
#include "OthelloWidget.h"
#include "OthelloWindow.h"

#include <QTime>
#include <CTime>
#include <CAssert>
using namespace std;

OthelloWidget::OthelloWidget(QWidget* parentWidget)
 :GameWidget(ROWS, COLUMNS, parentWidget) {
  // Empty.
}
```

当对局尚未开始时，BlackStarts 和 WhiteStarts 菜单项处于启用状态：

```
bool OthelloWidget::isBlackStartsEnabled() {
  return !isGameInProgress();
}

bool OthelloWidget::isWhiteStartsEnabled() {
  return !isGameInProgress();
}
```

onBlackStarts 和 onWhiteStarts 方法开始一局新对局，设置先走的那枚棋子（黑色或白色），清空棋盘，并更新窗口以绘制一个空的棋盘：

```
void OthelloWidget::onBlackStarts() {
  setGameInProgress(true);
  m_nextMark = Black;
  update();
}

void OthelloWidget::onWhiteStarts() {
  setGameInProgress(true);
  m_nextMark = White;
  update();
}
```

当玩家点击棋盘中的一个空位置时会调用 mouseMark 方法。我们用下一枚棋子设置该位置，翻转受这次走法影响的每一枚棋子，并更新窗口以反映变化：

```
void OthelloWidget::mouseMark(int row, int column) {
  set(row, column, m_nextMark);
  turn(row, column, m_nextMark);
  update();
```

我们检查这次走法是否使棋盘填满，并切换下一枚棋子：

```
  checkWinner();
  m_nextMark = (m_nextMark == Black) ? White : Black;
}
```

当棋盘中的某个位置需要重绘时会调用 drawMark 方法。如果该位置不为空，我们就绘制一个带黑色边框的黑色或白色椭圆。如果位置为空，我们什么也不做。注意，框架会在调用重绘之前清空窗口：

```
void OthelloWidget::drawMark(QPainter& painter,
     const QRect& markRect, int mark) {
  painter.setPen(Qt::black);
  painter.drawRect(markRect);
  painter.fillRect(markRect, Qt::lightGray);
```

```
  switch (mark) {
    case Black:
      painter.setPen(Qt::black);
      painter.setBrush(Qt::black);
      painter.drawEllipse(markRect);
      break;

    case White:
      painter.setPen(Qt::white);
      painter.setBrush(Qt::white);
      painter.drawEllipse(markRect);
      break;

    case Empty:
      break;
  }
}
```

checkWinner 方法统计被黑色和白色棋子占据的位置数量以及空位置的数量：

```
void OthelloWidget::checkWinner() {
  int blacks = 0, whites = 0, empties = 0;

  for (int row = 0; row < ROWS; ++row) {
    for (int column = 0; column < COLUMNS; ++column) {
      switch (get(row, column)) {
        case Black:
          ++blacks;
          break;

        case White:
          ++whites;
          break;

        case Empty:
          ++empties;
          break;
      }
    }
  }
```

如果没有空位置剩下，对局就结束了，我们宣布获胜者，除非是平局。获胜者是拥有最多己方颜色棋子的玩家：

```
  if (empties == 0) {
    QMessageBox messageBox(QMessageBox::Information,
        tr("Victory"), QString());
```

```
    QString text;

    if (blacks == whites) {
      text.sprintf("A Draw.");
    }
    else if (blacks > whites) {
      text.sprintf("The Winner: Black");
    }
    else {
      text.sprintf("The Winner: White");
    }

    messageBox.setText(text);
    messageBox.setStandardButtons(QMessageBox::Ok);
    messageBox.exec();
    setGameInProgress(false);

    clearGrid();
    update();
  }
}
```

turn 方法调用 calculateMark 来获得需要翻转棋子的位置集合。然后，集合中的每个位置都被设为相应的棋子。

在这个应用程序中，turn 是唯一调用 calculateMark 的方法。然而，在第 8 章「计算机对弈」中，calculateMark 还会被调用来计算计算机玩家的走法。因此，turn 和 calculateMark 的功能被分成了两个方法：

```
void OthelloWidget::turn(int row, int column, Mark mark) {
  QSet<QPair<int,int>> totalSet;
  calculateMark(row, column, mark, totalSet);

  for (QPair<int,int> pair : totalSet) {
    int row = pair.first, column = pair.second;
    set(row, column, mark);
  }
}
```

calculateMark 方法统计棋盘中每个位置在所有八个方向上将翻转的棋子数量：

```
void OthelloWidget::calculateMark(int row, int column,
    Mark playerMark, QSet<QPair<int,int>>& totalSet){
```

`directionArray` 中的每个整数对都按照罗盘方位对应一个方向：

```
QPair<int,int> directionArray[] =
  {QPair<int,int>(-1, 0),   // 北
   QPair<int,int>(-1, 1),   // 东北
   QPair<int,int>(0, 1),    // 东
   QPair<int,int>(1, 1),    // 东南
   QPair<int,int>(1, 0),    // 南
   QPair<int,int>(1, -1),   // 西南
   QPair<int,int>(0, -1),   // 西
   QPair<int,int>(-1, -1)}; // 西北
```

数组的大小可以用数组的总大小（以字节为单位）除以第一个值的大小来确定：

```
int arraySize =
  (sizeof directionArray) / (sizeof directionArray[0]);
```

我们遍历各个方向；对每个方向，只要找到的是对方的棋子，就沿该方向继续移动：

```
for (int index = 0; index < arraySize; ++index) {
  QPair<int,int> pair = directionArray[index];
```

在沿该方向迭代的过程中，row 和 column 字段保存当前的行和列：

```
int rowStep = pair.first, columnStep = pair.second,
    currRow = row, currColumn = column;
```

我们把迭代过程中找到的棋子收集在 directionSet 中：

```
QSet<QPair<int,int>> directionSet;

while (true) {
  currRow += rowStep;
  currColumn += columnStep;
```

如果我们到达棋盘的某条边界，或者找到空位置，就中断迭代：

```
if ((currRow < 0) || (currRow == ROWS) ||
    (currColumn < 0) || (currColumn == COLUMNS) ||
    (get(currRow, currColumn) == Empty)) {
  break;
}
```

如果我们找到玩家的棋子，就把方向集合并入总集合，并中断迭代：

```
else if (get(currRow, currColumn) == playerMark) {
  totalSet += directionSet;
  break;
}
```

如果我们既没有找到玩家的棋子，也没有找到空位置，那么找到的就是对方的棋子，我们就把

它的位置加入方向集合：

```
            else {
              directionSet.insert(QPair<int,int>(row, column));
            }
          }
        }
      }
```

main 函数

main 函数的工作方式与前面的 Qt 应用程序相同。它创建一个应用程序，显示奥赛罗棋窗口，并执行应用程序。执行一直持续到 exit 方法被调用——当用户关闭窗口或选择 Exit 菜单项时就会调用它。

Main.cpp

```
#include "OthelloWidget.h"
#include "OthelloWindow.h"
#include <QApplication>

int main(int argc, char *argv[]) {
  QApplication application(argc, argv);
  OthelloWindow othelloWindow;
  othelloWindow.show();
  return application.exec();
}
```

圈叉棋

圈叉棋应用程序搭建一个游戏棋盘，并允许两名玩家互相对弈。在圈叉棋中，两名玩家轮流往棋盘上添加圈和叉。最先成功把五枚棋子排成一行者获胜。棋子可以水平、垂直或沿对角线放置。当每名玩家试图把自己的五枚棋子排成一行时，也必须设法阻止对方排出五子。

在第 8 章「计算机对弈」中，计算机将与人类对弈。

NaCWindow 类

我们复用游戏部件一节中的 GameWidget。NaCWindow 类与 OthelloWindow 类相似。它向窗口的菜单栏添加 Nought Starts 和 Cross Starts 菜单项。

NaCWindow.h

```
#ifndef NACWINDOW_H
#define NACWINDOW_H

#include "..\MainWindow\MainWindow.h"
#include "NaCWidget.h"

class NaCWindow : public MainWindow {
  Q_OBJECT

  public:
    NaCWindow(QWidget *parentWidget = nullptr);
    ~NaCWindow();

  public:
    void closeEvent(QCloseEvent *eventPtr) override
                    {m_nacWidgetPtr->closeEvent(eventPtr);}

  private:
    NaCWidget* m_nacWidgetPtr;
};

#endif // NACWINDOW_H
```

NaCWindow.cpp 文件存放 NaCWindow 类各个方法的定义。

NaCWindow.cpp

```
#include "NaCWindow.h"
#include <QtWidgets>

NaCWindow::NaCWindow(QWidget *parentWidget /*= nullptr*/)
 :MainWindow(parentWidget) {
  setWindowTitle(tr("Noughts and Crosses"));
  resize(1000, 500);

  m_nacWidgetPtr = new NaCWidget(this);
  setCentralWidget(m_nacWidgetPtr);

  { QMenu* gameMenuPtr = menuBar()->addMenu(tr("&Game"));

    connect(gameMenuPtr, SIGNAL(aboutToShow()),
            this, SLOT(onMenuShow()));

    addAction(gameMenuPtr, tr("&Nought Starts"),
              SLOT(onNoughtStarts()), 0,
              tr("Nought Starts"), nullptr, tr("Nought Starts"),
              LISTENER(isNoughtStartsEnabled));
```

```
        addAction(gameMenuPtr, tr("&Cross Starts"),
                  SLOT(onCrossStarts()), 0,
                  tr("Cross Starts"), nullptr, tr("Cross Starts"),
                  LISTENER(isCrossStartsEnabled));

        gameMenuPtr->addSeparator();

        addAction(gameMenuPtr, tr("&Quit the Game"),
                  SLOT(onQuit()),
                  QKeySequence(Qt::CTRL + Qt::Key_Q), tr("Quit Game"),
                  nullptr, tr("Quit the Game"),
                  LISTENER(isQuitEnabled));

        addAction(gameMenuPtr, tr("E&xit"),
                  SLOT(onExit()), QKeySequence::Quit);
      }
    }

    NaCWindow::~NaCWindow() {
      delete m_nacWidgetPtr;
    }
```

NaCWidget 类

NaCWidget 类处理圈叉棋的功能。它允许两名玩家互相对弈。在第 8 章「计算机对弈」中，我们将编写一个计算机与人类对弈的游戏。

NaCWidget.h

```
#ifndef NACWIDGET_H
#define NACWIDGET_H

#include "..\MainWindow\GameWidget.h"

#define ROWS    26
#define COLUMNS 26
```

与奥赛罗棋应用程序类似，棋盘中的一个位置可以保存三个值之一：

- Empty（即零）；
- Nought（圈）；
- Cross（叉）。

Mark 枚举对应 Empty、Nought 和 Cross 这三个值：

```
enum Mark {Empty = 0, Nought, Cross};

class NaCWidget : public GameWidget {
  Q_OBJECT

  public:
    NaCWidget(QWidget* parentWidget);

    void mouseMark(int row, int column);
    void drawMark(QPainter& painter,
                  const QRect& markRect, int mark);

  public slots:
    DEFINE_LISTENER(NaCWidget, isNoughtStartsEnabled);
    void onNoughtStarts();

    DEFINE_LISTENER(NaCWidget, isCrossStartsEnabled);
    void onCrossStarts();

  private:
    void checkWinner(int row, int column, Mark mark);
    int countMarks(int row, int column, int rowStep,
                   int columnStep, Mark mark);

    Mark m_nextMark;
};

#endif // NACWIDGET_H
```

NaCWidget.cpp 文件存放 NaCWidget 类各个方法的定义。

NaCWidget.cpp

```
#include "NaCWidget.h"
#include <CTime>

NaCWidget::NaCWidget(QWidget* parentWidget)
 :GameWidget(ROWS, COLUMNS, parentWidget) {
  // Empty.
}
```

在 Game 菜单变为可见之前会调用 isNoughtStartsEnabled 和 isCrossStartsEnabled 方法。当没有对局正在进行时，Nought Starts 和 Cross Starts 菜单项处于启用状态：

```
bool NaCWidget::isCrossStartsEnabled() {
  return !isGameInProgress();
}
```

```
bool NaCWidget::isNoughtStartsEnabled() {
  return !isGameInProgress();
}
```

当用户选择 Nought Starts 和 Cross Starts 菜单项时会调用 onNoughtStarts 和 onCrossStarts 方法。它们把对局设为进行中，设置用来先走的那枚棋子（m_nextMark），并通过调用 update 强制重绘棋盘：

```
void NaCWidget::onNoughtStarts() {
  setGameInProgress(true);
  m_nextMark = Nought;
  update();
}

void NaCWidget::onCrossStarts() {
  setGameInProgress(true);
  m_nextMark = Cross;
  update();
}
```

当玩家点击棋盘中的某个位置时会调用 mouseMark 方法。我们在该位置设置下一枚棋子，检查是否有玩家赢得了对局，交换下一手棋，并通过调用 update 重绘窗口：

```
void NaCWidget::mouseMark(int row, int column) {
  set(row, column, m_nextMark);
  checkWinner(row, column, m_nextMark);
  m_nextMark = (m_nextMark == Nought) ? Cross : Nought;
  update();
}
```

当棋盘中的某个位置需要重绘时会调用 drawMark 方法：

```
void NaCWidget::drawMark(QPainter& painter,
    const QRect& markRect, int mark) {
```

我们把画笔颜色设为黑色；如果是圈，就画一个椭圆，如下所示：

```
painter.setPen(Qt::black);
switch (mark) {
  case Nought:
    painter.drawEllipse(markRect);
    break;
```

如果是叉，我们就画两条直线，分别连接左上角和右下角、右上角和左下角：

```
            case Cross:
              painter.drawLine(markRect.topLeft(),
                               markRect.bottomRight());
              painter.drawLine(markRect.topRight(),
                               markRect.bottomLeft());
              break;
```

如果是空位置，我们什么也不做。记住，框架会在重绘之前清空窗口：

```
            case Empty:
              break;
          }
        }
```

当一名玩家走了一步之后，我们检查这次走法是否导致了胜利。我们在四个方向上调用 countMarks 来做 checkWinner，看看这次走法是否造成了五枚棋子连成一行：

```
        void NaCWidget::checkWinner(int row, int column, Mark mark) {
```

南北方向的代码是：

```
          if ((countMarks(row, column, -1, 0, mark) >= 5) ||
```

东西方向的代码是：

```
              (countMarks(row, column, 0, -1, mark) >= 5) ||
```

西北—东南方向的代码是：

```
              (countMarks(row, column, -1, 1, mark) >= 5) ||
```

东南—西北方向的代码是：

```
              (countMarks(row, column, 1, 1, mark) >= 5)) {
```

如果这次走法造成了五子连珠，我们就显示一个带有获胜者（圈或叉）的消息框。在圈叉棋中不会有平局：

```
            QMessageBox messageBox(QMessageBox::Information,
                                   tr("Victory"), QString());
            QString text;
            text.sprintf("The Winner: %s.",
                         (mark == Nought) ? "Nought" : "Cross");

            messageBox.setText(text);
            messageBox.setStandardButtons(QMessageBox::Ok);
            messageBox.exec();
            setGameInProgress(false);
```

棋盘被清空，从而准备好下一局对局：

```
    clearGrid();
    update();
  }
}
```

countMarks 方法统计一行中棋子的数量。我们在两个方向上都调用 countMarks。例如，如果 rowStep 和 columnStep 都是负一，我们就在每次迭代中把当前行和当前列都减一。这意味着第一次迭代时我们在东北方向上调用 countMarks。第二次迭代时，我们在相反的方向上调用 countMarks，也就是西南方向：

```
int NaCWidget::countMarks(int row, int column, int rowStep,
                          int columnStep, Mark mark) {
  int countMarks = 0;
```

我们会一直计数，直到遇到棋盘的某条边界，或者找到一枚不是我们正在计数的棋子——也就是说，是对方的棋子或一枚空棋子：

```
{ int currentRow = row, currentColumn = column;
  while ((currentRow >= 0) && (currentRow < ROWS) &&
         (currentColumn >= 0) && (currentColumn < COLUMNS) &&
         (get(currentRow, currentColumn) == mark)) {
    ++countMarks;
    currentRow += rowStep;
    currentColumn += columnStep;
  }
}
```

在第二次迭代中，我们不是加上行和列的步长，而是减去它们。这样，我们就在相反的方向上调用 countMarks。我们还通过加上步长来初始化当前的行和列，这样就不会把中间那枚棋子计数两次：

```
  { int currentRow = row + rowStep,
        currentColumn = column + columnStep;
    while ((currentRow >= 0) && (currentRow < ROWS) &&
           (currentColumn >= 0) && (currentColumn < COLUMNS) &&
           (get(currentRow, currentColumn) == mark)) {
      ++countMarks;
      currentRow -= rowStep;
      currentColumn -= columnStep;
    }
  }

  return countMarks;
}
```

main 函数

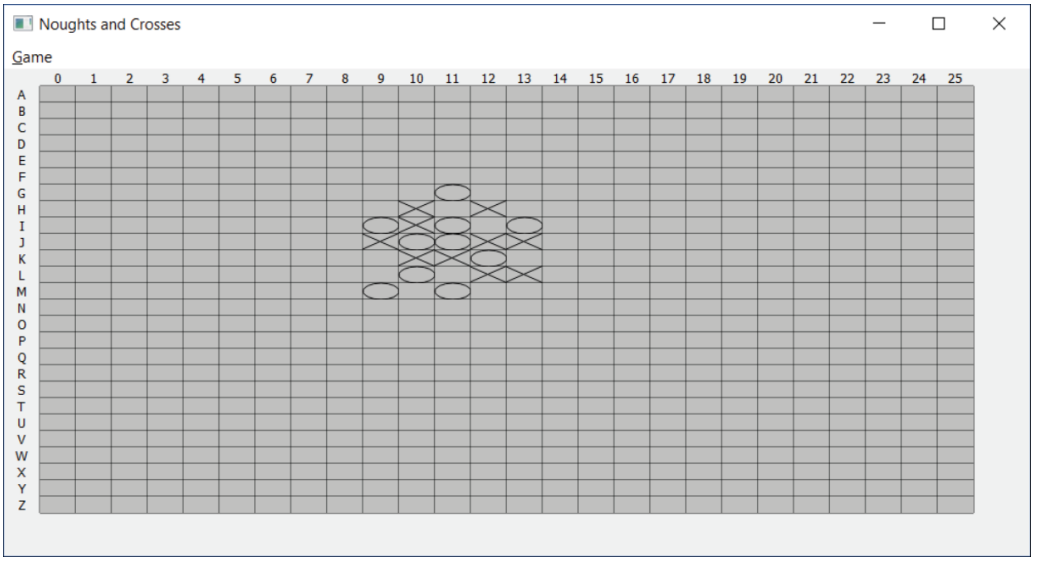
main 函数创建应用程序，显示窗口，并执行应用程序，直到用户关闭窗口或选择 Exit 菜单项。

Main.cpp

```
#include "NaCWidget.h"
#include "NaCWindow.h"
#include <QApplication>

int main(int argc, char *argv[]) {
  QApplication application(argc, argv);
  NaCWindow mainWindow;
  mainWindow.show();
  return application.exec();
}
```

运行前面的代码后，输出如下所示：



小结

本章我们开发了奥赛罗棋和圈叉棋这两个游戏。我们初步接触了博弈论，并开发了一个游戏棋盘，玩家在其中轮流添加自己的棋子。在奥赛罗棋中，我们开发了为每一步统计待翻转棋子数量的方法；在圈叉棋中，我们开发了识别是否有玩家成功把五枚棋子排成一行的方法——如果

有，我们就宣布该玩家为获胜者。

在第 8 章「计算机对弈」中，我们将开发这两个游戏的更高级版本，让计算机与人类对弈。

8
计算机对弈

本章我们继续开发奥赛罗棋和圈叉棋这两个游戏。本章的新内容是计算机与人类对弈：不再是两名人类玩家互相对弈，而是由计算机与一个人对弈。

本章将涵盖以下主题：

- 博弈推理。在两个游戏中，人类或计算机都可以先走，我们添加让计算机与人类对弈的代码。
- 在奥赛罗棋中，对每个走法，我们扫描棋盘，尝试找出能翻转最多人类棋子的走法。
- 在圈叉棋中，我们尝试找出棋盘中能形成最多连子的位置；或者，如果人类即将五子连珠，我们就必须把计算机的棋子放在能阻止这一情况发生的位置上。
- 随机数生成简介。如果计算机可以在若干等价走法之间进行选择，它将随机选择其中一个走法。
- 我们继续使用类、字段和方法等 C++ 特性，也继续使用窗口和部件等 Qt 特性。

奥赛罗棋

在本章的奥赛罗棋应用程序中，我们复用上一章的 `MainWindow` 和 `GameWidget` 类。

OthelloWindow 类

OthelloWindow 类与上一章的对应类颇为相似。不过，除了菜单和菜单项之外，这个版本的窗口还包含子菜单。子菜单通过调用 `addAction` 方法来添加，相关代码位于 OthelloWindow.cpp 文件中。

`OthelloWindow.h`

```
#ifndef OTHELLOWINDOW_H
#define OTHELLOWINDOW_H

#include "..\MainWindow\MainWindow.h"
#include "OthelloWidget.h"

class OthelloWindow : public MainWindow {
  Q_OBJECT

  public:
    OthelloWindow(QWidget *parentWidget = nullptr);
    ~OthelloWindow();

    void closeEvent(QCloseEvent *eventPtr)
                  {m_othelloWidgetPtr->closeEvent(eventPtr);}

  private:
    OthelloWidget* m_othelloWidgetPtr;
};

#endif // OTHELLOWINDOW_H
```

OthelloWindow.cpp 文件存放 OthelloWindow 类各个方法的定义。

OthelloWindow.cpp

```
#include "OthelloWidget.h"
#include "OthelloWindow.h"
#include <QtWidgets>
```

窗口的标题已经改为 Othello Advanced：

```
OthelloWindow::OthelloWindow(QWidget *parentWidget /*= nullptr*/)
 :MainWindow(parentWidget) {
  setWindowTitle(tr("Othello Advanced"));
  resize(1000, 500);

  m_othelloWidgetPtr = new OthelloWidget(this);
  setCentralWidget(m_othelloWidgetPtr);

  { QMenu* gameMenuPtr = menuBar()->addMenu(tr("&Game"));
    connect(gameMenuPtr, SIGNAL(aboutToShow()),
            this, SLOT(onMenuShow()));
```

Game 菜单有两个子菜单：Computer Starts 和 Human Starts：

```
{ QMenu* computerStartsMenuPtr =
    gameMenuPtr->addMenu(tr("&Computer Starts"));
  connect(computerStartsMenuPtr, SIGNAL(aboutToShow()),
          this, SLOT(onMenuShow()));
```

Computer Starts 子菜单包含 Computer Black 和 Computer White 两个菜单项:

```
  addAction(computerStartsMenuPtr, tr("Computer &Black"),
            SLOT(onComputerStartsBlack()), 0,
            tr("Computer Black"), nullptr,
            tr("Computer Black"),
            LISTENER(isComputerStartsBlackEnabled));

  addAction(computerStartsMenuPtr, tr("Computer &White"),
            SLOT(onComputerStartsWhite()), 0,
            tr("Computer White"), nullptr,
            tr("Computer White"),
            LISTENER(isComputerStartsWhiteEnabled));
}
```

Human Starts 子菜单包含 Human Black 和 Human White 两个菜单项:

```
  { QMenu* humanStartsMenuPtr =
      gameMenuPtr->addMenu(tr("&Human Starts"));
    connect(humanStartsMenuPtr, SIGNAL(aboutToShow()),
            this, SLOT(onMenuShow()));

    addAction(humanStartsMenuPtr, tr("Human &Black"),
              SLOT(onHumanStartsBlack()), 0, tr("Human Black"),
              nullptr, tr("Human Black"),
              LISTENER(isHumanStartsBlackEnabled));

    addAction(humanStartsMenuPtr, tr("Human &White"),
              SLOT(onHumanStartsWhite()), 0, tr("Human White"),
              nullptr, tr("Human White"),
              LISTENER(isHumanStartsWhiteEnabled));
  }

  gameMenuPtr->addSeparator();

  addAction(gameMenuPtr, tr("&Quit the Game"),
            SLOT(onQuit()),
            QKeySequence(Qt::CTRL + Qt::Key_Q), tr("Quit Game"),
            nullptr, tr("Quit the Game"),
            LISTENER(isQuitEnabled));

  addAction(gameMenuPtr, tr("E&xit"),
```

```
                SLOT(onExit()), QKeySequence::Quit);
  }
}

OthelloWindow::~OthelloWindow() {
  delete m_othelloWidgetPtr;
}
```

OthelloWidget 类

OthelloWidget 类持有奥赛罗棋的功能。它允许计算机与人类对弈：

OthelloWidget.h

```
#ifndef OTHELLOWIDGET_H
#define OTHELLOWIDGET_H

#include "..\MainWindow\GameWidget.h"

#define ROWS    8
#define COLUMNS 8

enum Mark {Empty = 0, Black, White};

class OthelloWidget : public GameWidget {
  Q_OBJECT

  public:
    OthelloWidget(QWidget* parentWidget);

    void mouseMark(int row, int column);
    void drawMark(QPainter& painter,
                  const QRect& markRect, int mark);

  public slots:
    DEFINE_LISTENER(OthelloWidget, isComputerStartsBlackEnabled);
    DEFINE_LISTENER(OthelloWidget, isComputerStartsWhiteEnabled);
    DEFINE_LISTENER(OthelloWidget, isHumanStartsBlackEnabled);
    DEFINE_LISTENER(OthelloWidget, isHumanStartsWhiteEnabled);

    void onComputerStartsBlack();
    void onComputerStartsWhite();
    void onHumanStartsBlack();
    void onHumanStartsWhite();
```

```
  private:
    bool checkWinner();
    void turn(int row, int column, Mark mark);
    void calculateComputerMove();
    void calculateTurns(int row, int column, Mark mark,
                        QSet<QPair<int,int>>& totalSet,
                        int& neighbours);
    Mark m_humanMark, m_computerMark;
};

#endif // OTHELLOWIDGET_H
```

OthelloWidget.cpp 文件存放 OthelloWidget 类各个方法的定义：

OthelloWidget.cpp

```
#include "OthelloWidget.h"
#include "OthelloWindow.h"

#include <QTime>
#include <CTime>
#include <CAssert>
using namespace std;

OthelloWidget::OthelloWidget(QWidget* parentWidget)
 :GameWidget(ROWS, COLUMNS, parentWidget) {
  // Empty.
}
```

isComputerStartsBlackEnabled、isComputerStartsWhiteEnabled、isHumanStartsBlackEnabled 和 isHumanStartsWhiteEnabled 方法在 Computer Starts 和 Human Starts 子菜单显示之前被调用。如果没有对局正在进行，它们就处于启用状态：

```
bool OthelloWidget::isComputerStartsBlackEnabled() {
  return !isGameInProgress();
}

bool OthelloWidget::isComputerStartsWhiteEnabled() {
  return !isGameInProgress();
}

bool OthelloWidget::isHumanStartsBlackEnabled() {
  return !isGameInProgress();
}

bool OthelloWidget::isHumanStartsWhiteEnabled() {
  return !isGameInProgress();
```

```
}
```

onComputerStartsBlack 和 onComputerStartsWhite 方法在用户选择 Computer Starts 子菜单中的某个菜单项时被调用。它们把计算机的棋子设为黑色或白色，通过在棋盘中央放置棋子来开始对局，并更新窗口：

```
void OthelloWidget::onComputerStartsBlack() {
  setGameInProgress(true);
  set(ROWS / 2, COLUMNS / 2, m_computerMark = Black);
  m_humanMark = White;
  update();
}

void OthelloWidget::onComputerStartsWhite() {
  setGameInProgress(true);
  set(ROWS / 2, COLUMNS / 2, m_computerMark = White);
  m_humanMark = Black;
  update();
}
```

onHumanStartsBlack 和 onHumanStartsWhite 方法在用户选择 Human Starts 子菜单中的某个菜单项时被调用。它们把计算机的棋子设为黑色或白色，并更新窗口。它们不会在棋盘上放置任何棋子，而是由人类来先走：

```
void OthelloWidget::onHumanStartsBlack() {
  setGameInProgress(true);
  m_humanMark = Black;
  m_computerMark = White;
  update();
}

void OthelloWidget::onHumanStartsWhite() {
  setGameInProgress(true);
  m_humanMark = White;
  m_computerMark = Black;

  update();
}
```

当用户点击棋盘中的一个空位置时会调用 mouseMark 方法。我们首先在该位置放置下一枚棋子，并翻转由这个走法引起的棋子：

```
void OthelloWidget::mouseMark(int row, int column) {
  set(row, column, m_humanMark);
  turn(row, column, m_humanMark);
  update();
```

如果人类的走法没有使棋盘填满，我们就调用 calculateComputerMove 来把计算机的棋子放到能翻转最多对方棋子的位置。然后我们更新窗口，并再次调用 checkWinner 来判断计算机的走法是否使棋盘填满了：

```
  if (!checkWinner()) {
    calculateComputerMove();
    update();
    checkWinner();
  }
}
```

当棋盘中某个位置需要重绘时会调用 drawMark 方法。它以与上一章相同的方式绘制棋子：

```
void OthelloWidget::drawMark(QPainter& painter,
                             const QRect& markRect, int mark) {
  painter.setPen(Qt::black);
  painter.drawRect(markRect);
  painter.fillRect(markRect, Qt::lightGray);

  switch (mark) {
    case Black:
      painter.setPen(Qt::black);
      painter.setBrush(Qt::black);
      painter.drawEllipse(markRect);
      break;

    case White:
      painter.setPen(Qt::white);
      painter.setBrush(Qt::white);
      painter.drawEllipse(markRect);
      break;

    case Empty:
      break;
  }
}
```

本章的 checkWinner 方法也与上一章的对应方法相似。它检查棋盘是否已满。如果已满，就宣布获胜者，否则就是平局：

```
bool OthelloWidget::checkWinner() {
  int blacks = 0, whites = 0, empties = 0;

  for (int row = 0; row < ROWS; ++row) {
    for (int column = 0; column < COLUMNS; ++column) {
      switch (get(row, column)) {
        case Black:
```

```
            ++blacks;
            break;

          case White:
            ++whites;
            break;

          case Empty:
            ++empties;
            break;
        }
      }
    }

    if (empties == 0) {
      QMessageBox messageBox(QMessageBox::Information,
                             tr("Victory"), QString());
      QString text;

      if (blacks > whites) {
        text.sprintf("The Winner: %s.", (m_computerMark == Black)
                                        ? "Computer" : "Human");
      }
      else if (whites > blacks) {
        text.sprintf("The Winner: %s.", (m_computerMark == White)
                                        ? "Computer" : "Human");
      }
      else {
        text.sprintf("A Draw.");
      }

      messageBox.setText(text);
      messageBox.setStandardButtons(QMessageBox::Ok);
      messageBox.exec();
      setGameInProgress(false);
      clearGrid();
      update();

      return true;
    }

    return false;
  }
```

calculateComputerMove 方法计算计算机的走法，使被翻转的对方棋子数量最多。我们遍历计算机可放置棋子的各个位置，对每个位置调用 calculateTurns 来获得如果把棋子放在该

位置将被翻转的对方棋子的最大数量。对每个位置，我们还获得相邻棋子的数量，在我们找不到任何可翻转的棋子时这一数据很有用。

maxTurnSetSize 和 maxNeighbours 字段保存可翻转棋子和相邻棋子的最大数量；maxTurnSetList 保存可翻转棋子的最大位置集合的列表，maxNeighboursList 保存相邻棋子数量最大位置列表：

```
void OthelloWidget::calculateComputerMove() {
  int maxTurnSetSize = 0, maxNeighbours = 0;
  QList<QSet<QPair<int,int>>> maxTurnSetList;
  QList<QPair<int,int>> maxNeighboursList;
```

我们遍历棋盘中的所有位置。对每个空位置，我们获得如果把我们的棋子放在该位置将被翻转的对方棋子数量。我们还获得相邻的对方棋子数量：

```
for (int row = 0; row < ROWS; ++row) {
  for (int column = 0; column < COLUMNS; ++column) {
    if (get(row, column) == Empty) {
      QSet<QPair<int,int>> turnSet;
      int neighbours = 0;
      calculateTurns(row, column, m_computerMark,
                     turnSet, neighbours);
      int turnSetSize = turnSet.size();
```

如果我们找到的可翻转棋子集合大于当前的最大集合，我们就把 maxTurnSetSize 字段设为新集合的大小，把当前位置插入集合中，清空 maxTurnSetList（因为我们不想要之前那些较小的集合），并加入新集合。

为了简单起见，我们把当前集合加入列表；把它加入集合比用其他任何方式保存都要容易：

```
if (turnSetSize > maxTurnSetSize) {
  maxTurnSetSize = turnSetSize;
  turnSet.insert(QPair<int,int>(row, column));
  maxTurnSetList.clear();
  maxTurnSetList.append(turnSet);
}
```

如果新集合非空且与最大集合大小相同，我们就直接把它加入 maxTurnSetList：

```
else if ((turnSetSize > 0) &&
         (turnSetSize == maxTurnSetSize)) {
  turnSet.insert(QPair<int,int>(row, column));
  maxTurnSetList.append(turnSet);
}
```

我们还检查当前位置的相邻棋子数量，处理方式与可翻转集合的情形相同。如果相邻棋子数量超过了最大相邻棋子数量，我们就清空 maxNeighboursList 并加入新位置：

```
      if (neighbours > maxNeighbours) {
        maxNeighbours = neighbours;
        maxNeighboursList.clear();
        maxNeighboursList.append(QPair<int,int>(row, column));
      }
```

如果至少有一个相邻棋子，且相邻棋子数量等于最大相邻棋子数量，我们就把它加入 maxNeighboursList 列表：

```
      else if ((neighbours > 0) &&
               (neighbours == maxNeighbours)) {
        maxNeighboursList.append(QPair<int,int>(row, column));
      }
    }
  }
}
```

如果至少有一个位置能翻转至少一枚对方棋子，我们就选择它。如果有多个位置能翻转相同数量的对方棋子，我们就随机选择其中一个。我们使用 C 标准函数 srand、rand 和 time 来获得一个随机整数。

随机数生成算法先取一个起始值，然后生成一列随机数。srand 函数用起始值初始化生成器，之后反复调用 rand 来获得新的随机数。为了避免每次都以相同的起始值调用 srand（那会导致每次产生相同的随机数序列），我们以 C 标准函数 time 的返回值来调用 srand，time 返回自 1970 年 1 月 1 日以来的秒数。这样，每局对弈随机数生成器都以一个新值初始化，我们通过反复调用 rand 获得新的随机数序列：

```
if (maxTurnSetSize > 0) {
  srand(time(NULL));
  int index = rand() % maxTurnSetList.size();
  QSet<QPair<int,int>> maxTurnSet = maxTurnSetList[index];
```

获得要翻转的位置集合之后，我们遍历该集合，把计算机的棋子放到其中的所有位置：

```
  for (QPair<int,int> position : maxTurnSet) {
    int row = position.first, column = position.second;
    set(row, column, m_computerMark);
  }
}
```

如果没有任何位置能导致对方棋子被翻转，我们就转而考察相邻棋子。同样地，我们在相邻棋子数量最多的位置中随机选择一个。注意，此时我们不需要遍历任何集合；在这种情况下，我们只放置一枚棋子：

```
  else {
    assert(!maxNeighboursList.empty());
    srand(time(NULL));
```

```
    int index = rand() % maxNeighboursList.size();
    QPair<int,int> position = maxNeighboursList[index];
    int row = position.first, column = position.second;
    set(row, column, m_computerMark);
  }
}
```

当人类走了一步之后会调用 turn 方法。它调用 calculateMark 来获得可翻转的对方棋子集合，然后遍历该集合，把棋盘中的各个位置都设置好：

```
void OthelloWidget::turn(int row, int column, Mark mark) {
  QSet<QPair<int,int>> turnSet;
  calculateMark(row, column, mark, turnSet);

  for (QPair<int,int> pair : turnSet) {
    int row = pair.first, column = pair.second;
    set(row, column, mark);
  }
}
```

calculateTurns 方法计算给定位置可翻转的对方棋子集合以及相邻棋子的数量：

```
void OthelloWidget::calculateTurns(int row, int column,
                   Mark playerMark,QSet<QPair<int,int>>& totalSet,
                   int& neighbours) {
```

directionArray 中的每个整数对都按照罗盘方位对应一个方向：

```
QPair<int,int> directionArray[] =
  {QPair<int,int>(-1, 0),   // 北
   QPair<int,int>(-1, 1),   // 东北
   QPair<int,int>(0, 1),    // 东
   QPair<int,int>(1, 1),    // 东南
   QPair<int,int>(1, 0),    // 南
   QPair<int,int>(1, -1),   // 西南
   QPair<int,int>(0, -1),   // 西
   QPair<int,int>(-1, -1)}; // 西北
```

数组的大小可以用数组的总大小（以字节为单位）除以第一个元素的大小来确定：

```
int arraySize =
  (sizeof directionArray) / (sizeof directionArray[0]);

neighbours = 0;
int opponentMark = (playerMark == Black) ? White : Black;
```

我们遍历各个方向；对每个方向，只要找到对方的棋子，就沿该方向继续移动：

```
for (int index = 0; index < arraySize; ++index) {
  QPair<int,int> pair = directionArray[index];
```

在沿一个方向迭代的过程中，row 与 column 字段保存当前的行和列：

```
int rowStep = pair.first, columnStep = pair.second,
    currRow = row, currColumn = column;
```

首先，我们检查紧邻的位置上是否有对方棋子的相邻棋子。如果我们尚未到达棋盘的某条边界，并且该位置上有对方棋子，我们就把 neighbours 加一：

```
if (((row + rowStep) >= 0) && ((row + rowStep) < ROWS) &&
    ((column + rowStep) >= 0) &&
    ((column + columnStep) < COLUMNS) &&
    (get(row + rowStep, column + rowStep) == opponentMark)) {
  ++neighbours;
}
```

我们把迭代过程中找到的棋子收集在 `directionSet` 中：

```
QSet<QPair<int,int>> directionSet;

while (true) {
  currRow += rowStep;
  currColumn += columnStep;
```

如果我们到达棋盘的某条边界，或者找到空位置，就中断迭代：

```
if ((currRow < 0) || (currRow == ROWS) ||
    (currColumn < 0) || (currColumn == COLUMNS) ||
    (get(currRow, currColumn) == Empty)) {
  break;
}
```

如果我们找到玩家的棋子，就把 `directionSet` 并入总集合，并中断迭代：

```
else if (get(currRow, currColumn) == playerMark) {
  totalSet += directionSet;
  break;
}
```

如果我们没有找到玩家的棋子或空位置，说明找到的就是对方的棋子，我们就把它的位置加入方向集合：

```
    else {
      directionSet.insert(QPair<int,int>(row, column));
    }
  }
```

```
    }
  }
```

main 函数

和往常一样，main 函数创建一个应用程序，显示窗口，并执行应用程序，直到用户关闭窗口或选择 Exit 菜单项。

Main.cpp

```
#include "OthelloWidget.h"
#include "OthelloWindow.h"
#include <QApplication>

int main(int argc, char *argv[]) {
  QApplication application(argc, argv);
  OthelloWindow othelloWindow;
  othelloWindow.show();
  return application.exec();
}
```

圈叉棋

本章的圈叉棋应用程序基于上一章的版本。不同之处在于，这个版本中由计算机与人类对弈。

NaCWindow 类

NaCWindow 类与上一节的 OthelloWindow 类相似（NaC 是 Noughts and Crosses 的缩写）。它向游戏菜单添加了两个子菜单，用来选择由计算机还是人类先走，并选择圈还是叉：

NaCWindow.h

```
#ifndef NACWINDOW_H
#define NACWINDOW_H

#include "..\MainWindow\MainWindow.h"
#include "NaCWidget.h"

class NaCWindow : public MainWindow {
  Q_OBJECT

  public:
```

```
    NaCWindow(QWidget *parentWidget = nullptr);
    ~NaCWindow();

  public:
    void closeEvent(QCloseEvent *eventPtr)
                  {m_nacWidgetPtr->closeEvent(eventPtr);}

  private:
    NaCWidget* m_nacWidgetPtr;
};

#endif // NACWINDOW_H
```

NaCWindow.cpp 文件存放 NaCWindow 类各个方法的定义：

NaCWindow.cpp

```
#include "NaCWindow.h"
#include <QtWidgets>
```

标题已经改为 Noughts and Crosses Advanced：

```
NaCWindow::NaCWindow(QWidget *parentWidget /*= nullptr*/)
 :MainWindow(parentWidget) {
  setWindowTitle(tr("Noughts and Crosses Advanced"));
  resize(1000, 500);

  m_nacWidgetPtr = new NaCWidget(this);
  setCentralWidget(m_nacWidgetPtr);

  { QMenu* gameMenuPtr = menuBar()->addMenu(tr("&Game"));
    connect(gameMenuPtr, SIGNAL(aboutToShow()),
            this, SLOT(onMenuShow()));

    { QMenu* computerStartsMenuPtr =
        gameMenuPtr->addMenu(tr("&Computer Starts"));
      connect(computerStartsMenuPtr, SIGNAL(aboutToShow()),
              this, SLOT(onMenuShow()));

      addAction(computerStartsMenuPtr, tr("Computer &Nought"),
                SLOT(onComputerStartsNought()), 0,
                tr("Computer Nought"), nullptr,
                tr("Computer Nought"),
                LISTENER(isComputerStartsNoughtEnabled));

        addAction(computerStartsMenuPtr, tr("Computer &Cross"),
                  SLOT(onComputerStartsCross()), 0,
```

```
                tr("Computer Cross"), nullptr,
                tr("Computer Cross"),
                LISTENER(isComputerStartsCrossEnabled));
    }

    { QMenu* humanStartsMenuPtr =
        gameMenuPtr->addMenu(tr("&Human Starts"));
      connect(humanStartsMenuPtr, SIGNAL(aboutToShow()),
              this, SLOT(onMenuShow()));

      addAction(humanStartsMenuPtr, tr("Human &Nought"),
                SLOT(onHumanNought()), 0, tr("Human Nought"),
                nullptr, tr("Human Nought"),
                LISTENER(isHumanNoughtEnabled));

      addAction(humanStartsMenuPtr, tr("Human &Cross"),
                SLOT(onHumanCross()), 0, tr("Human Cross"),
                nullptr, tr("Human Cross"),
                LISTENER(isHumanCrossEnabled));
    }

    gameMenuPtr->addSeparator();

    addAction(gameMenuPtr, tr("&Quit the Game"),
              SLOT(onQuit()),
              QKeySequence(Qt::CTRL + Qt::Key_Q), tr("Quit Game"),
              nullptr, tr("Quit the Game"),
              LISTENER(isQuitEnabled));

    addAction(gameMenuPtr, tr("E&xit"),
              SLOT(onExit()), QKeySequence::Quit);
  }
}

NaCWindow::~NaCWindow() {
  delete m_nacWidgetPtr;
}
```

NaCWidget 类

与上一章的版本相比，NaCWidget 类得到了改进。它持有 `calculateComputerMove` 和 `calculateMarkValue` 方法，让计算机与人类对弈：

NaCWidget.h

```
#ifndef NACWIDGET_H
#define NACWIDGET_H

#include "..\MainWindow\GameWidget.h"

#define ROWS    26
#define COLUMNS 26

enum Mark {Empty = 0, Nought, Cross};

class NaCWidget : public GameWidget {
  Q_OBJECT

  public:
    NaCWidget(QWidget* parentWidget);

    void mouseMark(int row, int column);
    void drawMark(QPainter& painter,
                  const QRect& markRect, int mark);

  public slots:
    DEFINE_LISTENER(NaCWidget, isComputerStartsNoughtEnabled);
    DEFINE_LISTENER(NaCWidget, isComputerStartsCrossEnabled);
    DEFINE_LISTENER(NaCWidget, isHumanStartsNoughtEnabled);
    DEFINE_LISTENER(NaCWidget, isHumanStartsCrossEnabled);

    void onComputerStartsNought();
    void onComputerStartsCross();
    void onHumanStartsNought();
    void onHumanStartsCross();

  private:
    bool checkWinner(int row, int column, Mark mark);
    int countMarks(int row, int column, int rowStep,
                   int columnStep, Mark mark);
    void calculateComputerMove(int& row, int &column);
    double calculateMarkValue(int row, int column, Mark mark);

    Mark m_humanMark, m_computerMark;
};

#endif // NACWIDGET_H
```

NaCWidget.cpp 文件存放 NaCWidget 类各个方法的定义：

NaCWidget.cpp

```
#include "NaCWidget.h"
#include <CTime>
#include <CAssert>

NaCWidget::NaCWidget(QWidget* parentWidget)
 :GameWidget(ROWS, COLUMNS, parentWidget) {
  // Empty.
}
```

isComputerStartsNoughtEnabled、isComputerStartsCrossEnabled、isHumanStartsNoughtEnabled 和 isHumanStartsCrossEnabled 方法决定是否启用 Computer Nought、Computer Cross、Human Nought 和 Human Cross 菜单项。当没有对局正在进行时，它们都是启用的：

```
bool NaCWidget::isComputerStartsNoughtEnabled() {
  return !isGameInProgress();
}

bool NaCWidget::isComputerStartsCrossEnabled() {
  return !isGameInProgress();
}

bool NaCWidget::isHumanStartsNoughtEnabled() {
  return !isGameInProgress();
}

bool NaCWidget::isHumanStartsCrossEnabled() {
  return !isGameInProgress();
}
```

onComputerStartsNought、onComputerStartsCross、onHumanStartsNought 和 onHumanStartsCross 方法在用户选择 Computer Noughts、Computer Cross、Human Noughts 和 Human Cross 菜单项时被调用。它们把对局设为进行中，把计算机和人类的棋子设为圈或叉，并更新窗口。在计算机先走的情况下，棋子放在棋盘中央，以便尽可能充分地利用棋盘：

```
void NaCWidget::onComputerStartsNought() {
  setGameInProgress(true);
  set(ROWS /2, COLUMNS / 2, m_computerMark = Nought);
  m_humanMark = Cross;
  update();
}

void NaCWidget::onComputerStartsCross() {
  setGameInProgress(true);
  set(ROWS /2, COLUMNS / 2, m_computerMark = Cross);
  m_humanMark = Nought;
```

```
  update();
}

void NaCWidget::onHumanStartsNought() {
  setGameInProgress(true);
  m_computerMark = Cross;
  m_humanMark = Nought;
  update();
}

void NaCWidget::onHumanStartsCross() {
  setGameInProgress(true);
  m_computerMark = Nought;
  m_humanMark = Cross;
  update();
}
```

当人类玩家点击棋盘中的一个空位置时会调用 mouseMark 方法。我们首先把棋子放到该位置，并更新窗口：

```
void NaCWidget::mouseMark(int row, int column) {
  set(row, column, m_humanMark);
  update();
```

如果人类的走法没有使他赢得对局，我们就计算计算机的下一步走法，把棋子放到该位置，检查这个走法是否使计算机赢得了对局，并更新窗口：

```
  if (!checkWinner(row, column, m_humanMark)) {
    calculateComputerMove(row, column);
    set(row, column, m_computerMark);
    checkWinner(row, column, m_computerMark);
    update();
  }
}
```

当某个位置需要重绘时会调用 drawMark 方法。它与上一章的对应方法相似，绘制一个圈或一个叉：

```
void NaCWidget::drawMark(QPainter& painter,
                         const QRect& markRect, int mark) {
  painter.setPen(Qt::black);

  switch (mark) {
    case Nought:
      painter.drawEllipse(markRect);
      break;
```

```
        case Cross:
          painter.drawLine(markRect.topLeft(),
                           markRect.bottomRight());
          painter.drawLine(markRect.topRight(),
                           markRect.bottomLeft());
          break;

        case Empty:
          break;
      }
    }
```

checkWinner 方法也与上一章的对应方法相似。它判断最近的走法是否造成了五子连珠。如果是，就宣布获胜者：

```
bool NaCWidget::checkWinner(int row, int column, Mark mark) {
  if ((countMarks(row, column, -1, 0, mark) >= 5) ||
      (countMarks(row, column, 0, -1, mark) >= 5) ||
      (countMarks(row, column, -1, 1, mark) >= 5) ||
      (countMarks(row, column, 1, 1, mark) >= 5)) {
    QMessageBox messageBox(QMessageBox::Information,
                           tr("Victory"), QString());
    QString text;
    text.sprintf("The Winner: %s.",
                 (mark == m_computerMark) ? "Computer" : "Human");

    messageBox.setText(text);
    messageBox.setStandardButtons(QMessageBox::Ok);
    messageBox.exec();
    setGameInProgress(false);
    clearGrid();
    update();
    return true;
  }

  return false;
}
```

countMarks 方法统计一行中棋子的数量。与上一章的对应方法相比，它得到了改进。在这个版本中，我们还统计该走法可能导致的最多的连子数量。由于 countMarks 会由 calculateComputerMove 调用，我们需要知道这个走法可能导致多少连子：

```
double NaCWidget::countMarks(int row, int column, int rowStep,
                             int columnStep, Mark mark) {
```

markCount 保存如果我们在给定位置放置棋子将得到的一行棋子数量；freeCount 保存如果我们继续在该行添加棋子，可能得到的一行棋子数量。原因是计算机不会向不可能形成五子连

珠的行添加棋子：

```
double markCount = 0;
int freeCount = 0;
```

我们在给定方向上遍历棋盘：

```
{ bool marked = true;
  int currentRow = row, currentColumn = column;

  while ((currentRow >= 0) && (currentRow < ROWS) &&
         (currentColumn >= 0) && (currentColumn < COLUMNS)) {
```

只要找到的是我们的棋子，我们就同时增加 markCount 和 freeCount：

```
if (get(currentRow, currentColumn) == mark) {
  if (marked) {
    ++markCount;
  }

  ++freeCount;
}
```

如果我们找到空位置，就向 markCount 加上 0.4（因为空闲的行比封闭的行好），并继续增加 freeCount：

```
else if (get(currentRow, currentColumn) == Empty) {
  if (marked) {
    markCount += 0.4;
  }

  marked = false;
  ++freeCount;
}
```

如果我们既没找到计算机的棋子也没找到空位置，那么一定是找到了人类的棋子，我们就中断迭代：

```
else {
  break;
}
```

在每次迭代的末尾，我们把行和列的步长加到当前的行和列上：

```
    currentRow += rowStep;
    currentColumn += columnStep;
  }
}
```

我们在相反的方向上进行类似的迭代。唯一的区别是，在每次迭代的末尾，我们从当前的行和列中减去行和列的步长，而不是加上它们：

```
{ bool marked = true;
  int currentRow = row + rowStep,
      currentColumn = column + columnStep;

  while ((currentRow >= 0) && (currentRow < ROWS) &&
         (currentColumn >= 0) && (currentColumn < COLUMNS)) {
    if (get(currentRow, currentColumn) == mark) {
      if (marked) {
        ++markCount;
      }
    }
    else if (get(currentRow, currentColumn) == Empty) {
      if (marked) {
        markCount += 0.4;
      }

      marked = false;
      ++freeCount;
    }
    else {
      break;
    }

    currentRow -= rowStep;
    currentColumn -= columnStep;
  }
}
```

如果空闲计数至少为五，我们就返回棋子计数。如果小于五，我们就返回零，因为在这个方向上不可能形成五子连珠：

```
  return (freeCount >= 5) ? markCount : 0;
}
```

`calculateComputerMove` 方法计算能形成最多连子的计算机走法。我们同时统计计算机和人类的连子，因为我们可能面临这样的情形：需要阻止人类获胜，而不是让计算机的获胜机会最大化。

`maxComputerValue` 和 `maxHumanValue` 字段保存到目前为止找到的最多连子数量。`maxComputerList` 和 `maxHumanList` 保存使计算机和人类形成最多连子的位置：

```
void NaCWidget::calculateComputerMove(int& maxRow,int &maxColumn){
  double maxComputerValue = 0, maxHumanValue = 0;
  QList<QPair<int,int>> maxComputerList, maxHumanList;
```

我们遍历棋盘。对每个空位置，我们尝试放置计算机或人类的棋子，看看会造成多少连子：

```
for (int row = 0; row < ROWS; ++row) {
  for (int column = 0; column < COLUMNS; ++column)  {
    if (get(row, column) == Empty) {
      set(row, column, m_computerMark);
```

我们获得计算机棋子和人类棋子所能形成的最多连子数量。如果它大于之前的最大数量，我们就清空列表并把该位置加入列表：

```
{ double computerValue =
    calculateMarkValue(row, column, m_computerMark);

  if (computerValue > maxComputerValue) {
    maxComputerValue = computerValue;
    maxComputerList.clear();
    maxComputerList.append(QPair<int,int>(row, column));
  }
```

如果新的连子数量大于零且等于最大数量，我们就直接把该位置加入列表：

```
  else if ((computerValue > 0) &&
           (computerValue == maxComputerValue)) {
    maxComputerList.append(QPair<int,int>(row, column));
  }
}
```

我们对人类棋子做与计算机棋子相同的处理：

```
set(row, column, m_humanMark);

{ double humanValue =
    calculateMarkValue(row, column, m_humanMark);

  if (humanValue > maxHumanValue) {
    maxHumanValue = humanValue;
    maxHumanList.clear();
    maxHumanList.append(QPair<int,int>(row, column));
  }
  else if ((humanValue > 0) &&
           (humanValue == maxHumanValue)) {
    maxHumanList.append(QPair<int,int>(row, column));
  }
}
```

最后，我们把该位置重置为空：

```
            set(row, column, Empty);
          }
        }
      }
```

无论是计算机还是人类，都必须至少存在一个能连子的位置：

```
      assert(!maxComputerList.empty() && !maxHumanList.empty());
```

如果计算机的价值至少为二且大于人类的价值，或者人类的价值小于四，我们就从计算机的最大走法中随机选择一个：

```
      if ((maxComputerValue >= 2) &&
          ((maxComputerValue >= maxHumanValue) ||
           (maxHumanValue < 3.8))) {
        srand(time(NULL));
        QPair<int,int> pair =
          maxComputerList[rand() % maxComputerList.size()];
        maxRow = pair.first;
        maxColumn = pair.second;
      }
```

但是，如果计算机无法连出至少两子，或者人类即将五子连珠，我们就从人类侧的最大走法中随机选择一个：

```
        else {
          srand(time(NULL));
          QPair<int,int> pair =
            maxHumanList[rand() % maxHumanList.size()];
          maxRow = pair.first;
          maxColumn = pair.second;
        }
      }
```

`calculateMarkValue` 方法通过计算四个方向中较大的值，来算出给定位置可能造成的最多连子数量：

```
      double NaCWidget::calculateMarkValue(int row, int column,
                                           Mark mark) {
        return qMax(qMax(countMarks(row, column, -1, 0, mark),
                         countMarks(row, column, 0, -1, mark)),
                    qMax(countMarks(row, column, -1, 1, mark),
                         countMarks(row, column, 1, 1, mark)));
      }
```

main 函数

最后，main 函数的工作方式与在 Qt 应用程序中一贯的做法相同：

Main.cpp

```
#include "NaCWidget.h"
#include "NaCWindow.h"
#include <QApplication>

int main(int argc, char *argv[]) {
  QApplication application(argc, argv);
  NaCWindow mainWindow;
  mainWindow.show();
  return application.exec();
}
```

小结

在本章中，我们开发了上一章各游戏的更高级版本。在奥赛罗棋和圈叉棋中，我们都添加了让计算机与人类对弈的代码。在奥赛罗棋中，我们寻找棋盘中能使对方棋子被翻转数量最多的位置。在圈叉棋中，我们搜索能给计算机带来最多连子的走法，最好是五子连珠。不过，我们也必须搜索对方潜在的连子数量，并在对方的下一步走法会导致其获胜时加以阻止。

现在，我建议你在进入下一章之前，先放松一下，和计算机对弈几局。

下一章我们将开始开发领域特定语言（Domain-Specific Language，DSL），这是一种面向特定领域的语言。我们将开发一个 DSL 来指定图形对象（如直线、矩形、椭圆和文本）的绘制方式，以及颜色、字体、画笔和刷子样式、对齐方式等设置。我们还会编写一个查看器来显示这些图形对象。

9
领域特定语言

前几章我们用 Qt 库开发了奥赛罗棋和圈叉棋两种游戏。本章我们将开始开发一种领域特定语言（Domain-Specific Language，DSL），即面向特定领域的语言。更具体地说，我们要开发一种在 Qt 部件中书写图形对象的语言。这门语言允许我们绘制直线、矩形、椭圆，并书写文本。此外，它还允许我们为图形对象选择颜色、笔样式和刷样式，也允许我们为文本选择字体和对齐方式。

本章将涵盖以下主题：

- 首先，我们通过一个示例来非正式地了解 DSL 的源代码。我们绘制图形对象，并设置它们的颜色、样式与字体。
- 我们用文法形式化地定义这门语言。
- 定义好文法之后，我们编写扫描器。扫描器读取源代码，识别出有意义字符序列，即词法单元（token）。
- 扫描器写好后，我们编写解析器。解析器可以看作 DSL 的核心，它在需要时向扫描器请求新的词法单元；它检查源代码是否符合文法，并生成一个动作序列。每个动作持有一条指令，如设置颜色或绘制直线。
- 最后，我们编写一个查看器：它读取解析器生成的动作序列，并在 Qt 部件中显示图形对象。

引入源语言——一个简单示例

DSL 的源语言由一系列指令构成。既有绘制直线、矩形、椭圆和文本等图形对象的指令，也有设置对象颜色与样式、文本字体与对齐方式的指令。最后，还有为名字赋值的指令。

我们来看一个示例。下面的代码绘制一个矩形并书写文本。注意，这门语言区分大小写——也就是说，代码中用小写还是大写字母并无关系。我们先定义矩形的左上角：

```
topLeft = point(100, 100);
```

我们用坐标运算符提取左上角点的 x 和 y 坐标，并定义右下角：

```
left = xCoordinate(topLeft);
top = yCoordinate(topLeft);
bottomRight = point(left + 100, top + 100);
```

我们用预定义值 DashLine 和 CrossPattern 来设置笔和刷的样式：

```
SetPenStyle(DashLine);
SetBrushStyle(CrossPattern);
```

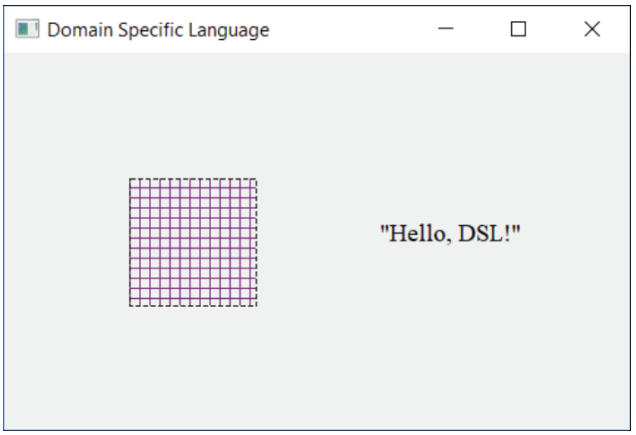
我们把预定义颜色 Black 用作笔的颜色，并为刷创建一个自己的颜色 Purple。我们可以用三个值——分别对应红、绿、蓝分量——来创建一种新颜色。每个分量的取值范围为 0 到 255（含两端）：

```
SetPenColor(Black);
PurpleColor = color(128, 0, 128);
SetBrushColor(PurpleColor);
DrawRectangle(topLeft, bottomRight);
```

我们接着添加一段文本，并设置字体和对齐方式。我们选用 12 磅的 Times New Roman、水平左对齐和垂直顶端对齐：

```
SetFont(font("Times New Roman", 12));
SetHorizontalAlignment(AlignLeft);
SetVerticalAlignment(AlignTop);
DrawText(point(300, 150), "Hello, DSL!");
```

本示例中的指令会被扫描器切分成有意义的部分；解析器将检查指令是否符合文法，并生成一个动作序列，查看器读取该序列后显示以下 Qt 部件：



源语言的文法

DSL 的源语言需要被精确定义，我们通过定义该语言的文法来做到这一点。文法由规则（以斜体显示）、关键字（以粗体显示）、分隔符和标点符号构成。

程序规则是开始规则。箭头（->）表示一个程序由指令列表构成。该箭头可以读作：

```
program -> instructionList
```

在文法中，星号（*）表示零个或多个。因此，指令列表由零条或多条指令构成：

```
instructionList -> instruction*
```

赋值指令接受一个名字，后跟赋值运算符（=）、一个表达式和一个分号。设置笔和刷颜色与样式的指令接受一个表达式，设置字体和对齐方式的指令也是如此。绘制直线、矩形和文本的指令接受两个表达式。注意，每条指令都以分号（;）结尾。

竖线（|）可以读作「或」。一条指令就是一次赋值，或者设置笔的颜色，或者设置刷的颜色，如此等等：

```
instruction -> name = expression;
             | SetPenColor(expression);
             | SetPenStyle(expression);
             | SetBrushColor(expression);
             | SetBrushStyle(expression);
             | SetFont(expression);
             | SetHorizontalAlignment(expression);
             | SetVerticalAlignment(expression);
             | DrawLine(expression, expression);
             | DrawRectangle(expression, expression);
             | DrawEllipse(expression, expression);
             | DrawText(expression, expression);
```

解析器接下来要定义的是表达式。我们先看表达式的运算符，还要考虑运算符的优先级。例如，乘法与除法的优先级高于加法与减法。文法中的运算符具有以下优先级：

表达式	运算符	优先级
加法、减法	+ -	最低
乘法、除法	* /	
基本表达式	point xCoordinate yCoordinate color font (expression) name value	最高

我们为加法与减法、乘法与除法各定义两条规则。先从最低的优先级入手，即加法与减法。在 expression 规则中，我们调用处理乘除表达式的 mulDivExpression 规则，并调用 expressionRest 规则来检查表达式的其余部分：

```
expression -> mulDivExpression expressionRest
```

在 expressionRest 规则中，我们检查下一个词法单元。如果它是加号或减号，就得到一个加法或减法表达式。我们调用 mulDivExpression 来处理优先级更高的表达式。最后，如果又遇到一个加号或减号，就再次调用 expressionRest。不过，如果第一个词法单元既不是加号也不是减号，我们什么也不做：

```
expressionRest -> + mulDivExpression expressionRest
                | - mulDivExpression expressionRest
                | /* empty */
```

mulDivExpression 与 mulDivExpressionRest 的工作方式与前面给出的 expression 和 expressionRest 相同：

```
mulDivExpression -> primaryExpression mulDivExpressionRest
mulDivExpressionRest -> * primaryExpression mulDivExpressionRest
                      | / primaryExpression mulDivExpressionRest
                      | /* empty */
```

基本表达式可以是一个点、x 或 y 坐标、颜色、字体、名字或值。点由两个表达式构成，分别存放该点的 x 和 y 坐标。坐标接受一个存放点的表达式，并给出它的 x 或 y 坐标：

```
primaryExpression -> point(expression, expression)
                   | xCoordinate(expression)
                   | yCoordinate(expression)
```

颜色表达式由其红、绿、蓝分量构成，而字体表达式由字体的名称和大小构成：

```
| color(expression, expression, expression)
| font(expression, expression)
```

表达式可以括在圆括号中，以改变表达式的优先级。例如，在表达式 `2 + 3 * 4` 中，乘法优先于加法；而在表达式 `(2 + 3) * 4` 中，加法优先于乘法：

```
| (expression)
```

最后，一个表达式可以是一个此前已与某个值关联的名字，或者干脆就是一个值：

```
| name
| value
```

目标语言

目标语言由一系列动作定义。通俗地说，动作对应于文法中的指令。既有设置笔或刷的颜色与样式的动作，也有设置文本水平或垂直对齐方式的动作，还有真正绘制图形中的直线、矩形、椭圆和文本的动作。本章后面我们会编写一个解析器来生成动作序列，再编写一个查看器来读取这些动作，并在 Qt 部件中显示图形对象。

一个 `Action` 对象存放动作的标识（由 `Token` 类中的 `TokenId` 枚举定义如下），以及至多两个值。

Action.h

```
#ifndef ACTION_H
#define ACTION_H

#include "Token.h"
#include "Value.h"

class Action {
  public:
    Action(TokenId actionId, const Value& value1 = Value(),
           const Value& value2 = Value());
    Action(const Action& action);
    Action operator=(const Action& action);

    TokenId id() const {return m_actionId;}
    const Value& value1() const {return m_value1;}
    const Value& value2() const {return m_value2;}

  private:
    TokenId m_actionId;
    Value m_value1, m_value2;
};
```

```
#endif // ACTION_H
```

Action.cpp 文件存放 Action 类各方法的定义。

Action.cpp

```
#include "Action.h"
```

构造函数接受动作标识和至多两个值：

```
Action::Action(TokenId actionId,
               const Value& value1 /*= Value()*/,
               const Value& value2 /*= Value()*/ )
  :m_actionId(actionId),
   m_value1(value1),
   m_value2(value2) {
  // Empty.
}
```

颜色

设置笔或刷的颜色时，我们需要随指令提交颜色。我们可以用前面文法中的 color 规则创建自己的颜色。不过，Qt 的 QColor 类提供了一组预定义颜色。下面的扫描器定义了一组预定义 QColor 对象（Aqua、Black 等），并把它们映射到各自的名称。例如，用户可以在源代码中写出如下指令：

```
SetPenColor(Aqua);
```

此时，由于名称 Aqua 与 QColor 对象 Aqua 相关联，笔的颜色就被设置为 Aqua。

Colors.h

```
#ifndef COLOR_H
#define COLOR_H

#include <QWidget>

extern QColor
  Aqua, Black, Blue, Brown, Cyan, Gray, Green, Lime, Magenta,
  Navyblue, Orange, Orchid, Pink, Purple, Red, Silver, Snow,
  SteelBlue, SystemColor, Turquoise, Violet, White, Yellow;

#endif // COLOR_H
```

Colors.cpp 文件存放 Colors.h 中各颜色的定义。

Colors.cpp

```
#include "Colors.h"
```

每种颜色由其红、绿、蓝分量定义。每个分量的取值范围为 0 到 255（含两端）。例如，Blue 的蓝色分量取满值，其余分量为零；而 Yellow 是红与绿的混合：

```
QColor
  Aqua(0, 255, 255), Black(0, 0, 0), Blue(0, 0, 255),
  Brown(165, 42, 42), Cyan(0, 255, 255), Gray(127, 127, 127),
  Green(0, 128, 0), Lime(0, 255, 0), Magenta(255, 0, 255),
  Navyblue(159, 175, 223), Orange(255, 165, 0),
  Orchid(218, 112, 214), Pink(255, 192, 203),
  Purple(128, 0, 128), Red(255, 0, 0), Silver(192, 192, 192),
  Snow(255, 250, 250), SteelBlue(70, 130, 180),
  SystemColor(0, 0, 0), Turquoise(64, 224, 208),
  Violet(238, 130, 238), White(255, 255, 255),
  Yellow(255, 255, 0);
```

错误处理

有几个用于错误处理的函数：`check` 检查某个条件是否为真，若不为真则报告错误；`syntaxError` 与 `semanticError` 分别报告语法错误和语义错误，而 `error` 抛出异常，由 `main` 函数捕获并报告。

Error.h

```
#ifndef ERROR_H
#define ERROR_H

#include <QString>

void error(const QString& message);
void syntaxError();
void syntaxError(const QString& message);
void semanticError(const QString& message);
void check(bool condition, const QString& message);

#endif // ERROR_H
```

Error.cpp 文件存放 Error.h 中的定义。

Error.cpp

```
#include <SStream>
#include <Exception>
```

```
using namespace std;
#include "Error.h"

extern int g_lineNo = 1;

void error(const QString& message) {
  throw exception(message.toStdString().c_str());
}
```

我们用 C++ 的 stringstream 标准类来拼接错误信息：

```
void syntaxError() {
  stringstream stringStream;
  stringStream << "Syntax error at line " << g_lineNo << ".";
```

str 方法返回一个 C++ 的 string 标准类对象，c_str 返回一个字符指针，它在 error 调用中被转换为 QString 对象：

```
  error(stringStream.str().c_str());
}
```

当扫描器遇到不构成词法单元的字符序列，或者解析器检测到词法单元序列不符合文法时，就发生语法错误。我们很快就会讨论这个话题；眼下只需记住，扫描器也可能报告错误：

```
void syntaxError(const QString& message) {
  stringstream stringStream;
  stringStream << "Syntax error at line " << g_lineNo
               << ": " << message.toStdString() << ".";
  error(stringStream.str().c_str());
}
```

当发现未知名字，或表达式的类型不符合要求时，就发生语义错误：

```
void semanticError(const QString& message) {
  stringstream stringStream;
  stringStream << "Sematic error: "
               << message.toStdString() << ".";
  error(stringStream.str().c_str());
}
```

check 方法的作用与 assert 宏类似：它检查条件是否为真，若不为真则调用 semanticError，后者最终抛出错误异常：

```
void check(bool condition, const QString& message) {
  if (!condition) {
    semanticError(message);
  }
}
```

值

这门语言中有若干种值，用于设置笔或刷的颜色与样式、设置直线的端点、设置字体的名称或文本的对齐方式：数值（double）、字符串（QString）、颜色（QColor）、字体（QFont）、点（QPoint）、笔样式（Qt::PenStyle）、刷样式（Qt::BrushStyle），以及水平或垂直对齐（Qt::AlignmentFlag）。

Value.h

```
#ifndef VALUE_H
#define VALUE_H

#include <IOStream>
using namespace std;

#include <QtWidgets>

enum TypeId {NumericalTypeId, StringTypeId, ColorTypeId,
             PenStyleTypeId, BrushStyleId, AlignmentTypeId,
             FontTypeId, PointTypeId};

class Value {
  public:
    Value();
    Value(double numericalValue);
    Value(const QString& stringValue);
    Value(const QPoint& pointValue);
    Value(const QColor& colorValue);
    Value(const QFont& fontValue);
    Value(const Qt::PenStyle& penStyleValue);
    Value(const Qt::BrushStyle& brushStyleValue);
    Value(const Qt::AlignmentFlag& alignment);

    Value(const Value& value);

    Value& operator=(const Value& value);

    bool isNumerical() const {return (m_typeId==NumericalTypeId);}
    bool isString() const { return (m_typeId == StringTypeId); }
    bool isColor() const { return (m_typeId == ColorTypeId); }
    bool isFont() const { return (m_typeId == FontTypeId); }
    bool isPoint() const { return (m_typeId == PointTypeId); }
    bool isPenStyle() const {return (m_typeId == PenStyleTypeId);}
    bool isBrushStyle() const {return (m_typeId == BrushStyleId);}
    bool isAlignment() const {return (m_typeId==AlignmentTypeId);}
```

```
    double numericalValue() const { return m_numericalValue; }
    const QString& stringValue() const { return m_stringValue; }
    const QColor& colorValue() const { return m_colorValue; }
    const QFont& fontValue() const { return m_fontValue; }
    const QPoint& pointValue() const { return m_pointValue; }
    const Qt::PenStyle& penStyleValue() const
                        { return m_penStyleValue; }
    const Qt::BrushStyle& brushStyleValue() const
                          { return m_brushStyleValue; }
    const Qt::AlignmentFlag& alignmentValue() const
                             { return m_alignmentValue; }

  private:
    TypeId m_typeId;
    double m_numericalValue;
    QString m_stringValue;
    QPoint m_pointValue;
    QColor m_colorValue;
    QFont m_fontValue;
    Qt::PenStyle m_penStyleValue;
    Qt::BrushStyle m_brushStyleValue;
    Qt::AlignmentFlag m_alignmentValue;
};

#endif // VALUE_H
```

Value.cpp 文件存放 Value 类各方法的定义。

Value.cpp

```
#include <CAssert>
using namespace std;
#include "Value.h"

Value::Value() {
  // Empty.
}
```

非默认构造函数用相应的值初始化 Value 对象：

```
Value::Value(double numericalValue)
  :m_typeId(NumericalTypeId),
   m_numericalValue(numericalValue) {
  // Empty.
}

Value::Value(const QPoint& pointValue)
```

```
  :m_typeId(PointTypeId),
   m_pointValue(pointValue) {
  // Empty.
}
```

扫描器

扫描器是应用程序的一部分，它接受源代码并生成词法单元序列。词法单元是源代码中最小的有意义部分。例如，字符 f、o、n、t 构成了关键字 `font`，字符 1、2、3 构成了数值 123。

不过首先需要一个 `Token` 类来记录词法单元。`m_tokenId` 字段被设为 `TokenId` 枚举中的一个值。对于名字，`m_name` 字段存放该名字；对于值，`m_value` 字段存放该值。

`Token.h`

```
#ifndef TOKEN_H
#define TOKEN_H

#include <QWidget>
#include "Value.h"
```

`TokenId` 枚举容纳扫描器的所有词法单元，分为关键字、运算符、标点与分隔符，以及名字和值。为了避免在不同枚举之间转换，`TokenId` 枚举被扫描器、解析器和查看器共同使用。扫描器用 `TokenId` 枚举区分不同的词法单元，解析器在类型检查和求值表达式时也用它，`Action` 类则用它区分不同的动作。

第一部分（ColorId 到 YCoordinateId）是这门语言的关键字：

```
enum TokenId {ColorId, DrawEllipseId, DrawLineId,
              DrawRectangleId, DrawTextId, FontId,
              PointId, SetBrushColorId, SetBrushStyleId,
              SetFontId, SetHorizontalAlignmentId,
              SetPenColorId, SetPenStyleId,
              SetVerticalAlignmentId,
              XCoordinateId, YCoordinateId,
```

第二部分（AddId 到 DivideId）是运算符：

```
AddId, SubtractId, MultiplyId, DivideId,
```

接下来一部分是圆括号、赋值符（=）、逗号和分号：

```
LeftParenthesisId, RightParenthesisId,
AssignId, CommaId, SemicolonId,
```

最后一部分是名字、值和文件结束标记：

```
                    NameId, ValueId, EndOfFileId};

class Token{
  public:
    Token();
    Token(TokenId tokenId);
    Token(TokenId tokenId, const QString& name);
    Token(TokenId tokenId, const Value& value);
```

每个词法单元都可以带一个名字或值：

```
    TokenId id() const {return m_tokenId;}
    const QString& name() const { return m_name; }
    const Value& value() const { return m_value; }

  private:
    TokenId m_tokenId;
    QString m_name;
    Value m_value;
};

#endif // TOKEN_H
```

Token.cpp 文件存放 Token 类各方法的定义。

Token.cpp

```
#include "Token.h"
```

默认词法单元用一个文件结束词法单元初始化：

```
Token::Token()
  :m_tokenId(EndOfFileId) {
  // Empty.
}
```

大多数词法单元只存放一个 TokenId 枚举值：

```
Token::Token(TokenId tokenId)
  :m_tokenId(tokenId) {
  // Empty.
}
```

词法单元也可以存放一个名字或值：

```
Token::Token(TokenId tokenId, const QString& name)
  :m_tokenId(tokenId),
   m_name(name) {
```

```
  // Empty.
}

Token::Token(TokenId tokenId, const Value& value)
  :m_tokenId(tokenId),
   m_value(value) {
  // Empty.
}
```

Scanner 类接受源代码，把它切分成词法单元。一个词法单元也可以关联一个名字或值。

Scanner.h

```
#ifndef SCANNER_H
#define SCANNER_H

#include "Token.h"
#include "Colors.h"
```

init 方法初始化关键字和运算符的名称：

```
class Scanner {
  public:
    static void init();
    Scanner(QString& buffer);
```

nextToken 方法扫描缓冲区并返回下一个词法单元。若找不到可识别的词法单元，就抛出错误异常，稍后由 main 函数捕获：

```
public:
  Token nextToken();
```

m_buffer 字段存放源代码；m_bufferIndex 存放缓冲区中下一个待检查字符的下标（下标初始化为零）；m_keywordMap 存放关键字的名称；m_valueMap 存放颜色、对齐方式以及笔和刷样式值的映射；m_operatorList 存放运算符列表：

```
private:
  QString m_buffer;
  int m_bufferIndex = 0;
```

在之前的章节中，我们使用过 C++ 标准类 map、set、list、vector 和 stack。本章改用 Qt 的 QMap、QSet、QList、QVector 和 QStack，它们的工作方式大致相同：

```
    static QMap<QString,TokenId> m_keywordMap;
    static QMap<QString,Value> m_valueMap;
    static QList<pair<QString,TokenId>> m_operatorList;
};
```

```
#endif // SCANNER_H
```

Scanner.cpp 文件存放 Scanner 类各方法的定义。

Scanner.cpp

```
#include <SStream>
#include <IOStream>
#include <Exception>
using namespace std;
#include "Error.h"
#include "Scanner.h"

QMap<QString,Value> Scanner::m_valueMap;
QMap<QString,TokenId> Scanner::m_keywordMap;
QList<pair<QString, TokenId>> Scanner::m_operatorList;
```

全局字段 g_lineNo 记录源代码中的当前行，以便错误信息能够给出正确的行号：

```
extern int g_lineNo;
```

宏 ADD_TO_OPERATOR_LIST 向运算符列表添加一个词法单元。例如，ADD_TO_OPERATOR_LIST("+", AddId) 把「+」与 AddId 这一对加入列表：

```
#define ADD_TO_OPERATOR_LIST(text, token)
  m_operatorList.push_back(pair<QString,TokenId>(text, token));

void Scanner::init() {
  ADD_TO_OPERATOR_LIST("+", AddId)
  ADD_TO_OPERATOR_LIST("-", SubtractId)
  ADD_TO_OPERATOR_LIST("*", MultiplyId)
  ADD_TO_OPERATOR_LIST("/", DivideId)
  ADD_TO_OPERATOR_LIST("(", LeftParenthesisId)
  ADD_TO_OPERATOR_LIST(")", RightParenthesisId)
  ADD_TO_OPERATOR_LIST("=", AssignId)
  ADD_TO_OPERATOR_LIST(",", CommaId)
  ADD_TO_OPERATOR_LIST(";", SemicolonId)
```

宏 ADD_TO_KEYWORD_MAP 向关键字映射添加一个关键字。例如，ADD_TO_KEYWORD_MAP(ColorId) 把 Color 与 ColorId 这一对加入映射。注意，关键字中 Id 部分（最后两个字符）已被去掉：

```
#define ADD_TO_KEYWORD_MAP(x) {
  QString s(#x);
  m_keywordMap[s.toLower().left(s.length() - 2)] = x; }

  ADD_TO_KEYWORD_MAP(ColorId)
```

```
  ADD_TO_KEYWORD_MAP(DrawEllipseId)
  ADD_TO_KEYWORD_MAP(DrawLineId)
  ADD_TO_KEYWORD_MAP(DrawRectangleId)
  ADD_TO_KEYWORD_MAP(DrawTextId)
  ADD_TO_KEYWORD_MAP(FontId)
  ADD_TO_KEYWORD_MAP(PointId)
  ADD_TO_KEYWORD_MAP(SetBrushColorId)
  ADD_TO_KEYWORD_MAP(SetBrushStyleId)
  ADD_TO_KEYWORD_MAP(SetFontId)
  ADD_TO_KEYWORD_MAP(SetHorizontalAlignmentId)
  ADD_TO_KEYWORD_MAP(SetPenColorId)
  ADD_TO_KEYWORD_MAP(SetPenStyleId)
  ADD_TO_KEYWORD_MAP(SetVerticalAlignmentId)
  ADD_TO_KEYWORD_MAP(XCoordinateId)

ADD_TO_KEYWORD_MAP(YCoordinateId)
```

宏 ADD_TO_VALUE_MAP 向值映射添加一个值。例如，ADD_TO_VALUE_MAP(Aqua) 把 aqua 与 QColor 对象 Aqua 这一对加入映射。注意，文本被转换为小写；还要注意，只包含最后一对冒号（::）之后的部分：

```
#define ADD_TO_VALUE_MAP(x) {
  QString s(#x);
  QString t = s.toLower();
  int i = t.lastIndexOf("::");
  m_valueMap[(i == -1) ? t : t.mid(i + 2)] = Value(x); }
```

ADD_TO_VALUE_MAP(Qt::AlignLeft) 把 align left 与 Qt::PenStyle 值这一对加入映射。再次注意，值名只把最后一段作为文本存放：

```
  ADD_TO_VALUE_MAP(Qt::AlignLeft)
  ADD_TO_VALUE_MAP(Qt::AlignTop)
  ADD_TO_VALUE_MAP(Qt::PenStyle::NoPen)
  ADD_TO_VALUE_MAP(Qt::PenStyle::SolidLine)
  ADD_TO_VALUE_MAP(Qt::BrushStyle::NoBrush)
  ADD_TO_VALUE_MAP(Qt::BrushStyle::SolidPattern)
  ADD_TO_VALUE_MAP(Aqua)
  ADD_TO_VALUE_MAP(Black)
}
```

在构造函数中，我们把缓冲区装载到 m_buffer 字段，并添加空字符（\0），以便更容易找到缓冲区末尾：

```
Scanner::Scanner(QString& buffer)
  :m_buffer(buffer) {
  m_buffer.append('\0');
}
```

nextToken 方法扫描缓冲区并返回找到的词法单元。首先，只要遇到换行、空白或行注释，我们就继续迭代。遇到换行时，把行号加一：

```
Token Scanner::nextToken() {
  while (true) {
    if (m_buffer[m_bufferIndex] == '\n') {
      ++g_lineNo;

    ++m_bufferIndex;
  }
```

空白可以是普通空格、水平或垂直制表符、回车符或换行。我们用 isSpace 方法判断某个字符是否为空白：

```
  else if (m_buffer[m_bufferIndex].isSpace()) {
    ++m_bufferIndex;
  }
```

若遇到行注释的开头（//），就继续，直到遇到行尾（'n'）或缓冲区末尾（''）：

```
  else if (m_buffer.indexOf("//", m_bufferIndex) ==
           m_bufferIndex) {
    while ((m_buffer[m_bufferIndex] != QChar('\n')) &&
           (m_buffer[m_bufferIndex] != QChar('\0'))) {
      ++m_bufferIndex;
    }
  }
```

若既不是换行、空白，也不是行注释，就跳出迭代，继续查找下一个词法单元：

```
    else {
      break;
    }
  }
```

扫过可能的空白与注释之后，我们开始查找真正的词法单元。先检查缓冲区中的下一个字符是否为空字符（''）。若是空字符，说明已到达源代码末尾，返回文件结束词法单元。请记住，我们在构造函数中于缓冲区末尾添加了一个空字符，正是为了能够识别文件末尾：

```
  if (m_buffer[m_bufferIndex] == QChar('\0')) {
    return Token(EndOfFileId);
  }
```

若下一个词法单元不是文件结束，就检查它是否为运算符。我们遍历运算符列表，检查缓冲区是否以其中某个运算符的文本开头。例如，加法运算符的文本是 +：

```
  for (const pair<QString,TokenId>& pair : m_operatorList) {
    const QString& operatorText = pair.first;
```

```
    TokenId tokenId = pair.second;

    if (m_buffer.indexOf(operatorText, m_bufferIndex) ==
        m_bufferIndex) {
      m_bufferIndex += operatorText.length();
      return Token(tokenId);
    }
  }
```

若缓冲区不以运算符开头，我们查找表示关键字的名字、一个值，或干脆是一个名字。先检查缓冲区是否以字母或下划线字符（_）开头，因为名字允许以字母或下划线开头；不过其余字符除字母和下划线外还可以是数字：

```
  if (m_buffer[m_bufferIndex].isLetter() ||
      (m_buffer[m_bufferIndex] == '_')) {
    int index = m_bufferIndex;
```

我们继续迭代，直到遇到既不是字母、也不是数字或下划线的字符：

```
    while (m_buffer[index].isLetterOrNumber() ||
           (m_buffer[index] == '_')) {
      ++index;
    }
```

我们提取文本并把缓冲区下标向前推进：

```
    int size = index - m_bufferIndex;
    QString text = m_buffer.mid(m_bufferIndex, size).toLower();
    m_bufferIndex += size;
```

这段文本可能是关键字、值或名字。先检查文本是否存在于关键字映射中；若存在，就直接返回与该关键字文本关联的词法单元：

```
    if (m_keywordMap.contains(text)) {
      return Token(m_keywordMap[text]);
    }
```

接着检查文本是否存在于值映射中。若存在，就返回一个带上该值标注的值词法单元，解析器稍后可以取出这个值：

```
    else if (m_valueMap.contains(text)) {
      return Token(ValueId, m_valueMap[text]);
    }

    else {
      return Token(NameId, text);
    }
  }
```

若文本既不是关键字也不是值，我们就认为它是一个名字，返回一个带上该名字标注的名字词法单元。解析器稍后可以取出这个名字：

若查找名字无果，我们就转而查找字符串。字符串是用双引号（”’）括起来的文本。若缓冲区中的下一个字符是双引号，就说明是一段文本的开头。我们去掉双引号，然后迭代，直到找到这段文本的结束引号：

```
if (m_buffer[m_bufferIndex] == '"') {
  int index = m_bufferIndex + 1;

  while (m_buffer[index] != '"') {
```

若在文本结束前遇到空字符，就报告语法错误，因为我们在文本内部遇到了文件末尾：

```
if (m_buffer[index] == QChar('\0')) {
  syntaxError("unfinished string");
}

  ++index;
}
```

找到结束引号之后，我们把缓冲区下标向前推进，并返回一个以该文本为标注值的值词法单元。解析器稍后可以取出这段文本：

```
  int size = index - m_bufferIndex + 1;
  QString text = m_buffer.mid(m_bufferIndex, size);
  m_bufferIndex += size;
  return Token(ValueId, Value(text));
}
```

若缓冲区中的下一个字符是数字，就说明找到了一个数值，带不带小数都可以。首先，只要在缓冲区中遇到数字就继续迭代：

```
if (m_buffer[m_bufferIndex].isDigit()) {
  int index = m_bufferIndex;

  while (m_buffer[index].isDigit()) {
    ++index;
  }
```

不再遇到数字时，检查缓冲区中的下一个字符是否为小数点（'.'）。若是小数点，就继续迭代，只要遇到数字就继续：

```
if (m_buffer[index] == '.') {
  ++index;

  while (m_buffer[index].isDigit()) {
    ++index;
```

```
    }
}
```

不再遇到数字时，我们推进缓冲区下标，并返回一个带标注值的值词法单元。解析器稍后可以取出该值：

```
    int size = index - m_bufferIndex;
    QString text = m_buffer.mid(m_bufferIndex, size);
    m_bufferIndex += size;
    return Token(ValueId, Value(text.toDouble()));
}
```

最后，若前面各种情形都不成立，说明源代码在语法上不正确，我们报告语法错误：

```
syntaxError();
```

我们返回一个文件结束词法单元，纯粹是因为这个方法必须有返回值。不过由于 `syntaxError` 调用抛出了异常，我们永远不会到达代码的这一处：

```
    return Token(EndOfFileId);
}
```

至此我们已经看过扫描器，下一节将接着研究解析器。

构建解析器

看过扫描器之后，该转向解析器了。解析器检查源代码是否符合文法，同时进行类型检查，并生成动作列表，稍后由查看器显示。`Parser` 类与文法保持一致，即每条文法规则对应一个方法。

Parser.h

```
#ifndef PARSER_H
#define PARSER_H

#include "Action.h"
#include "Scanner.h"
```

构造函数接受一个文法对象和动作列表，动作列表一开始为空。解析器每次需要新词法单元时都会调用扫描器：

```
class Parser {
  public:
    Parser(Scanner& m_scanner, QList<Action>& actionList);
```

`match` 方法检查给定的词法单元是否等于扫描器取得的下一词法单元。若不相等，就报告语法

错误：

```
private:
  void match(TokenId tokenId);
```

Parser 类其余的方法分为两类：处理文法中指令的方法，以及处理表达式的方法，此外还包括类型检查和表达式求值的方法：

```
void instructionList();
void instruction();
```

我们还为文法中的每条表达式规则各加一个方法：

```
Value expression();
Value expressionRest(Value leftValue);
Value mulDivExpression();
Value mulDivExpressionRest(Value leftValue);
Value primaryExpression();
Value primaryExpression();
```

求值表达式时需要检查值的类型。例如，两个值相加时，两个操作数都必须是数值：

```
void checkType(TokenId operatorId, const Value& value);
void checkType(TokenId operatorId, const Value& leftValue,
               const Value& rightValue);
Value evaluate(TokenId operatorId, const Value& value);
Value evaluate(TokenId operatorId, const Value& leftValue,
               const Value& rightValue);

  private:
    Token m_lookAHead;
    Scanner& m_scanner;
    QList<Action>& m_actionList;
    QMap<QString,Value> m_assignMap;
};

#endif // PARSER_H
```

m_lookAHead 字段存放扫描器取得的下一词法单元，m_scanner 存放扫描器本身。m_actionList 字段存放构造函数中给出的动作列表的引用。最后，m_assignMap 存放赋值规则为名字分配的值。

Parser.cpp 文件存放 Parser 类各方法的定义。

Parser.cpp

```
#include <CAssert>
using namespace std;
```

```
#include "Value.h"
#include "Token.h"
#include "Scanner.h"
#include "Parser.h"
#include "Error.h"
```

构造函数初始化扫描器与动作列表的引用，并把 m_lookAHead 字段设为扫描器取得的第一个词法单元。随后调用 instructionList 开始解析过程。指令列表解析完毕后，剩下的唯一词法单元应当是文件结束词法单元：

```
Parser::Parser(Scanner& m_scanner, QList<Action>& actionList)
  :m_scanner(m_scanner),
   m_actionList(actionList) {
  m_lookAHead = m_scanner.nextToken();
  instructionList();
  match(EndOfFileId);
}
```

全局字段 g_lineNo 记录源代码的当前行，以便语法错误能报告正确的行号：

```
extern int g_lineNo;
```

instructionList 方法不断迭代，直到遇到文件结束词法单元：

```
void Parser::instructionList() {
  while (m_lookAHead.id() != EndOfFileId) {
    instruction();
  }
}
```

match 方法把扫描器取得的下一词法单元与给定的词法单元比较。若不符合，就报告语法错误；若符合，就由扫描器取得下一个词法单元：

```
void Parser::match(TokenId tokenId) {
  if (m_lookAHead.id() != tokenId) {
    syntaxError();
  }

  m_lookAHead = m_scanner.nextToken();
}
```

解析语言中的指令

instruction 方法包含一串 switch 分支，每一类指令各对应一个分支。

```
void Parser::instruction() {
  TokenId tokenId = m_lookAHead.id();
```

若是名字，我们就解析名字、赋值符（=）、其后的表达式和分号：

```
switch (tokenId) {
  case NameId: {
      QString assignName = m_lookAHead.name();
      match(NameId);
      match(AssignId);
      Value assignValue = expression();
      match(SemicolonId);
```

若该名字已经关联了某个值，就报告语义错误：

```
    check(!m_assignMap.contains(assignName),
          "the name "" + assignName + "" defined twiced");
    m_assignMap[assignName] = assignValue;
  }
  break;

  case SetPenColorId:
  case SetPenStyleId:
  case SetBrushColorId:
  case SetBrushStyleId:
  case SetFontId:
  case SetHorizontalAlignmentId:
  case SetVerticalAlignmentId: {
      match(tokenId);
      match(LeftParenthesisId);
      Value value = expression();
      match(RightParenthesisId);
      match(SemicolonId);
      checkType(tokenId, value);
      m_actionList.push_back(Action(tokenId, value));
    }
    break;
```

设置笔和刷的颜色与样式，以及字体和对齐方式，要稍微复杂一些。我们调用 `expression` 来解析并求值一个表达式，检查该表达式的类型，并向动作列表添加一个 `Action` 对象：

绘制直线、矩形、椭圆和文本接受两个表达式，它们的值会被求值并做类型检查：

```
  case DrawLineId:
  case DrawRectangleId:
  case DrawEllipseId:
  case DrawTextId: {
      match(tokenId);
```

```
      match(LeftParenthesisId);
      Value firstValue = expression();
      match(CommaId);
      Value secondValue = expression();
      match(RightParenthesisId);
      match(SemicolonId);
      checkType(tokenId, firstValue, secondValue);
      m_actionList.push_back(Action(tokenId, firstValue,
                                    secondValue));
    }
    break;
```

若前面的词法单元都不适用，就报告语法错误：

```
    default:
      syntaxError();
  }

  }
```

解析语言中的表达式

一个表达式在最低优先级上由两个乘法或除法表达式构成。我们先调用 `mulDivExpression`（按优先级递增顺序的下一个表达式），取得可能的加法或减法表达式的左值，然后调用 `expressionRest` 检查是否真的存在这样的表达式：

```
Value Parser::expression() {
  Value leftValue = mulDivExpression ();
  return expressionRest(leftValue);
}
```

`expressionRest` 方法检查下一个词法单元是否为加号或减号。若是，就得到一个加法或减法表达式：匹配该词法单元，对左值和右值做类型检查，求值所得表达式并返回：

```
Value Parser::expressionRest(Value leftValue) {
  TokenId tokenId = m_lookAHead.id();

  switch (tokenId) {
    case AddId:
    case SubtractId: {
        match(tokenId);
        Value rightValue = mulDivExpression();
        check(leftValue.isNumerical() && rightValue.isNumerical(),
              "non-numerical values in arithmetic expression");
        Value resultValue =
          evaluate(tokenId, leftValue, rightValue);
```

```
        return expressionRest(resultValue);
      }

    default:
      return leftValue;
  }
}
```

mulDivExpression 方法的工作方式与前面给出的 expression 类似：它调用 primaryExpression 和 mulDivExpressionRest，后两者查找乘法与除法。乘法与除法的优先级高于加法与减法。正如前面「源语言的文法」一节所述，我们需要在文法中增加一对新规则，并在解析器中为加/减表达式和乘/除表达式各增加一对方法：

```
Value Parser::mulDivExpression() {
  Value leftValue = primaryExpression();
  return mulDivExpressionRest(leftValue);
}

Value Parser::mulDivExpressionRest(Value leftValue) {
  TokenId tokenId = m_lookAHead.id();

  switch (tokenId) {
    case MultiplyId:
    case DivideId: {
        match(tokenId);
        Value rightValue = primaryExpression();
        check(leftValue.isNumerical() && rightValue.isNumerical(),
              "non-numerical values in arithmetic expression");
        Value resultValue =
          evaluate(tokenId, leftValue, rightValue);
        return mulDivExpressionRest (resultValue);
      }

    default:
      return leftValue;
  }
}
```

最后，基本表达式由点、坐标、颜色或字体表达式构成，也可以由括在圆括号中的表达式、一个名字（此时查其值）或一个值构成：

```
Value Parser::primaryExpression() {
  TokenId tokenId = m_lookAHead.id();
```

坐标表达式接受一个点，返回其 x 或 y 坐标。我们匹配关键字和圆括号，并在中间调用表达式；然后检查表达式的值是否为点，最后调用 evaluate 取出 x 或 y 坐标：

```
switch (tokenId) {
  case XCoordinateId:
  case YCoordinateId: {
      match(tokenId);
      match(LeftParenthesisId);
      Value value = expression();
      match(RightParenthesisId);
      check(value.isPoint(),
            "not a point in coordinate expression");
      checkType(tokenId, value);
      return evaluate(tokenId, value);
    }
    break;

break;
```

点表达式由关键字 point 和两个数值表达式构成：x 坐标与 y 坐标：

```
case PointId: {
    match(PointId);
    match(LeftParenthesisId);
    Value xValue = expression();
    match(CommaId);
    Value yValue = expression();
    match(RightParenthesisId);
    check(xValue.isNumerical() && yValue.isNumerical(),
          "non-numerical values in point expression");
    return Value(QPoint(xValue.numericalValue(),
                        yValue.numericalValue()));
}
```

颜色表达式由关键字 color 和三个数值表达式构成：红、绿、蓝分量：

```
case ColorId: {
    match(ColorId);
    match(LeftParenthesisId);
    Value redValue = expression();
    match(CommaId);
    Value greenValue = expression();
    match(CommaId);
    Value blueValue = expression();
    match(RightParenthesisId);
    check(redValue.isNumerical() && greenValue.isNumerical()
          && blueValue.isNumerical(),
          "non-numerical values in color expression");
    return Value(QColor(redValue.numericalValue(),
                        greenValue.numericalValue(),
```

```
                                blueValue.numericalValue()));
    }
```

字体表达式由关键字 font 和两个表达式构成：字体名称（字符串）与字体大小（数值）：

```
case FontId: {
    match(FontId);
    match(LeftParenthesisId);
    Value nameValue = expression();
    match(CommaId);

  Value sizeValue = expression();
  match(RightParenthesisId);
  check(nameValue.isString() && sizeValue.isNumerical(),
        "invalid types in font expression");
  return Value(QFont(nameValue.stringValue(),
                     sizeValue.numericalValue()));
}
```

表达式可以括在圆括号中。这种情况下，我们匹配圆括号，并在中间调用 expression 取得表达式的值：

```
case LeftParenthesisId: {
    match(LeftParenthesisId);
    Value value = expression();
    match(RightParenthesisId);
    return value;
  }
```

若是名字，我们在赋值映射中查其值并返回值；若没有值，就报告语义错误：

```
case NameId: {
    QString lookupName = m_lookAHead.name();
    match(NameId);
    check(m_assignMap.contains(lookupName ),
          "unknown name: "" + lookupName + "".");
    return m_assignMap[lookupName ];
  }
```

若是值，就直接返回该值：

```
case ValueId: {
    Value value = m_lookAHead.value();
    match(ValueId);
    return value;
  }
```

其他情况下报告语法错误：

```
    default:
      syntaxError();
      return Value();
  }
}
```

表达式的类型检查

第一个 checkType 方法检查只有一个值的表达式的类型。设置笔或刷的样式时，类型必须分别是笔样式或刷样式：

```
void Parser::checkType(TokenId codeId, const Value& value) {
  switch (codeId) {
    case SetPenStyleId:
      check(value.isPenStyle(), "not a pen-style value");
      break;

    case SetBrushStyleId:
      check(value.isBrushStyle(), "not a brush-style value");
      break;
```

设置颜色或字体时，值必须分别是颜色或字体：

```
case SetPenColorId:
case SetBrushColorId:
  check(value.isColor(), "not a color value");
  break;

case SetFontId:
  check(value.isFont(), "not a font value");
  break;
```

设置对齐方式时，值必须是某种对齐方式：

```
case SetHorizontalAlignmentId:
case SetVerticalAlignmentId:
  check(value.isAlignment(), "not an alignment value");
  break;
```

从点中取出 x 或 y 坐标时，值必须是点：

```
    case XCoordinateId:
    case YCoordinateId:
      check(value.isPoint(), "not a point value");
      break;
  }
```

```
}
```

第二个 checkType 方法接受两个值。绘制指令必须接受两个点：

```
void Parser::checkType(TokenId codeId, const Value& leftValue,
                       const Value& rightValue) {

switch (codeId) {
  case DrawLineId:
  case DrawRectangleId:
  case DrawEllipseId:
    check(leftValue.isPoint() && rightValue.isPoint(),
          "non-point values in draw expression");
    break;
```

绘制文本的指令必须接受一个点和一个字符串：

```
    case DrawTextId:
      check(leftValue.isPoint() && rightValue.isString(),
            "invalid values in text-drawing expression");
      break;
  }
}
```

求值表达式的值

第一个 evaluate 方法返回只有一个值的表达式的值。x 与 y 坐标运算符返回点的 x 或 y 坐标：

```
Value Parser::evaluate(TokenId codeId, const Value& value) {
  switch (codeId) {
    case XCoordinateId:
      return Value((double) value.pointValue().x());

    case YCoordinateId:
      return Value((double) value.pointValue().y());
```

断言只用于调试，返回 false 只是因为方法必须有返回值：

```
    default:
      assert(false);
      return false;
  }
}
```

最后，第二个 evaluate 方法对含两个值的表达式求值。先取出数值并求值算术表达式：

```
Value Parser::evaluate(TokenId codeId, const Value& leftValue,
                       const Value& rightValue) {
  double leftNumericalValue = leftValue.numericalValue(),
         rightNumericalValue = rightValue.numericalValue();

switch (codeId) {
  case AddId:
    return Value(leftNumericalValue + rightNumericalValue);

  case SubtractId:
    return Value(leftNumericalValue - rightNumericalValue);

  case MultiplyId:
    return Value(leftNumericalValue * rightNumericalValue);
```

若除数为零，就报告语义错误：

```
case DivideId:
  if (rightNumericalValue == 0) {
    semanticError("division by zero");
  }

  return Value(leftNumericalValue / rightNumericalValue);
```

最后，在点表达式中，我们返回一个存放两个数值的点值，它们即为该点的 x 与 y 坐标：

```
case PointId:
  return Value(QPoint(leftNumericalValue,
                      rightNumericalValue));
```

与前面第一个 evaluate 分支一样，断言只用于调试，返回 false 只是因为方法必须有返回值：

```
    default:
      assert(false);
      return Value();
  }
}
```

查看器

最后，该编写 DSL 的最后一部分——查看器了。查看器遍历动作并显示图形对象。`ViewerWidget` 类继承 Qt 的 `QWidget` 类，后者在屏幕上显示一个部件。

`ViewerWidget.h`

```
#ifndef MAINWIDGET_H
#define MAINWIDGET_H

#include <QWidget>
#include <QtWidgets>
#include "Value.h"
#include "Colors.h"
#include "Action.h"

class ViewerWidget : public QWidget {
  Q_OBJECT
```

构造函数调用基类 QWidget 的构造函数，并存下动作列表的引用：

```
public:
  ViewerWidget(const QList<Action>& actionList,
               QWidget *parentWidget = nullptr);
```

这个类的主体是 paintEvent 方法。每当部件需要重绘时都会调用它，它遍历动作列表：

```
void paintEvent(QPaintEvent *eventPtr);
```

我们调用 QFont 的默认构造函数，它把字体初始化为合适的系统字体。水平对齐和垂直对齐都居中。最后，m_actionList 存放解析器生成的动作列表的引用：

```
  private:
    Qt::Alignment m_horizontalAlignment = Qt::AlignHCenter,
                  m_verticalAlignment = Qt::AlignVCenter;
    const QList<Action>& m_actionList;
};

#endif // MAINWIDGET_H
```

ViewerWidget.cpp 文件存放 ViewerWidget 类各方法的定义。

ViewerWidget.cpp

```
#include <QtWidgets>
#include "ViewerWidget.h"
```

构造函数以父部件调用基类 QWidget 的构造函数，初始化 m_actionList 引用，设置部件标题，并设置合适的大小：

```
ViewerWidget::ViewerWidget(const QList<Action>& actionList,
                           QWidget *parentWidget)

  :QWidget(parentWidget),
   m_actionList(actionList) {
```

```
  setWindowTitle(tr("Domain Specific Language"));
  resize(500, 300);
}
```

每当部件需要重绘时都会调用 paintEvent 方法。它先定义 QPainter 对象 painter，然后遍历动作列表：

```
void ViewerWidget::paintEvent(QPaintEvent* /*event*/) {
  QPainter painter(this);

  for (const Action& action : m_actionList) {
    switch (action.id()) {
```

SetPenColor 动作用新颜色和当前样式创建一个新的笔，并把它设置给 painter。同样地，SetPenStyle 动作用新样式和当前颜色创建一个笔：

```
case SetPenColorId: {
    QColor penColor = action.value1().colorValue();
    QPen pen(penColor);
    pen.setStyle(painter.pen().style());
    painter.setPen(pen);
  }
  break;

case SetPenStyleId: {
    Qt::PenStyle penStyle = action.value1().penStyleValue();
    QPen pen(penStyle);
    pen.setColor(painter.pen().color());
    painter.setPen(pen);
  }
  break;
```

我们以与前面设置笔相同的方式设置刷的颜色与样式，唯一的区别在于创建的是刷而不是笔：

```
case SetBrushColorId: {
    QColor brushColor = action.value1().colorValue();
    QBrush brush(brushColor);
    brush.setStyle(painter.brush().style());
    painter.setBrush(brush);
  }
  break;

case SetBrushStyleId: {

  Qt::BrushStyle brushStyle =
    action.value1().brushStyleValue();
  QBrush brush(brushStyle);
```

```
    brush.setColor(painter.brush().color());
    painter.setBrush(brush);
  }
  break;
```

对于字体，我们调用 painter 的 `setFont`。此后字体就与 painter 关联，书写文本时将被使用：

```
case SetFontId: {

  QFont font = action.value1().fontValue();
  painter.setFont(font);
}
break;
```

水平与垂直对齐存放在 `m_horizontalAlignment` 和 `m_verticalAlignment` 中，书写文本时会用到这两个值：

```
case SetHorizontalAlignmentId:
  m_horizontalAlignment = action.value1().alignmentValue();
  break;

case SetVerticalAlignmentId:
  m_verticalAlignment = action.value1().alignmentValue();
  break;
```

现在该真正绘制一些图形对象了。直线只是连接两个点，而矩形或椭圆的左上角和右下角会被放进一个矩形，作为 `drawRect` 与 `drawEllipse` 调用的参数：

```
case DrawLineId:
  painter.drawLine(action.value1().pointValue(),
                   action.value2().pointValue());
  break;

case DrawRectangleId: {
    QRect rect(action.value1().pointValue(),
               action.value2().pointValue());

    painter.drawRect(rect);
  }
  break;

case DrawEllipseId: {

  QRect rect(action.value1().pointValue(),
             action.value2().pointValue());

  painter.drawEllipse(rect);
```

```
        }
        break;
```

最后,我们书写文本。先取出用来居中文本的点,以及要绘制的文本;然后用 Qt 的 QFontMetrics 类取得文本的尺寸（以像素为单位）:

```
        case DrawTextId:  {
            QPoint point = action.value1().pointValue();
            const QString& text = action.value2().stringValue();
            QFontMetrics metrics(painter.font());
            QSize size = metrics.size(0, text);
```

对于水平左对齐，文本的左侧就是该点的 x 坐标；对于居中对齐，文本左侧要向左移半个文本宽度；对于右对齐，文本左侧要向左移整个文本宽度:

```
        switch (m_horizontalAlignment) {
          case Qt::AlignHCenter:
            point.rx() -= size.width() / 2;
            break;

          case Qt::AlignRight:
            point.rx() -= size.width();
            break;
        }
```

同样地，对于垂直顶端对齐，文本的上侧就是该点的 y 坐标；对于居中对齐，文本上侧要向上移半个文本高度；对于底端对齐，文本上侧要向上移整个文本高度:

```
        switch (m_verticalAlignment) {
          case Qt::AlignVCenter:
            point.ry() -= size.height() / 2;
            break;

          case Qt::AlignBottom:
            point.ry() -= size.height();
            break;
        }

        painter.drawText(point, text);

                }
                break;
            }
          }
        }
```

main 函数

最后，main 函数调用扫描器的静态方法 init 初始化其词法单元、关键字和值。然后创建 QApplication 对象，读取并解析源代码，创建查看器部件。它执行动作列表并显示图形对象；应用程序一直运行，直到用户点击右上角的关闭按钮。

Main.cpp

```
#include <QApplication>
#include <QMessageBox>

#include "Action.h"
#include "Error.h"
#include "Scanner.h"
#include "Parser.h"
#include "ViewerWidget.h"

int main(int argc, char *argv[]) {
  Scanner::init();
  QApplication application(argc, argv);

  try {
    QString path = "C:\Input.dsl";
    QFile file(path);

    if (!file.open(QIODevice::ReadOnly)) {
      error("Cannot open file "" + path + "" for reading.");
    }

    QString buffer(file.readAll());
    Scanner scanner(buffer);

    QList<Action> actionList;
    Parser(scanner, actionList);

    ViewerWidget mainWidget(actionList);
    mainWidget.show();
    return application.exec();

}
```

若出现语法或语义错误，就把错误信息显示在一个消息框中：

```
  catch (exception e) {
    QMessageBox messageBox(QMessageBox::Information,
                           QString("Error"), QString(e.what()));
```

```
            messageBox.exec();
        }
    }
```

小结

本章我们开始开发一种 DSL，它生成一串创建图形对象的动作，这些对象显示在一个部件中。我们的 DSL 支持绘制直线、矩形、椭圆和文本等图形对象的指令，以及设置对象颜色、样式和对齐方式的指令。它还支持带算术运算符的表达式。

DSL 的语言由文法定义，并由扫描器扫描文本中的有意义部分，解析器检查源代码是否符合文法并生成一串动作，查看器读取并执行这些动作。

下一章我们将继续开发 DSL。本章的 DSL 只支持按直线顺序执行的代码；下一章我们将加入函数调用，以及选择与迭代（`if` 与 `while` 指令）。

10
高级领域特定语言

在上一章中，我们开发了一种领域特定语言（Domain-Specific Language，DSL）。本章将从以下几个方面改进这门语言：

我们将添加选择和迭代。具体来说，我们将加入 `if` 和 `while` 指令。在上一章的语言中，动作是按顺序直接执行的；而在本章中，可以在多个分支之间做出选择，也可以对一部分代码进行迭代。

我们将添加变量。在上一章中，一个名字只能被赋值一次；而在本章中，赋给名字的值可以在程序执行过程中被重新赋值。

我们将添加带有参数和返回值的函数。在上一章中，程序由一连串指令组成；而在本章中，程序由一连串函数组成。与 C++ 一样，程序必须有一个 `main` 函数作为执行的起点。

最后，我们将在从源代码到查看器的处理流程中再增加一个模块。在上一章中，解析器生成一连串动作，由查看器显示；而在本章中，解析器生成一连串指示（directive），指示再由求值器求值为动作。

由于本章的语言支持选择、迭代、变量和函数调用，它已经开始像一门传统的编程语言了。

本章将涵盖以下主题：

- 与上一章一样，我们先通过一个示例非正式地查看 DSL 的源代码。不过，这个示例会用到变量和函数调用，还会用到 `if` 和 `while` 指令。
- 然后，我们用文法（grammar）正式定义这门语言。文法是上一章文法的扩展，我们将加入函数定义、函数调用和函数返回的指令，以及选择（`if`）和迭代（`while`）。
- 定义好文法之后，我们编写扫描器（scanner）。本章的扫描器与上一章几乎完全相同，唯一的区别是增加了几个关键字。
- 写好扫描器之后，我们编写解析器（parser）。解析器是上一章解析器的扩展，我们为函数、选择和迭代添加了相应的方法。不过，上一章的解析器生成的是一连串动作，由查看器读取并执行；而本章的解析器生成的则是一连串指示，由求值器读取。

- 本章接下来的环节是求值器（evaluator），而不是查看器。求值器接收解析器生成的指示序列，生成一连串动作，由查看器读取并执行。求值器使用为名字赋值的映射。其中有一个值映射栈（value map stack），确保每个被调用的函数都获得自己的全新值映射；还有一个值栈（value stack），在求值表达式时存放临时值；最后还有一个调用栈（call stack），存放函数调用的返回地址。
- 最后，查看器的工作方式与上一章相同：它遍历求值器生成的动作列表，在 Qt 部件中显示图形对象。

通过示例改进源语言

我们来看一个新示例：定义并调用一个名为 `triangle` 的函数，用不同的画笔以不同的尺寸绘制三角形。注意，函数不必按任何特定顺序出现。

我们先把 `left` 和 `length` 变量设为 50，它们分别存放第一个三角形最左侧角的 x 坐标和底边长度。接着把 `index` 变量设为 0，它的值将用于 `while` 迭代：

```
function main() {
  left = 50;
  length = 50;
  index = 0;
```

只要 `index` 小于 4，我们就继续迭代。注意，本章在 Value 类中加入了布尔值。当 `index` 为偶数时，我们把画笔样式设为实线；当 `index` 为奇数时，则把画笔样式设为虚线。注意，我们还用关系表达式和取模运算符（`%`）扩展了这门语言：

```
while (index < 4) {
  if ((index % 2) == 0) {
    SetPenStyle(SolidLine);
  }
  else {
    SetPenStyle(DashLine);
  }
```

我们设置三角形的左上角点，并调用 `drawTriangle` 函数来实际绘制三角形：

```
topLeft = point(left, 25);
call drawTriangle(topLeft, length);
```

调用 `triangle` 之后，我们增大下一个三角形的底边长度，并移动最左侧角的位置：

```
    length = length + 25;
    left = left + length;
    index = index + 1;
  }
```

```
}
```

在 drawTriangle 函数中，我们调用 getTopRight 和 getBottomMiddle 函数来获得三角形的右上点和底边中点。最后，我们调用 drawLine 来绘制三角形的三条边：

```
function drawTriangle(topLeft, length) {
  topRight = call getTopRight(topLeft, length);
  bottomMiddle = call getBottomMiddle(topLeft, length);
  drawLine(topLeft, topRight);
  drawLine(topRight, bottomMiddle);

  drawLine(bottomMiddle, topLeft);
}
```

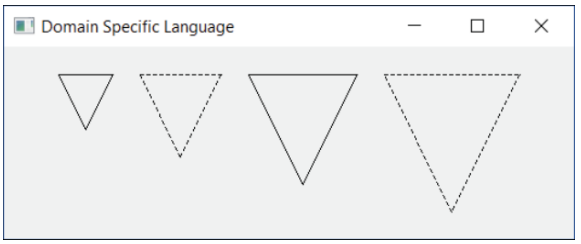
getTopRight 函数提取左上角点的 x 和 y 坐标，并返回一个 x 坐标增加了三角形底边长度的点：

```
function getTopRight(topLeft, length) {
  return point(xCoordinate(topLeft) + length,
               yCoordinate(topLeft));
}
```

getBottomMiddle 函数同样提取左上角点的 x 和 y 坐标，然后计算出底边中点的 x 和 y 坐标并返回该点：

```
function getBottomMiddle(topLeft, length) {
  left = xCoordinate(topLeft);
  top = yCoordinate(topLeft);
  middle = left + length / 2;
  bottom = top + length;
  return point(middle, bottom);
}
```

运行代码后，输出如下截图所示：



改进文法

本章将改进我们这门语言的文法。首先，程序由一连串函数而非指令组成。严格来说，程序可以不含任何函数，但此时语义错误会报告缺少 `main` 函数：

```
program -> functionDefinitionList
functionDefinitionList -> functionDefinition*
```

函数的定义由关键字 `function`、括号括起的一列名字以及花括号括起的一列指令组成。nameList 由零个或多个名字组成，名字之间用逗号分隔：

```
functionDefinition -> function name(nameList) { instructionList }
```

在指令方面，我们加入了函数调用。我们既可以把函数作为一条指令直接调用（前面示例中的 call drawTriangle），也可以把它作为表达式的一部分来调用（call getTopRight 和 call getBottomMiddle）。

我们还加入了 `while` 指令以及 `if` 指令——后者可以带也可以不带 `else` 部分。最后还有块指令：由花括号括起的一列指令：

```
instruction -> callExpression ;
             | while (expression) instruction
             | if (expression) instruction
             | if (expression) instruction else instruction
             | { instructionList }
             | ...
callInstruction -> callExpression ;
```

至于表达式，唯一的区别是我们加入了函数调用。expressionList 是一列由零个或多个表达式组成的列表，表达式之间用逗号分隔：

```
primaryExpression -> call name(expressionList)
                   |
```

Token 与扫描器

与上一章一样，这门语言最终生成的目标代码仍是动作，只不过这次是由求值器而非解析器生成。Action 类与上一章的类完全相同，Value 类和 ViewerWidget 类以及颜色和错误处理也是如此。不过，Token 类和 Scanner 类做了扩展，TokenId 枚举增加了更多的词法单元（token）标识。

`Token.h`

```
class Token {
  // ...
```

```
    enum TokenId {BlockId, CallId, ElseId, FunctionId, GotoId,
                  IfId, IfNotGotoId, ReturnId, WhileId, // ...
                 };
    // ...
};
```

同样，Scanner 中的 init 也加入了这些关键字：

Scanner.cpp

```
void Scanner::init() {
  ADD_TO_KEYWORD_MAP(CallId)
  ADD_TO_KEYWORD_MAP(ElseId)
  ADD_TO_KEYWORD_MAP(FunctionId)
  ADD_TO_KEYWORD_MAP(IfId)
  ADD_TO_KEYWORD_MAP(ReturnId)
  ADD_TO_KEYWORD_MAP(WhileId)
// ...
}
```

解析器

解析器增加了与文法新规则相对应的方法。此外，本章的解析器不再生成动作，而是生成指示。原因在于：上一章的源代码中的指令是从头到尾顺序执行的，而本章的源代码包含选择、迭代和函数调用，它们会改变指令的执行流程。因此，引入一个中间层是合理的——解析器生成指示，指示再被求值为动作。

由于本章的语言支持函数，我们需要一个 Function 类来存放函数。它存放形式参数的名字和函数的起始地址。

Function.h

```
#ifndef FUNCTION_H
#define FUNCTION_H
#include <QtWidgets>
#include "Value.h"
#include "Action.h"
class Function {
  public:
    Function() {}
    Function(const QList<QString>& nameList, int address);
    const QList<QString>& nameList() const {return m_nameList;}
    int address() {return m_address;}
    Function(const Function& function);
    Function operator=(const Function& function);
```

```
  private:
    QList<QString> m_nameList;
    int m_address;
};
#endif // FUNCTION_H
```

Function.cpp 文件存放 Function 类方法的定义。

Function.cpp

```
#include "Function.h"
Function::Function(const QList<QString>& nameList, int address)
 :m_nameList(nameList),
  m_address(address) {
  // Empty.
}
Function::Function(const Function& function)
 :m_nameList(function.m_nameList),
  m_address(function.m_address) {
  // Empty.
}
Function Function::operator=(const Function& function) {
  m_nameList = function.m_nameList;
  m_address = function.m_address;
  return *this;
}
```

由于本章的解析器生成的是一连串指示而非动作，我们还需要一个 Directive 类来存放指示。大多数情况下，Directive 对象只存放 TokenId 枚举中的标识。不过，在函数调用的场合，我们需要存放函数名和实际参数的个数；在函数定义的场合，我们存放对 Function 对象的引用；在表达式由一个名字或一个值构成的场合，我们需要存放名字或值；最后还有几种跳转指示，此时我们需要存放地址。

Directive.h

```
#ifndef DIRECTIVE_H
#define DIRECTIVE_H
#include <QtWidgets>
#include "Token.h"
#include "Value.h"
#include "Function.h"
class Directive {
  public:
    Directive(TokenId tokenId);
    Directive(TokenId tokenId, int address);
    Directive(TokenId tokenId, const QString& name);
    Directive(TokenId tokenId, const QString& name,
```

```
                int parameters);
    Directive(TokenId tokenId, const Value& value);
    Directive(TokenId tokenId, const Function& function);
    Directive(const Directive& directive);
    Directive operator=(const Directive& directive);
    TokenId directiveId() {return m_directiveId;}
    const QString& name() {return m_name;}
    const Value& value() {return m_value;}
    const Function& function() {return m_function;}
    int parameters() const {return m_parameters;}
    int address() const {return m_address;}
    void setAddress(int address) {m_address = address;}
  private:
    TokenId m_directiveId;
    QString m_name;
    int m_parameters, m_address;
    Value m_value;
    Function m_function;
};
#endif // DIRECTIVE_H
```

Directive.cpp 文件存放 Directive 类方法的定义。

Directive.cpp

```
#include "Directive.h"
```

大多数情况下，我们只创建一个带有指示标识的 Directive 对象：

```
Directive::Directive(TokenId directiveId)
 :m_directiveId(directiveId) {
  // Empty.
}
```

跳转指示需要跳转地址：

```
Directive::Directive(TokenId directiveId, int address)
 :m_directiveId(directiveId),
  m_address(address) {
  // Empty.
}
```

给变量赋值时，我们需要变量的名字；不过不需要值，因为值将存放在栈上。另外，当表达式由一个名字构成时，我们需要存放这个名字：

```
Directive::Directive(TokenId directiveId, const QString& name)
 :m_directiveId(directiveId),
```

```
  m_name(name) {
  // Empty.
}
```

函数调用的指示需要函数名和实际参数的个数：

```
Directive::Directive(TokenId directiveId, const QString& name,
                     int parameters)
 :m_directiveId(directiveId),
  m_name(name),
  m_parameters(parameters) {
  // Empty.
}
```

当表达式仅由一个值构成时，我们直接把值存放在指示中：

```
Directive::Directive(TokenId directiveId, const Value& value)
 :m_directiveId(directiveId),
  m_value(value) {
  // Empty.
}
```

最后，在函数定义中我们存放一个 Function 类的对象：

```
Directive::Directive(TokenId directiveId,
                     const Function& function)
 :m_directiveId(directiveId),
  m_function(function) {
  // Empty.
}
```

Parser 类增加了与文法新规则相对应的方法：函数定义以及 `if`、`while`、`call` 和 `return` 指令。

`Parser.h`

```
// ...
class Parser {
  private:
    void functionDefinitionList();
    void functionDefinition();
```

nameList 方法收集函数的形式参数，而 expressionList 收集函数调用的实际参数：

```
QList<QString> nameList();
int expressionList();
```

Parser 类中还加入了 callExpression 方法，因为函数既可以作为一条指令被显式调用，也可以

作为表达式的一部分被调用：

```
    void callExpression();
    // ...
};
```

Parser.cpp 文件存放 Parser 类方法的定义。本章解析器的起始方法是 functionDefinitionList，它不断调用 functionDefinition，直到到达文件末尾为止。

Parser.cpp

```
void Parser::functionDefinitionList() {
  while (m_lookAHead.id() != EndOfFileId) {
    functionDefinition();
  }
}
```

functionDefinition 方法解析一个函数定义。我们先匹配 function 关键字，并把函数名存放起来：

```
void Parser::functionDefinition() {
  match(FunctionId);
  QString name = m_lookAHead.name();
  match(NameId);
```

函数名后面是括号括起的参数名字列表。我们把名字列表存放在 nList 字段中。不能把这个字段命名为 nameList，因为这个名字已经被同名方法占用了：

```
match(LeftParenthesisId);
QList<QString> nList = nameList();
match(RightParenthesisId);
```

我们把指示列表的当前大小存为函数的起始地址，用名字列表和起始地址创建一个 Function 对象，并把带有该函数的 Directive 对象加入指示列表：

```
int startAddress = (int) m_directiveList.size();
Function function(nList, startAddress);
m_directiveList.push_back(Directive(FunctionId, function));
```

名字列表后面是由花括号括起的一列指令：

```
match(LeftBracketId);
instructionList();
match(RightBracketId);
```

为了确保函数确实把控制权交还给调用它的函数，我们加入一个带有 return 词法单元标识的 Directive 对象：

```
    m_directiveList.push_back(Directive(ReturnId));
```

函数定义完毕后，我们检查没有其他同名函数：

```
    check(!m_functionMap.contains(name),
          "function "" + name + "" already defined");
```

如果函数名为 main，它就是程序的起始函数，不能带有参数：

```
    check(!((name == "main") && (nList.size() > 0)),
          "function "main" cannot have parameters");
```

最后，我们把函数加入 functionMap：

```
      m_functionMap[name] = function;
    }
```

nameList 方法解析括号内以逗号分隔的名字列表：

```
    QList<QString> Parser::nameList() {
      QList <QString> nameList;
```

只要没有遇到右括号，我们就继续：

```
    while (m_lookAHead.id() != RightParenthesisId) {
      QString name = m_lookAHead.name();
      nameList.push_back(name);
      match(NameId);
```

匹配完名字后，我们检查下一个词法单元是否为右括号。如果是，说明已到达名字列表末尾，我们便中断迭代：

```
    if (m_lookAHead.id() == RightParenthesisId) {
      break;
    }
```

如果下一个词法单元不是右括号，我们就假定它是逗号，匹配它，然后继续迭代处理下一个表达式：

```
      match(CommaId);
    }
```

最后，在返回名字列表之前，我们需要检查列表中没有名字出现两次。我们遍历名字列表，把名字加入一个集合：

```
      QSet<QString> nameSet;
      for (const QString& name : nameList) {
        if (nameSet.contains(name)) {
          semanticError("parameter "" + name + "" defined twice");
```

```
    }
    nameSet.insert(name);
  }
  return nameList;
}
```

本章的 instructionList 方法看起来略有不同，因为它位于一个指令块内部。只要没有遇到右花括号，我们就继续迭代：

```
void Parser::instructionList() {
  while (m_lookAHead.id() != RightBracketId) {
    instruction();
  }
}
```

由于函数既可以作为指令被显式调用，也可以作为表达式的一部分，因此在调用指令的场合，我们只需调用 callExpression 并匹配分号：

```
void Parser::instruction() {
  switch (m_lookAHead.id()) {
    case CallId:
      callExpression();
      match(SemicolonId);
      break;
```

在 return 指令中，我们匹配 return 关键字，并检查其后是否跟着分号。如果没有跟着分号，我们就解析一个表达式，然后假定下一个词法单元是分号。注意，我们不存放表达式的结果，求值器稍后会把这个值放到栈上：

```
case ReturnId:
  match(ReturnId);
  if (m_lookAHead.id() != SemicolonId) {
    expression();
  }
  m_directiveList.push_back(Directive(ReturnId));
  match(SemicolonId);
  break;
```

对于 if 关键字，我们匹配它并解析一个由括号括起的表达式：

```
case IfId: {
    match(IfId);
    match(LeftParenthesisId);
    expression();
    match(RightParenthesisId);
```

如果表达式求值为假，我们就要跳过 if 表达式后面的那条指令。因此，我们加入一条 IfNotGoto

指示，打算跳过 if 关键字后面的指令：

```
int ifNotIndex = (int) m_directiveList.size();
m_directiveList.push_back(Directive(IfNotGotoId, 0));
instruction();
```

如果这条指令后面跟着 else 关键字，我们就匹配它，并加入一条 Goto 指示——当 if 指令的表达式为真时，打算跳过 else 部分：

```
if (m_lookAHead.id() == ElseId) {
  match(ElseId);
  int elseIndex = (int) m_directiveList.size();
  m_directiveList.push_back(Directive(GotoId, 0));
```

然后我们设置前面那条 IfNotGoto 指示的跳转地址。如果表达式不为真，程序就应跳到这一点：

```
m_directiveList[ifNotIndex].
  setAddress((int) m_directiveList.size());
instruction();
```

另一方面，如果 if 指令的表达式为真，程序就应跳过 else 部分到达这一点：

```
  m_directiveList[elseIndex].
    setAddress((int) m_directiveList.size());
}
```

如果 if 指令后面没有跟 else 关键字，那么当表达式不为真时，程序就应跳到程序中的这一点：

```
  else {
    m_directiveList[ifNotIndex].
      setAddress((int) m_directiveList.size());
  }
}
break;
```

对于 while 关键字，我们匹配它，并把指示列表的当前下标存放起来，以便程序在每次迭代之后都能跳回这一点：

```
case WhileId: {
    match(WhileId);
    int whileIndex = (int) m_directiveList.size();
```

然后我们解析表达式及其两侧的括号：

```
match(LeftParenthesisId);
expression();
match(RightParenthesisId);
```

当表达式不为真时，我们加入一条 IfNotGoto 指示，让程序跳出迭代：

```
int ifNotIndex = (int) m_directiveList.size();
m_directiveList.push_back(Directive(IfNotGotoId, 0));
instruction();
```

我们在 `while` 表达式后面的那条指令之后加入一条 Goto 指示，让程序在每次迭代结束时都能跳回表达式处：

```
m_directiveList.push_back(Directive(GotoId, whileIndex));
```

最后，我们把 `while` 指令开头的 IfNotGoto 指示的地址设置好，这样当表达式不为真时，它就能跳到程序中的这一点：

```
  m_directiveList[ifNotIndex].
    setAddress((int) m_directiveList.size());
}
break;
```

遇到左花括号时，我们面对的是由花括号括起的一串指令。我们解析这一对花括号，并调用 instructionList：

```
case LeftBracketId:
  match(LeftBracketId);
  instructionList();
  match(RightBracketId);
  break;
```

最后，遇到名字时，我们面对的是一条赋值。我们匹配名字词法单元和赋值运算符（=），解析表达式，然后匹配分号。接着，我们向指示列表加入一个 Assign 对象，其中存放着将要被赋值的名字。注意，我们不存放表达式的值，因为求值器稍后会把它压入值栈：

```
    case NameId: {
        QString name = m_lookAHead.name();
        match(NameId);
        match(AssignId);
        expression();
        match(SemicolonId);
        m_directiveList.push_back(Directive(AssignId, name));
      }
      break;
      // ...
  }
}
```

callExpression 方法匹配 `call` 关键字，存放函数名，解析参数表达式，并向指示列表加入一条携带该调用的 Directive 对象。注意，此时我们不检查函数是否存在，也不统计参数个数，因

为函数可能尚未定义。所有类型检查都由后面的求值器负责完成：

```
void Parser::callExpression() {
  match(CallId);
  QString name = m_lookAHead.name();
  match(NameId);
  match(LeftParenthesisId);
  int size = expressionList();
  match(RightParenthesisId);
  m_directiveList.push_back(Directive(CallId, name, size));
}
```

expressionList 方法解析一列表达式。与前面的名字列表不同，这里我们不返回列表本身，只返回它的大小。这些表达式各自生成自己的指示，它们的值由求值器稍后存放到栈上：

```
int Parser::expressionList() {
  int size = 0;
```

只要没有遇到右括号，我们就继续迭代：

```
while (m_lookAHead.id() != RightParenthesisId) {
  expression();
  ++size;
```

解析完表达式后，我们检查下一个词法单元是否为右括号。如果是，表达式列表就结束了，我们便中断迭代：

```
if (m_lookAHead.id() == RightParenthesisId) {
  break;
}
```

如果下一个词法单元不是右括号，我们就假定它是逗号，匹配它，然后继续迭代：

```
  match(CommaId);
}
```

最后，迭代结束后，我们返回表达式的个数：

```
  return size;
}
```

求值器

求值器对一串指示进行求值，生成一列动作，供查看器稍后读取并执行。求值从第一行上的指示开始，这一指示是一条跳到 `main` 函数起始地址的跳转。当求值遇到一条不带返回地址的 `return` 指示时，求值便停止。此时我们已经到达 `main` 的末尾，执行应当结束。

求值器在一个值栈上工作。每当求出一个值，就把它压入栈中；每当求值表达式需要用到值时，就从栈中弹出。

Evaluator.h

```
#ifndef EVALUATOR_H
#define EVALUATOR_H
#include <QtWidgets>
#include "Error.h"
#include "Directive.h"
#include "Action.h"
#include "Function.h"
```

Evaluator 类的构造函数借助函数映射对指示列表进行求值：

```
class Evaluator {
  public:
    Evaluator(const QList<Directive>& directiveList,
              QList<Action>& actionList,
              QMap<QString,Function> functionMap);
```

checkType 和 evaluate 方法与上一章的完全相同，它们已从 Parser 移到了 Evaluator 中。checkType 方法检查与词法单元相关联的表达式是否类型正确，evaluate 方法则对表达式求值：

```
private:
  void checkType(TokenId tokenId, const Value& value);
  void checkType(TokenId tokenId, const Value& leftValue,
                 const Value& rightValue);
  Value evaluate(TokenId tokenId, const Value& value);
  Value evaluate(TokenId tokenId, const Value& leftValue,
                 const Value& rightValue);
```

对表达式求值时，它的值被压入 m_valueStack。给变量赋值时，变量名和值存放在 m_valueMap 中。注意，在本章中，一个变量可以被多次赋值。当一个函数调用另一个函数时，调用者的值映射被压入 m_valueMapStack，以便被调用函数获得一个全新的值映射；同时返回地址被压入 m_returnAddressStack：

```
    QStack<Value> m_valueStack;
    QMap<QString,Value> m_valueMap;
    QStack<QMap<QString,Value>> m_valueMapStack;
    QStack<int> m_returnAddressStack;
};
#endif // EVALUATOR_H
```

Evaluator.cpp 文件存放 Evaluator 类方法的定义：

Evaluator.cpp

```
#include <CAssert>
using namespace std;
#include "Error.h"
#include "Evaluator.h"
```

Evaluator 类的构造函数可以看作求值器的心脏。构造函数中的 directiveIndex 字段是当前 Directive 对象在指示列表中的下标。通常，每轮迭代它都会增加；不过，由于 `if` 或 `while` 指令以及函数调用和返回的存在，它也可以被赋予不同的值：

```
Evaluator::Evaluator(const QList<Directive>& directiveList,
                     QList<Action>& actionList,
                     QMap<QString,Function> functionMap) {
  int directiveIndex = 0;
  while (true) {
    Directive directive = directiveList[directiveIndex];
    TokenId directiveId = directive.directiveId();
```

函数被调用时，我们先在函数映射中查找函数名，找不到就报告语义错误。然后我们检查实际参数的个数是否等于形式参数的个数（即 Function 对象中名字列表的大小）：

```
switch (directiveId) {
  case CallId: {
      QString name = directive.name();
      check(functionMap.contains(name),
            "missing function: "" + name + """);
      Function function = functionMap[name];
      check(directive.parameters() ==
            function.nameList().size(),
            "invalid number of parameters");
```

调用函数时，我们把下一条指示的下标压入返回地址栈，让被调用函数能返回到正确的地址；把调用者的值映射压入值映射栈，以便调用结束后能取回它；然后清空值映射，让它焕然一新供被调用函数使用。最后，我们把指示下标设为被调用函数的起始地址，把控制转移到被调用函数的开头。注意，我们对实际参数表达式不做任何处理——它们早已求值完毕，值也已经压入值栈：

```
    m_returnAddressStack.push(directiveIndex + 1);
    m_valueMapStack.push(m_valueMap);
    m_valueMap.clear();
    directiveIndex = function.address();
  }
  break;
```

在函数的开头，我们为每个参数弹出值栈，把每个参数的名字与它的值关联到值映射中。别忘了，参数表达式是在调用函数之前求值的，它们的值已压入值栈；还要记住，第一个参数最先压入，位于栈中其他参数的下方，这就是我们按相反顺序为参数赋值的原因；最后记住，调用

者的值映射已被压入值映射栈，而值映射在函数调用期间已被清空，因此函数开头处的当前值映射是空的：

```
case FunctionId: {
    const Function& function = directive.function();
    const QList<QString>& nameList = function.nameList();
    for (int listIndex = ((int) nameList.size() - 1);
         listIndex >= 0; --listIndex) {
      const QString& name = nameList[listIndex];
      m_valueMap[name] = m_valueStack.pop();
    }
  }
  ++directiveIndex;
  break;
```

从函数返回时，我们首先检查返回地址栈是否为空。如果不为空，就执行一次普通的函数返回：弹出值映射栈来恢复调用者的值映射，同时弹出返回地址栈，把指示下标设为函数调用之后的那条地址：

```
case ReturnId:
  if (!m_returnAddressStack.empty()) {
    m_valueMap = m_valueMapStack.pop();
    directiveIndex = m_returnAddressStack.pop();
  }
```

不过，如果返回地址栈为空，我们就遇到了一种特殊情况——已经到达 `main` 函数的末尾。此时我们不再返回到调用函数（因为根本没有调用函数），而是通过调用 `return` 来结束求值器的执行。别忘了，我们此刻正处在 Evaluator 类的构造函数中，我们是从构造函数返回的：

```
else {
  return;
}
break;
```

IfNotGoto 指示是解析器在解析 `if` 或 `while` 指令时加入的。我们弹出值栈；如果值为假，就通过调用指示的 address 方法设置指示下标来执行跳转。别忘了，本章已在 Value 类中加入了布尔值：

```
case IfNotGotoId: {
    Value value = m_valueStack.pop();
    if (!value.booleanValue()) {
      directiveIndex = directive.address();
    }
```

如果值为真，我们不执行跳转，只是简单地增加指示下标：

```
      else {
        ++directiveIndex;
      }
    }
    break;
```

Goto 指示执行无条件跳转；我们只需设置新的指示下标。由于 IfNotGoto 和 Goto 指示都是解析器生成的，我们不需要做任何类型检查：

```
    case GotoId:
      directiveIndex = directive.address();
      break;
```

set 指示的工作方式与上一章的解析器相对应。表达式的值已在早前某条指示求值时被压入值栈。我们弹出值栈中的值，检查它的类型是否正确，然后把带该值的动作加入动作列表，并增加指示下标：

```
    case SetPenColorId:
    case SetPenStyleId:
    case SetBrushColorId:
    case SetBrushStyleId:
    case SetFontId:
    case SetHorizontalAlignmentId:
    case SetVerticalAlignmentId: {
        Value value = m_valueStack.pop();
        checkType(directiveId, value);
        actionList.push_back(Action(directiveId, value));
        ++directiveIndex;
      }
      break;
```

同样，draw 指示也与上一章的解析器相似。它们的第一个值和第二个值按相反的顺序弹出，因为第一个值最先压入，因而位于栈中第二个值的下方。接着我们检查两个值的类型是否正确，把动作加入动作列表，并增加指示下标：

```
    case DrawLineId:
    case DrawRectangleId:
    case DrawEllipseId:
    case DrawTextId: {
        Value secondValue = m_valueStack.pop();
        Value firstValue = m_valueStack.pop();
        checkType(directiveId, firstValue, secondValue);
        actionList.push_back(Action(directiveId, firstValue,
                                    secondValue));
        ++directiveIndex;
      }
      break;
```

赋值指示把一个名字与值关联到值映射中。注意，如果该名字已经关联了一个值，先前的值会被覆盖；还要注意，值映射是当前函数局部的，调用它的函数有各自的值映射，压在值映射栈上：

```
case AssignId: {
    Value value = m_valueStack.pop();
    m_valueMap[directive.name()] = value;
    ++directiveIndex;
  }
  break;
```

对于只含一个值的表达式，我们从栈中弹出它的值，检查类型，然后对表达式求出结果值并压入值栈。最后，增加指示下标：

```
case XCoordinateId:
case YCoordinateId: {
    Value value = m_valueStack.pop();
    checkType(directiveId, value);
    Value resultValue = evaluate(directiveId, value);
    m_valueStack.push(resultValue);
    ++directiveIndex;
  }
  break;
```

对于含两个值的表达式，它的第一个值和第二个值从栈中弹出（按相反顺序），检查类型，然后对表达式求出结果值并压入值栈。最后，增加指示下标：

```
case AddId:
case SubtractId:
case MultiplyId:
case DivideId:
case PointId: {
    Value rightValue = m_valueStack.pop();
    Value leftValue = m_valueStack.pop();
    checkType(directiveId, leftValue, rightValue);
    Value resultValue =
      evaluate(directiveId, leftValue, rightValue);
    m_valueStack.push(resultValue);
    ++directiveIndex;
  }
  break;
```

对于颜色表达式，红、绿、蓝分量值从值栈中弹出（按相反顺序），检查类型，然后把得到的颜色压入值栈。最后，增加指示下标：

```
case ColorId: {
    Value blueValue = m_valueStack.pop();
    Value greenValue = m_valueStack.pop();
    Value redValue = m_valueStack.pop();
    checkColorType(redValue, greenValue, blueValue);
    QColor color(redValue.numericalValue(),
                 greenValue.numericalValue(),
                 blueValue.numericalValue());
    m_valueStack.push(Value(color));
    ++directiveIndex;
  }
  break;
```

对于字体表达式，名字和大小两个值从值栈中弹出（按相反顺序）并检查类型。得到的字体压入值栈，指示下标增加：

```
case FontId: {
    Value sizeValue = m_valueStack.pop();
    Value nameValue = m_valueStack.pop();
    checkFontType(nameValue, sizeValue,
                  boldValue, italicValue);
    QFont font(nameValue.stringValue(),
               sizeValue.numericalValue());
    m_valueStack.push(Value(font));
    ++directiveIndex;
  }
  break;
```

遇到名字时，我们查找它的值，压入值栈，并增加指示下标。如果该名字没有关联任何值，就报告语义错误：

```
case NameId: {
    QString name = directive.name();
    check(m_valueMap.contains(name),
          "unknown name: "" + name +"""");
    m_valueStack.push(m_valueMap[name]);
    ++directiveIndex;
  }
  break;
```

最后，遇到值时，我们直接把它压入值栈，并增加指示下标：

```
    case ValueId:
      m_valueStack.push(directive.value());
      ++directiveIndex;
      break;
  }
```

```
  }
}
```

main 函数

最后，main 函数与上一章的几乎完全相同。

Main.cpp

```
#include <QApplication>
#include <QMessageBox>
#include <IOStream>
using namespace std;
#include "Action.h"
#include "Error.h"
#include "Scanner.h"
#include "Parser.h"
#include "Evaluator.h"
#include "ViewerWidget.h"

int main(int argc, char *argv[]) {
  Scanner::init();
  QApplication application(argc, argv);
  try {
    QString path = "C:\Input.dsl";
    QFile file(path);
    if (!file.open(QIODevice::ReadOnly)) {
      error("Cannot open file "" + path + "" for reading.");
    }
    QString buffer(file.readAll());
    Scanner scanner(buffer);
```

唯一的区别在于：解析器生成的是一连串指示而非动作，同时还会生成一个函数映射；函数映射被送往求值器，求值器生成最终的动作列表，交给查看器读取并执行，由查看器显示图形对象：

```
    QList<Directive> directiveList;
    QMap<QString,Function> functionMap;
    Parser(scanner, directiveList, functionMap);
    QList<Action> actionList;
    Evaluator evaluator(directiveList, actionList, functionMap);
    ViewerWidget mainWidget(actionList);
    mainWidget.show();
    return application.exec();
  }
```

```
    catch (exception e) {
      QMessageBox messageBox(QMessageBox::Information,
                             QString("Error"), QString(e.what()));
      messageBox.exec();
    }
  }
```

小结

本章改进了我们在上一章开始编写的 DSL：加入了选择、迭代、变量和函数调用；还加入了求值器——它接收解析器生成的指示，生成由查看器读取并执行的动作。在执行指示时，表达式的值存放在栈上，赋给名字的值存放在映射中，函数调用的返回地址存放在栈上。

这是全书的最后一章，希望你喜欢这本书！

其他你可能会喜欢的书籍

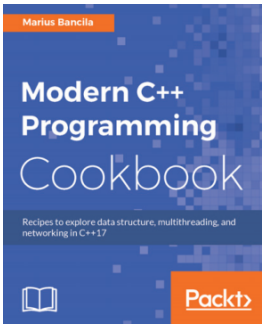
如果你喜欢这本书，可能也会对 Packt 出版的这些其他书籍感兴趣：



Beginning C++ Programming
Richard Grimes

ISBN：978-1-78712-494-3

- 熟悉 C++ 项目的结构
- 识别语言中的主要结构：函数与类
- 自信地找出代码的执行流程
- 了解标准库提供的各种功能
- 深入理解面向对象的基本概念
- 学会调试自己的程序
- 熟悉标准 C++ 库



Modern C++ Programming Cookbook
Marius Bancila

ISBN：978-1-78646-518-4

- 了解新的核心语言特性以及它们旨在解决的问题
- 理解对线程与并发的标准支持，并了解如何把它们用于日常的基础任务
- 利用 C++ 的特性提升健壮性与性能
- 探索广泛使用的 C++ 测试框架，并实现各种有用的模式与惯用法
- 处理各种类型的字符串，并考察编译的各个方面
- 探索函数与可调用对象，重点关注现代特性
- 使用标准库，配合容器、算法与迭代器工作
- 使用正则表达式进行查找与替换字符串操作
- 利用新的文件系统库处理文件与目录
- 使用标准库新增的工具解决开发者遇到的常见问题，包括 string_view、any、optional 与 variant 类型

留一篇书评——让其他读者了解你的看法

请在你购买本书的站点上留下书评，把你的想法分享给其他人。如果你是在 Amazon 购买的本书，请在这本书的 Amazon 页面上给我们留下真诚的评论。这样做很重要，其他潜在读者可以看到并使用你的客观意见做出购买决策，我们也可以了解客户对我们产品的看法，我们的作者同样能看到他们与 Packt 合作创作的这本书的反馈。这只会占用你几分钟时间，却对其他潜在客户、我们的作者和 Packt 都很有价值。谢谢！

索引

A

actions 327
高级领域特定语言
  main 函数 343
高级搜索算法 76
高级排序算法 76

B

基部件类 146, 147, 148, 150, 152, 154
基窗口类 139, 143, 144
块注释 15
粗体样式 296
Book class
  概述 81, 82, 83, 106, 107
  借阅图书 85, 108
  显示图书 85, 108
  读取图书 84, 107
  预约图书 85, 108
  写入图书 83, 107
冒泡排序算法 52, 54

C

调用栈 327
Car class
  概述 11, 15
  扩展 16, 19
Caret class 176, 178
Cell class 31
类 11
类层次结构 19, 21, 23
时钟应用
  Clock 类 133, 137, 191, 193
  创建 132
  增强 190
  环境搭建 133
  main 函数 137, 193
复合赋值 14
Customer class
  概述 86, 88, 109
  借阅图书 89, 111
  预约图书 89, 111
  显示读者 89, 111
  读取读者 111
  从文件读取读者 88
  写入读者 111
  写读者到文件 89

D

掷骰子
  掷骰子 6, 9, 11
差集方法 48
指令 326, 330
领域特定语言（DSL）
  概述 294, 326
  main 函数 324
画图程序
  构建 155
  DrawingWidget 类 212, 215, 218, 222, 225
  DrawingWindow 类 210
  Ellipse 派生类 164, 165
  EllipseX 类 206, 209
  增强 193
  Figure 基类 155, 157
  Figure 类 194, 196
  Line 类 196, 198
  Line 派生类 157, 160, 161
  main 函数 175, 225
  Rectangle 类 201, 205
  Rectangle 派生类 161
  绘制部件 167, 169, 171, 172, 174, 175
  绘制窗口 165, 166

E

编辑器
  构建 176
  Caret 类 176, 178
  EditorWidget 类 229, 231, 234, 236, 239, 241, 244, 246
  EditorWindow 类 226, 228
  增强 226
  main 函数 188, 246
  绘制部件 180, 181, 182, 184, 186, 187
  绘制窗口 178, 179
Ellipse 派生类 164, 165
EllipseX 类 206, 209
求值器
  概述 326, 338
  工作原理 340, 341, 343

F

字段 11
Figure 基类 155, 157
Figure 类 194
函数 326

G

文法
  概述 327
  增强 328

I

插入排序算法 51
交集方法 48
迭代 326
迭代器类 32

L

Library class
  概述 90, 92, 112
  读取图书对象 127
  写入图书对象 124
  添加图书 95, 115
  借阅图书 98, 119
  删除图书 95, 115
  预约图书 100, 120, 121
  归还图书 101, 122
  列出图书 95, 117
  搜索图书 94, 114, 122
  读取借阅索引 128
  写入借阅索引 125
  读取读者对象 128
  写入读者对象 125
  添加读者 96, 117
  删除读者 97, 118
  列出读者 98, 118
  搜索读者 94, 114, 122
  从文件加载图书馆信息 102, 127
  保存图书馆信息到文件 102, 124
  写入外借图书索引 126
  读取外借图书索引 129
  main 函数 130
  编写 main 函数 103
  marshmallowing 124
  释放内存 130
  读取预约图书索引 129
  写入预约图书索引 126
  读取预约索引 129
  写入预约索引 125
Line 类 196, 198
Line 派生类 157, 160, 161
List class
  概述 31, 34, 35, 37
  Cell 类 31
  扩展 54, 56
  迭代器类 32
  添加列表 38, 40
  ReverseIterator 类 56, 62, 65, 68, 68
  擦除列表中的值 40, 41
监听器
  添加 139

M

marshmallowing 105, 124
归并排序算法 76
方法
  概述 11
  构造函数 11
  析构函数 11
  读取器 11
  修改器 11

N

NaCWindow 类 264
圈叉棋应用
  概述 263, 282
  main 函数 269, 292
  NaCWidget 类 265, 268, 269, 284, 286, 289, 290, 292
  NaCWindow 类 264, 282

O

面向对象编程（OOP） 5
奥赛罗棋
  概述 248, 271
  游戏部件 249, 252, 255
  main 函数 263, 282
  OthelloWidget 类 257, 261, 263, 274, 276, 278, 282
  OthelloWindow 类 256, 272
重载 12

P

paintEvent 方法 134
参数 11
解析器
  概述 327, 330, 332, 334, 337, 338
  构建 311, 312
  求值表达式 319
  类型检查表达式 318
  解析语言表达式 315, 317
  解析语言指令 313, 314
枢轴值 78

Q

队列 5, 27, 28, 29
快速排序算法 78

R

read 方法 48
Rectangle 类 201, 205
Rectangle 派生类 161
可复用类
  添加监听器 139
  为部件搭建 139
  为窗口搭建 139
ReverseIterator 类 56, 62, 65, 68, 68

S

扫描器 303, 306, 307, 309, 311, 327
Scanner 类 329
搜索算法 49
选择排序算法 50
选择 326
Set class
  概述 42, 45, 46
  差集 71, 75
  差集运算 48
  扩展 69, 71
  交集 48, 71, 75
  并集 48, 71, 75
排序算法
  概述 49
  冒泡排序算法 52, 54
  插入排序算法 51
  选择排序算法 50
源语言
  概述 295
  颜色 299
  增强 327, 328
  错误处理 300, 302
  文法 296
  目标语言 298
  值 302, 303
栈 5, 23, 25, 27

T

记号 294, 303
Token 类 329

U

unionSet 函数 48

V

值栈 327
变量 326
查看器
  概述 327
  编写 320, 322, 323

W

部件
  基部件类 146, 147, 148, 150, 152, 154
  搭建可复用类 139
窗口
  基窗口类 139, 143, 144, 145
  搭建可复用类 139
write 方法 48